S4D400

Basic ABAP Programming

PARTICIPANT HANDBOOK
INSTRUCTOR-LED TRAINING

Course Version: 2601
Course Duration: 4 Days

SAP Copyrights, Trademarks and Disclaimers

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see https://www.sap.com/corporate/en/legal/copyright.html for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials may have been machine translated and may contain grammatical errors or inaccuracies.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.

Typographic Conventions

American English is the standard used in this handbook.

The following typographic conventions are also used.

This information is displayed in the instructor's presentation	
Demonstration	
Procedure	
Warning or Caution	
Hint	
Related or Additional Information	
Facilitated Discussion	
User interface control	*Example text*
Window title	*Example text*

© Copyright. All rights reserved.

Contents

vii	Course Overview	
1	Unit 1:	Getting Started
3		Lesson: Preparing the Development Environment
8		Lesson: Taking a First Look at ABAP
11		Exercise 1: Create an ABAP Cloud Project and Investigate ABAP Coding
16		Lesson: Understanding Software Structure and Logistics
21		Exercise 2: Create an ABAP Package
25		Lesson: Developing Your First ABAP Application
31		Exercise 3: Create a 'Hello World' Application
35		Lesson: Amplifying ABAP Development with AI
45	Unit 2:	Applying Basic Techniques and Concepts
47		Lesson: Understanding the Basics of ABAP
53		Lesson: Working With Basic Data Objects and Data Types
64		Lesson: Processing Data
69		Exercise 4: Declare Variables and Process Data
75		Lesson: Using Control Structures in ABAP
83		Exercise 5: Implement Conditional Branching
91		Lesson: Working with Simple Internal Tables
97		Exercise 6: Work with Simple Internal Tables
104		Lesson: Debugging an ABAP Program
113		Exercise 7: Debug ABAP Code
127	Unit 3:	Working with Local Classes
128		Lesson: Defining a Local Class
133		Exercise 8: Define a Local Class
137		Lesson: Creating Instances of a Class
143		Exercise 9: Create and Manage Instances
150		Lesson: Defining and Calling Methods
159		Exercise 10: Define and Call Methods
168		Lesson: Using Encapsulation to Ensure Consistency
175		Exercise 11: Use Private Attributes and a Constructor
189	Unit 4:	Reading Data from the Database
191		Lesson: Investigating a Table Definition
194		Lesson: Implementing Basic SELECT Statements
201		Exercise 12: Read Data from a Database Table
209		Lesson: Working with CDS Views
215		Exercise 13: Analyze and Use a CDS View Entity

© Copyright. All rights reserved.

227	**Unit 5:**	**Working with Structured Data Objects**
229		Lesson: Declaring a Structured Data Object
235		Lesson: Working with Structured Data Objects
245		Exercise 14: Use a Structured Data Object
253	**Unit 6:**	**Working with Complex Internal Tables**
255		Lesson: Declaring a Complex Internal Table
262		Lesson: Working with Complex Internal Tables
275		Exercise 15: Use a Complex Internal Table
287	**Unit 7:**	**Implementing Database Updates Using Business Objects**
288		Lesson: Analyzing a Business Object
295		Exercise 16: Analyze a Business Object
304		Lesson: Using the Entity Manipulation Language
311		Exercise 17: Modify Data Using EML
319	**Unit 8:**	**Describing the ABAP RESTful Application Programming Model**
321		Lesson: Introducing the Programming Model
325		Lesson: Exploring the Architecture
333		Exercise 18: Copy a Database Table
345		Exercise 19: Generate and Preview an OData UI Service
353		Lesson: Adding ABAP logic
367		Exercise 20: Validate the Flight Price
375		Lesson: Improving the User Experience
381		Exercise 21: Adjust the User Interface

© Copyright. All rights reserved.

Course Overview

TARGET AUDIENCE

This course is intended for the following audiences:

- Development Consultant
- Developer

© Copyright. All rights reserved.

© Copyright. All rights reserved.

UNIT 1 Getting Started

Lesson 1

Preparing the Development Environment 3

Lesson 2

Taking a First Look at ABAP 8

Exercise 1: Create an ABAP Cloud Project and Investigate ABAP Coding 11

Lesson 3

Understanding Software Structure and Logistics 16

Exercise 2: Create an ABAP Package 21

Lesson 4

Developing Your First ABAP Application 25

Exercise 3: Create a 'Hello World' Application 31

Lesson 5

Amplifying ABAP Development with AI 35

UNIT OBJECTIVES

- Create an ABAP Cloud project.
- Work with a development object.
- Create an ABAP package.
- Create a 'Hello World' application.
- Explain how ABAP AI Capabilities can assist ABAP developers during application development.
- Briefly describe how the ABAP AI SDK can be used to add AI features to business applications.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 1
Lesson 1

Preparing the Development Environment

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Create an ABAP Cloud project.

ABAP Cloud Project

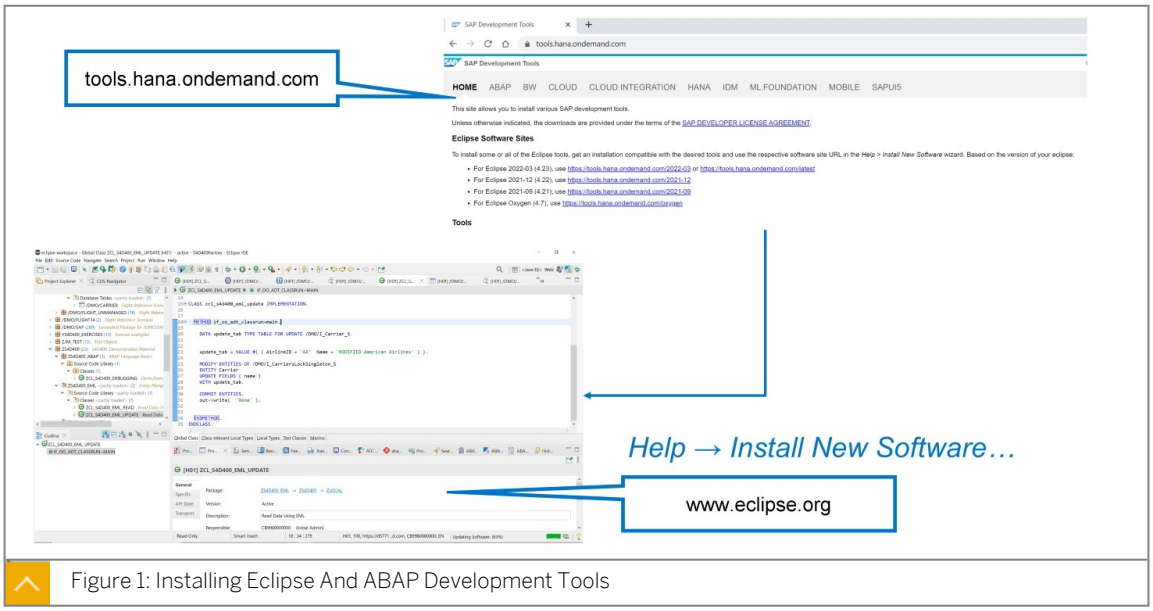


Figure 1: Installing Eclipse And ABAP Development Tools

To develop ABAP applications for SAP BTP, you need the Eclipse development environment. You can download it free of charge from eclipse.org. The default Eclipse installation does not contain the ABAP Development Tools (ADT) - you must download these separately. To do so, open Eclipse and choose *Help -> Install New Software*. A dialog box appears, in which you enter the URL, https://tools.hana.ondemand.com/latest. This downloads the ABAP tools and installs them in Eclipse.

You should periodically check for updates to both Eclipse and the ABAP tools. You do this in Eclipse by choosing *Help -> Check for Updates*.

Note:
In most classroom trainings, we use a Windows Terminal Server (WTS) with a pre-configured Eclipse IDE installation.

© Copyright. All rights reserved.

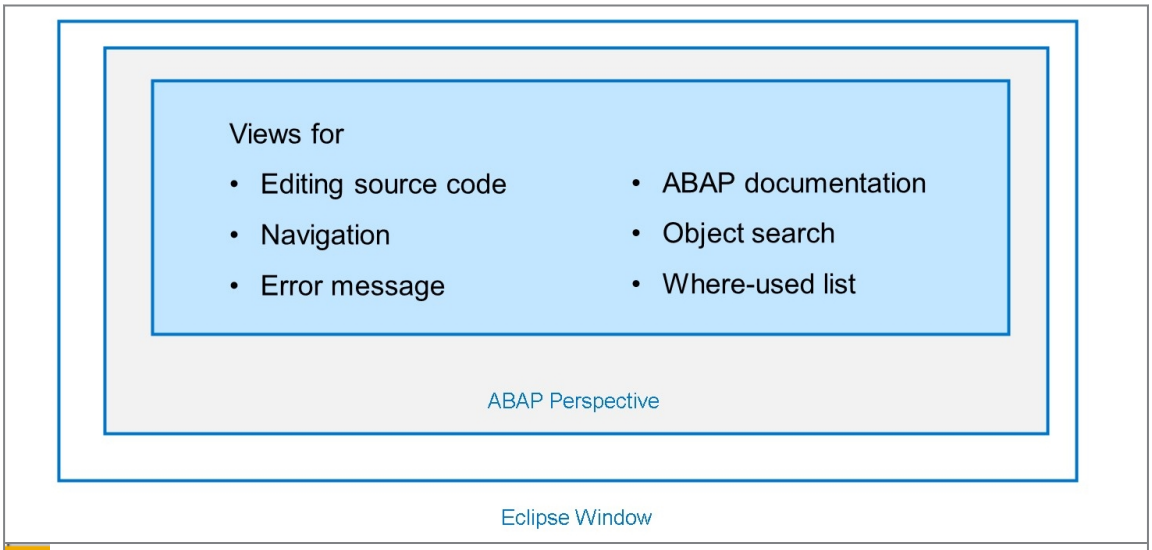


Figure 2: The ABAP Perspective

When you have installed Eclipse and ADT, you need to switch to the ABAP perspective. A perspective in Eclipse is a window in which you can perform a certain task. The default perspective in a fresh Eclipse installation is the Java development perspective. To develop ABAP applications, choose *Window → Perspective → Open Perspective → Other*. A dialog box appears with a list of all of the available perspectives. Double-click *ABAP*.

A perspective is made up of a set of views, which appear as tabs in the Eclipse window. The most important views are the source code editor and the project explorer, which you use to navigate between different development objects. There is a wide range of other views that you will need, such as the Problems view to display error messages, the documentation view, and views for searching for objects or displaying where in the system a particular object is used.

Different Types of ABAP Project

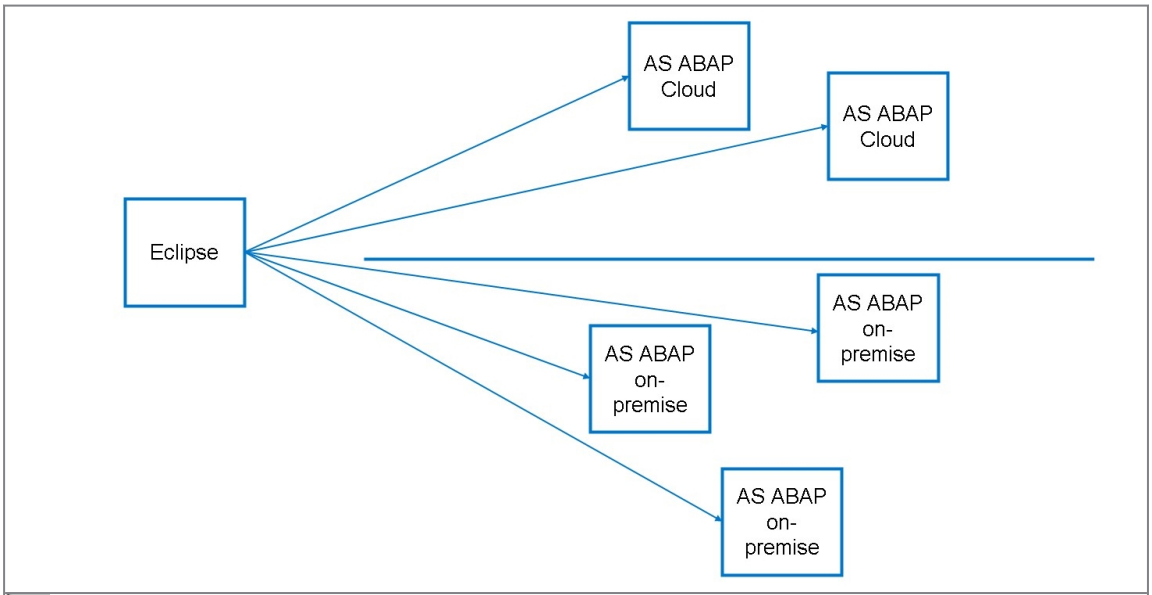


Figure 3: Different Types of ABAP Project

Eclipse as a development environment is not embedded in the ABAP system. Instead, you have to connect to each ABAP system in which you want to work, and each connection is represented in Eclipse by a project. There are two kinds of project in ADT - *ABAP Projects*,

© Copyright. All rights reserved.

which you use to connect to an on-premise ABAP system or to SAP S/4HANA Private Cloud, and *ABAP Cloud Projects*, which you use to connect either to SAP BTP ABAP or to SAP S/4HANA Public Cloud.

> Note:
> In the following, we are only going to discuss how to access a cloud system.

Locating the ABAP Environment

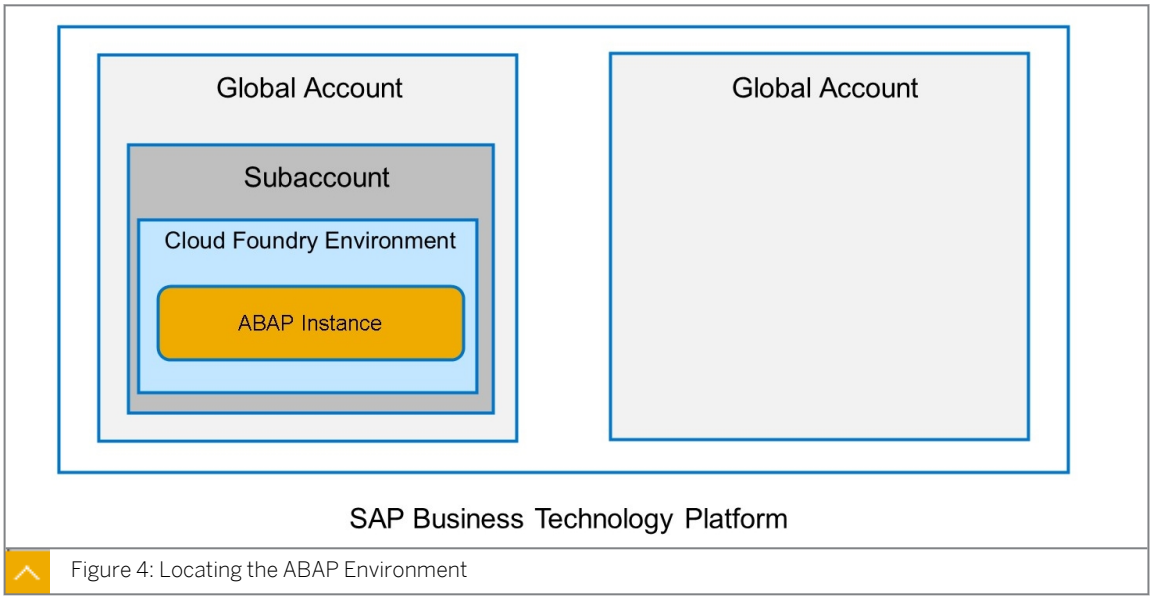


Figure 4: Locating the ABAP Environment

The SAP Business Technology Platform is SAP's platform as a service (PAAS). To access it, you need to create a global account. There are various subscription models available, depending on whether you need to run large-scale productive environments or just a single-user environment for your own continuing professional development.

Within a global account, there are one or more subaccounts. Each subaccount can be configured differently, so that a single enterprise can run several different platforms but manage their subscription using just the single global account. Inside the subaccount, you deploy a runtime such as Cloud Foundry or Kyma. Once you have done this, you can deploy an ABAP instance.

> Note:
> In this course we are using an ABAP instance deployed on the SAP Business Technology Platform (SAP BTP). However, the material is also relevant for other ABAP deployments, such as an on-premise SAP S/4HANA system or an SAP S/4HANA Cloud system.

© Copyright. All rights reserved.

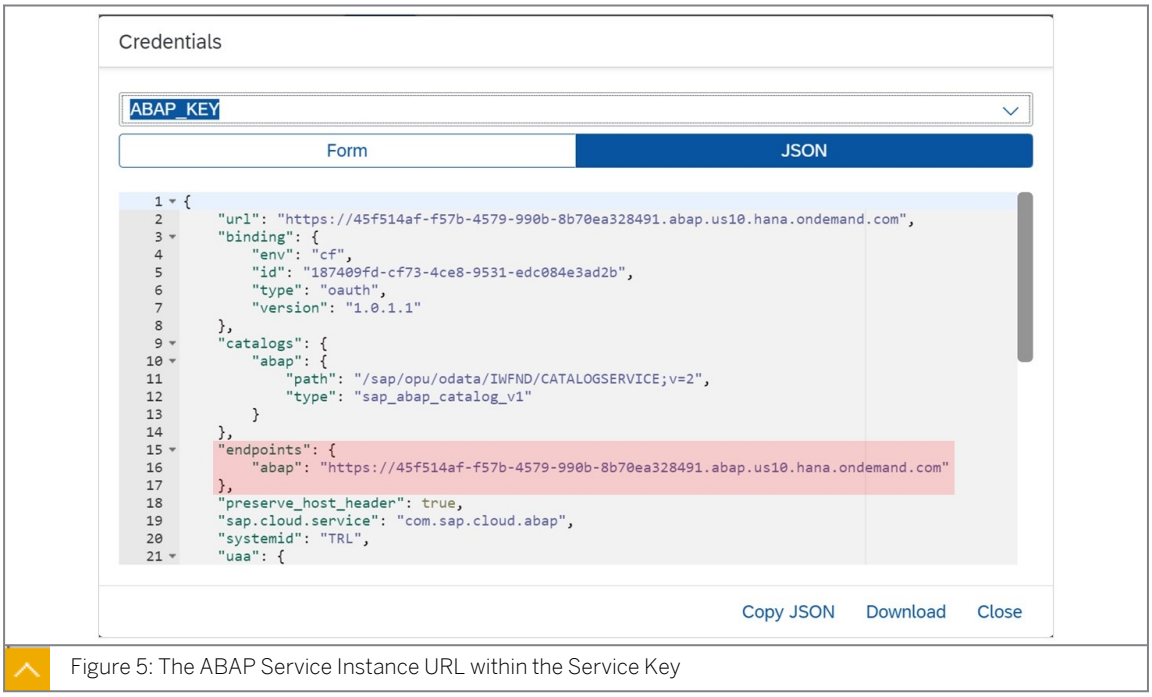


Figure 5: The ABAP Service Instance URL within the Service Key

To log on to an ABAP instance on SAP BTP, you need the *ABAP Service Instance URL* as well as a user and password. The *ABAP Service Instance URL* is specified within the *Service Key*. The *Service Key* is specific for the ABAP instance and is created in the SAP BTP where it can be downloaded to a text file.

Note:
For this training, it is not necessary to create or download a *Service Key*. Your instructor will provide you with the *ABAP Service Instance URL*, a user name, and a password.

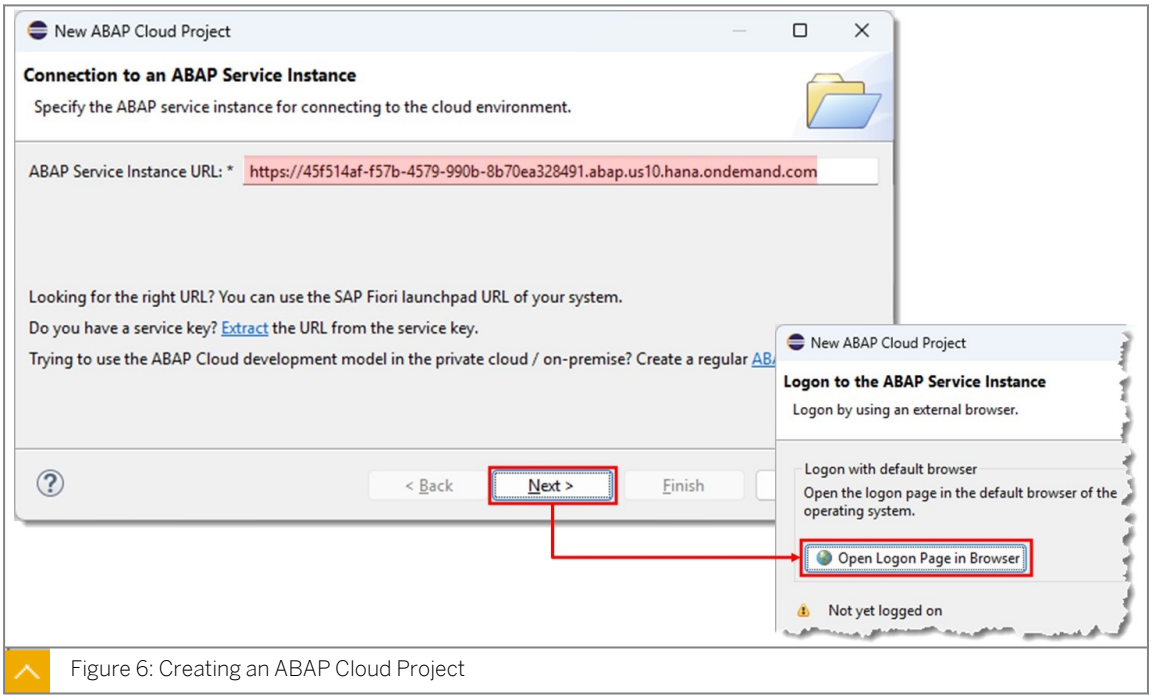


Figure 6: Creating an ABAP Cloud Project

© Copyright. All rights reserved.

In order to develop ABAP, you must log on to your ABAP account. To do this, proceed as follows:

1. In Eclipse, choose *File* → *New* → *ABAP Cloud Project*.
2. A dialog box appears, in which you enter the *ABAP Service Instance URL*.
3. Choose *Next*, and then select *Open Logon Page in Browser*.
4. Log on with user and password.
5. Close the browser window and return to Eclipse. Set the language that you want to use, and choose *Finish*.

LESSON SUMMARY

You should now be able to:

- Create an ABAP Cloud project.

© Copyright. All rights reserved.

Unit 1
Lesson 2

Taking a First Look at ABAP

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Work with a development object.

Investigation of ABAP Code

Working in the Project Explorer

To work with ABAP development objects, you usually display the contents of a package in the project explorer. Packages are containers for development objects that logically belong together.

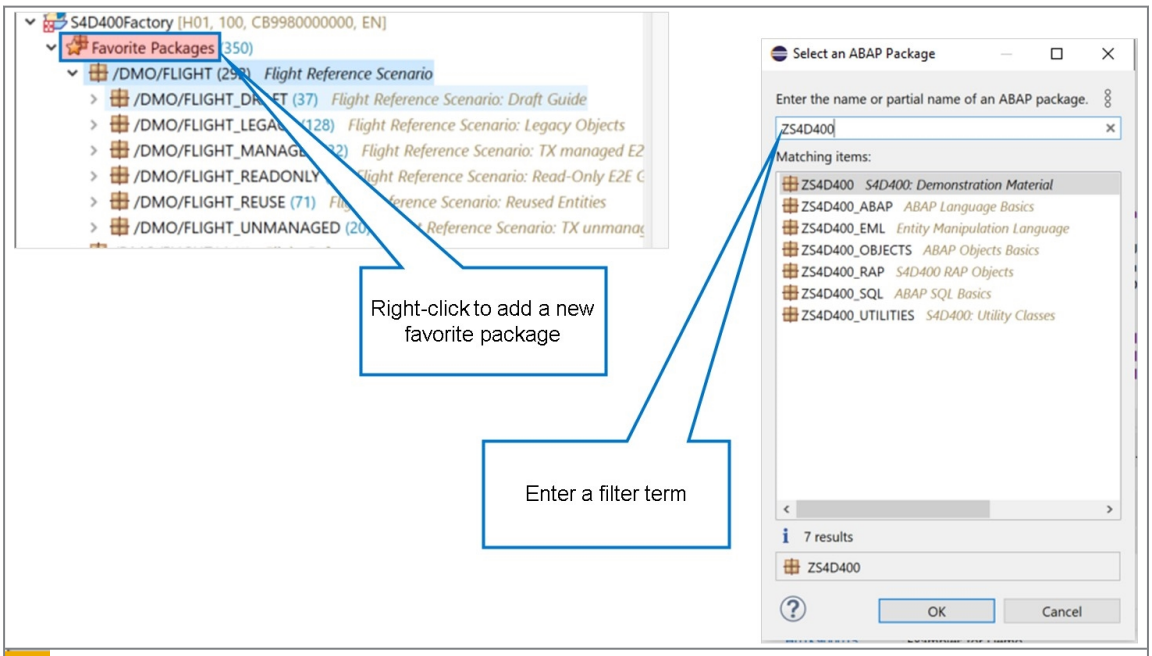


Figure 7: The Project Explorer

To add a package to your favorites, right-click the *Favorite Packages* node in the project explorer and choose *Add Package*. A dialog box appears, in which you can enter a filter term. The system then displays only the packages that contain this term. Double-click the package that you want to add to your favorites.

Opening Development Objects

There are two ways to open a development object in ADT.

© Copyright. All rights reserved.

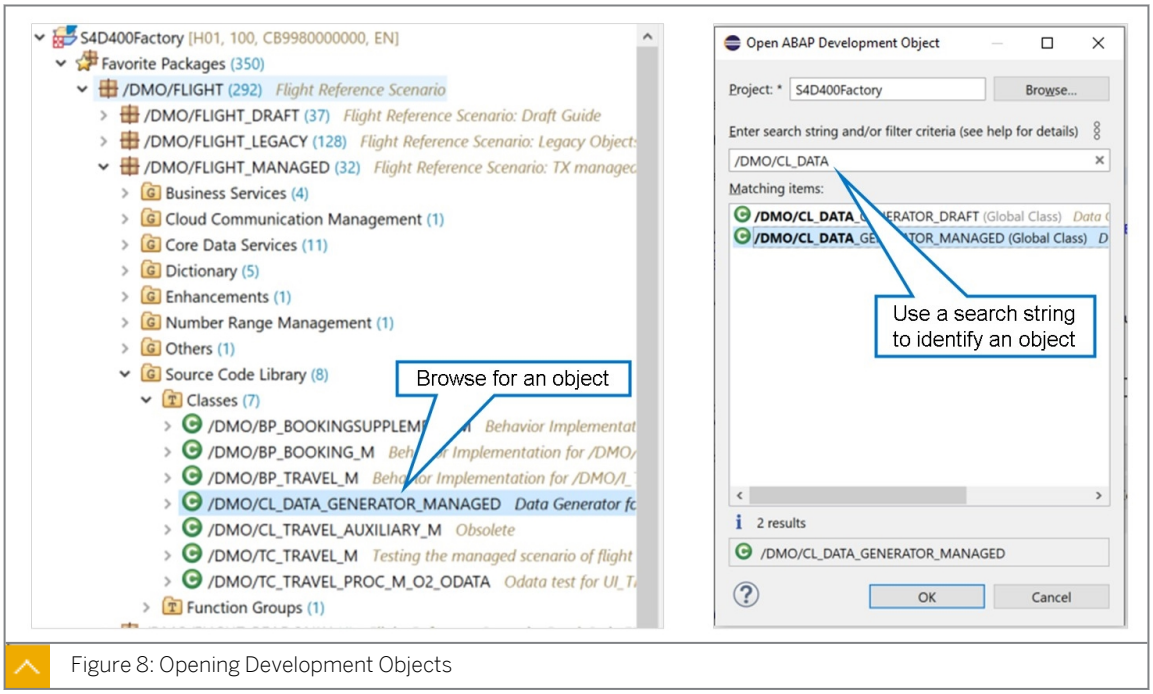


Figure 8: Opening Development Objects

The first is to locate the object name in the project explorer and double-click it. The other is to use the keyboard shortcut, Ctrl + Shift + A. This opens a dialog box in which you can enter part of the name of an object.

Investigating a Development Object

When you are working with ABAP code, there are certain function keys in ADT that will help you.

Let's look at these function keys.

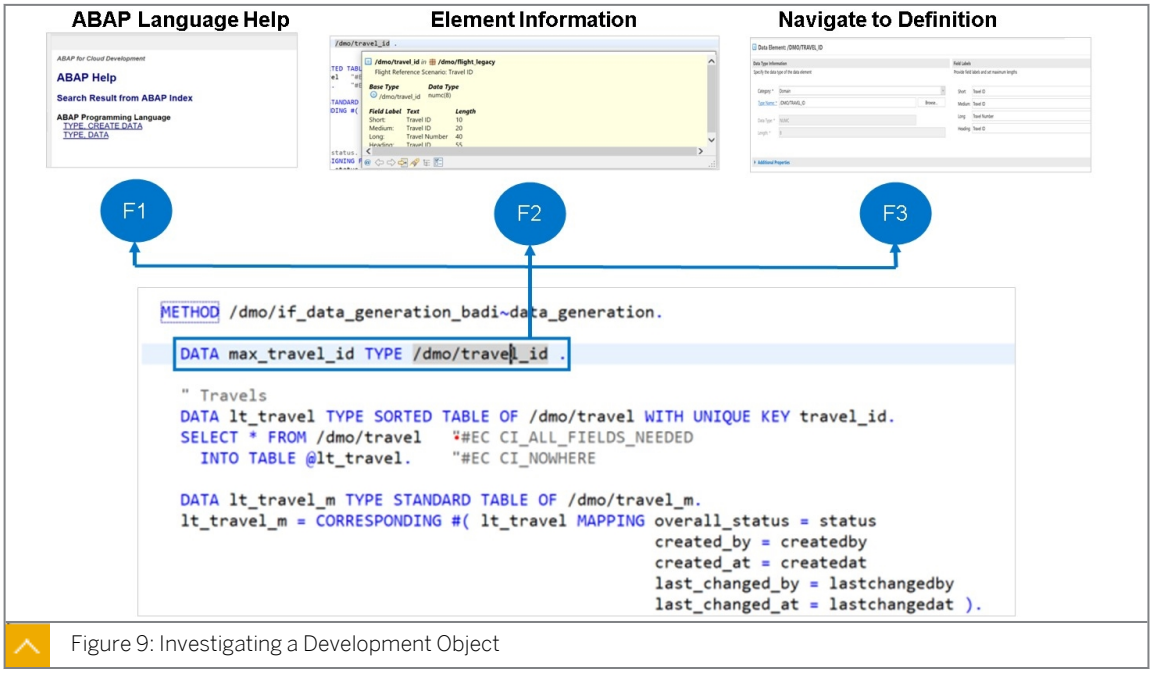


Figure 9: Investigating a Development Object

F1

© Copyright. All rights reserved.

The F1 key displays the ABAP language help for the current statement. A statement is the name for a command in ABAP.

F2

The F2 key displays information about the element on which the cursor is placed.

F3

The F3 key navigates to the definition of the object on which the cursor is placed. You can return from there to the original object using the key combination, `ALT + LEFT ARROW`, on your keyboard.

© Copyright. All rights reserved.

Unit 1
Exercise 1

Create an ABAP Cloud Project and Investigate ABAP Coding

In this exercise, you start your ABAP cloud project and analyze some existing ABAP code to get familiar with some basic features of the ABAP development environment.

Task 1: Create an ABAP Cloud Project
Open Eclipse, switch to the ABAP perspective and create an ABAP Cloud Project.

1. Open Eclipse and switch to the ABAP perspective.
2. Create an *ABAP Cloud Project*.

Task 2: Analyze ABAP Class /DMO/CL_FLIGHT_LEGACY
Use some functions of the *ABAP Development Tools* to analyze the source code of ABAP class **`/DMO/CL_FLIGHT_LEGACY`**.

Note:
You are not required to read the code in this exercise. Instead, concentrate on the navigation and display functions in the development environment.

1. Add the **`/DMO/FLIGHT`** package to your *Favorite Packages*.
2. Open the ABAP class **`/DMO/CL_FLIGHT_LEGACY`** in the editor view.
3. Open the *Properties* tab to display the administrative data of development object **`/DMO/CL_FLIGHT_LEGACY`**.
4. Locate the ABAP class **`/DMO/CL_FLIGHT_LEGACY`** in the *Project Explorer* view on the left of the display.
5. Find the string **`get_instance( )->get`** in the source code of ABAP class **`/DMO/CL_FLIGHT_LEGACY`**.

Note:
There has to be exactly one blank between the brackets.

6. In the current code line, navigate to the implementation of code element `get`.
7. A few code lines down there is a line starting with `SELECT`. Without navigating away, display some information about code element `/dmo/travel` after keyword `FROM`.
8. Display the ABAP Language Help for the keyword `SELECT`.

© Copyright. All rights reserved.

Unit 1
Solution 1

Create an ABAP Cloud Project and Investigate ABAP Coding

In this exercise, you start your ABAP cloud project and analyze some existing ABAP code to get familiar with some basic features of the ABAP development environment.

Task 1: Create an ABAP Cloud Project
Open Eclipse, switch to the ABAP perspective and create an ABAP Cloud Project.

1. Open Eclipse and switch to the ABAP perspective.
   a) Open Eclipse.
   b) Choose *Window* → *Perspective* → *Open Perspective* → *Other...*.
   c) In the dialog box, double-click *ABAP*.
2. Create an *ABAP Cloud Project*.
   a) Choose *File* → *New* → *ABAP Cloud Project*.
   b) Proceed as follows on the *New ABAP Cloud Project* dialog that appears:
      If you have been given the *ABAP Service Instance URL*:
      i. Copy and paste this URL in the corresponding field on the dialog.
      ii. Choose *Next*.
      If you have been given a file containing the *Service Key*:
      i. Select the *Extract* link on the dialog.
      ii. Choose *Import...* and select the file with the *Service Key*.
      iii. After importing click *Copy to Clipboard* to copy the *Service Instance URL*.
      iv. Choose *Close*.
      v. In the top of the popup box in the *ABAP Service Instance URL:** field, paste the *ABAP Service Instance URL* with the key combination *Ctrl + V*.
      vi. Choose *Next*.
   c) Choose *Open Logon Page in Browser*.
   d) You will be prompted to logon to the system.
      Enter the user and password provided by your instructor.
   e) When you see the message, *You have been successfully logged on*, close the browser window and return to Eclipse.
   f) To finish creating the project, choose *Finish*.

© Copyright. All rights reserved.

Task 2: Analyze ABAP Class /DMO/CL_FLIGHT_LEGACY

Use some functions of the *ABAP Development Tools* to analyze the source code of ABAP class **`/DMO/CL_FLIGHT_LEGACY`**.

> Note:
> You are not required to read the code in this exercise. Instead, concentrate on the navigation and display functions in the development environment.

1. Add the **`/DMO/FLIGHT`** package to your *Favorite Packages*.
   a) In the *Project Explorer* view, right-click on the *Favorite Packages* folder under your project folder.
   b) Choose *Add Package...* from the context menu.
   c) In the input field, enter **`/DMO/FLIGHT`** as search string.
   d) In the list of matching items, click on the */DMO/FLIGHT* entry and choose *OK*.
2. Open the ABAP class **`/DMO/CL_FLIGHT_LEGACY`** in the editor view.
   a) In the Eclipse menu, choose *Navigate* → *Open ABAP Development Object...* or press **`Ctrl + Shift + A`**.
   b) In the input field, enter **`/DMO/CL_FLIGHT`** as search string.
   c) In the list of matching items, click on */DMO/CL_FLIGHT_LEGACY (Class)* and choose *OK*.
3. Open the *Properties* tab to display the administrative data of development object **`/DMO/CL_FLIGHT_LEGACY`**.
   a) Place the cursor anywhere in the source code of class **`/DMO/CL_FLIGHT_LEGACY`**.
   b) In the tab strip below the editor view, navigate to *Properties* tab.
   c) Analyze the administrative data that is displayed there, for example the original language or the timestamp of the last change.
4. Locate the ABAP class **`/DMO/CL_FLIGHT_LEGACY`** in the *Project Explorer* view on the left of the display.
   a) Place the cursor anywhere in the source code of the class **`/DMO/CL_FLIGHT_LEGACY`**.
   b) From the toolbar of the *Project Explorer*, choose *Link with Editor*. This should expand a part of the tree under your ABAP project with the ABAP class **`/DMO/CL_FLIGHT_LEGACY`** as its end point.
5. Find the string **`get_instance( )->get`** in the source code of ABAP class **`/DMO/CL_FLIGHT_LEGACY`**.

> Note:
> There has to be exactly one blank between the brackets.

© Copyright. All rights reserved.

a) Place the cursor in the first row of the source code of class **`/DMO/CL_FLIGHT_LEGACY`**.

b) Press **`Ctrl + F`** to open the *Find/Replace* dialog.

c) In the *Find* input field that appears, enter **`get_instance( )->get`**.

6. In the current code line, navigate to the implementation of code element `get`.

a) Press the *Ctrl* key and click on *get*. From the appearing menu choose *Navigate to Implementation*. Alternatively, place the cursor in the word *get* and press **`F3`**.

7. A few code lines down there is a line starting with `SELECT`. Without navigating away, display some information about code element `/dmo/travel` after keyword `FROM`.

a) In the code line starting with `SELECT`, place the cursor on `/dmo/travel` and press **`F2`**.

8. Display the ABAP Language Help for the keyword `SELECT`.

a) At the beginning of the current code line, place the cursor on `SELECT` and press **`F1`**.

Hint:
If the ABAP Language Help view that opens indicates that the result set of the search query is not supported in the current ABAP language version, enter **`SELECT`** in the *search term* field of the ABAP Language Help view and press *Enter*. This search should provide the desired help.

Note:
There is no need to read or understand the documentation at this point.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Work with a development object.

© Copyright. All rights reserved.

Unit 1
Lesson 3

Understanding Software Structure and Logistics

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Create an ABAP package.

ABAP Package

Organizing Development

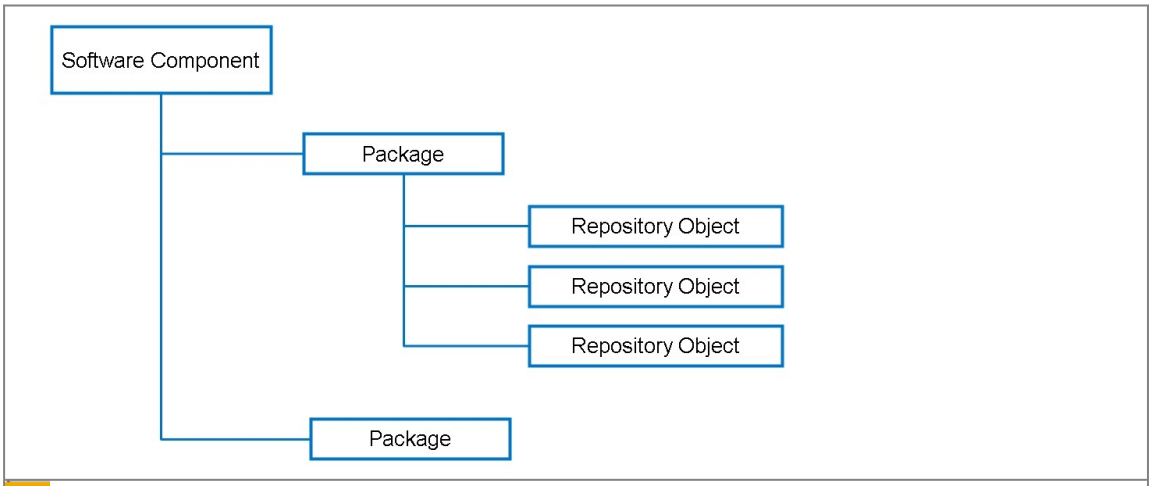


Figure 10: Organizing Development

When you create a development object in the ABAP environment, you must assign it to a package. Packages are development objects that serve as containers for all of the development objects that logically belong together. Each package is also assigned to a software component. The complete set of development objects in the system is referred to as the ABAP Repository. Consequently, development objects are also often called repository objects.

© Copyright. All rights reserved.

Note:
Development objects such as packages are organized using namespaces. Namespaces ensure a clear separation between SAP standard objects and customer or partner developments. Customer developments must be created in the customer namespace, which starts with Z or Y. This means that the first letter of the name of a development object, such as a package name, must be Z or Y. Customers and partners can also request their own namespaces from SAP. The names of the development objects are then prefixed with /<namespace>/. The namespace concept prevents name conflicts and protects customer code during upgrades and updates.

You develop your applications in a development environment, but must then ensure that they can be tested in an appropriate test environment before being moved on to the production environment. Typically, you will have a single global account and a subaccount for each of the development, test, and production environments. Software components allow you to transport your objects.

Transport Request

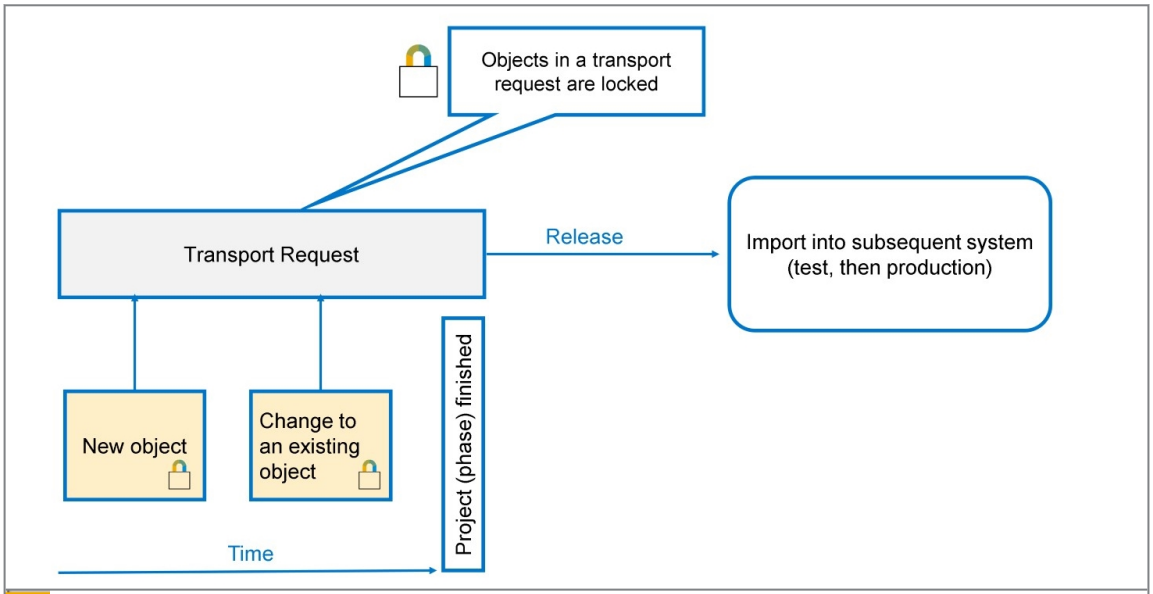


Figure 11: Transport Request

When you create a new development object or change an existing one, you must assign it to a transport request. Transport requests ensure that all development objects that logically belong together are transported together into the test, and subsequently the production system.

Each transport request has an owner, and the owner can assign other users to the request. In this way, transport requests support team development.

When an object is included in a transport request, it is locked. This means that it can only be edited by a user who is assigned to the same request.

When work on all of the objects in the request is finished, all of the developers assigned to it must release their work. After this, the owner of the request can release the entire request. The system administrator can then import it into the test system for testing.

© Copyright. All rights reserved.

When you release any kind of transport request, the system releases the locks on the objects in the request, so that any developer can access them again.

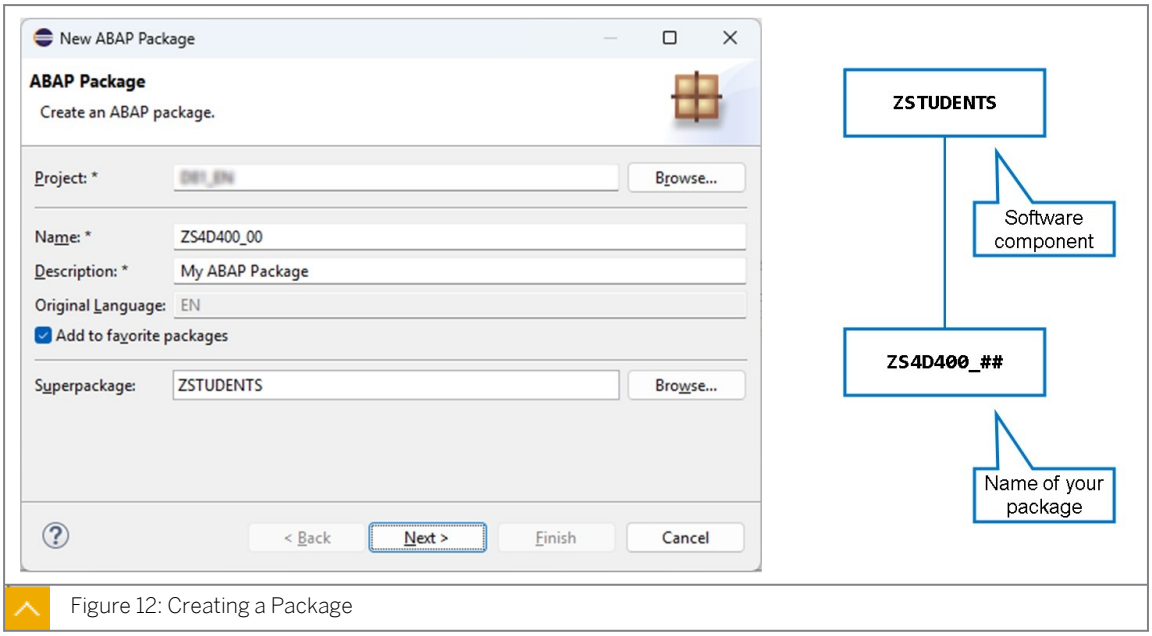


Figure 12: Creating a Package

To create a new package, choose *File → New → ABAP Package*. Enter the name of the package, which must begin with the letter Z or Y. This is a requirement for all ABAP development in the customer namespace, and serves to avoid naming collisions, as objects delivered by SAP never use Z or Y as the first character of the name.

Enter a description for the package and a superpackage. In this class, we will use ZSTUDENTS as the superpackage. ZSTUDENTS is a generated package that belongs to the software component of the same name. This means that by specifying ZSTUDENTS as the superpackage, you assign a package to the software component ZSTUDENTS.

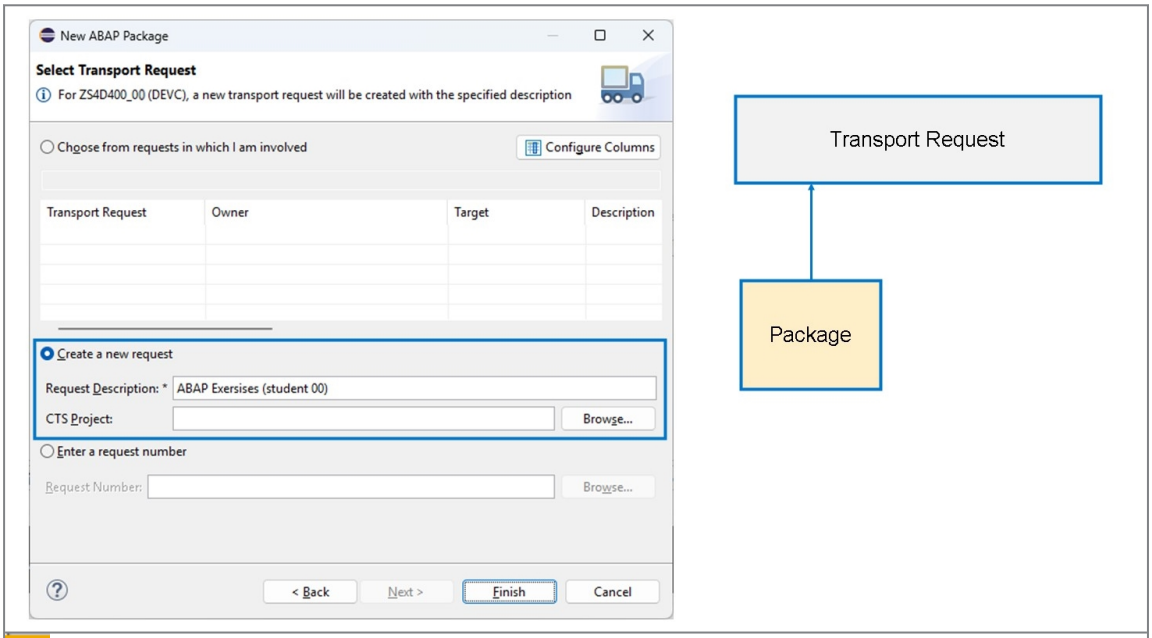


Figure 13: Assigning a Package to a Transport Request

© Copyright. All rights reserved.

You must assign your package to a transport request. As you have not yet worked with transport requests, select *Create a new request*. Enter a description for the transport request and choose *Finish*.

You are now ready to start developing. You will assign all of the further objects that you create to your package and to the same transport request.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 1
Exercise 2

Create an ABAP Package

In this exercise, you first add package **/LRN/S4D400_EXERCISE** to the list of *Favorite Packages*. You then create a new package under the superpackage **ZSTUDENTS** (suggested name: **ZS4D400_##** where ## is your group number).

Note:
This package will be the home for all development objects that you will create in this course.

Your new package should have the following attributes:

Attribute	Value
Name	**ZS4D400_##**, where **##** is your group number
Description	**My ABAP Package**
Add to favorite packages	Checked
Superpackage	**ZSTUDENTS**

1. Add package **/LRN/S4D400_EXERCISE** to the list of *Favorite Packages*.
2. In your ABAP Cloud project, create a new package with the attributes listed above. When you are prompted for a transport choose the transport request in which you are involved. If no transport request is listed, create a new request.
3. Ensure that your new package has been added to the *Favorite Packages*.

© Copyright. All rights reserved.

Unit 1
Solution 2

Create an ABAP Package

In this exercise, you first add package **/LRN/S4D400_EXERCISE** to the list of *Favorite Packages*. You then create a new package under the superpackage **ZSTUDENTS** (suggested name: **ZS4D400_##** where ## is your group number).

Note:
This package will be the home for all development objects that you will create in this course.

Your new package should have the following attributes:

Attribute	Value
Name	**ZS4D400_##**, where **##** is your group number
Description	**My ABAP Package**
Add to favorite packages	Checked
Superpackage	**ZSTUDENTS**

1. Add package **/LRN/S4D400_EXERCISE** to the list of *Favorite Packages*.
   a) In the *Project Explorer* on the left, expand your ABAP Cloud project.
   b) Right-click *Favorite Packages* and choose *Add Package...*.
   c) In the search field, enter **/LRN/S4D400**.
   d) From the list of matching items, select */LRN/S4D400_EXERCISE*, then choose *OK*.
2. In your ABAP Cloud project, create a new package with the attributes listed above. When you are prompted for a transport choose the transport request in which you are involved. If no transport request is listed, create a new request.
   a) In the *Project Explorer*, right-click on your ABAP Cloud project and choose *New* → *ABAP Package*.
   b) Enter the package name **ZS4D400_##** where **##** is your group number.
   c) Enter the description **My ABAP Package**.
   d) Mark the checkbox *Add to favorite packages*.
   e) Enter the superpackage **ZSTUDENTS**.

© Copyright. All rights reserved.

f) Choose *Next*.

g) Ensure the *Software Component* is set to **`ZSTUDENTS`**.

h) Choose *Next*.

i) Check if there is a transport request listed under option *Choose from requests in which I am involved*. If this is the case, choose this option. If the list is empty, choose the option *Create a new request* and enter a request description, for example **`ABAP Exercises`**.

j) Choose *Finish*.

3. Ensure that your new package has been added to the *Favorite Packages*.

a) In the *Project Explorer* on the left, expand your ABAP Cloud project and then sub node *Favorite Packages*.

b) Make sure your own package, **`ZS4D400_##`**, is listed here. If not, add it like you added package **`/LRN/S4D400_EXERCISE`** earlier.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Create an ABAP package.

© Copyright. All rights reserved.

Developing Your First ABAP Application

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Create a 'Hello World' application.

A 'Hello World' App in ABAP

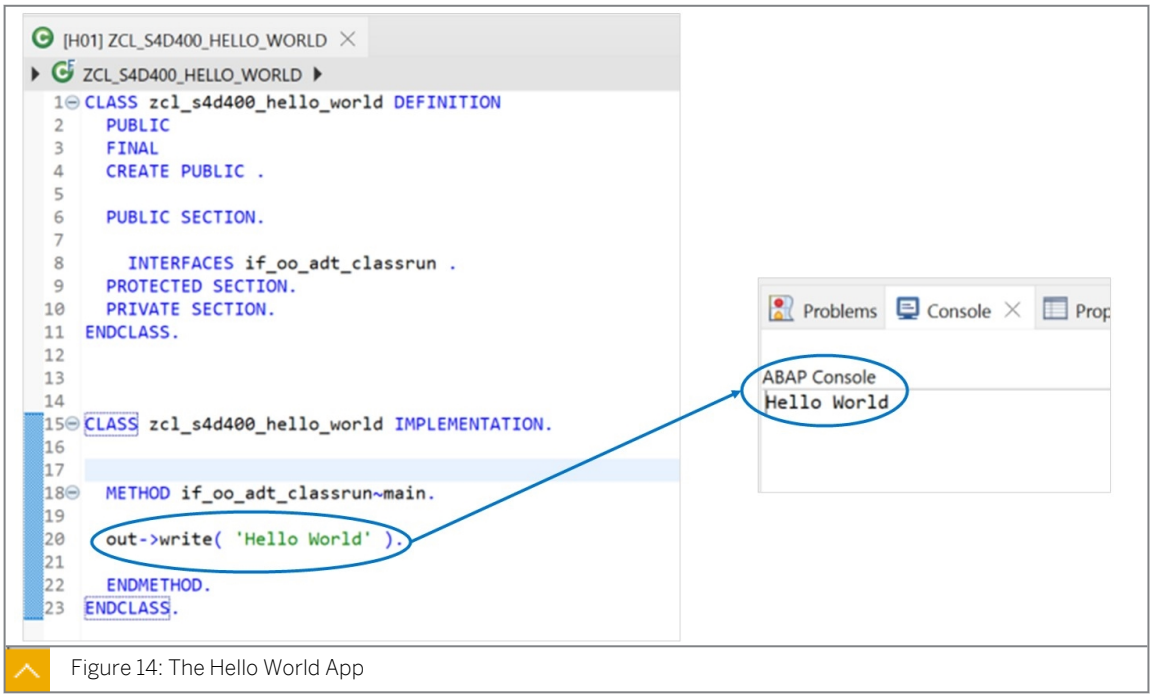


Figure 14: The Hello World App

As in all other programming languages, the first thing that you should do in ABAP is familiarize yourself with the development environment and the most elementary aspects of the language by writing a short “Hello World” app.

The main user interface technology that you will use in modern ABAP programming is SAP Fiori elements. However, ADT provides a console for you to create output quickly and simply in test applications.

© Copyright. All rights reserved.

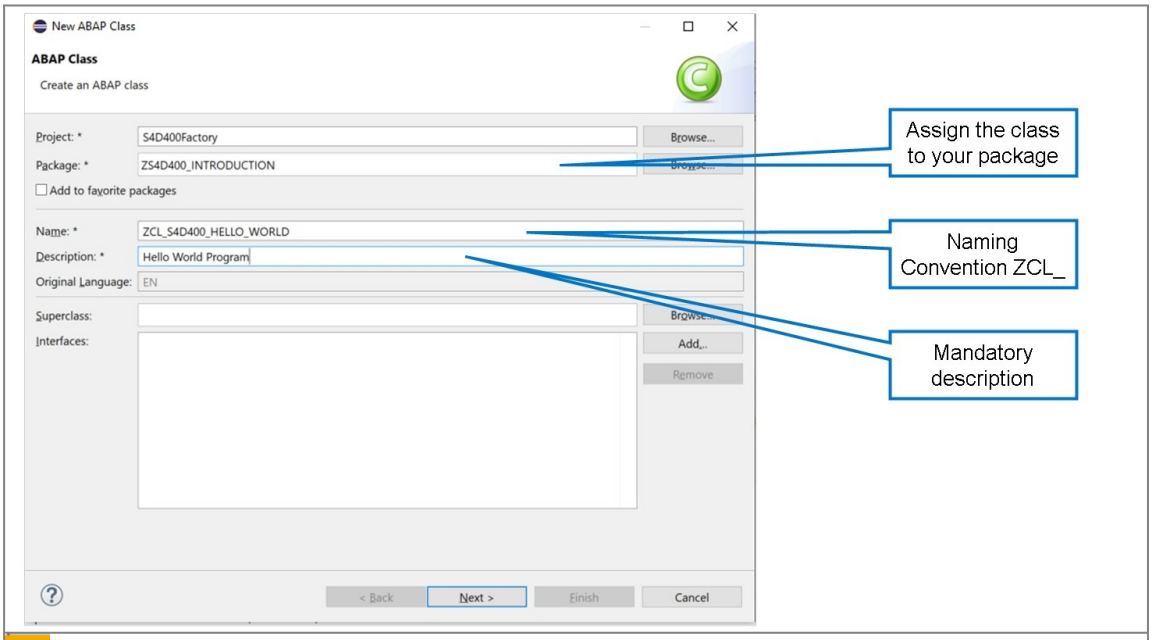


Figure 15: Creating a Class

You will write your ABAP code in a class. To create one, choose *File → New → ABAP Class*. A dialog box appears. Ensure that the project is correct, and enter the name of the package that you have already created.

Next you must name your class, following ABAP naming conventions; the name of the class must start with Z or Y to avoid naming collisions with SAP's own objects. Naming conventions also require that the name of the class is also prefixed with *CL* for class.

Following the prefix *ZCL*, enter the actual name of the class. This should be as easily understandable as possible. The name of the class (including prefix) can be up to 30 characters long and may consist of the letters A-Z, the digits 0-9, and the underscore symbol _. You use the underscore to separate the individual words that you use in the name of the class.

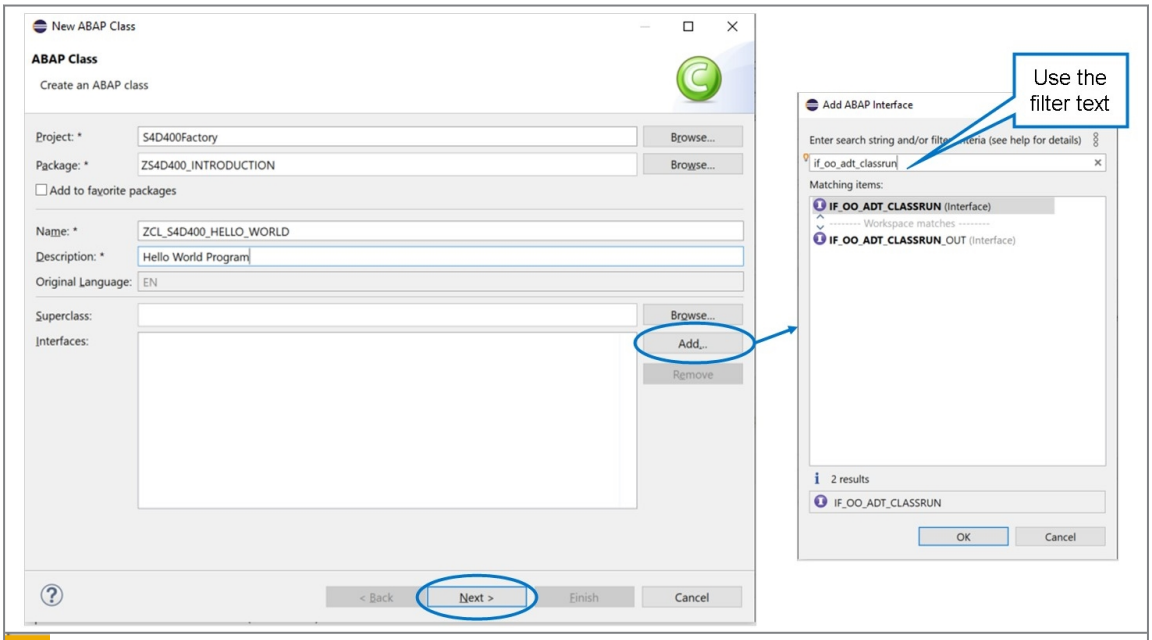


Figure 16: Adding the Interface IF_OO_ADT_CLASSRUN

© Copyright. All rights reserved.

Once you have entered the name of your class, choose *Add...* to add an interface to the class. Interfaces ensure that a user of the class can use it in a particular way. The interface IF_OO_ADT_CLASSRUN ensures that the development environment can run the class and display output in the Eclipse console. In the *Add ABAP Interface* dialog box, use the filter field to restrict the number of entries in the hit list. Once the interface IF_OO_ADT_CLASSRUN is displayed, double-click it. When you return to the *New ABAP Class* dialog box, the interface appears in the list of interfaces.

Choose *Next*.

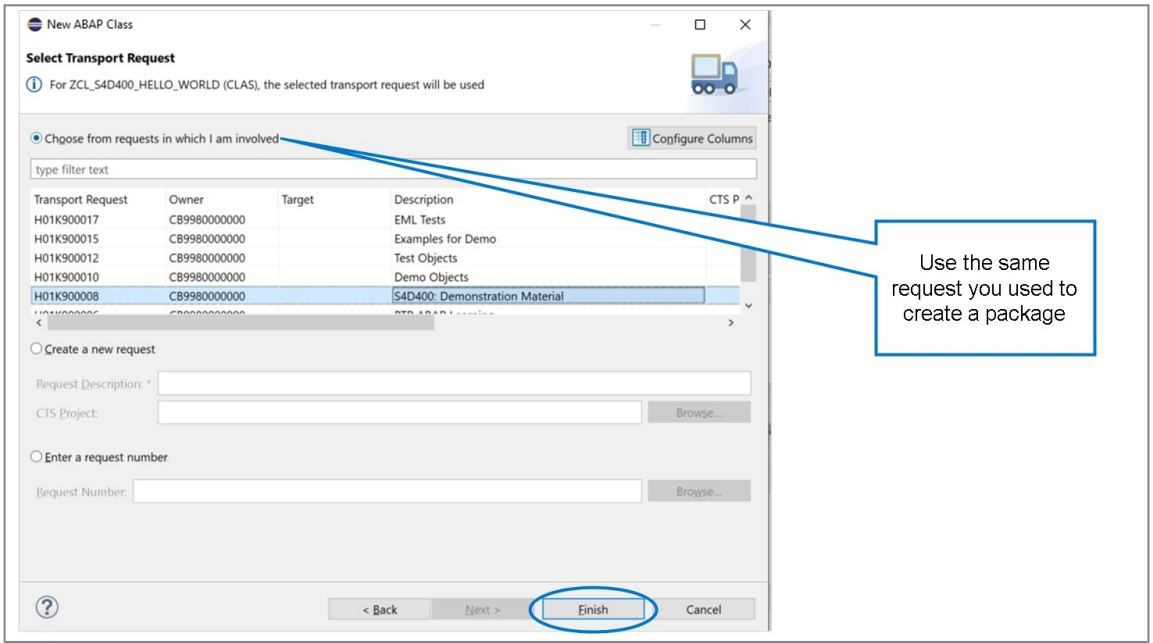


Figure 17: Assigning the Class to a Transport Request

On the last screen, you assigned the class to a package. Now you must assign it to a transport request. Under *Choose from requests in which I am involved*, mark the request that you used to create your package and choose *Finish*.

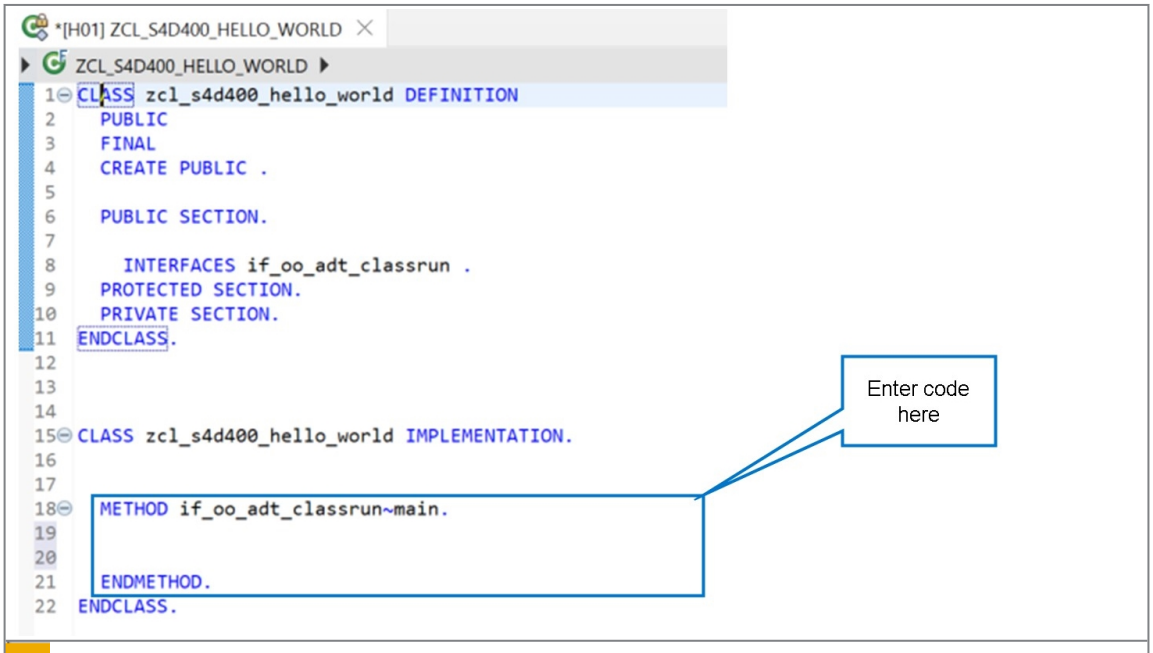


Figure 18: Editing the Class

© Copyright. All rights reserved.

The interface IF_OO_ADT_CLASSRUN allows you to run a class in ADT using the *F9* key. When you do so, the system executes the code between `METHOD if_oo_adt_classrun~main` and `ENDMETHOD`. In this code block you can output information in the ADT console.

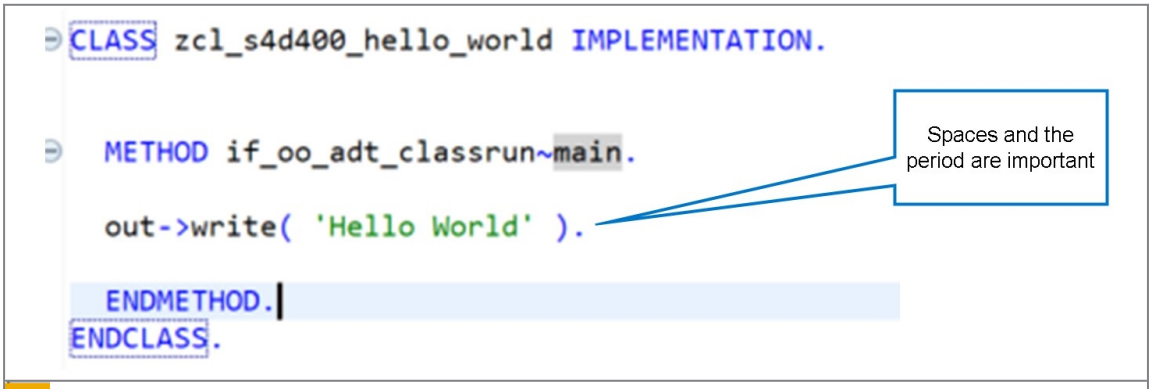


Figure 19: Using out->write( )

In your code block, you can use `out->write( )` to display information in the console. The line `out->write( 'Hello World' ).` prints *Hello World* to the console. Crucially, you do not have to know at this point how it works, you just have to type in the code, ensuring the following:

- There is no space between `write` and the opening parenthesis
- There is at least one space after the opening parenthesis
- There is at least one space before the closing parenthesis
- There is a period at the end of the line.

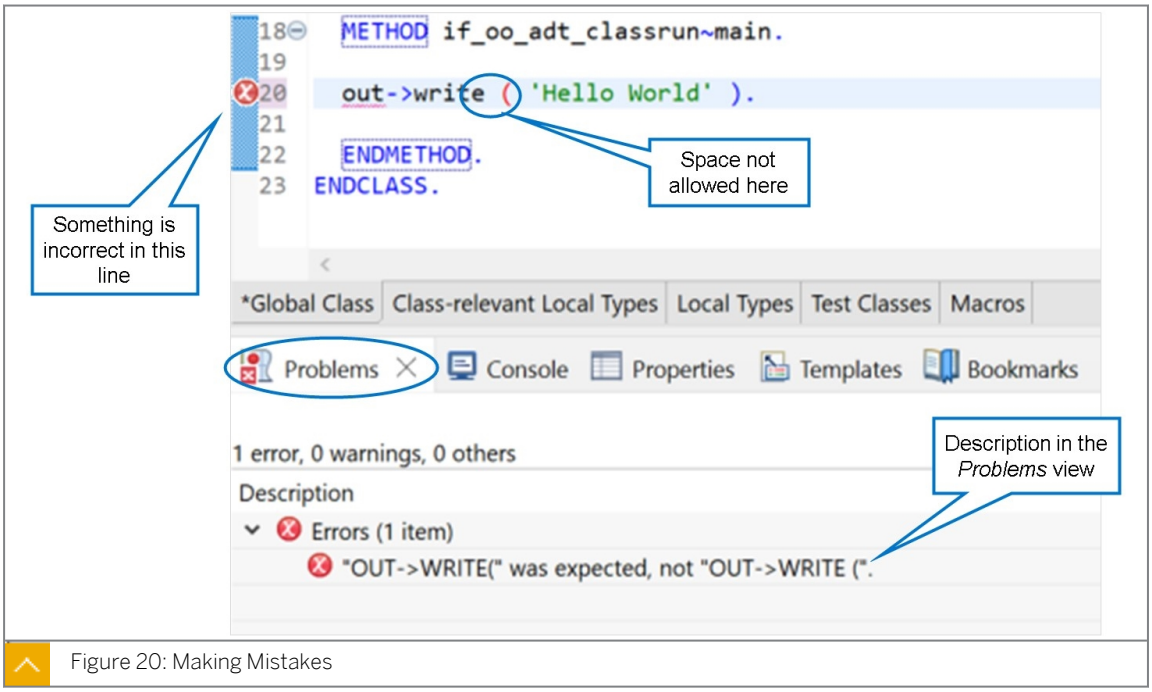


Figure 20: Making Mistakes

When you write ABAP code, you will inevitably make mistakes. ADT checks your code as you go along, and flags up errors in the left-hand margin of the editor with a white cross on a red background.

© Copyright. All rights reserved.

You can see the corresponding error messages in the *Problems* view below the editor. ADT also displays the same message as a pop-up if you move the mouse over the error symbol in the editor.

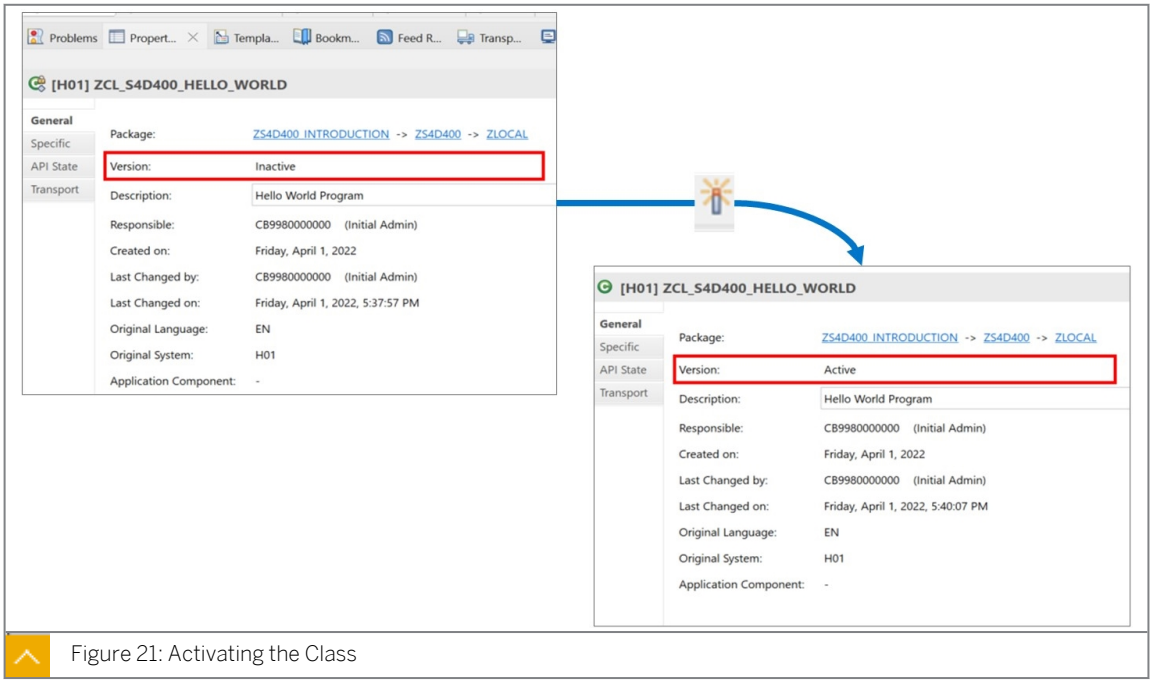


Figure 21: Activating the Class

In order to run an ABAP object, you must activate it. To do this, choose the *Activate* icon in the toolbar or use the keyboard shortcut **`Ctrl + F3`**. You can see whether an object is active or not by looking in the *Properties* view, usually located in the tab below the ABAP editor.

During the activation, the system compiles the object into a form that the ABAP runtime system can understand.

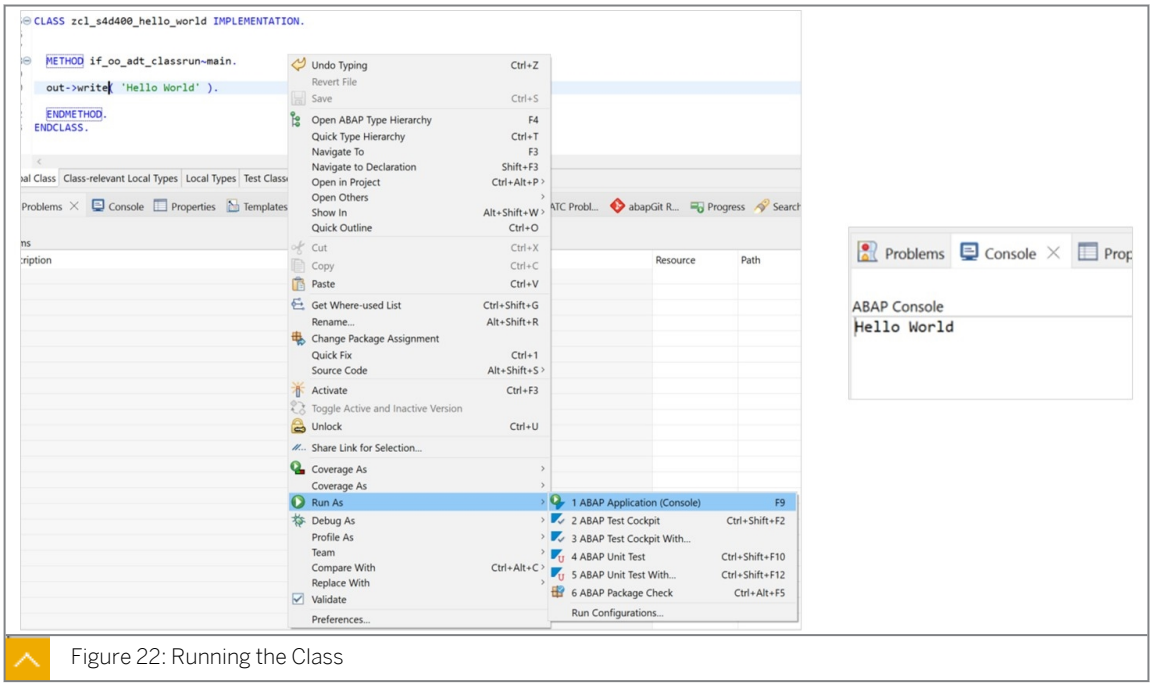


Figure 22: Running the Class

© Copyright. All rights reserved.

To run the class, press **F9** or right-click in the editor and choose *Run As → ABAP Application (Console)*. The output *Hello World* appears in the console. If you cannot see the *Console* view, choose *Window → Show View → Other...* and select the *Console* view.

© Copyright. All rights reserved.

Unit 1
Exercise 3

Create a 'Hello World' Application

Create a Hello World Application
In your package, create an new ABAP class. Let the class implement interface **`IF_OO_ADT_CLASSRUN`** so that you can use the class as the main program for an Eclipse console app.

1. In your package, create a new ABAP class with the name **`ZCL_##_HELLO_WORLD`**. Ensure that it uses the interface **`IF_OO_ADT_CLASSRUN`**. When you are prompted to assign the class to a transport request, use the transport request that you created in the previous task.
2. In the **`if_oo_adt_classrun~main( )`** method, use `out->write( )` to output the phrase *Hello World*.
3. Activate and test your class.
4. Check the output in the *Console* view of Eclipse.

© Copyright. All rights reserved.

Unit 1
Solution 3

Create a 'Hello World' Application

Create a Hello World Application
In your package, create an new ABAP class. Let the class implement interface **`IF_OO_ADT_CLASSRUN`** so that you can use the class as the main program for an Eclipse console app.

1. In your package, create a new ABAP class with the name **`ZCL_##_HELLO_WORLD`**. Ensure that it uses the interface **`IF_OO_ADT_CLASSRUN`**. When you are prompted to assign the class to a transport request, use the transport request that you created in the previous task.
   a) Choose *File* → *New* → *ABAP Class*.
   b) Enter your package **`ZS4D400_##`**, where **`##`** is your group number.
   c) Enter the name **`ZCL_##_HELLO_WORLD`** where **`##`** is your group number, and enter a description for your class.
   d) Choose *Add...* (next to the *Interfaces* group box).
   e) Enter the filter text **`IF_OO_ADT_CLASSRUN`**. Double-click the matching entry in the hit list.
   f) Choose *Next*.
   g) Select *Choose from requests in which I am involved* and your own transport request.
   h) Choose *Finish*.
2. In the **`if_oo_adt_classrun~main( )`** method, use `out->write( )` to output the phrase *Hello World*.
   a) In the editor, enter the following coding between `METHOD if_oo_adt_classrun~main` and `ENDMETHOD.`:

```
out->write( 'Hello World' ).
```

3. Activate and test your class.
   a) Activate the class with the keyboard shortcut **`Ctrl + F3`**.
   b) Run the class with the **`F9`** key.
4. Check the output in the *Console* view of Eclipse.
   a) Check the *Console* view that should have opened as a new tab below the editor view.
   b) If the *Console* view is not visible, open it by choosing *Window* → *Show view* → *Other*. Double-click *Console* in the hit list.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Create a 'Hello World' application.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 1
Lesson 5

Amplifying ABAP Development with AI

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Explain how ABAP AI Capabilities can assist ABAP developers during application development.
- Briefly describe how the ABAP AI SDK can be used to add AI features to business applications.

Joule for Developers: Your AI Assistant

Introduction

Imagine you are a new ABAP developer on a project. You are faced with understanding existing code, generating initial versions of ABAP code, or implementing new features. Instead of spending a lot of time manually analyzing code or writing everything from scratch, you can use ABAP AI Capabilities to help you speed up your development and improve the quality of your code from the start.

Generative AI is changing ABAP development by providing tools that can help developers be more productive and create smarter business applications. This lesson will give you a general idea of how you can use these AI features in your ABAP projects.

AI Coding Assistance - Main Features of Joule

Joule for Developers is a set of AI-powered tools that are built into the ABAP development tools in Eclipse. It acts like your assistant, helping you with different development tasks to make you more efficient.

© Copyright. All rights reserved.

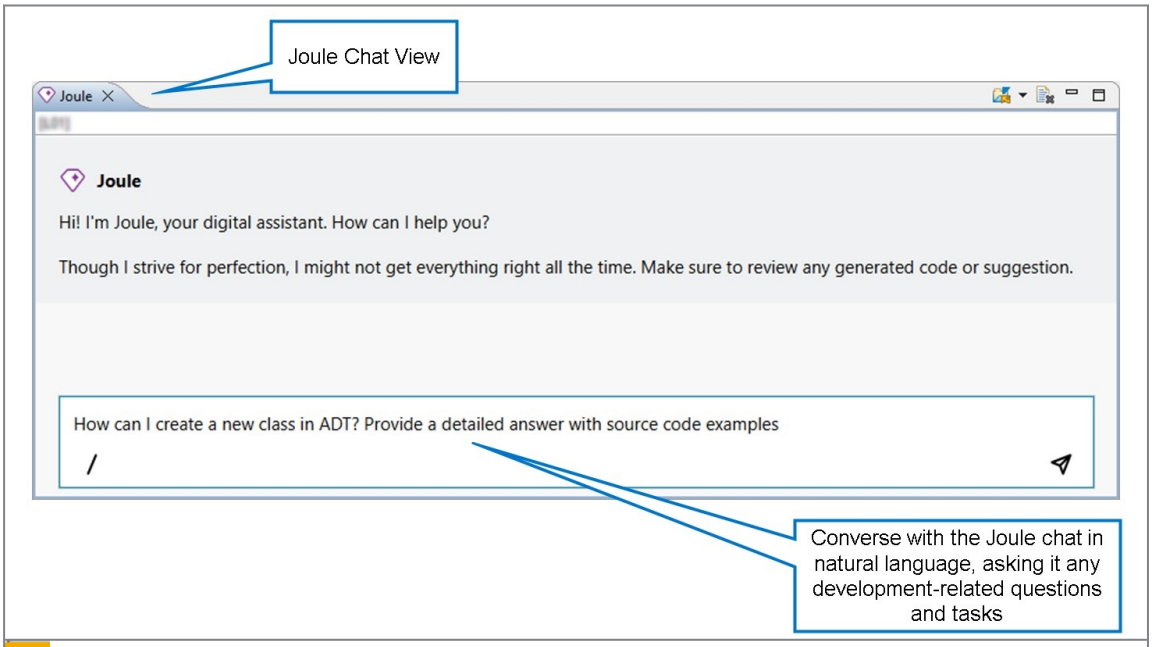


Figure 23: Joule Chat

You can chat with an AI to get help with development-related questions. Ask questions, request explanations, or generate code examples using simple language. To open the Joule chat view, choose *Window* → *Show View* → *Other...* → *Joule*.

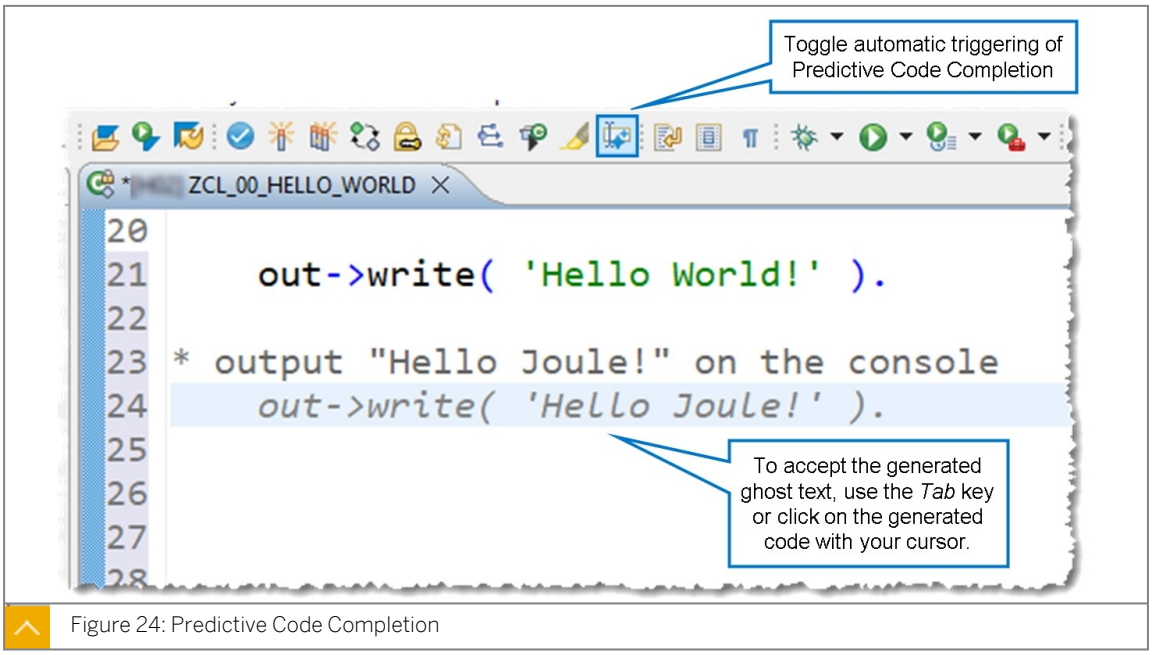


Figure 24: Predictive Code Completion

You can incorporate ghost text in your source code during your writing process. Ghost texts are predicted based on your cursor position and the content in the ABAP source code editor. The predictive code completion currently supports the following object types:

- Classes
- Interfaces
- Programs

© Copyright. All rights reserved.

The predictive code completion is enabled by default, but you can disable it manually if you prefer. When enabled, ghost text appears continuously during your coding process and is displayed in grey in the ABAP source code editor.

Select *Toggle Automatic Triggering of Predictive Code Completion* from the toolbar in ABAP development tools for Eclipse to enable or disable the predictive code completion.

Begin typing in the source code editor and take a short break. If predictive code completion is enabled, ghost text appears in the ABAP source code editor at the point where you paused. You can then incorporate it into your code.

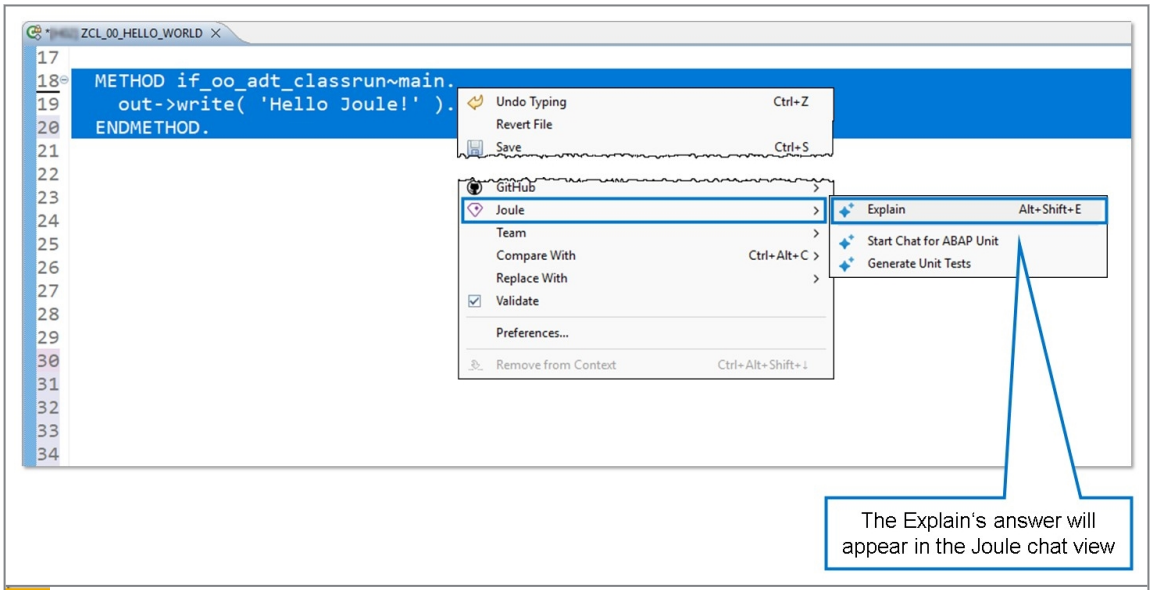


Figure 25: Explain Capability

The *Explain* capability enables you to get detailed explanations of selected development objects or lines of source code.

To open the *Explain* capability, select a development object from the *Project Explorer*, the source code editor (by selecting lines of code), or other views such as the *Outline* view.

Then choose *Joule* → *Explain* from the context menu or open the Joule chat view and select **`/explain`** located in the input field at the bottom of the view.

Using the *Explain* capability is a collaborative process. You can specify to the *Explain* capability in natural language how the explanation should be generated, for instance, whether it should be detailed or concise, with or without examples.

© Copyright. All rights reserved.

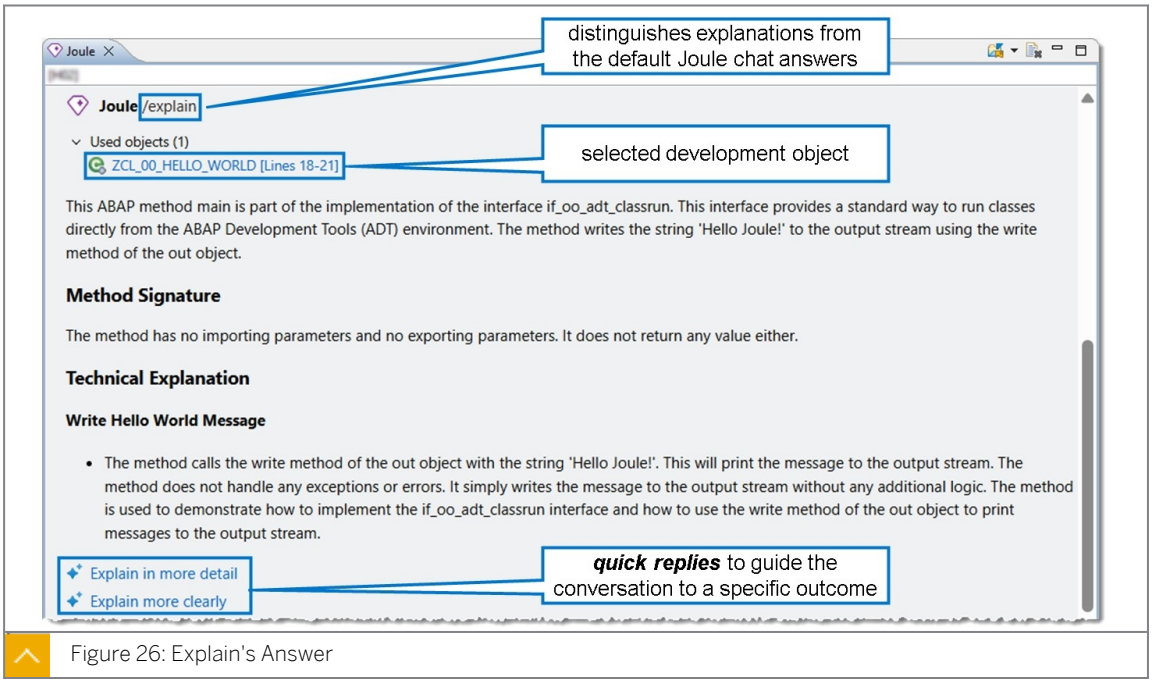


Figure 26: Explain's Answer

The *Explain*'s answer will appear in the Joule chat view with the selected development object listed under *Used objects*. The explanations are marked with */explain* to distinguish them from the default Joule chat answers. Via the Joule chat view you have the option of follow-up questions. For some development object types, you will even get the option to guide the conversation to a specific outcome with *quick replies*. These *quick replies* will appear at the end of the *Explain*'s response in the Joule chat view. For example, you may have the option to generate a more detailed or more understandable explanation through the use of such *quick replies*.

In addition, SAP Joule for Developers offers the following ABAP AI capabilities:

- ABAP Unit Test Generation

  The AI can help you generate ABAP Unit Tests. This allows you to create tests that check your code more easily.

- CDS Test Generation

  The AI can help you create test classes for CDS entities. This ensures the quality and correctness of your CDS views.

- OData UI Service from Scratch

  Create all RAP service-related repository objects from scratch with the aid of the Joule chat.

- RAP Business Logic Prediction

  Implement determinations or validations in the implementation class of your behavior definition.

- Consume

  Generate an OData request and ABAP code for consuming a corresponding OData call via the OData Client Proxy.

- GenAI-Driven Extensibility Assistant

© Copyright. All rights reserved.

The extensibility assistant can answer questions related to field extensibility and field logic. It can assist you in various tasks related to field extensibility, including creating custom fields and finding value help views.

Caution:
Important points to remember:

- AI results may vary.
- Always check AI-generated code for correctness.
- Make sure AI-generated code meets your requirements.

Building Your Own AI-Enabled Applications

ABAP AI SDK

While Joule provides AI help inside the development environment, the ABAP AI SDK (Software Development Kit) lets you add AI features directly into your own business applications.

The ABAP AI SDK provides an ABAP library that allows you to use large language models (LLMs).

With the ABAP AI SDK, you can:

- Use LLMs from your ABAP code.
- Add GenAI features to your apps or services.
- Utilize features like prompt completion and function calls.

The ABAP AI SDK enables you to use the following features:

- Completion API

  This API provides access to the completion API functionality of LLMs, meaning that it uses an LLM to generate a text response from a prompt.

- Prompt Library API

  Using the prompt library API, you can use predefined prompt templates to generate new prompts.

- Tracing

  If you encounter problems when calling the API, you can initiate the ABAP Cross Trace. You could do this for instance with the completion API to inspect the call stack, to identify the cause of the error.

Setting Up the ABAP AI SDK

To use the ABAP AI SDK, your system administrator needs to set up system connections with the generative AI hub. As a developer, you will define the LLM to be used and create intelligent scenarios and models.

© Copyright. All rights reserved.

Summary

- ABAP AI Capabilities help developers be more productive by providing AI assistance within the ABAP development environment.
- Key features of Joule include Joule Chat, Predictive Code Completion, and Explain, which simplify development tasks.
- The ABAP AI SDK allows developers to build custom AI-enabled business applications by using large language models.

Further reading

- SAP Joule for Developers, ABAP AI Capabilities | SAP Help Portal
- Developing Your Own AI-Enabled Applications | SAP Help Portal
- Workshop RAP120 - Build SAP Fiori Apps with ABAP Cloud and SAP Joule for developers

LESSON SUMMARY

You should now be able to:

- Explain how ABAP AI Capabilities can assist ABAP developers during application development.
- Briefly describe how the ABAP AI SDK can be used to add AI features to business applications.

© Copyright. All rights reserved.

Learning Assessment

1. An ABAP Cloud Project in ABAP Development Tools (ADT) allows you to connect to what kinds of system?

   Choose the correct answers.

   - ☐ A SAP BTP ABAP instance
   - ☐ B On-Premise ABAP system
   - ☐ C Embedded ABAP instance in SAP S/4HANA Cloud

2. In ABAP source code, how do you navigate to the definition of a development object?

   Choose the correct answer.

   - ☐ A Double-click the name of the object
   - ☐ B Position the cursor on the name of the object and press F2
   - ☐ C Position the cursor on the name of the object and press F3

3. What must you do when you create an ABAP package?

   Choose the correct answers.

   - ☐ A Use the letter Z or Y as the first character in the name
   - ☐ B Assign the package to an application component
   - ☐ C Assign the package to a software component
   - ☐ D Declare what other packages can use the contents of the package.

4. Which of the following characters is not allowed in the name of an ABAP class?

   Choose the correct answer.

   - ☐ A 8
   - ☐ B _
   - ☐ C $

© Copyright. All rights reserved.

5. Which of the following is a key feature of ABAP AI Capabilities?

   Choose the correct answer.

   - ☐ A Automated deployment to SAP BTP
   - ☐ B Predictive code completion
   - ☐ C Real-time monitoring of application performance
   - ☐ D Automatic translation of ABAP code to other languages

6. Which of the following can the Explain capability in Joule provide?

   Choose the correct answer.

   - ☐ A Simple explanations of ABAP code
   - ☐ B Automatic generation of UIs
   - ☐ C Performance analysis of ABAP code
   - ☐ D Suggestions for database optimization

7. What is the primary purpose of the ABAP AI SDK?

   Choose the correct answer.

   - ☐ A To provide a development environment for creating ABAP code
   - ☐ B To manage user authorizations in ABAP systems
   - ☐ C To enable the development of custom AI-enabled business applications
   - ☐ D To automate the deployment of ABAP applications to the cloud

© Copyright. All rights reserved.

Learning Assessment - Answers

1. An ABAP Cloud Project in ABAP Development Tools (ADT) allows you to connect to what kinds of system?

   Choose the correct answers.

   - [X] A SAP BTP ABAP instance
   - [ ] B On-Premise ABAP system
   - [X] C Embedded ABAP instance in SAP S/4HANA Cloud

   That is correct.

2. In ABAP source code, how do you navigate to the definition of a development object?

   Choose the correct answer.

   - [ ] A Double-click the name of the object
   - [ ] B Position the cursor on the name of the object and press F2
   - [X] C Position the cursor on the name of the object and press F3

   That is correct

3. What must you do when you create an ABAP package?

   Choose the correct answers.

   - [X] A Use the letter Z or Y as the first character in the name
   - [ ] B Assign the package to an application component
   - [X] C Assign the package to a software component
   - [ ] D Declare what other packages can use the contents of the package.

   That is correct.

© Copyright. All rights reserved.

4. Which of the following characters is not allowed in the name of an ABAP class?

   Choose the correct answer.

   - [ ] A 8
   - [ ] B _
   - [x] C $

   That is correct

5. Which of the following is a key feature of ABAP AI Capabilities?

   Choose the correct answer.

   - [ ] A Automated deployment to SAP BTP
   - [x] B Predictive code completion
   - [ ] C Real-time monitoring of application performance
   - [ ] D Automatic translation of ABAP code to other languages

   Correct! Predictive code completion is a key feature of ABAP AI Capabilities.

6. Which of the following can the Explain capability in Joule provide?

   Choose the correct answer.

   - [x] A Simple explanations of ABAP code
   - [ ] B Automatic generation of UIs
   - [ ] C Performance analysis of ABAP code
   - [ ] D Suggestions for database optimization

   Correct! The Explain capability in Joule can provide simple explanations of ABAP code.

7. What is the primary purpose of the ABAP AI SDK?

   Choose the correct answer.

   - [ ] A To provide a development environment for creating ABAP code
   - [ ] B To manage user authorizations in ABAP systems
   - [x] C To enable the development of custom AI-enabled business applications
   - [ ] D To automate the deployment of ABAP applications to the cloud

   Correct! The primary purpose of the ABAP AI SDK is to enable the development of custom AI-enabled business applications.

© Copyright. All rights reserved.

UNIT 2 Applying Basic Techniques and Concepts

Lesson 1

Understanding the Basics of ABAP 47

Lesson 2

Working With Basic Data Objects and Data Types 53

Lesson 3

Processing Data 64

Exercise 4: Declare Variables and Process Data 69

Lesson 4

Using Control Structures in ABAP 75

Exercise 5: Implement Conditional Branching 83

Lesson 5

Working with Simple Internal Tables 91

Exercise 6: Work with Simple Internal Tables 97

Lesson 6

Debugging an ABAP Program 104

Exercise 7: Debug ABAP Code 113

UNIT OBJECTIVES

- Describe the evolution of ABAP.
- Describe the basics of ABAP syntax.
- Declare data objects.
- Assign values.
- Perform arithmetic calculations.
- Apply string processing.

© Copyright. All rights reserved.

- Implement conditional branching.
- Handle Exceptions.
- Implement Iterations.
- Define simple internal tables.
- Process data using simple internal tables.
- Enter debugging mode.
- Control the execution of code.
- Analyze the content of data objects.

© Copyright. All rights reserved.

Unit 2
Lesson 1

Understanding the Basics of ABAP

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Describe the evolution of ABAP.
- Describe the basics of ABAP syntax.

ABAP Then And Now

Origin and Evolution of ABAP

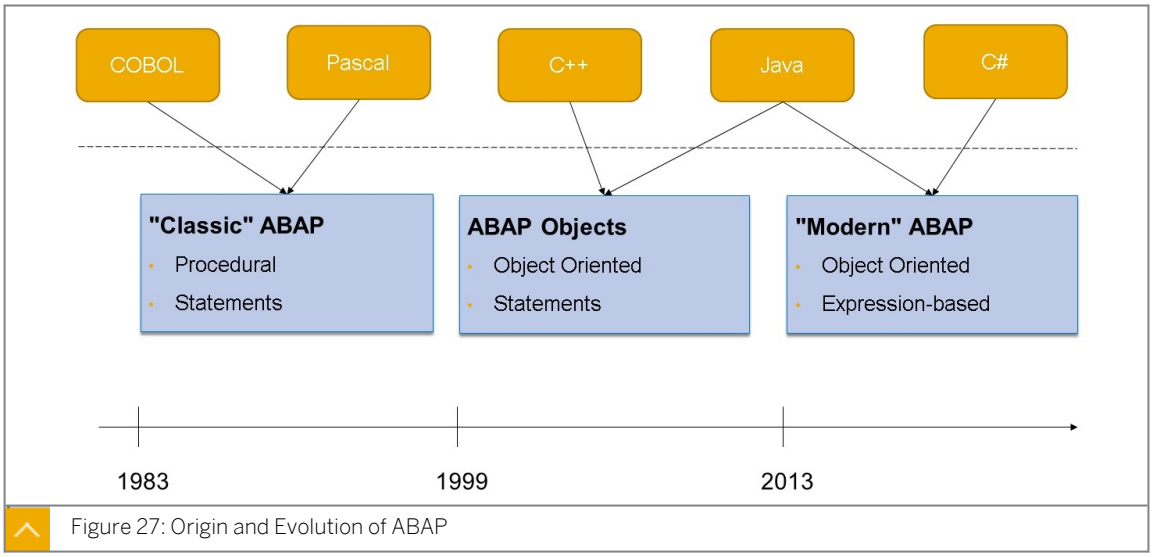


Figure 27: Origin and Evolution of ABAP

ABAP is a programming language developed by SAP for the development of business applications in the SAP environment.

When ABAP was first published in 1983, it was designed as a procedural programming language based on statements. Although relatively independent, it was influenced by other popular programming languages of the time, mostly COBOL and Pascal.

By the end of the 1990s, the ABAP language was extended to allow for object-oriented programming. Important concepts of popular object-oriented programming languages, like C ++ and Java, were integrated into the ABAP language. ABAP Objects, as the programming language was called now, combined procedural and object-oriented parts in one language.

Even though ABAP allowed for object-oriented programming, the syntax remained dominated by statements. Compared to other modern programming languages, like Java and C#, ABAP code appeared lengthy, complicated and involved a lot of helper variables to link the various statement. To amend this, ABAP was extended with modern expressions to replace or extend

© Copyright. All rights reserved.

the existing statements. Again, ABAP combines traditional statement-based syntax elements and modern, expression-based syntax elements in one programming language.

For new ABAP developments, we recommended using object-oriented and expression-based syntax elements wherever possible.

ABAP Language Versions

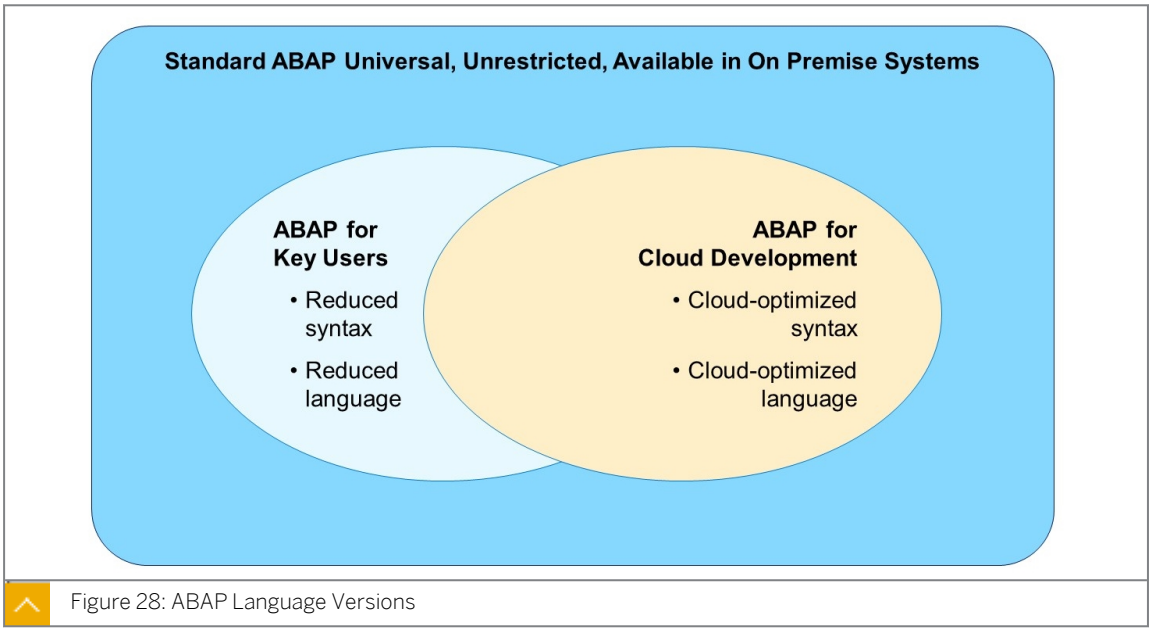


Figure 28: ABAP Language Versions

Each ABAP program has the program attribute “ABAP language version”, which is defined internally by a version ID. The version of a program determines which language elements and which repository objects can be used in the program and which syntax rules apply. The following versions are currently available:

Standard ABAP

This version of ABAP is the universal basic version. It is an unrestricted ABAP language version that covers the entire language scope of ABAP that can be used in Unicode systems and, apart from the static package concept, access to all other repository objects is allowed. The syntax check for Standard ABAP is performed as Unicode check, which is the minimum requirement for a Unicode system.

ABAP for Key Users

This version of ABAP is intended for the secure implementation of enhancements by key users within the scope of enhancement options provided by SAP. ABAP for Key Users is a restricted ABAP language version where the general rules for Standard ABAP apply but only a very restricted set of language elements is supported and access to repository objects is restricted.

ABAP for Cloud Development

This version of ABAP is intended for developments in the SAP BTP ABAP environment and in the S/4HANA Cloud ABAP environment. ABAP for Cloud Development is a restricted ABAP language version where the general rules for Standard ABAP apply but only a restricted set of language elements is supported and access to repository objects is restricted.

© Copyright. All rights reserved.

Note:
This course gives a universal introduction to ABAP development. It restricts itself to syntax elements and language features available in all three language versions. An exception is the introduction to the *ABAP RESTful Application Programming Model* which is not part of *ABAP for key users*.

ABAP Cloud

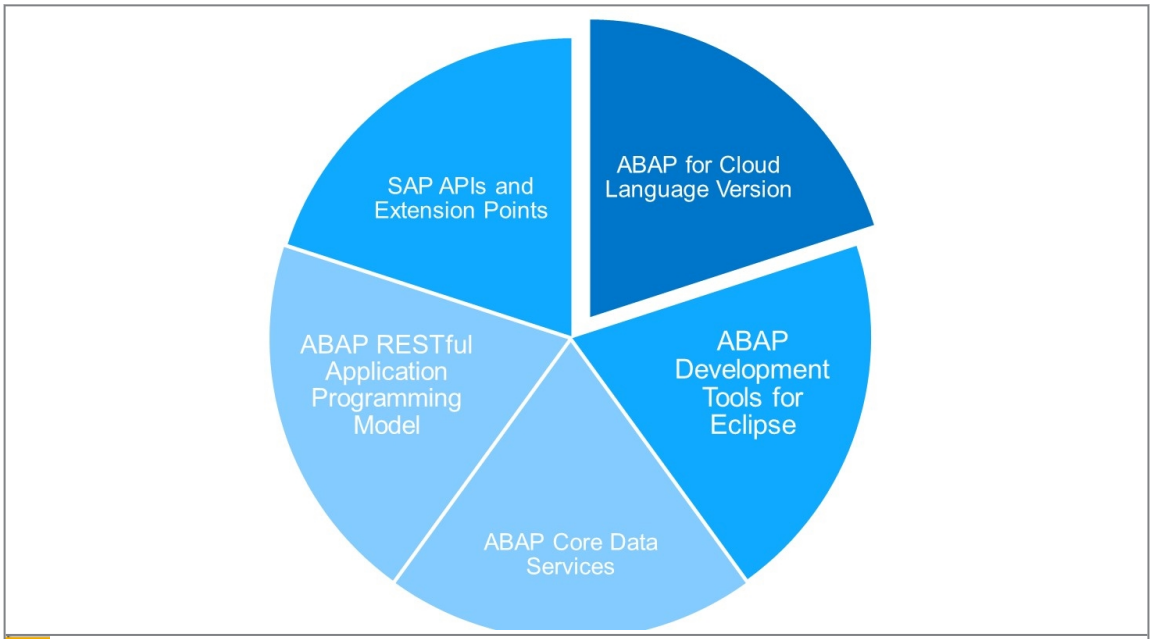


Figure 29: ABAP Cloud Elements

The ABAP for Cloud Development language version is a key element of **ABAP Cloud**. The technological core of ABAP Cloud defines the design-time and runtime architecture of all extensions, services, and applications. The main ABAP Cloud elements are:

- **ABAP for Cloud Development** as the cloud-optimized ABAP language for the business logic
- **ABAP Development Tools** for Eclipse as the ABAP integrated development environment
- **ABAP Core Data Services (CDS)** for the data model and for embedded analytics
- The **ABAP RESTful Application Programming Model** for building apps and services
- **Mandatory public SAP APIs and extension points** to allow automated cloud operations and lifecycle stable extensibility.

© Copyright. All rights reserved.

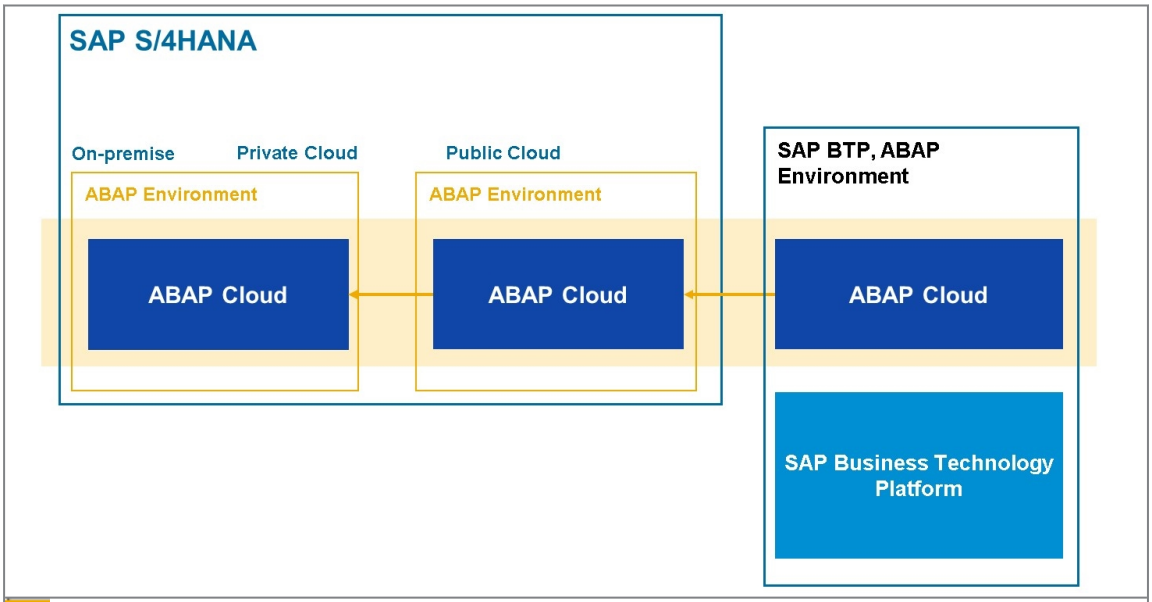


Figure 30: ABAP Cloud Availability

Using ABAP Cloud is mandatory on the *SAP Business Technology Platform* and on *SAP S/4HANA Cloud, public edition*. It is not mandatory but strongly recommended to follow the principles of ABAP Cloud on *SAP S/4HANA, private edition* and in on-premise systems. By implementing your custom development with ABAP Cloud, your developments are cloud ready by default and you are not changing the composition of the core system, which is an important principle for system maintenance in the cloud environment (**Clean Core** approach).

The Basics of the ABAP Syntax

Some Basic Features of the ABAP Language

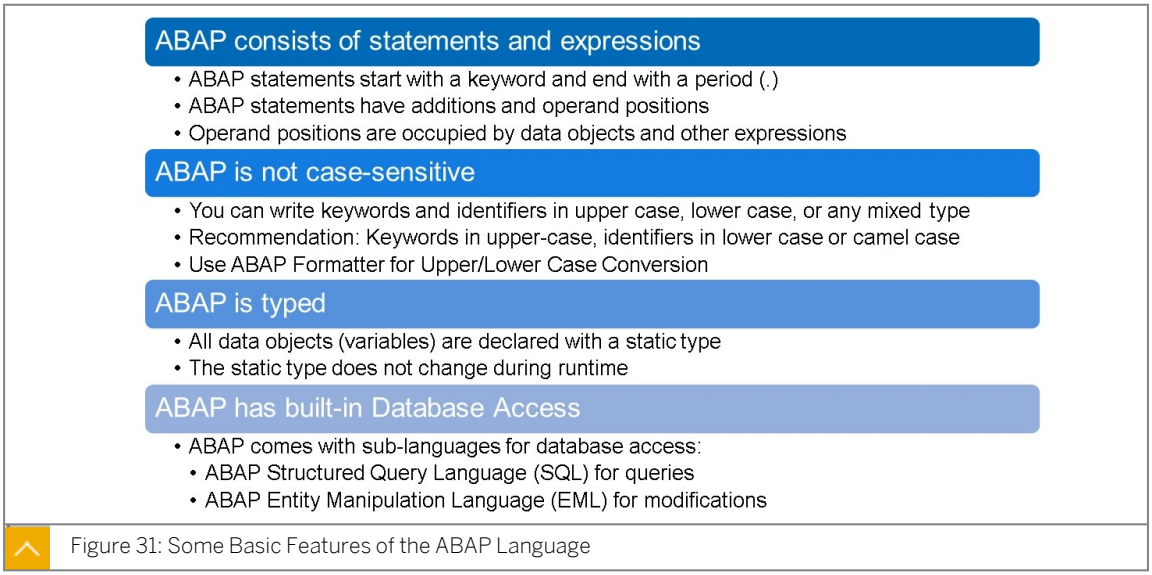


Figure 31: Some Basic Features of the ABAP Language

© Copyright. All rights reserved.

Examples of ABAP Syntax

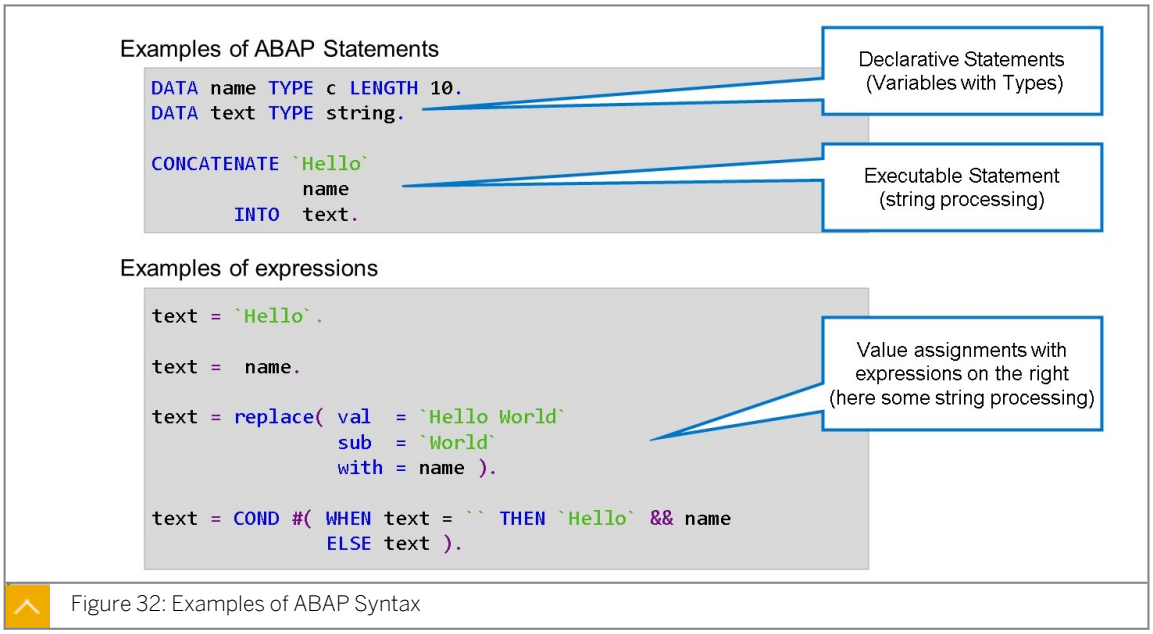


Figure 32: Examples of ABAP Syntax

These code examples illustrate some of the basic features of the ABAP programming language. The upper code consists of three statements, each of them ends with a period (.) .

The DATA statement is a declarative statement to define variables. The CONCATENATE statement is a data processing statement for combining char-like data.

The lower code example consists of four statements.

The lower code consists of four value assignments. The name of the target variable (text) is followed by the equals sign (=) that serves as the assignment operator. The operand position on the right-hand side is occupied by expressions of different complexity: A literal, a variable, a string processing function (replace), and an expression for a conditional assignment (COND).

Comments in ABAP

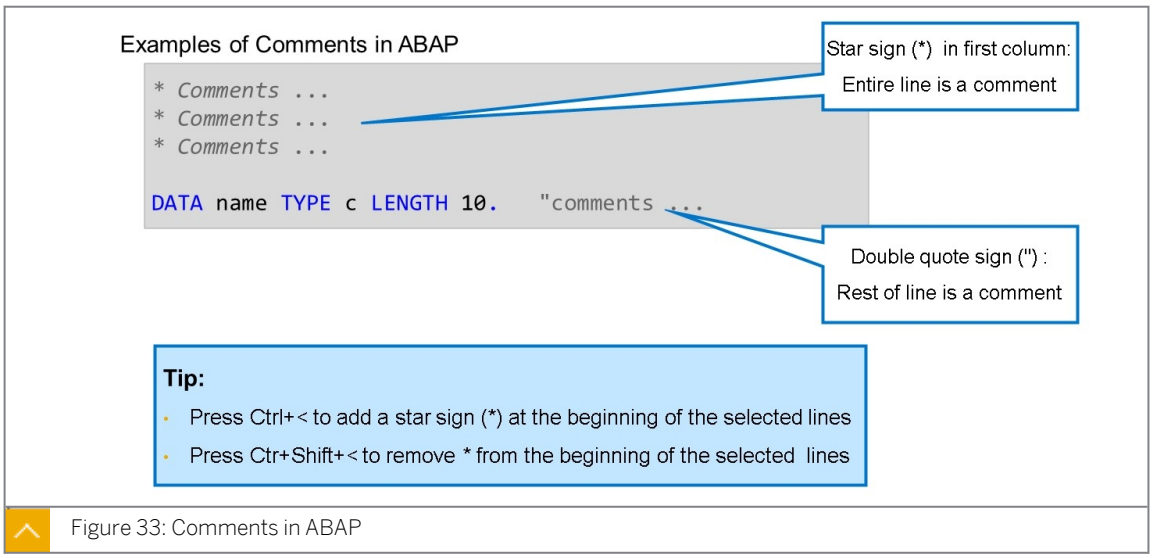


Figure 33: Comments in ABAP

© Copyright. All rights reserved.

A comment in a programming language is a piece of source code that is ignored by the compiler or interpreter

In ABAP, there are two ways to define a piece of source code as comment:

- A star sign (*) in the first column identifies the entire line as a comment
- A double quote (") sign identifies the rest if the line, that is, the code to the right, as comment.

Note:
You can place the " sign in any column. The star sign (*) only works if placed in the first column. In other positions it causes a syntax error, unless it is a syntactically correct part of an ABAP statement.

Hint:
To comment one or several code lines, that is, to add star sign (*) in the first column, select the code lines and press **`Ctrl + <`**. Alternatively, from the context menu, you can choose *Source Code → Add Comment*. To uncomment the selected code lines, that is, to remove the star sign (*) from the first column, press **`Ctrl + >`** or choose *Souce Code → Remove Comment* from the context menu.

LESSON SUMMARY
You should now be able to:

- Describe the evolution of ABAP.
- Describe the basics of ABAP syntax.

© Copyright. All rights reserved.

Unit 2
Lesson 2

Working With Basic Data Objects and Data Types

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Declare data objects.
- Assign values.

Data Objects in ABAP

A data object in an ABAP program represents a reserved section of the program memory.

ABAP knows three types of data objects: Variables, Constants, and Literals.

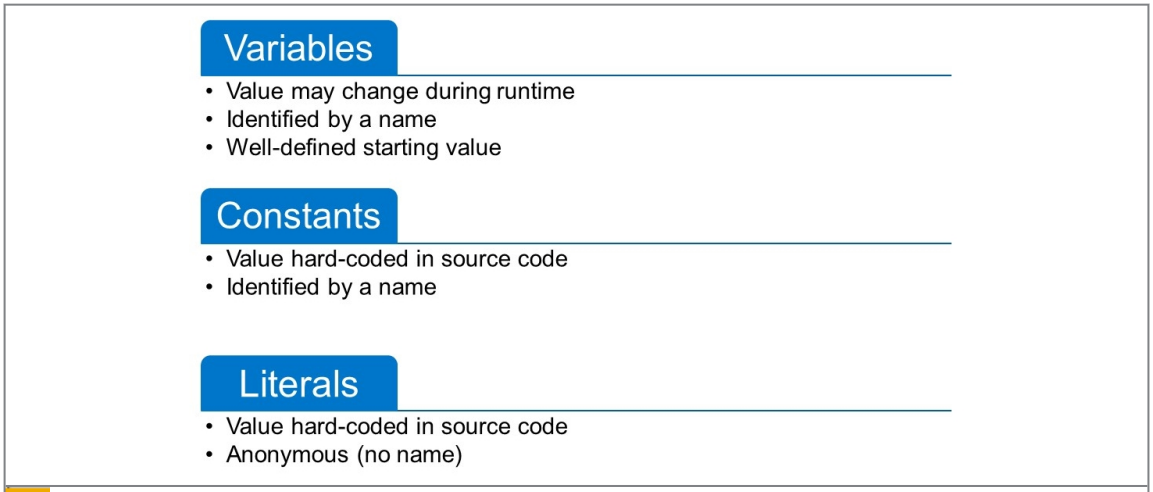


Figure 34: Data Objects in ABAP

Variables

A variable is a data object with content that can change during runtime. A variable is identified by a name. The name is also used to address the data object at runtime. The starting value of an ABAP variables is always well-defined.

Constants

Constants are similar to variables. But in contrast to variables the value is hard coded in the source code and must not change during runtime. Like variables, constants have a name by which they can be re-used.

Literals

© Copyright. All rights reserved.

The value of literals is also hard-coded in the source code. In contrast to constants, literals don't have a name. Because of that you cannot reuse a literal. Only use literals to specify the values for constants and the starting values for variables.

ABAP data objects are always typed: Every data object is based on a data type which determines the kind of information they can contain. The data type of an ABAP data object stays the same throughout a program execution.

Declaration of Variables

A variable in an ABAP program is declared with keyword DATA.

A DATA statement consists of three parts. Let's look at each part in more detail.

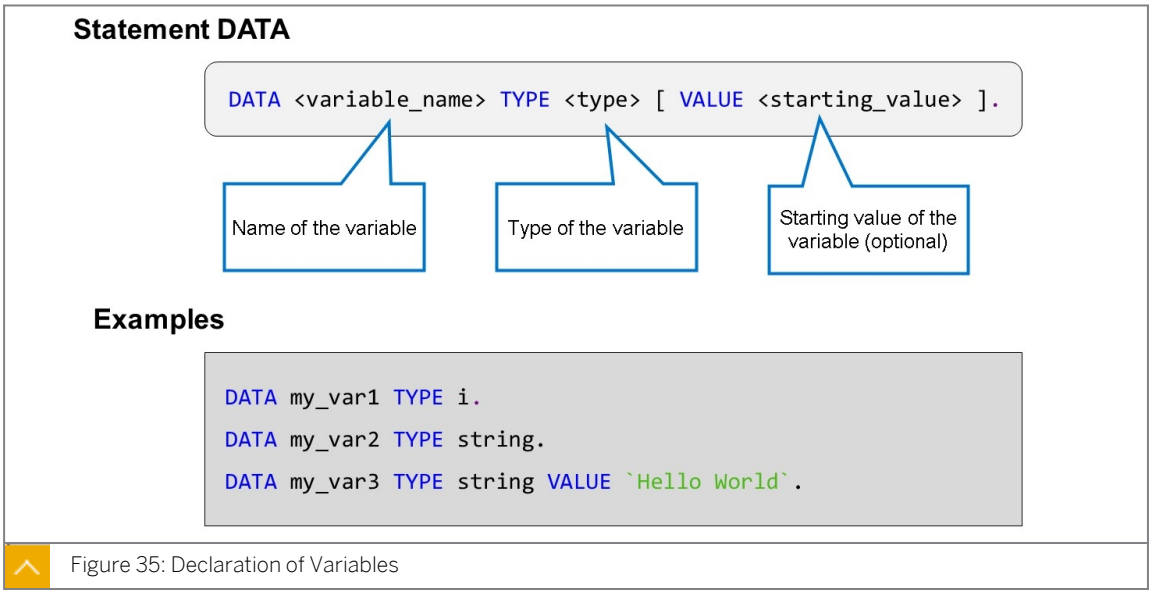


Figure 35: Declaration of Variables

DATA

Keyword DATA is followed by the name of the variable. The name of a variable may be up to 30 characters long. It may contain the characters A-Z, the digits 0-9, and the underscore character. The name must begin with a letter or an underscore.

TYPE

The type of the variable is specified after addition TYPE. In the example, built-in types i (for integer numbers) and string (character string with variable length) are used.

VALUE

Addition VALUE is optional and you can use it to specify a start value for the variable. If VALUE is missing, the variable is created with an initial value that depends on the technical type of the variable.

© Copyright. All rights reserved.

Sources of ABAP Data Types

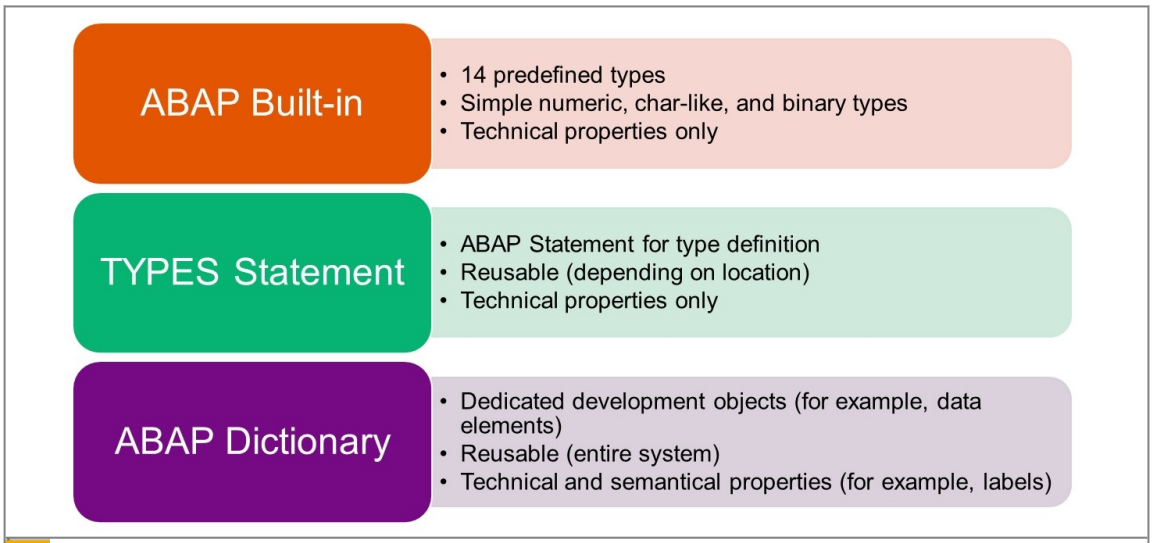


Figure 36: Sources Of ABAP Data Types

ABAP offers the following sources of Data Types:

ABAP Built-in

ABAP has a set of 14 predefined data types for simple numeric, char-like, and binary data objects.

TYPES Statement

Statement TYPES allows you to define data types and reuse them in different places, depending on the location of the definition.

ABAP Dictionary

The ABAP Dictionary is a part of the ABAP Repository. Among other things, it manages global data types which are available throughout the system. ABAP Dictionary types not only define technical properties, they add semantic information, for example, labels. ABAP Dictionary types are particularly useful when implementing user interfaces.

© Copyright. All rights reserved.

Predefined ABAP Types

```
TYPES <variable> TYPE <type>.
```

Type	Value Range	Initial Value
STRING	Any characters, any length	empty
I	-2,147,483,648 to +2,147,483,647	0
D	Date in format YYYYMMDD	'00000000'
T	Time in format HHMMSS	'000000'

```
TYPES <variable> TYPE <type> LENGTH <len> DECIMALS <dec>.
```

Type	Values for <len>	Values for <dec>	Value Range	Initial Value
C	1 – 262,143	n.a.	Any characters	Sequence of Blanks
N	1 – 262,143	n.a.	Digits only	Sequence of character '0'
P	1 – 16	0 – 14	Depends on length	0

Figure 37: Some Predefined ABAP Types

Some important ABAP types are listed in the figure, Some Predefined ABAP Types. For some of these types you can add addition LENGTH to specify a fixed length. These types are called incomplete types. In the case of type P, you may also specify a number of decimal places. The ABAP types for which additions LENGTH and DECIMALS are not supported are called complete types.

Complete ABAP Types

TYPE STRING

A field of type string is a character-like field of variable length. The ABAP runtime system allocates and releases memory to optimize the management of string variables. You cannot influence this behavior directly. The maximum length of a string is governed by settings that the system administrator makes. However, in practice, we talk about strings being unlimited in length.

TYPE I

A field of type I is a numeric field that contains a whole number. The system allocates 4 bytes (32 bits) for such a field which allows for values between - 2^31 and + 2^31.

TYPE D

A field of type D represents a date. In ABAP, the date always has the format YYYYMMDD (without separators). The system converts this format according to the current locale before the value is displayed on the user interface. Likewise, when the user enters a date, the system converts it into the ABAP format before you start to process it.

TYPE T

A field of type Type T represents a time. In ABAP, this has the format HHMMSS (without separators in 24 hour format). If the current locale uses 12 hour format, the system converts the values automatically.

Incomplete ABAP Types

Type C

© Copyright. All rights reserved.

A field of type C is a character-like field of specific length. You specify its length in characters; the runtime system then assigns double the number of bytes in order to accommodate the field. You use this type when a fixed length is important.

Type N

A field of type N is a character field of specific length that only contains digits. This field should contain sequence of digits that you do not want to regard as a number and perform calculations with. For example, this could be a personnel number or cost center.

Type P

A field of type P (P for “packed number”) is a field that contains a numeric value with a specified number of digits and decimal places. Use this type for numbers with decimal places or where the value range type I is not sufficient.

Local Types and Global Types

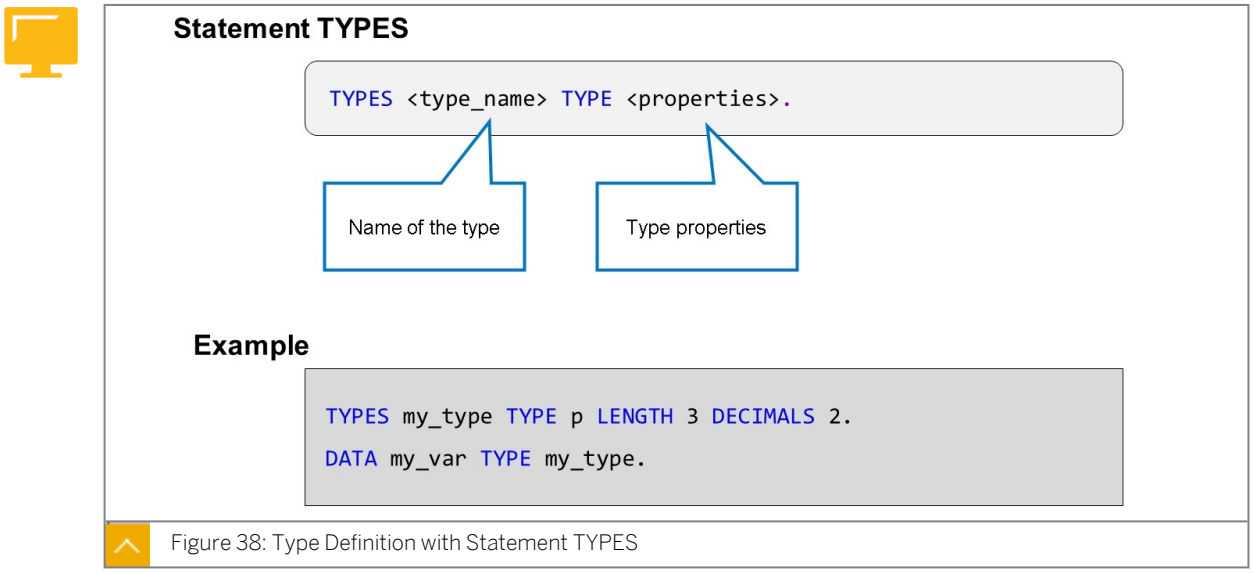


Figure 38: Type Definition with Statement TYPES

Instead of using built-in types in the DATA statement directly, you can use statement TYPES to define the type first. You can then use the type in a DATA statement after the TYPE addition.

© Copyright. All rights reserved.

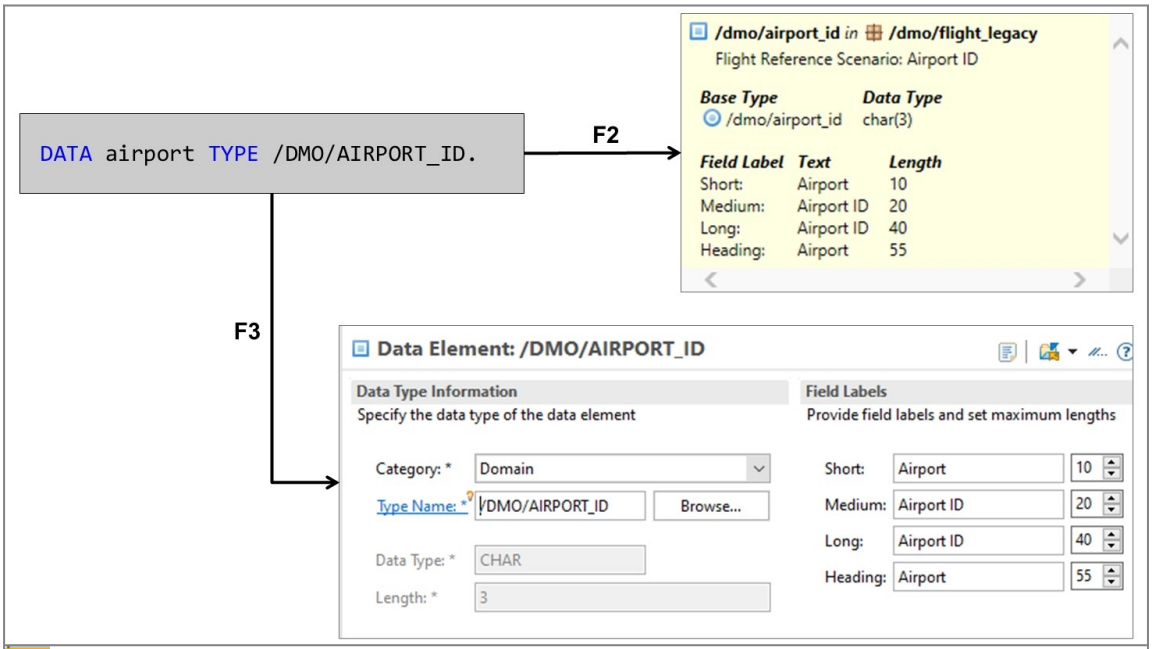


Figure 39: Data Elements in the ABAP Dictionary

In an SAP system, there are thousands of business entities, such as country code, plant, material number, fiscal year, cost center, and so on. Technically, it would be possible to define these entities in every single program using the built-in ABAP types that you have just seen. However, this would be extremely labor-intensive and error-prone. Instead, SAP provides the ABAP Dictionary, which is a central store for important data types and also the tool that you use to create database tables.

In the ABAP Dictionary, single business entities are described by data elements. In the example, variable airport is typed with data element /DMO/AIRPORT_ID.

When you press the F2 key to display the details of this data type you can see that technically this type is a character of length 3. In addition, the data element provides the description “Flight Reference Scenario: Airport ID” and four field labels of different length.

When you press the F3 key to navigate to the definition of the type, a new view opens with the editor for data elements.

© Copyright. All rights reserved.

Constants and Literals

Declaration of Constants

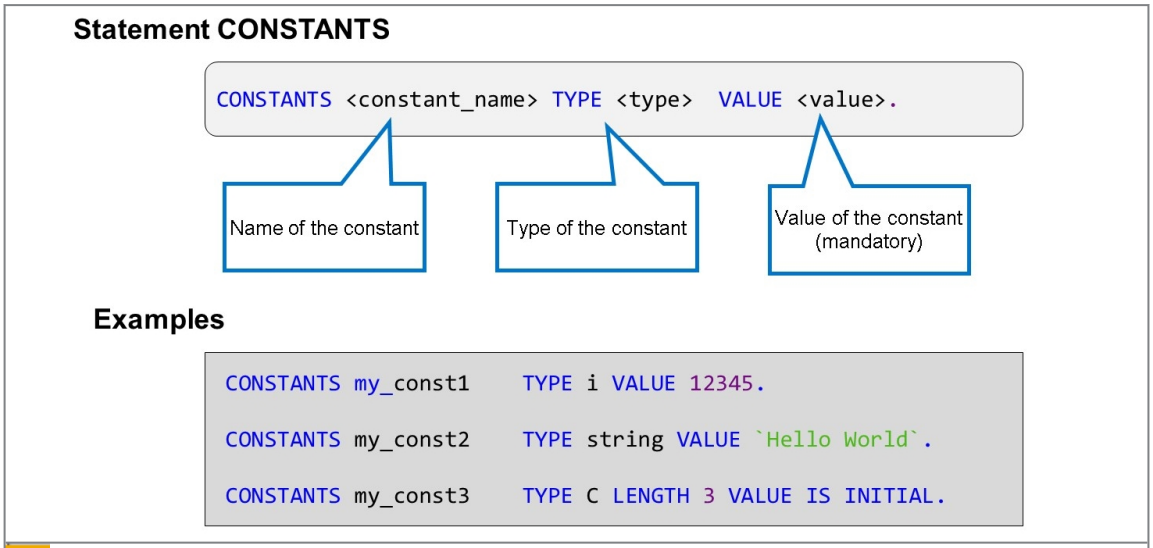


Figure 40: Declaration of Constants

A constant is a data object with a hard-coded value that must not be changed during runtime. Any write access to a constant leads to a syntax error.

In ABAP, you declare a constant using keyword CONSTANTS. A CONSTANT statement consists of the same parts as a DATA statement. The only difference is, that the VALUE addition is mandatory.

You can use the VALUE addition in the special form VALUE IS INITIAL, if the value of the constant should be the type-specific initial value.

Literals in ABAP

Literals are anonymous data objects with a hard-coded value. Literals are often used to define non-initial values for constants and non-initial starting values for variables.

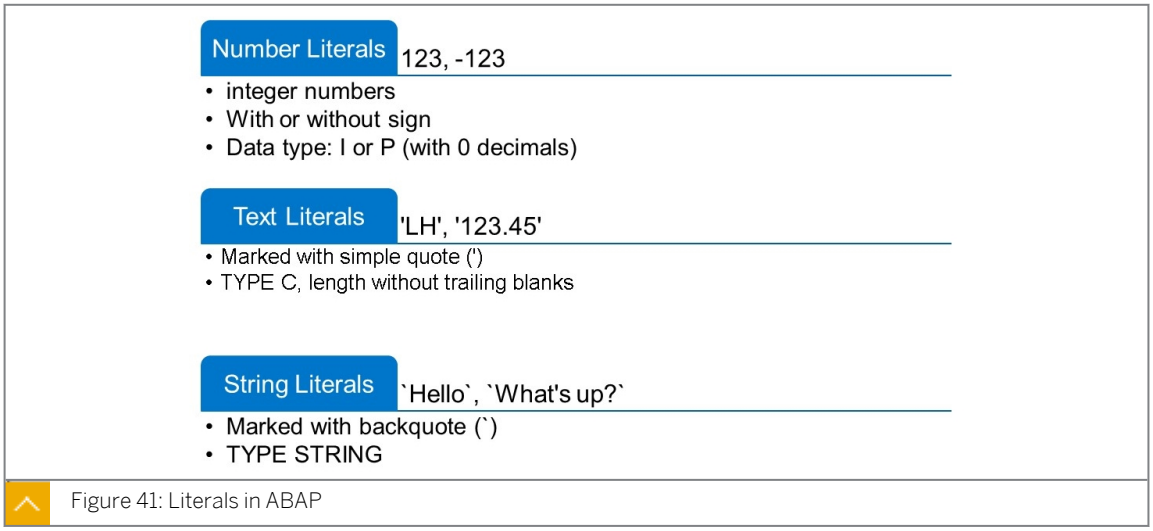


Figure 41: Literals in ABAP

Technically, you can use literals anywhere in your code. To support readability and maintainability, it is recommended to define and use constants, instead.

© Copyright. All rights reserved.

ABAP knows three types of literals:

- Number Literals are integer numbers with or without sign. Number literals usually have data type I. Only if the value is too large for data type I, type P is used, with a sufficient length and without decimal places.
- Text Literals are character strings in simple quotes. Text literals have type C. The length is derived from the content in quotes. Trailing blanks are ignored.
- String Literals are character strings in a pair of back quotes (`). String literals are of type STRING. They should be used to provide values for string typed data objects.

Value Assignments to Data Objects

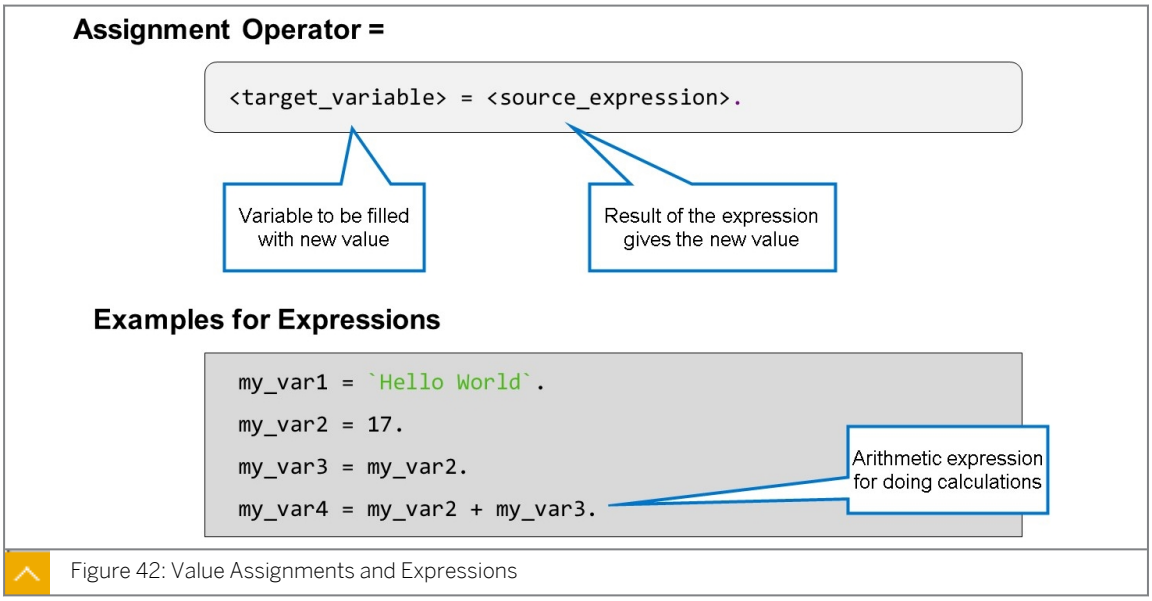


Figure 42: Value Assignments and Expressions

Use value assignments to change the value of variables.

In ABAP, the operator for value assignments is a simple equals sign (=). The variable on the left hand side is filled with the value of the expression on the right-hand side.

The simplest possible expression is a single data object, for example a literal or another variable. But ABAP supports more sophisticated expressions too, like the arithmetic expression in the example.

© Copyright. All rights reserved.

Implicit Type Conversions

```
DATA my_int    TYPE i.
DATA my_pack   TYPE p LENGTH 8 DECIMALS 2.
DATA my_char10 TYPE c LENGTH 10 VALUE 'abcdefghij'.
DATA my_char4  TYPE c LENGTH 4.
DATA my_string TYPE string.

* Implicit type conversions
  my_int = '1234'.          " C -> I
  my_pack = '-273.15'.      " C -> P
  my_int = my_char.         " C -> I       Potential runtime error
  my_char4 = my_char10      " C -> C       Potential information loss
  my_string = 'Hello   '. " C -> STRING  Ignores trailing blanks
```

Figure 43: Implicit Type Conversions

As shown in the figure, if possible, try to avoid type conversions for the following reasons:

- Additional Runtime consumption: Values with type conversions require more runtime than value assignments with identical types.
- Potential Runtime Errors: Some combinations of source type and target type can lead to runtime errors. If, for example, the target variable has a numeric type I and the source expression has a character like type, then the runtime will raise an error if it cannot translate the text into a number.
- Potential Information Loss: Some combinations of source type and target type do not cause runtime errors but can lead to a loss of data. If, for example, source and target are of both of type C but the type of the target variable is shorter. Then the runtime simply truncates the value of the source.

Resetting Variables

Statement CLEAR

```
CLEAR <variable_name>.
```

Examples

```
DATA my_var1 TYPE i.
DATA my_var2 TYPE i VALUE 1234.
DATA my_var3 TYPE string.

my_var1 = my_var2.
my_var3 = `Hello World`.

CLEAR my_var1.      " Value  is reset to 0.
CLEAR my_var2.      " Value  is reset to 0!
CLEAR my_var3.      " String is now empty again
```

Figure 44: Resetting Variables

© Copyright. All rights reserved.

Note:
The CLEAR statement disregards the starting value from the VALUE addition. After CLEAR the variable always contains the type-specific initial value.

Inline Declarations in Value Assignments

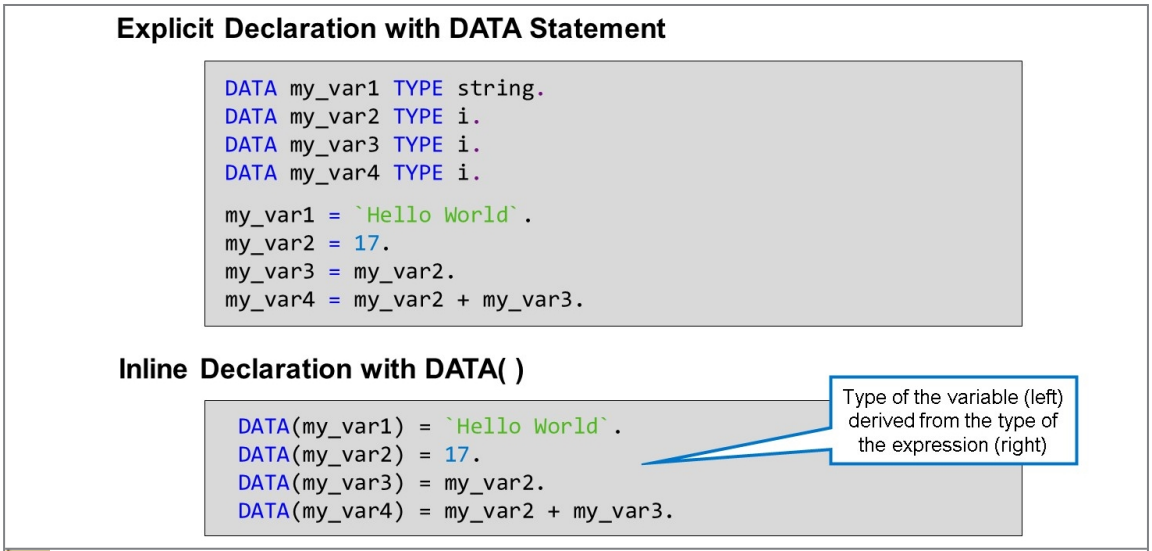


Figure 45: Inline Declarations in Value Assignments

Until now, we used the DATA statement to explicitly declare our variables before we used them in executable coding.

As an alternative, you can declare a variable at the place where you first fill it with data. We call this an Inline Declaration. For an inline declaration you surround the name of the new variable with brackets and add keyword DATA before the opening bracket.

Note:
No blanks are allowed inside the brackets or between DATA and the opening bracket.

After the inline declaration, you can use the variable just like any explicitly declared variable.

The ABAP compiler derives the type for the inline declared variable from the context of the operand position. If, for example, an inline declaration is used on the left hand side of a value assignment the type of the new variable is the same as the type of the expression on the right-hand side.

In the example shown in the figure Inline Declarations in Value Assignments, my_var1 is of type string, because that is the type of the string literal on the right hand side. The other three variables are of type integer.

LESSON SUMMARY

You should now be able to:

- Declare data objects.

© Copyright. All rights reserved.

- Assign values.

© Copyright. All rights reserved.

Unit 2
Lesson 3

Processing Data

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Perform arithmetic calculations.
- Apply string processing.

Arithmetic Calculations

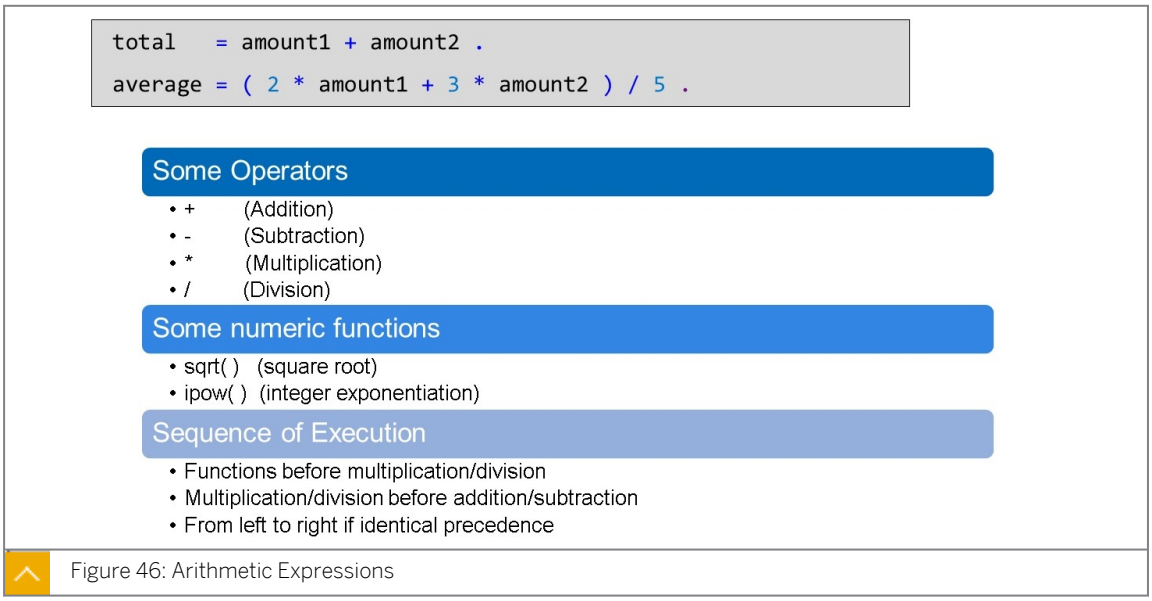


Figure 46: Arithmetic Expressions

Arithmetic expressions are ABAP expressions with a combination of values, operators, and functions that the runtime system processes to calculate a result. For arithmetic expressions the result type depends on the type of the operands used as input to the expression.

You can use an arithmetic expression in any reading operand position, for example, the right-hand side of a value assignment.

The first example is a simple addition. The contents of amount1 and amount2 are added together and the result placed in variable total.

The second example is a bit more sophisticated. Before adding them, the contents of amount1 and amount2 are weighed with factors 2 and 3. The result of this addition is then divided by 5 to calculate a weighed average.

For basic arithmetic, ABAP provides the operators + for addition, - for subtraction, * for multiplication and / for division. Additionally, you can use the operators DIV for whole-number division and MOD for the whole-number remainder of a division. Thus, 6 DIV 4 is 1, and 6 MOD 4 is 2.

© Copyright. All rights reserved.

ABAP has a range of built-in functions for various tasks. Many of these are used for string processing, but here you can see some examples for numeric functions. You use sqrt( ) function to pull the square root and ipow( ) function to raise a number to a whole-numbered power.

In complex expressions involving more than one operator, multiplication and division take precedence over addition and subtraction. Expressions with identical precedence are processed from left to right.

Note:
ABAP syntax requires at least one blank between operators and operands. 1 + 1 is correct. 1+1 leads to a syntax error.

Blanks are also needed after opening brackets and before closing brackets.

String Processing

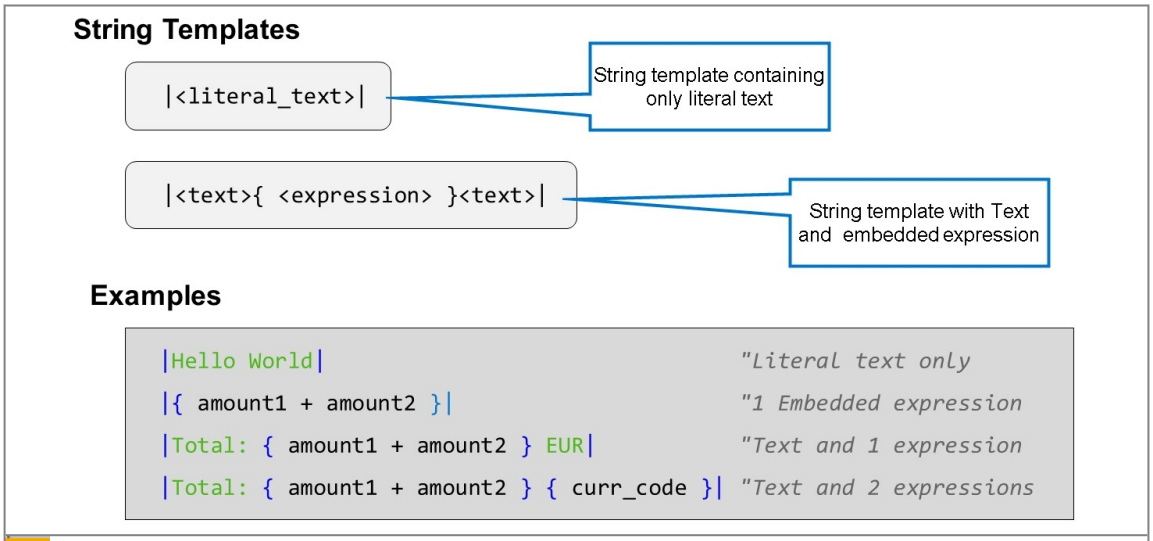


Figure 47: String Templates

String templates are ABAP expressions of result type string. You can use string templates in any reading operand position, for example, the right-hand side of a value assignment.

A string template begins and ends with a pipe-symbol (|). The simplest possible string template contains nothing but literal text. In this form a string template is not really different from a string literal.

What distinguishes a string template from a string literal is the ability to embed expressions. An embedded expression is an ABAP expression surrounded by a pair of curly brackets ({ and }). At runtime, ABAP evaluates the embedded expression and translates the result into a string. In the final result, this string replaces the embedded expression (together with the surrounding curly brackets).

Note:
ABAP syntax requires at least one blank after the opening bracket and at least one blank before the closing bracket.

© Copyright. All rights reserved.

Of course one string template can contain more than one embedded expression.

Inside the curly brackets you can place any kind of ABAP expression: arithmetic expressions, like in the example above, but single variables or even literals can serve as embedded expressions.

Format Option DATE

```
DATA the_date TYPE d VALUE '19891130'.

|{ the_date }|             "=> '19891130', internal format
|{ the_date DATE = ISO  }| "=> '1989-11-30', output in ISO format
|{ the_date DATE = USER }| "=> '11/30/1989', '30.11.1989', …
```

Format Options for Numbers

```
DATA my_number TYPE p LENGTH 3 DECIMALS 2 VALUE '-273.15'.

|{ my_number  }|
|{ my_number NUMBER = USER       }| "=> '-273.15', or '-273,15', …
|{ my_number SIGN   = RIGHT      }| "=> '273.15-'
|{ my_number STYLE  = SCIENTIFIC }| "=> '-2.7315E+02'
```

Figure 48: Format Options in String Templates

One important use case for string templates is the controlled formatting of data for output.

In the first example, variable the_date is of type d and contains a date in the internal (raw) format YYYYMMDD (where YYYY stands for the year, MM for the two-digit month and DD for the two-digit date). When you use variable the_date as embedded expression in a string template the result will be the same as the internal format. But when you add format option DATE = <date_format> within the curly brackets, the system will format the value as a date. If you add DATE = ISO, the output will be in ISO format. With DATE = USER the output format depends on the user settings of the current user.

The second example illustrates some of the options you can use for formatting numbers. Using NUMBER you control the general formatting of numbers, for example, whether a decimal point is used or a decimal comma. Using SIGN you control the position of the sign and whether a plus sign (+) is displayed or not. Using STYLE you can choose from several pre-defined styles, like a scientific style or an engineering style.

© Copyright. All rights reserved.

Operator &&

```
DATA result TYPE string.
DATA part1  TYPE string VALUE `Hello` .
DATA part2  TYPE string VALUE `World` .

result = part1 && part2.

result = part1 && | | && part2.
```

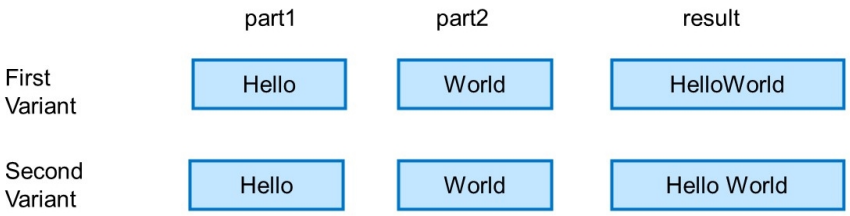


Figure 49: Joining Strings

You can join fields together using the concatenation operator &&. You can join any combination of data objects and string expressions.

The constituent parts of the expression are joined with no space or other separator between them. If you need spaces or another separator character, you must remember to insert it yourself as part of the expression as shown in the second example.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 2
Exercise 4

Declare Variables and Process Data

In this exercise, you create a program that does a calculation based on two numbers, then formats and outputs the result.

Template:

none

Solution:

/LRN/CL_S4D400_BTS_COMPUTE (global Class)

Task 1: Variable Declaration
In your own package, create a new global class and define numeric variables for the input.

1. Create a new global class that implements the interface **`IF_OO_ADT_CLASSRUN`** (suggested name: **`ZCL_##_COMPUTE`**, where ## stands for your group number).
2. In the *IF_OO_ADT_CLASSRUN~MAIN* method, declare two variables for integer values (suggested names: **`number1`** and **`number2`**).
3. Implement a value assignment for each of the two numeric variables. Assign a negative value to the first variable and a positive value to the second variable.

Task 2: Data Processing
Calculate a ratio and write the result to the console.

1. Implement an arithmetic expression that calculates the ratio of the two numbers. Store the result in a variable that you declare using an inline declaration (suggested name: **`result`**).
2. Implement a string template that puts together a text like this:“6 / 3 = 2”, with the actual values of the variables instead of 6, 3, and 2. Store the result in a variable that you declare using an inline declaration (suggested name: **`output`**).
3. Call method **`out->write( … )`** to write the text to the console.

Task 3: Activate and Test
Activate and test the console app.

1. Activate the class.
2. Execute the console app.
3. Test the app with different input values.

© Copyright. All rights reserved.

What is the result of -8 / 3?

_____

_____

_____

How can you explain this result?

_____

_____

_____

Task 4: Control the Result Precision

Make sure the result is rounded to a value with 2 decimal places, and not to an integer value.

1. Instead of an inline declaration, use an explicit declaration of variable **`result`**. Use a numeric type that allows to specify the number of decimal places.
2. Activate the class and test again.

© Copyright. All rights reserved.

Unit 2
Solution 4

Declare Variables and Process Data

In this exercise, you create a program that does a calculation based on two numbers, then formats and outputs the result.

Template:

none

Solution:

/LRN/CL_S4D400_BTS_COMPUTE (global Class)

Task 1: Variable Declaration

In your own package, create a new global class and define numeric variables for the input.

1. Create a new global class that implements the interface **`IF_OO_ADT_CLASSRUN`** (suggested name: **`ZCL_##_COMPUTE`**, where ## stands for your group number).
   a) Choose *File* → *New* → *ABAP Class*.
   b) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_COMPUTE`**, where ## stands for your group number.
   c) Enter a description.
   d) In the *Interfaces* group box, choose *Add*.
   e) Enter **`IF_OO_ADT_CLASSRUN`**. When the interface appears in the hit list, double-click it to add it to the class definition.
   f) Choose *Next*.
   g) Select your transport request and choose *Finish*.
2. In the *IF_OO_ADT_CLASSRUN~MAIN* method, declare two variables for integer values (suggested names: **`number1`** and **`number2`**).
   a) Add the following code:

```
DATA number1 TYPE i.
DATA number2 TYPE i.
```

3. Implement a value assignment for each of the two numeric variables. Assign a negative value to the first variable and a positive value to the second variable.
   a) Add the following code:

```
number1 = -8.
number2 =  3.
```

Task 2: Data Processing

Calculate a ratio and write the result to the console.

© Copyright. All rights reserved.

1. Implement an arithmetic expression that calculates the ratio of the two numbers. Store the result in a variable that you declare using an inline declaration (suggested name: **`result`**).
   a) Add the following code:

```
DATA(result) = number1 / number2.
```

2. Implement a string template that puts together a text like this:“6 / 3 = 2”, with the actual values of the variables instead of 6, 3, and 2. Store the result in a variable that you declare using an inline declaration (suggested name: **`output`**).
   a) Add the following code:

```
DATA(output) = |{ number1 } / { number2 } = { result }|.
```

3. Call method **`out->write( … )`** to write the text to the console.
   a) Proceed as you have done in the previous exercise.

Task 3: Activate and Test
Activate and test the console app.

1. Activate the class.
   a) Press **`Ctrl + F3`** to activate the class.
2. Execute the console app.
   a) Press **`F9`** to execute the console app.
3. Test the app with different input values.
   a) Edit the value assignments, then activate and execute as before.

   What is the result of -8 / 3?

   The result is -3.

   How can you explain this result?

   Due to the inline declaration, variable **`result`** is of integer type and ABAP rounds the result using the “half-up” rounding rule.

Task 4: Control the Result Precision
Make sure the result is rounded to a value with 2 decimal places, and not to an integer value.

1. Instead of an inline declaration, use an explicit declaration of variable **`result`**. Use a numeric type that allows to specify the number of decimal places.
   a) At the beginning of method **`if_oo_adt_classrun~main`**, add the following code:

```
DATA result TYPE p LENGTH 8 DECIMALS 2.
```

   b) In the code line with the inline declaration, replace `DATA(result)` with `result`.
2. Activate the class and test again.
   a) Proceed as you have done before.

© Copyright. All rights reserved.

b) Compare your code to the following extract from the model solution:

```
 METHOD if_oo_adt_classrun~main.

* Declarations
**************************

    DATA number1 TYPE i.
    DATA number2 TYPE i.

    DATA result TYPE p LENGTH 8 DECIMALS 2.


* Input Values
**************************

    number1 = -8.
    number2 =  3.

* Calculation
**************************

*    DATA(result) = number1 / number2.   "with inline declaration

   result = number1 / number2.

    DATA(output) = |{ number1 } / { number2 } = { result }|.

* Output
**************************

    out->write( output ).

  ENDMETHOD.
```

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Perform arithmetic calculations.
- Apply string processing.

© Copyright. All rights reserved.

Using Control Structures in ABAP

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Implement conditional branching.
- Handle Exceptions.
- Implement Iterations.

Conditional Branching

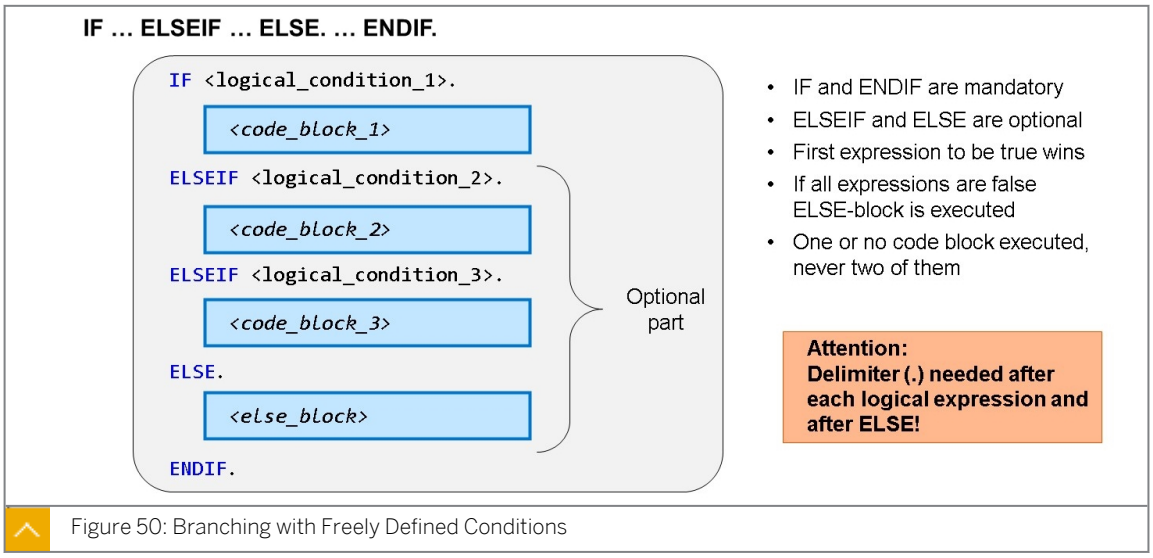


Figure 50: Branching with Freely Defined Conditions

A conditional branching is a control structure that allows you to make the execution of code dependent on logical conditions.

The most common conditional branching consists of a pair of keywords IF and ENDIF. The code block between IF and ENDIF is only executed, if the condition after IF is fulfilled.

You can add more code blocks by extending the IF ... ENDIF structure with up to one keyword ELSE and an arbitrary number of keywords ELSEIF. By adding keyword ELSE you ensure that always exactly one of the code blocks is executed. If ELSE is missing it can happen that none of the code blocks is executed.

The code block to be executed is determined as follows:

- First, the IF condition is evaluated. If it is fulfilled, the related code block is executed and the program continues after ENDIF.

© Copyright. All rights reserved.

- Only if the IF condition is not fulfilled, the condition after the first ELSEIF is evaluated. If it is fulfilled, the related code block is executed and the program continues after ENDIF.
- This is done consecutively for all ELSEIF conditions. If none of the conditions is fulfilled and the structure contains ELSE, the code block after ELSE is executed. Otherwise the none of the code blocks is executed.

Hint:
As opposed to many other programming languages, ABAP requires a delimiter (.) after each of the logical conditions and even after keyword ELSE.

```
IF x = y.
IF ( y <= x AND x < 2 * y ) OR ( y >= x AND x > 2 * y ).
IF abs( x ) BETWEEN abs( y ) AND abs( 2 * y ).
```

Comparison Operators

• =	(equal)	<>	(non-equal)
• <	(less than)	>	(greater than)
• <=	(less than or equal)	>=	(greater than or equal

Logical Operators

- NOT, AND, OR

Sequence of Evaluation

- NOT before AND
- AND before OR

Special Logical Expressions and Predicate Functions

- <data object> IS INITIAL, <data object> IS NOT INITIAL
- <expression> BETWEEN <expression1> AND <expression2>
- contains(...), line_exists(...)

Figure 51: Logical Conditions

Logical conditions are a combination of comparisons, logical operations, expressions and functions that the runtime system evaluates to decide whether the condition is true or false.

The most common use-case for logical conditions is after keywords IF or ELSEIF in an IF ... ENDIF. structure.

The first example is a simple comparison: The condition is true if the two data objects x and y have the same value.

The second example is a bit more sophisticated: Either the value of x is greater than or equal to y and less than twice the value of y or it is less than or equal to y and greater than twice the value of y.

The third example makes use of arithmetic function abs( ) and logical expression BETWEEN <expression1> AND <expression2> . The condition is true if the absolute value of x lies between the absolute value of y and the absolute value of two times y.

For simple value comparisons you can use operators =, <>, >, <, >=, and <=. You can not only compare the values of data objects, but the values of many other expressions, like the arithmetic expression 2 * y in the example.

© Copyright. All rights reserved.

Note:
ABAP uses the same symbol (=) for value assignments and for value comparisons. The distinction is made based on the position.

You can use operators AND and OR to combine logical expressions and operator NOT to negate an expression. Without brackets, NOT binds stronger than AND and AND stronger than OR.

ABAP knows some special logical expressions:

- <data object> IS INITIAL is true if <data object> contains its type-specific initial value
- <data object> IS NOT INITIAL is true if <data object> contains a value that is different from the type-specific initial value
- <data object> BETWEEN <expression1> AND <expression2>

Some special ABAP functions are predicate functions. This means that they are logical conditions themselves. Contains( ) is a function that compares character-like values and line_exists( ) performs an existence check for a row in an internal table.

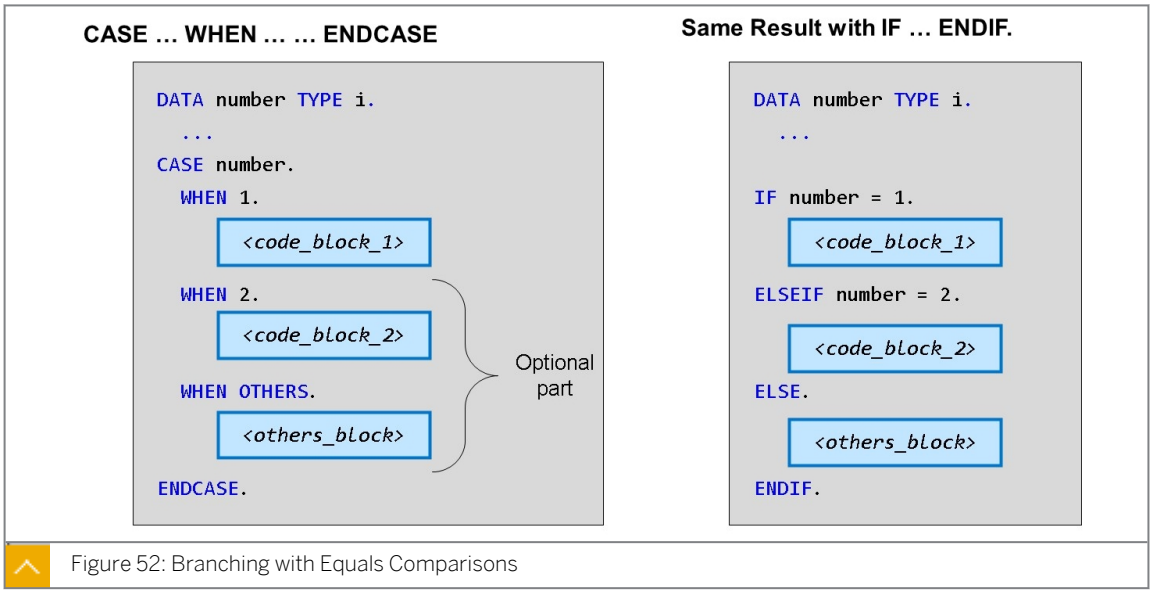


Figure 52: Branching with Equals Comparisons

A second technique for conditional branching is the CASE ... WHEN .. ENDCASE control structure.

Conditional branching with CASE .. ENDCASE is a special case of the more general branching with IF ... ENDIF. You can use CASE in situations where the branching depends on the value of a single data object, which you consecutively compare to a set of possible values, using an equals comparison each time.

In the example, the value of data object number is compared to values 1 and 2. If the value equals 1, <code_block_1> is executed and if the value euqals 2, <code_blocl_2> is executed instead. For any other value, the code block after WHEN OTHERS is executed.

Any conditional branching with CASE ... ENDCASE could be implemented with an IF ... ENDIF structure, as well. This is illustrated with the example on the right.

© Copyright. All rights reserved.

Hint:
You should use CASE ... ENDCASE when dealing with the special case to increase readability of your code.

Exception Handling

Exceptions

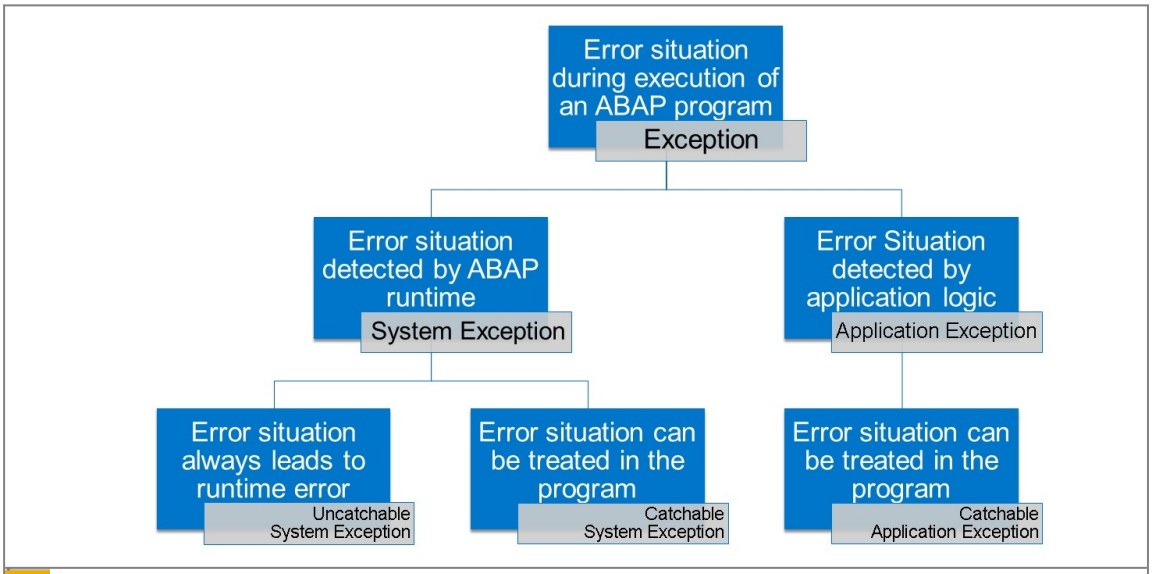


Figure 53: Exceptions

In ABAP, an exception is an error situation during execution of an ABAP program. An exception is raised by the code that detects the error situation.

Depending on who raises the exception, we distinguish between system exceptions and application exceptions.

Without further measures exceptions result in runtime errors. A runtime error terminates the program and is documented by default in a short dump.

You can avoid the runtime error if the exception in question is catchable. A catchable exception can be treated in the program using statements TRY ... CATCH ... ENDTRY.

All application exceptions and many system exceptions are catchable. Later in this course you will learn how to raise application exceptions. At this point we will focus on the handling of catchable system exceptions.

© Copyright. All rights reserved.

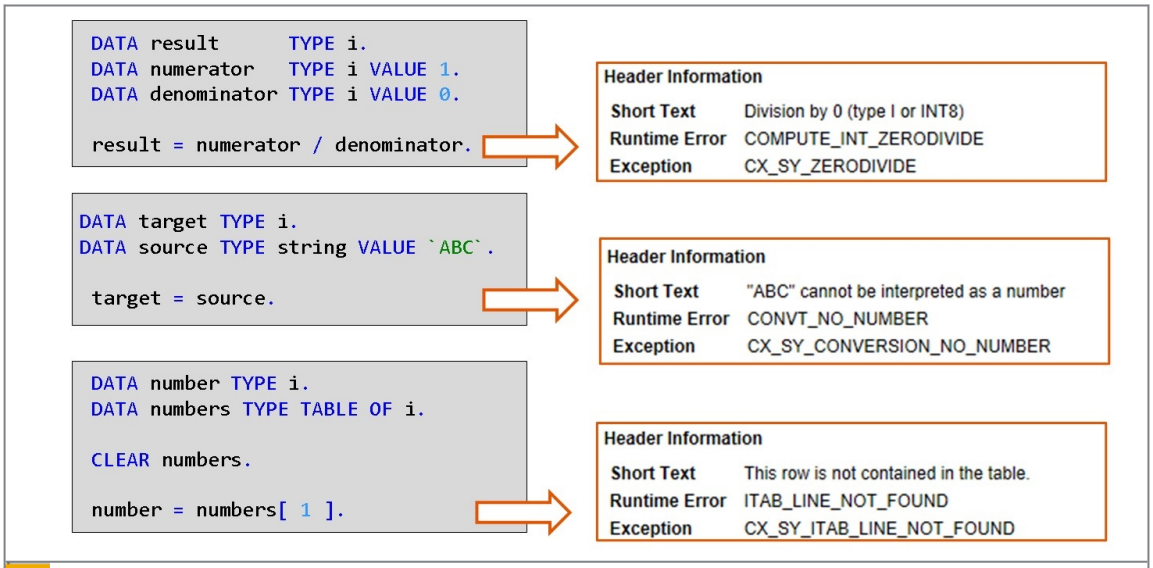


Figure 54: Some Examples of Catchable System Exceptions

In the figure, Some Examples for Catchable System Exceptions, you can see some examples of runtime errors:

- The first code example attempts a division with zero as the denominator. This is not defined.
- The second code example wants to convert a character string into an integer number but the character string cannot be interpreted as a number.
- The third code example tries to read the first row from an internal table. But the internal table is initial and therefore no first row can be found.

When you execute the code examples on the left, the ABAP runtime raises catchable system exceptions. These exceptions are not treated by the program yet, which leads to runtime errors.

The screenshots on the right are snippets from the short dumps created by these runtime errors. Beside a short text to describe the situation, you see the ID of the runtime error and the ID of the uncaught exception.

© Copyright. All rights reserved.

Exception Handling

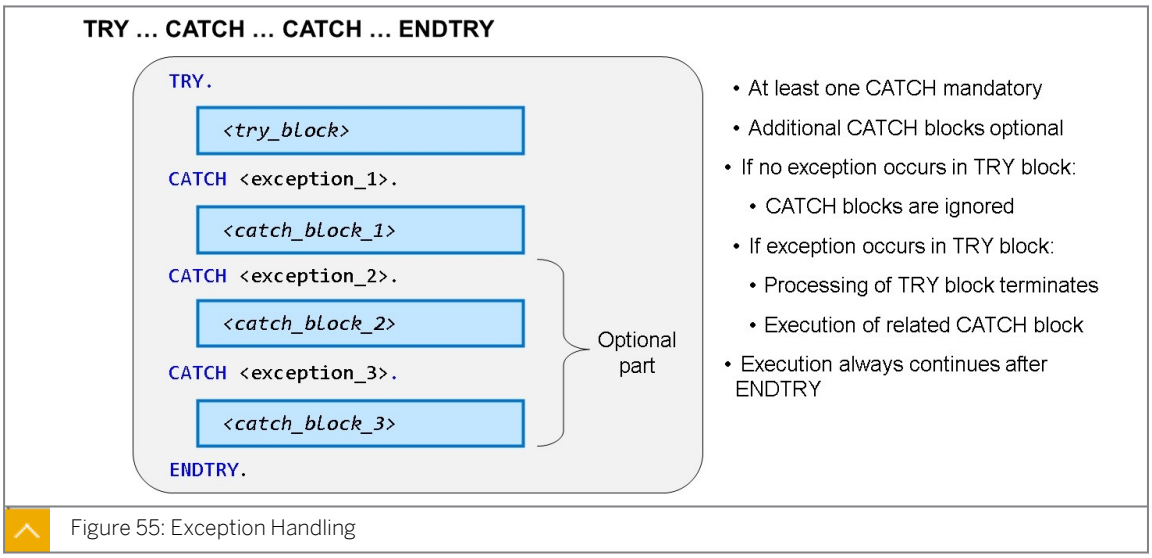


Figure 55: Exception Handling

To prevent a program from terminating because of a catchable exception, you have to surround the code from where the exception originates with statements TRY and ENDTRY. By doing so, the code becomes part of the TRY block of the TRY ... ENDTRY structure.

Before the ENDTRY statement you have to add a CATCH statement followed by the ID of the exception you want to handle. Optionally, you can add more than one CATCH statement to handle several different exceptions. Each CATCH statement should be followed by code to handle this exception. This code is called the CATCH block.

When program execution reaches the TRY statement, it continues with the code in the TRY block. Three things can happen then:

1. If no exception is raised during the TRY block, the CATCH blocks are ignored. Execution continues after the ENDTRY statement.
2. If an exception is raised during the TRY block, for which a matching CATCH exists, execution of the TRY block is terminated and the CATCH block for this exception is executed. Afterward, execution continues after the ENDTRY statement.
3. If an exception is raised during the TRY block for which no matching CATCH exists, the program terminates with a runtime error.

© Copyright. All rights reserved.

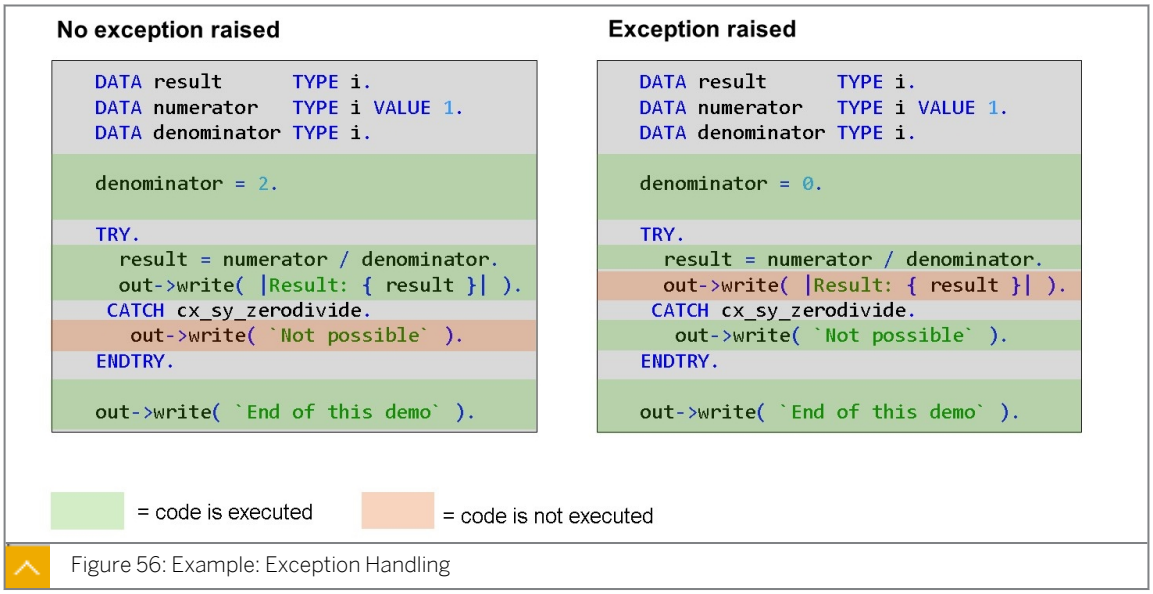


Figure 56: Example: Exception Handling

The example illustrates the handling of exception CX_SY_ZERODIVIDE which is raised if in a division the denominator is zero.

The source code on the left and on the right is identical - except for the value that is assigned to variable denominator. Red and green background colors are used to distinguish code that is executed and code that is not executed.

On the left, the entire TRY block is executed and the CATCH block is ignored. On the right, the TRY block is executed but only up to the code line, where the system raises the exception. The rest of the TRY block is not executed. Instead, the CATCH block is executed.

The code after ENDTRY is executed in both cases. It is independent from the successful or unsuccessful execution of the TRY block.

Iterations

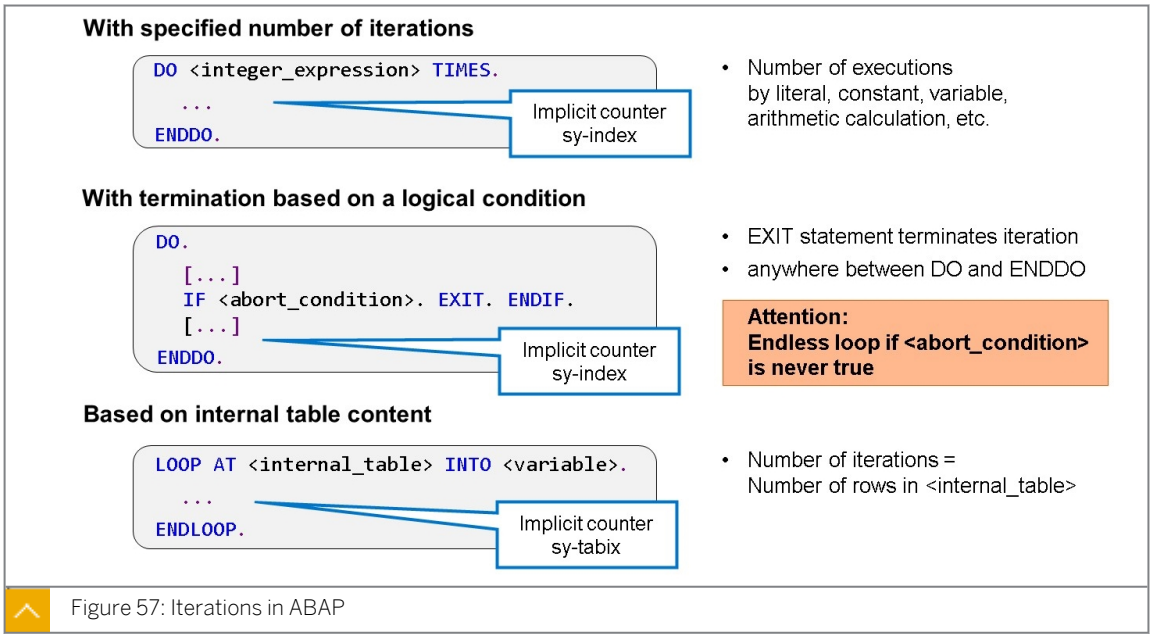


Figure 57: Iterations in ABAP

Iterations are control structures that define a block of code which is executed several times.

© Copyright. All rights reserved.

The simplest form of iteration consists of a code block surrounded by pair of statements DO and ENDDO. Without further measures this establishes an endless loop which must be avoided by one of the following possibilities:

Specified number of iterations

By extending the DO statement with an integer expression followed by keyword TIMES, you can specify explicitly how often the code block is to be iterated. The integer expression can be as simple as number literal, but also arithmetic calculations are an option. If the value of the expression equals 0, the code block between DO and ENDDO is not executed at all and the program immediately continues with the code after ENDDO.

Abort based on a logical condition

You can abort an iteration any time using the EXIT statement. The program then continues with the code after ENDDO. Be aware that outside of iterations EXIT has a different effect. There it terminates the processing of the current processing block, for example the current method.

Usually, EXIT is surrounded by IF and ENDIF to terminate the iteration depending on an abort condition. Be aware that such iterations can turn into endless loops, if the abort condition never gets true.

Of course it is possible to combine the two techniques, that is, explicitly specify the number of iterations and then leave the iteration with EXIT. Thus the number of iterations becomes a maximum number that might not be reached at runtime.

Based on an internal table

A third type of iteration is the LOOP ... ENDLOOP structure that is used to consecutively read the rows of an internal table. Here, the number of iterations is determined by the number of rows in the internal table.

In the code block between DO and ENDDO, you can implement read-accesses to ABAP built-in data object sy-index. This integer variable serves as an iteration counter, that is, the ABAP runtime increases it by one at the beginning of each new iteration.

Note:
In contradiction to what you might be used to from other programming languages, sy-index starts with value 1 during the first iteration.

ABAP built-in variable sy-tabix can fulfill a similar purpose for iterations with LOOP. But be aware that strictly speaking sy-tabix is not really a counter but it identifies the position of the table row that is processed in the current iteration. Later we will see the difference when not all rows of an internal table are processed in a LOOP ... ENDLOOP structure.

© Copyright. All rights reserved.

Unit 2
Exercise 5

Implement Conditional Branching

In this exercise, you enable the program for the four basic calculation operations and handle error situations.

Template:

/LRN/CL_S4D400_BTS_COMPUTE (global Class)

Solution:

/LRN/CL_S4D400_BTS_BRANCH (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_COMPUTE`**.

1. Open the source code of global class **`/LRN/CL_S4D400_BTS_COMPUTE`** in the ABAP editor.
2. Link the *Project Explorer* view with the editor.
3. Copy class **`/LRN/CL_S4D400_BTS_COMPUTE`** to a class in your own package (suggested name: **`ZCL_##_BRANCH`**, where ## stands for your group number).

Task 2: Variable for Operator
Declare a variable for the calculation operator and make the calculation dependent on that variable.

1. Declare a variable for the calculation operator (suggested name: **`op`**). The variable needs a type of fixed length to hold one of the following values: “+”, “-”, “*”, or “/”.
2. Directly after the value assignment for variables **`number1`** and **`number2`**, add a value assignment to set the new variable **`op`** to value “/”.
3. Use a suitable control structure to fill the variable **`result`** with different arithmetic expressions, depending on the value of the variable **`op`**.
4. Adjust the string template to use the content of **`op`**, rather than the literal character “/”.
5. Activate and test the application with different values for **`number1`**, **`number2`**, and **`op`**.

   What happens if you set **`op`** to value “%”?

   _____

   _____

   _____

Task 3: Handle Invalid Operator
Handle an invalid value of **`op`**. If **`op`** contains a value other than “+”, “-”, “*”, or “/”, write an error message to the console instead of the calculation and the result value.

© Copyright. All rights reserved.

1. In the CASE control structure, add a branch that is executed for any other values of **op**.
2. In this branch, fill **output** with a suitable text to describe the error situation.

Note:
We suggest that at this point, you replace the inline declaration of the variable **output** with an explicit declaration at the beginning of the method.

3. Use a suitable control structure to make sure the **output** variable is only filled with the calculation result if it is not already filled with an error text.
4. Activate and test the application with different values for **number1**, **number2**, and **op**.

   What happens if you set **number1** to **1**, **number2** to **0**, and **op** to **/**?

Task 4: Handle Division by Zero

Prevent the program termination by handling the exception.

1. Inside the branch for division (**op** has value "/"), surround the calculation with a pair of TRY and ENDTRY statements.
2. Before the ENDTRY statement, add a CATCH-block for the exception that is raised in case of a division by zero error.
3. In the CATCH-block, fill the **output** variable with a suitable error text.
4. Activate and test the application with different values for **number1**, **number2**, and **op**. In particular, test the handling of the error situations: invalid operator and division by zero.

© Copyright. All rights reserved.

Unit 2
Solution 5

Implement Conditional Branching

In this exercise, you enable the program for the four basic calculation operations and handle error situations.

Template:

/LRN/CL_S4D400_BTS_COMPUTE (global Class)

Solution:

/LRN/CL_S4D400_BTS_BRANCH (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_COMPUTE`**.

1. Open the source code of global class **`/LRN/CL_S4D400_BTS_COMPUTE`** in the ABAP editor.
   a) In the *Eclipse* toolbar, choose *Open ABAP Development Object*. Alternatively, press **`Ctrl + Shift + A`**.
   b) Enter **`/LRN/CL_S4D400_BTS`** as search string.
   c) From the list of development objects choose */LRN/CL_S4D400_BTS_COMPUTE*, then choose *OK*.
2. Link the *Project Explorer* view with the editor.
   a) In the *Project Explorer* toolbar, find the *Link with Editor* button. If it is not yet selected, choose it. As a result, the development object, that is open in the editor, should be highlighted in the *Project Explorer*.
3. Copy class **`/LRN/CL_S4D400_BTS_COMPUTE`** to a class in your own package (suggested name: **`ZCL_##_BRANCH`**, where ## stands for your group number).
   a) In the *Project Explorer* view, right-click class **`/LRN/CL_S4D400_BTS_COMPUTE`** to open the content menu.
   b) From the context menu, choose *Duplicate ...*.
   c) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_BRANCH`**, where ## stands for your group number.
   d) Adjust the description and choose *Next*.
   e) Confirm the transport request and choose *Finish*.

Task 2: Variable for Operator
Declare a variable for the calculation operator and make the calculation dependent on that variable.

© Copyright. All rights reserved.

1. Declare a variable for the calculation operator (suggested name: **op**). The variable needs a type of fixed length to hold one of the following values: “+”, “-”, “*”, or “/”.
   a) Add the following code:

```
DATA op      TYPE c LENGTH 1.
```

2. Directly after the value assignment for variables **number1** and **number2**, add a value assignment to set the new variable **op** to value “/”.
   a) Adjust the code as follows:

```
number1 = 123.
number2 = 0.
op      = '/'.
```

3. Use a suitable control structure to fill the variable **result** with different arithmetic expressions, depending on the value of the variable **op**.
   a) Adjust the code as follows:

```
CASE op.
   WHEN '+'.
     result = number1 + number2.
   WHEN '-'.
     result = number1 - number2.
   WHEN '*'.
     result = number1 * number2.
   WHEN '/'.
     result = number1 / number2.
ENDCASE.
```

4. Adjust the string template to use the content of **op**, rather than the literal character “/”.
   a) Adjust the string template as follows:

```
output = |{ number1 } { op } { number2 } = { result }|.
```

5. Activate and test the application with different values for **number1**, **number2**, and **op**.

   What happens if you set **op** to value “%”?

   The result is always zero (0.00) because none of the four WHEN branches is executed.

   a) Proceed as you have done in previous exercises.

Task 3: Handle Invalid Operator

Handle an invalid value of **op**. If **op** contains a value other than “+”, “-”, “*”, or “/”, write an error message to the console instead of the calculation and the result value.

1. In the CASE control structure, add a branch that is executed for any other values of **op**.
   a) Adjust the code as follows:

```
CASE op.
     WHEN '+'.
       result = number1 + number2.
     WHEN '-'.
```


© Copyright. All rights reserved.

```
      result = number1 - number2.
    WHEN '*'.
      result = number1 * number2.
    WHEN '/'.
      result = number1 / number2.
    WHEN OTHERS.

  ENDCASE.
```

2. In this branch, fill **output** with a suitable text to describe the error situation.

Note:
We suggest that at this point, you replace the inline declaration of the variable **output** with an explicit declaration at the beginning of the method.

a) At the beginning of the method, add the following code:

```
DATA output TYPE string.
```

b) Replace the inline declaration `DATA(output)` with the explicitly declared variable **output**.

c) After the `WHEN OTHERS.` statement, add the following code:

```
output = |'{ op }' is not a valid operator!|.
```

3. Use a suitable control structure to make sure the **output** variable is only filled with the calculation result if it is not already filled with an error text.

a) Adjust the code as follows:

```
IF output IS INITIAL.  "no error so far
  output = |{ number1 } { op } { number2 } = { result }|.
ENDIF.
```

4. Activate and test the application with different values for **number1**, **number2**, and **op**.

What happens if you set **number1** to **1**, **number2** to **0**, and **op** to **/**?

The application terminates with exception CX_SY_ZERODIVIDE.

a) Proceed as you have done in previous exercises.

Task 4: Handle Division by Zero

Prevent the program termination by handling the exception.

1. Inside the branch for division (**op** has value "/"), surround the calculation with a pair of TRY and ENDTRY statements.

a) Adjust the code as follows:

```
  WHEN '/'.

     TRY.
         result = number1 / number2.
```

© Copyright. All rights reserved.

```
        ENDTRY.

      WHEN OTHERS.
```

2. Before the ENDTRY statement, add a CATCH-block for the exception that is raised in case of a division by zero error.
   a) Adjust the code as follows:

```
        TRY.
            result = number1 / number2.

          CATCH cx_sy_zerodivide.
         ENDTRY.
```

3. In the CATCH-block, fill the **output** variable with a suitable error text.
   a) Adjust the code as follows:

```
         TRY.
            result = number1 / number2.

          CATCH cx_sy_zerodivide.
            output = |Division by zero is not defined|.
        ENDTRY.
```

4. Activate and test the application with different values for **number1**, **number2**, and **op**. In particular, test the handling of the error situations: invalid operator and division by zero.
   a) Proceed as you have done before.
   b) Compare your code to the following extract from the model solution:

```
METHOD if_oo_adt_classrun~main.

* Declarations
**************************

    DATA number1 TYPE i.
    DATA number2 TYPE i.

    DATA result  TYPE p LENGTH 8 DECIMALS 2.

    DATA op      TYPE c LENGTH 1.

    DATA output TYPE string.

* Input Values
**************************

    number1 = 123.
    number2 = 0.
    op      = '/'.

* Calculation
**************************
    CASE op.
      WHEN '+'.
        result = number1 + number2.
      WHEN '-'.
        result = number1 - number2.
```


© Copyright. All rights reserved.

```
      WHEN '*'.
        result = number1 * number2.
      WHEN '/'.

        TRY.
            result = number1 / number2.
          CATCH cx_sy_zerodivide.
            output = |Division by zero is not defined|.
        ENDTRY.

      WHEN OTHERS.

        output = |'{ op }' is not a valid operator!|.

    ENDCASE.

    IF output IS INITIAL.  "no error so far

      output = |{ number1 } { op } { number2 } = { result }|.

    ENDIF.

* Output
**************************

    out->write( output ).

  ENDMETHOD.
```

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Implement conditional branching.
- Handle Exceptions.
- Implement Iterations.

© Copyright. All rights reserved.

Unit 2
Lesson 5

Working with Simple Internal Tables

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Define simple internal tables.
- Process data using simple internal tables.

Definition of Simple Internal Tables

Internal Tables

Internal tables are variable data objects in which you can store several values of identical type. This type has to be specified in the declaration and is called the row type of the internal table.

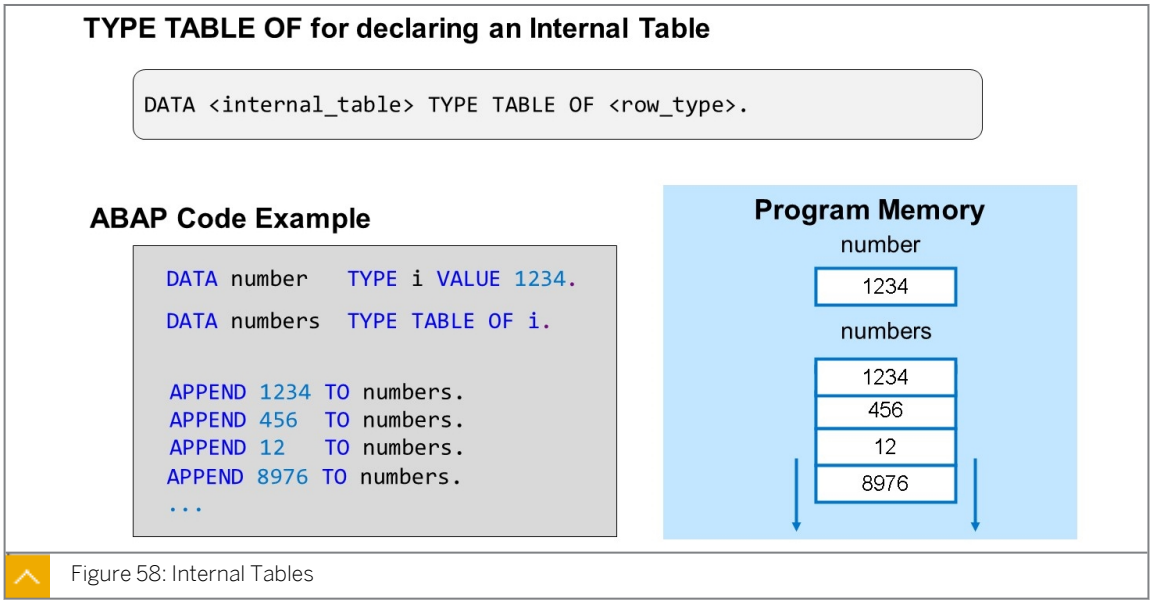


Figure 58: Internal Tables

Each value occupies one row of the internal table. The number of rows is not restricted. Theoretically, you can store any number of values in one internal table. Limitations only come from technical boundaries like available memory or system configuration.

The initial value of an internal table is an empty table or, in other words, a table with 0 lines. There are different techniques for filling an internal table. The example uses the APPEND statement to add a new row at the end of the internal table and fill it with a value.

© Copyright. All rights reserved.

Table Types

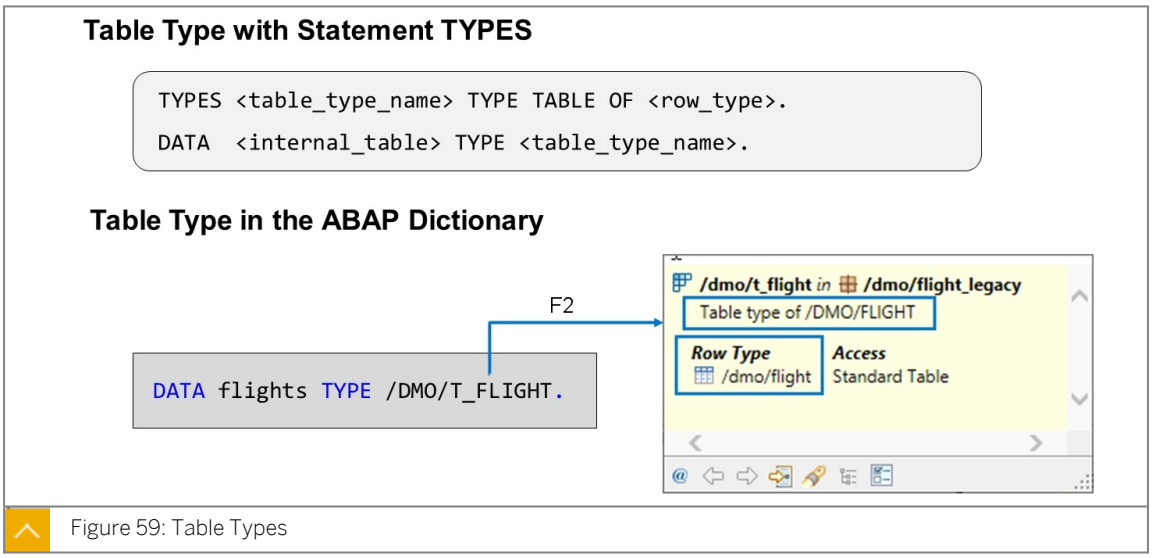


Figure 59: Table Types

There are also table types in the ABAP Dictionary. These table type are maintained with a dedicated editor. They are called global table types because they are visible anywhere in the system.

Data Processing with Simple Internal Tables

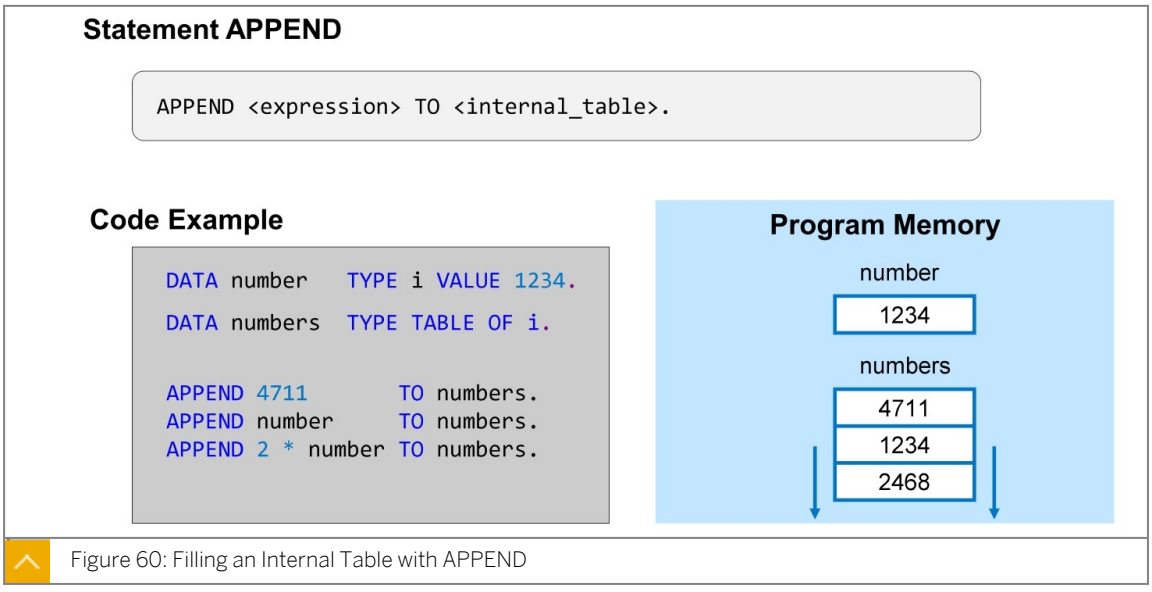


Figure 60: Filling an Internal Table with APPEND

One way to fill an internal table with data is the APPEND statement. With each APPEND statement a new row is added to the internal table. This new row is then filled with the value of the expression after keyword APPEND.

Note:

The type of the expression has to be compatible with the line type of the internal table. If it is not identical, the system attempts an implicit type conversion.

© Copyright. All rights reserved.

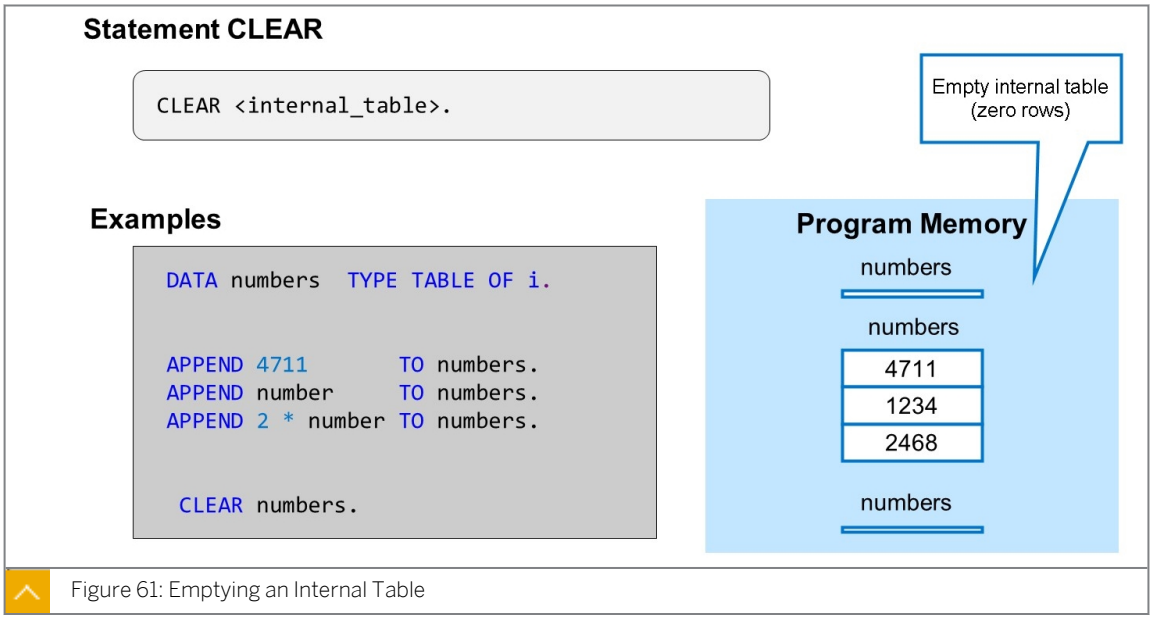


Figure 61: Emptying an Internal Table

The initial value of an internal table is an empty table, that is, an internal table with zero rows.

You already learned that with statement CLEAR you can reset an ABAP variable to its type-specific initial value.

When you use CLEAR for an internal table, you delete all its content and set the number of rows to zero.

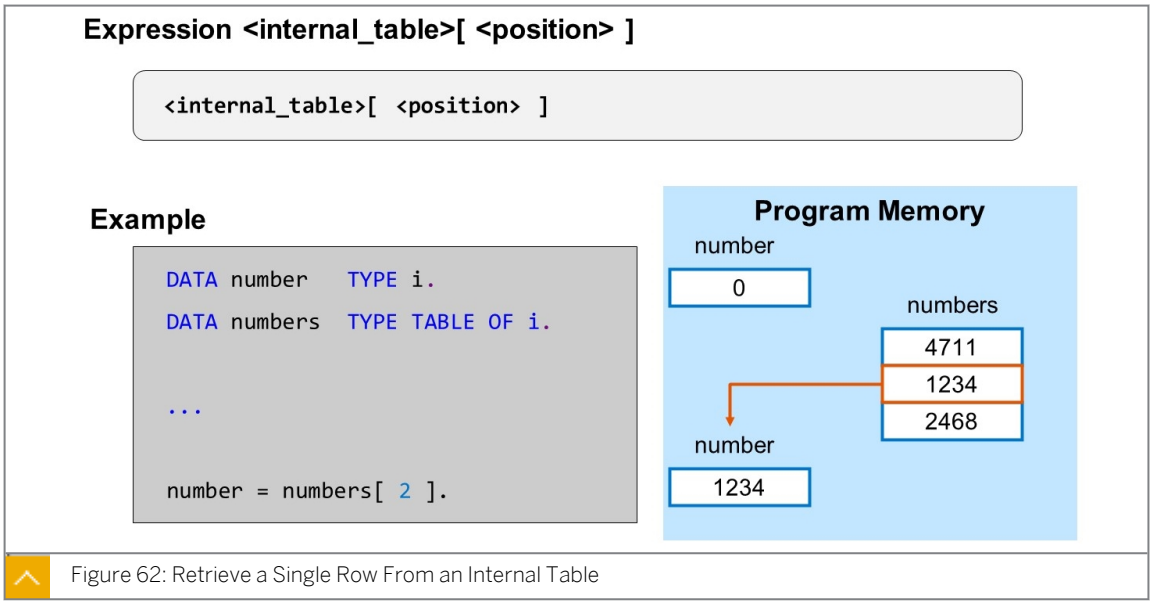


Figure 62: Retrieve a Single Row From an Internal Table

There are various ways to retrieve data from an internal table. This example retrieves the content of a single row using an internal table expression. In the table expression, the name of the internal table is followed immediately by a pair of square brackets. An integer expression inside the brackets specifies the position of the row to be read.

Note:

Correct syntax requires at least one blank after the opening bracket and before the closing bracket.

© Copyright. All rights reserved.

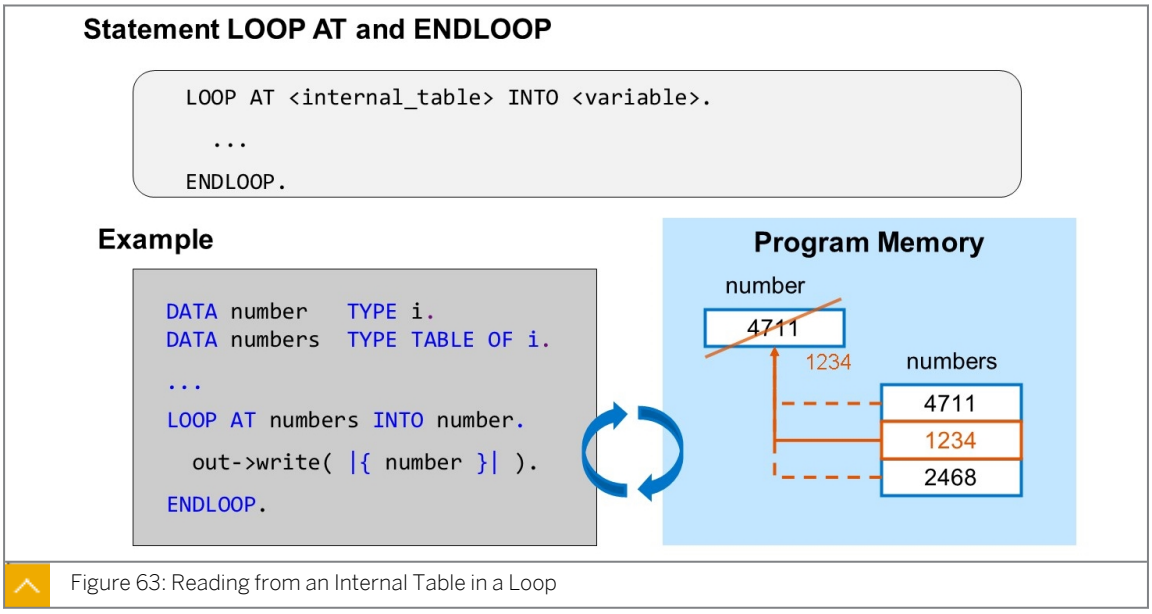


Figure 63: Reading from an Internal Table in a Loop

You can read all rows of an internal table and process them in turn by using a combination of statements LOOP AT and ENDLOOP. LOOP AT and ENDLOOP always come in pairs, that is, for each LOOP AT statement there has to be exactly one corresponding ENDLOOP statement.

Between LOOP AT and ENDLOOP, you can place any amount of ABAP code. The code you place there is processed several times, once for each row of the internal table.

Note:
If the internal table is empty, the coding between LOOP AT and ENDLOOP is not processed at all.

The data object listed after addition INTO is called the work area for the internal table. The type of this data object must be identical to, or at least compatible with, the line type of the internal table.

At the beginning of each round, the work area is filled with the data from the next table row.

In the example internal table `numbers` contains 3 rows. Therefore the coding between LOOP AT and ENDLOOP is processed 3 times. During the first round, variable `number` contains 4711, the value of the first row. This value is written to the console. At the beginning of the second round, the value in variable `number` is replaced with value 1234 from the second row, and so on.

© Copyright. All rights reserved.

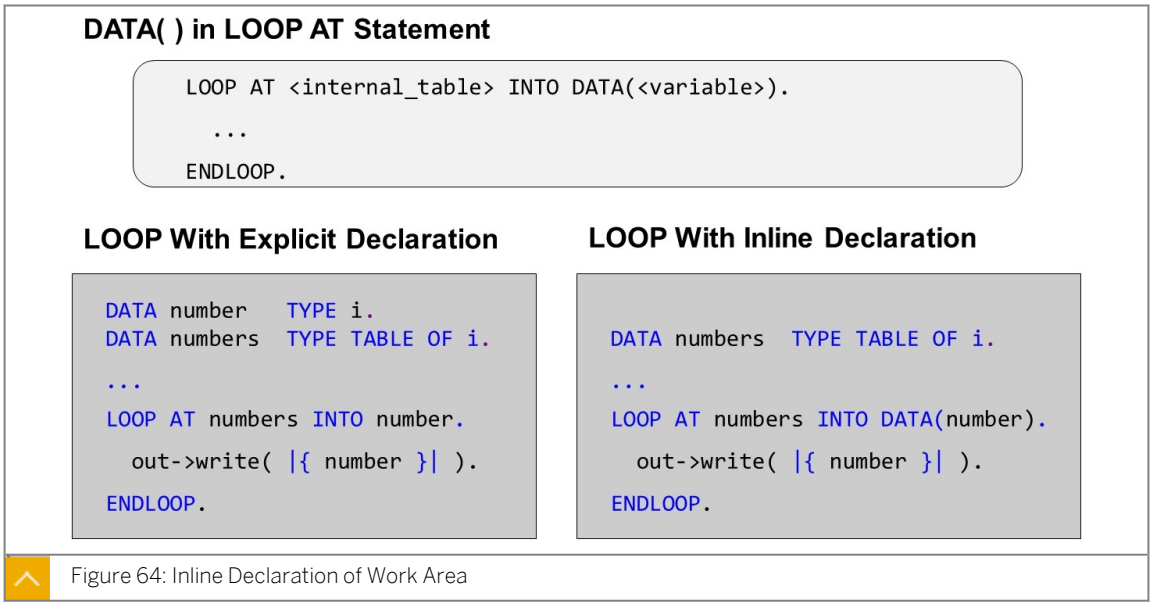


Figure 64: Inline Declaration of Work Area

When you implement a loop over an internal table, you can use an inline declaration after addition INTO instead of declaring the work area explicitly with a DATA statement.

By doing so, you not only reduce the amount of code you have to type, you also ensure that the type of work area fits the line type of the internal table, because the type of the inline declared data object is derived from the context, which, in this case, is the internal table.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 2
Exercise 6

Work with Simple Internal Tables

In this exercise, you use iterations and simple internal tables to calculate and display the Fibonacci numbers.

Note:
The Fibonacci numbers are a sequence of numbers in which each number is the sum of the two preceding ones. The sequence starts with 0 and 1, and each subsequent number is the sum of the two previous numbers. The sequence has various mathematical properties, patterns, and applications in fields such as mathematics, computer science, nature, and art.

Template:

none

Solution:

/LRN/CL_S4D400_BTS_ITERATE (global Class)

Task 1: Calculate the Numbers

In your own package, create a new global class. Define a constant for the number of iterations and an internal table for the Fibonacci numbers. Then calculate the Fibonacci numbers in an iteration and store them in the internal table.

1. Create a new global class that implements the interface **`IF_OO_ADT_CLASSRUN`** (suggested name: **`ZCL_##_ITERATE`**, where ## stands for your group number).
2. In the *IF_OO_ADT_CLASSRUN~MAIN* method, define a constant for the number of iterations (suggested name: **`max_count`**) with a small value, for example **`20`**.
3. Declare a simple internal table to store the Fibonacci numbers (suggested name: **`numbers`**).
4. Implement an iteration that is executed **`max_count`** times.
5. Inside the iteration, implement a case distinction based on the built-in iteration counter: In the first iteration, add the value **`0`** to the internal table **`numbers`**. In the second iteration, add the value **`1`** to the end of internal table **`numbers`**.

   Hint:
   The built-in iteration counter is system field **`sy-index`**.

6. Add a branch that is executed for all other values. In this branch, calculate the new entry as the sum of the two preceding entries in **`numbers`**.

© Copyright. All rights reserved.

Hint:
Access the preceding entries using index **`sy-index - 1`** and **`sy-index - 2`**.

Task 2: Prepare a Formatted Output

In a loop over the internal table, prepare a formatted output that lists each Fibonacci number with its respective sequential number. Prepare the output in another internal table of row type *string*.

1. At the beginning of the method **`if_oo_adt_classrun~main`**, declare an internal table of row type *string* (suggested name: **`output`**).
2. Implement a loop over the internal table **`numbers`** to read the Fibonacci numbers one by one into a variable **`number`**.

Hint:
Use an inline declaration for **`number`**.

3. Declare an integer variable (suggested name: **`counter`**), that you set to zero before the loop and increase by 1 in each iteration.

Hint:
You can use an inline declaration for **`counter`**.

4. In each iteration, add a new row to the internal table **`output`** which contains the content of **`counter`** (4 characters wide, left-justified), a colon (:) and the content of **`number`** (10 characters wide, right justified).

Hint:
Use a string template with suitable format options to adjust the format.

Which format options are used to set the width and the alignment?

Task 3: Output and Test

Write the result to the console. Then activate and test your class as a console app.

1. At the end of the method, call the method **`out->write`** to write the content of **`output`** to the console. Supply the importing parameter **`name`** with a suitable caption.
2. Activate the class.
3. Execute your class as a console app.

© Copyright. All rights reserved.

Unit 2
Solution 6

Work with Simple Internal Tables

In this exercise, you use iterations and simple internal tables to calculate and display the Fibonacci numbers.

Note:
The Fibonacci numbers are a sequence of numbers in which each number is the sum of the two preceding ones. The sequence starts with 0 and 1, and each subsequent number is the sum of the two previous numbers. The sequence has various mathematical properties, patterns, and applications in fields such as mathematics, computer science, nature, and art.

Template:

none

Solution:

/LRN/CL_S4D400_BTS_ITERATE (global Class)

Task 1: Calculate the Numbers

In your own package, create a new global class. Define a constant for the number of iterations and an internal table for the Fibonacci numbers. Then calculate the Fibonacci numbers in an iteration and store them in the internal table.

1. Create a new global class that implements the interface **`IF_OO_ADT_CLASSRUN`** (suggested name: **`ZCL_##_ITERATE`**, where ## stands for your group number).
   a) Choose *File* → *New* → *ABAP Class*.
   b) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_ITERATE`**, where ## stands for your group number.
   c) Enter a description.
   d) In the *Interfaces* group box, choose *Add*.
   e) Enter **`IF_OO_ADT_CLASSRUN`**. When the interface appears in the hit list, double-click it to add it to the class definition.
   f) Choose *Next*.
   g) Select your transport request and choose *Finish*.
2. In the *IF_OO_ADT_CLASSRUN~MAIN* method, define a constant for the number of iterations (suggested name: **`max_count`**) with a small value, for example **`20`**.

© Copyright. All rights reserved.

a) Add the following code:

```
CONSTANTS max_count TYPE i VALUE 20.
```

3. Declare a simple internal table to store the Fibonacci numbers (suggested name: **numbers**).

a) Add the following code:

```
DATA numbers TYPE TABLE OF i.
```

4. Implement an iteration that is executed **max_count** times.

a) After the declarations, add the following code:

```
DO max_count TIMES.

ENDDO.
```

5. Inside the iteration, implement a case distinction based on the built-in iteration counter: In the first iteration, add the value **0** to the internal table **numbers**. In the second iteration, add the value **1** to the end of internal table **numbers**.

Hint:
The built-in iteration counter is system field **sy-index**.

a) Between DO and ENDDO, add the following code:

```
CASE sy-index.
   WHEN 1.
     APPEND 0 TO numbers.
   WHEN 2.
     APPEND 1 TO numbers.
ENDCASE.
```

6. Add a branch that is executed for all other values. In this branch, calculate the new entry as the sum of the two preceding entries in **numbers**.

Hint:
Access the preceding entries using index **sy-index - 1** and **sy-index - 2**.

a) Adjust the code as follows:

```
CASE sy-index.
   WHEN 1.
     APPEND 0 TO numbers.
   WHEN 2.
     APPEND 1 TO numbers.
   WHEN OTHERS.
       APPEND numbers[  sy-index - 2 ]
            + numbers[  sy-index - 1 ]
         TO numbers.
ENDCASE.
```


© Copyright. All rights reserved.

Task 2: Prepare a Formatted Output

In a loop over the internal table, prepare a formatted output that lists each Fibonacci number with its respective sequential number. Prepare the output in another internal table of row type *string*.

1. At the beginning of the method **`if_oo_adt_classrun~main`**, declare an internal table of row type *string* (suggested name: **`output`**).
2. Implement a loop over the internal table **`numbers`** to read the Fibonacci numbers one by one into a variable **`number`**.

Hint:
Use an inline declaration for **`number`**.

a) At the end of the method, add the following code:

```
LOOP AT numbers INTO DATA(number).

ENDLOOP.
```

3. Declare an integer variable (suggested name: **`counter`**), that you set to zero before the loop and increase by 1 in each iteration.

Hint:
You can use an inline declaration for **`counter`**.

a) Adjust the code as follows:

```
DATA(counter) = 0.
LOOP AT numbers INTO DATA(number).

   counter = counter + 1.

ENDLOOP.
```

4. In each iteration, add a new row to the internal table **`output`** which contains the content of **`counter`** (4 characters wide, left-justified), a colon (:) and the content of **`number`** (10 characters wide, right justified).

Hint:
Use a string template with suitable format options to adjust the format.

Which format options are used to set the width and the alignment?

Format options WIDTH and ALIGN.

© Copyright. All rights reserved.

a) Adjust the code as follows:

```
  DATA(counter) = 0.
  LOOP AT numbers INTO DATA(number).

     counter = counter + 1.

     APPEND |{ counter WIDTH = 4 ALIGN = LEFT }: { number WIDTH = 10
ALIGN = RIGHT }|
          TO output.

  ENDLOOP.
```

Task 3: Output and Test

Write the result to the console. Then activate and test your class as a console app.

1. At the end of the method, call the method **`out->write`** to write the content of **`output`** to the console. Supply the importing parameter **`name`** with a suitable caption.
   a) After `ENDLOOP.`, add the following code:

```
  out->write(
         data   = output
         name   = |The first { max_count } Fibonacci Numbers|
                ) .
```

2. Activate the class.
   a) Press **`Ctrl + F3`** to activate the class.
3. Execute your class as a console app.
   a) Press **`F9`** to execute the class as console app.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Define simple internal tables.
- Process data using simple internal tables.

© Copyright. All rights reserved.

Unit 2
Lesson 6

Debugging an ABAP Program

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Enter debugging mode.
- Control the execution of code.
- Analyze the content of data objects.

The Debugging Mode

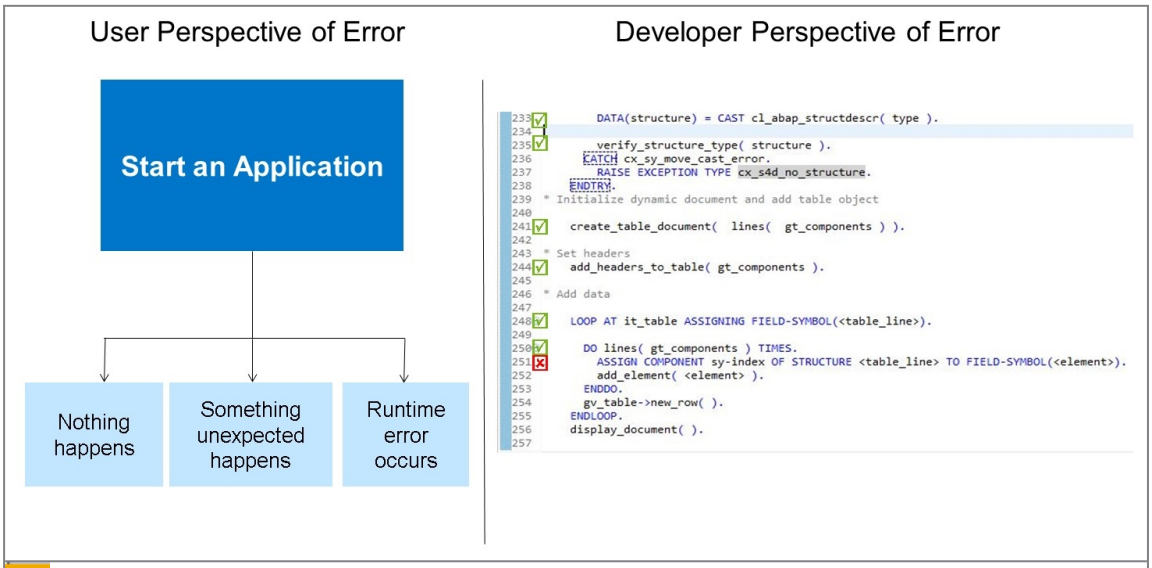


Figure 65: The Need For Debugging

There is no way around the fact that errors occur in programs. However, they manifest themselves in different ways. When a user starts a faulty application, it may crash, something unexpected may happen, or nothing at all may happen. From the user's point of view, at user interface level, it is impossible to say just how and why this error occurred.

As a developer, you now need to examine the program more closely - line-by-line in fact - to establish just what statements and combinations of values in the different program variables caused the error. This is where the Debugger comes in.

© Copyright. All rights reserved.

Starting the Debugger

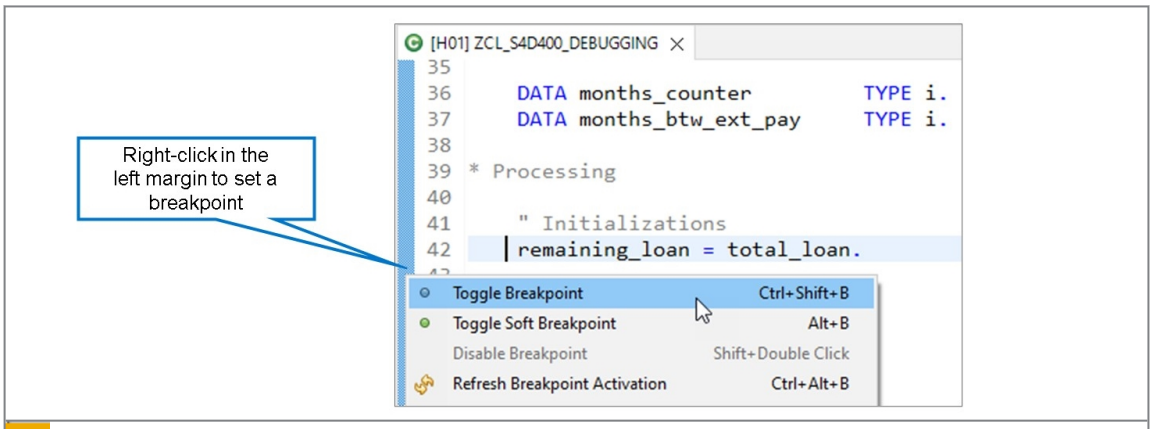


Figure 66: Starting the Debugger

To debug an ABAP program, you set a breakpoint then run the program normally. When the program reaches the breakpoint, the system interrupts it and opens the ABAP Debug perspective in ADT. You can then execute each subsequent statement individually to see what effect it has on the program. You can also inspect the contents of all of the variables in the program to see if any of the values are unexpected.

To set or remove a breakpoint, right-click the left margin of the editor and choose Toggle Breakpoint. As an alternative you can double-click the left margin. Note that the program has to be activated before you can set breakpoints.

Breakpoints are user-specific and are persistent - they remain active even after you have logged off from ADT and back on again. To prevent the debugger from starting at a breakpoint you must either delete the breakpoint (using the Toggle Breakpoint function) or deactivate it using the corresponding function in the context menu.

Note:
Depending on your personalization settings, ADT will ask for confirmation, first, before automatically opening the debug perspective.

© Copyright. All rights reserved.

The Debug Perspective in ADT

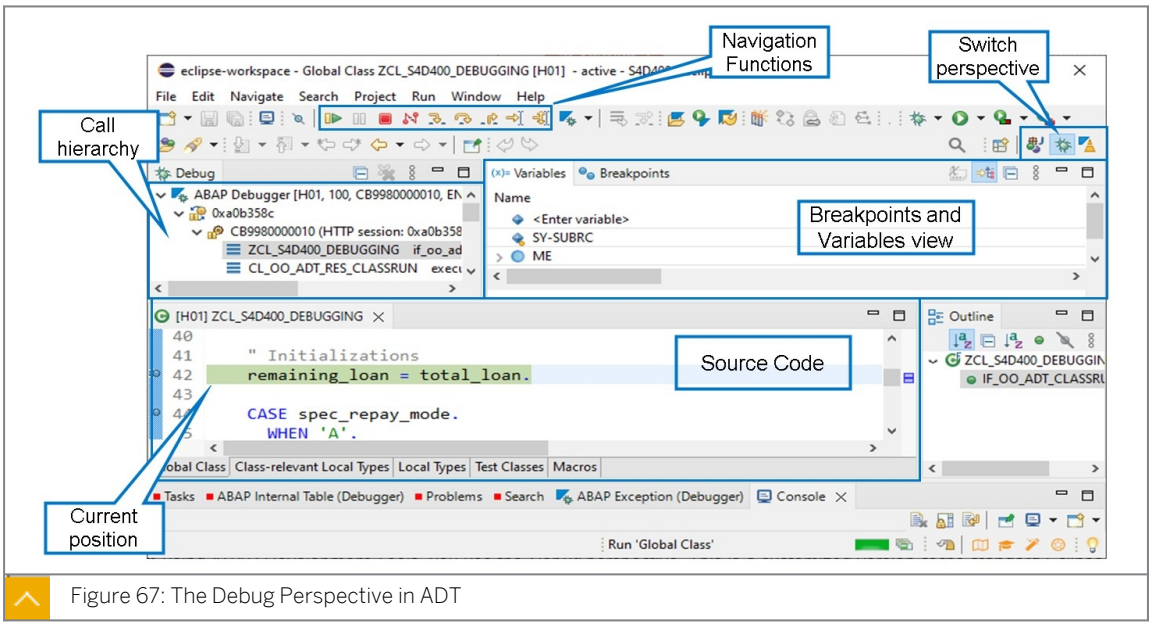


Figure 67: The Debug Perspective in ADT

When you debug an ABAP program using ABAP Development Tools, you use the Debug perspective. This is a customized version of the standard Eclipse Debug perspective, and it contains views and functions that are particularly important for debugging.

Some important elements of the debugger perspective are as follows:

Source Code View

The *Source Code* view is the central part of the debugger perspective. It displays the source code and highlights the current position in the program.

Variables View

The *Variables* view is also very important. You use this view to display the current values of variables.

Breakpoints View

The *Breakpoints* view is displayed next to the *Variables* view (not selected in the figure above). You use this view to display, delete or create breakpoints. Breakpoints are points in the program at which normal processing is interrupted and the system shows you the Debugger so that you can analyze the state of the program at exactly that moment.

Navigation Functions

While debugging a program, you use the navigation functions to control the execution of the code.

Debug View

The *Debug* view on the upper left shows the debugging session and the call hierarchy. You will need this later when you debug calls of modularization units, for example, methods.

Perspective Selector

You can switch back to the ABAP perspective with the *Perspective Selector* buttons in the right upper corner.

© Copyright. All rights reserved.

Hint:
Because many important editor tools are missing from the Debugger perspective it is recommended to switch back to the ABAP perspective once you finished debugging.

Control of Code Execution

Some Navigation Functions

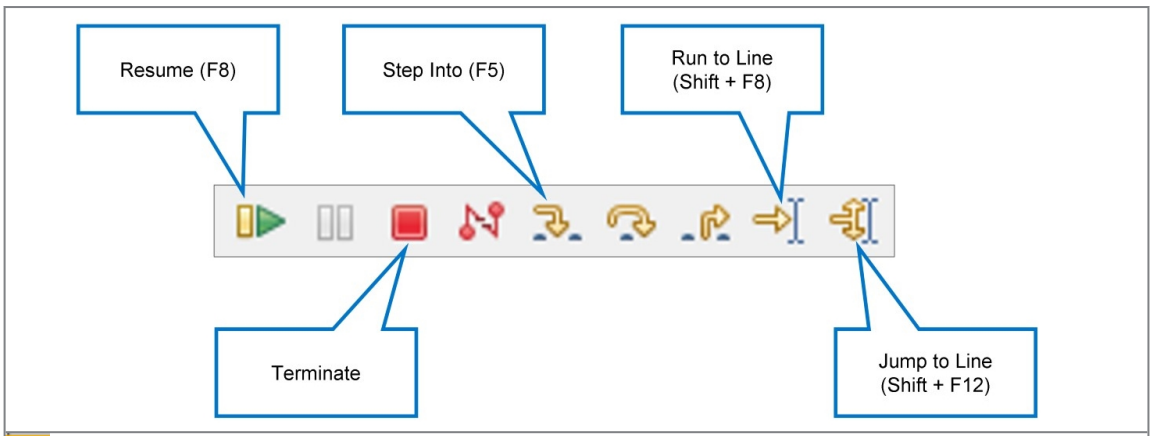


Figure 68: Navigation Functions

Once you started debugging, use the navigation functions to control the execution of the code.

Some important navigation functions are as follows:

Step Into (`F5`)

Choose *Step Into* or press **`F5`** to execute a single step. Use this function for a step-by-step analysis. If, for example, you want to see which code block of a control structure is actually executed.

Resume (`F8`)

Choose *Resume* or press **`F8`** to execute the program up to the next breakpoint. If the debugger does not hit any more breakpoints, the program is executed to the end and the debugging session terminates.

Run to Line (`Shift+F8`)

Choose *Run to Line* or press **`Shift+F8`** to execute the program up to the current cursor position. Clicking on a code line and choosing this function is a convenient alternative to setting a breakpoint, choosing Resume and removing the breakpoint again.

Jump to Line (`Shift+F12`)

Choose *Jump to Line* or press **`Shift+F12`** to skip some lines of code or to jump backwards to some already executed code. This function can be helpful to simulate what would happen if a certain piece of code was removed or to repeat debugging a bit of code you missed analysis the first time. Keep in mind that this is actually jumping, not executing coding. When you jump backwards, changes to data objects are not reverted!

Terminate

© Copyright. All rights reserved.

Choose *Terminate* if you are done with debugging and you do not want to execute the remaining program. The debug session terminates immediately.

Special Breakpoints

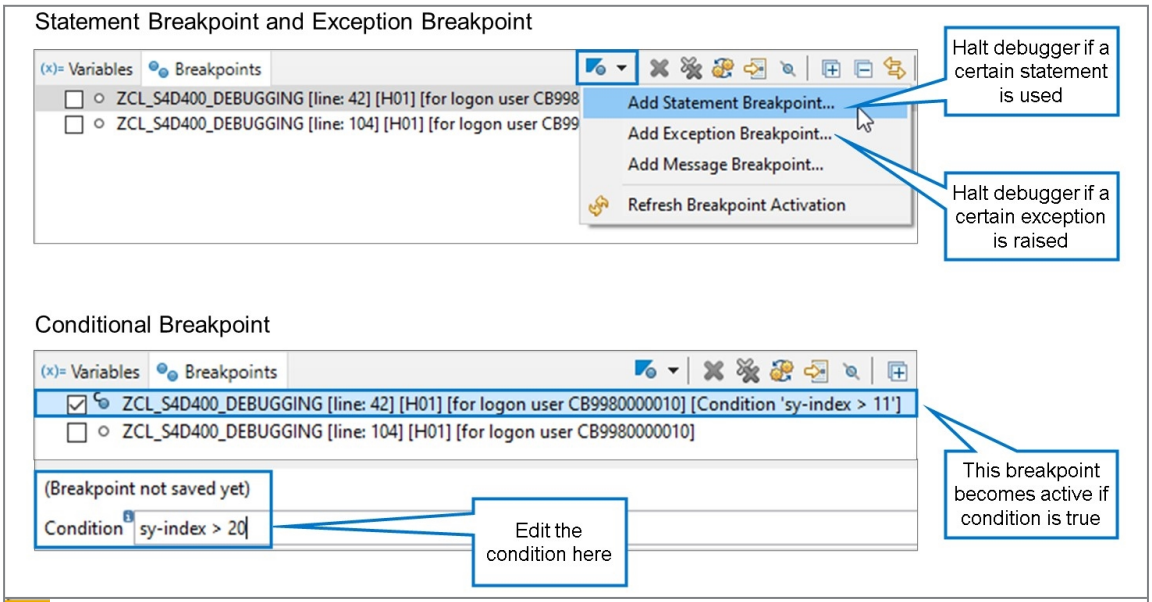


Figure 69: Creating Special Breakpoints

You learned that you can create and manage breakpoints by clicking on the left margin of the ABAP editor view. This also works with the ABAP editor view in the Debug perspective.

In addition, you can switch to the Breakpoints view and manage your breakpoints there.

In the Breakpoints view you can also create special breakpoints:

Statement Breakpoint

A statement breakpoint is not attached to a specific line of code but to a specific ABAP statement. A statement breakpoint on statement CLEAR, for example, causes the program to stop in the debugger whenever a CLEAR statement is executed - no matter where this statement is located.

To create a statement breakpoint, open the dropdown list from the toolbar of the Breakpoints view and choose *Add Statement Breakpoint ...*

Exception Breakpoint

An exception breakpoint is attached to a specific exception. It causes the program to stop in the debugger whenever this particular exception is raised - no matter if this exception is handled by the program or causes a runtime error. To create an exception breakpoint, open the dropdown list from the toolbar of the Breakpoints view and choose Add Exception Breakpoint ... .

Conditional Breakpoint

You turn a breakpoint into a conditional breakpoint by adding a condition. If program execution hits a conditional breakpoint, the program only stops in the debugger, if the condition is fulfilled. If, for example, a breakpoint is located between DO and ENDDO it will cause the program to stop in the debugger in every iteration. But if you add a condition sy-index > 20 the debugger will ignore this breakpoint during the first 20 iterations and only stop in the following iterations.

© Copyright. All rights reserved.

To add a condition to a breakpoint, choose it in the list of breakpoints and enter the condition in field Condition. Press Enter to save the breakpoint with the condition.

Watchpoints

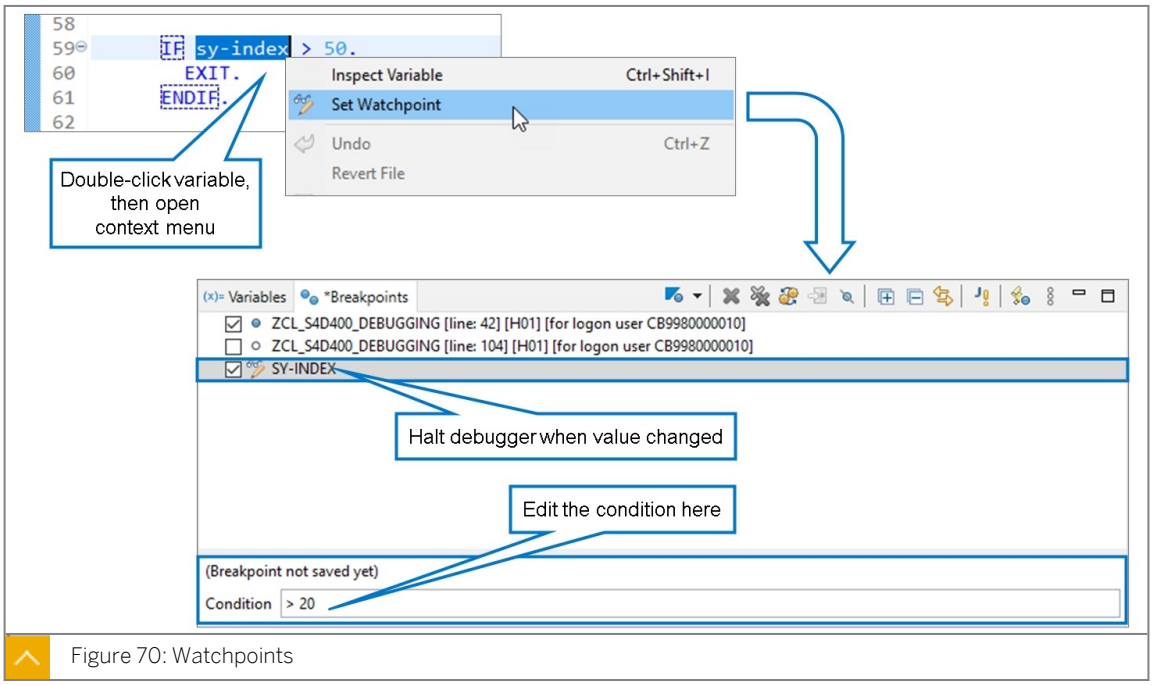


Figure 70: Watchpoints

If an unexpected value of a variable is causing you problems, you can track its value during the course of the program using a watchpoint.

A simple watchpoint on a variable causes the program to stop in the debugger whenever the value of this variable changes. By adding a condition, you can achieve that the program does not stop at every value change of the variable but only in those cases where also the condition is fulfilled.

To set a watchpoint on a variable, double-click the variable in the source code, then right-click it and choose *Set Watchpoint*. This creates a watchpoint on this variable, which you can then see in the *Breakpoints* view.

To add a condition to a watchpoint, choose it in the list of breakpoints and enter the condition in field *Condition*. Choose *Save* to save the watchpoint with the condition.

© Copyright. All rights reserved.

Analysis of Data Objects

Display Content of Data Objects

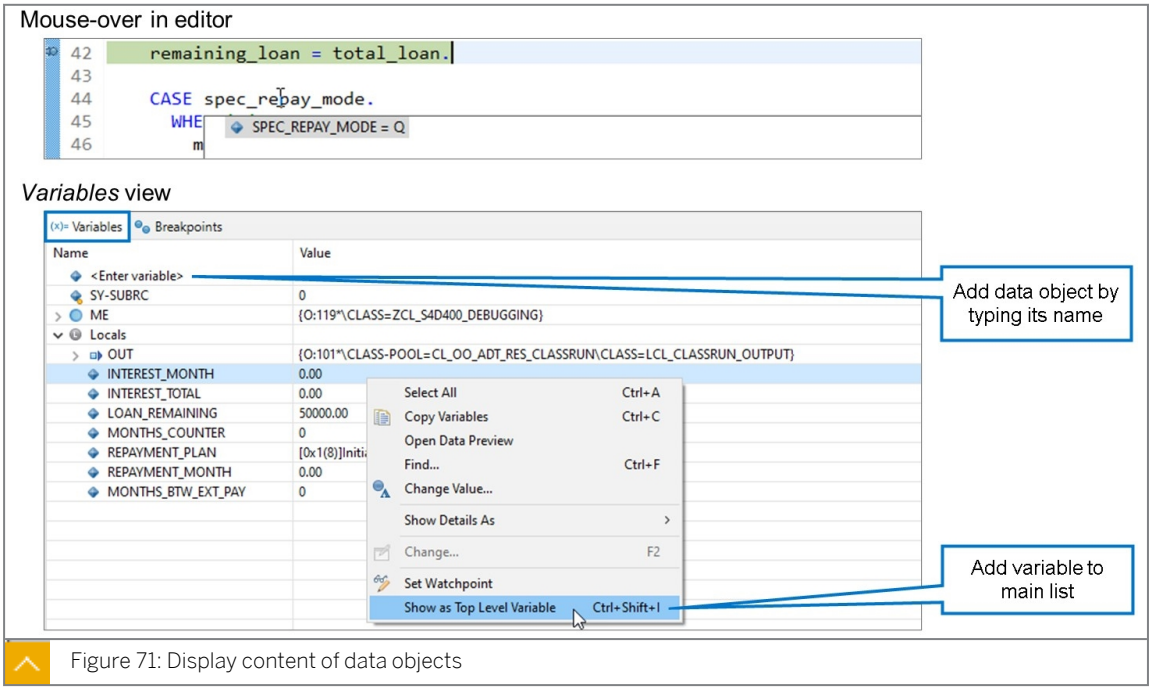


Figure 71: Display content of data objects

One way to analyze the content of data objects in the debugger is the mouse-over functionality of the ABAP Editor. While in debugging mode, place the cursor on the name of a data object and wait a moment. A dialog box opens with the content of the data object.

Another way to analyze the content of data objects in the debugger is the Variables view. This view displays a list of data objects and their current values. The main list, the so-called top-level variables, contains some built-in data objects, by default. In the example, these are SY-SUBRC and ME. Expand node Locals to see a list of all variable data objects defined in the current processing block.

There are three ways to add data objects to the main list on the Variables view:

- In the editor, double-click on the name of a data object
- In the variables list, left-click on placeholder <Enter variable> and enter the name of the data object
- Right-click on a variable in Locals node and choose Show as Top Level Variable

Hint:
To remove a data object from the list, right-click on it and choose *Remove*.

© Copyright. All rights reserved.

Display Content of Internal Tables

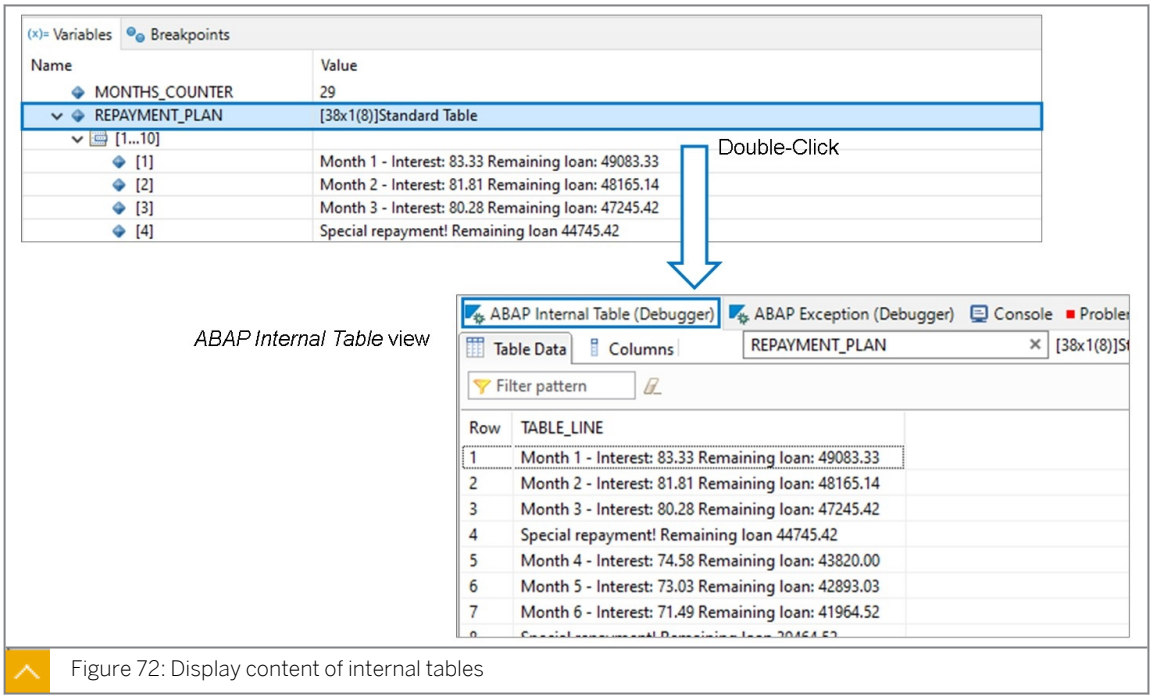


Figure 72: Display content of internal tables

For internal tables, the *Variables* view displays an overview with the number of rows and the number of columns. Expand the hierarchy below the name of the internal table to see the content of the rows. Double-click the name of the internal table to analyze it in the *ABAP Internal Table* view.

The *ABAP Internal Table* view is a debugger tool to analyze the content of internal tables. In the default configuration of the *Debugger* perspective the *ABAP Internal Table* view is one of the views below the source code. You can filter the table content by entering a filter pattern and pressing *Enter*.

Changing the Values of Variables

Depending on your authorizations, you can change the value of variables during debugging.

© Copyright. All rights reserved.

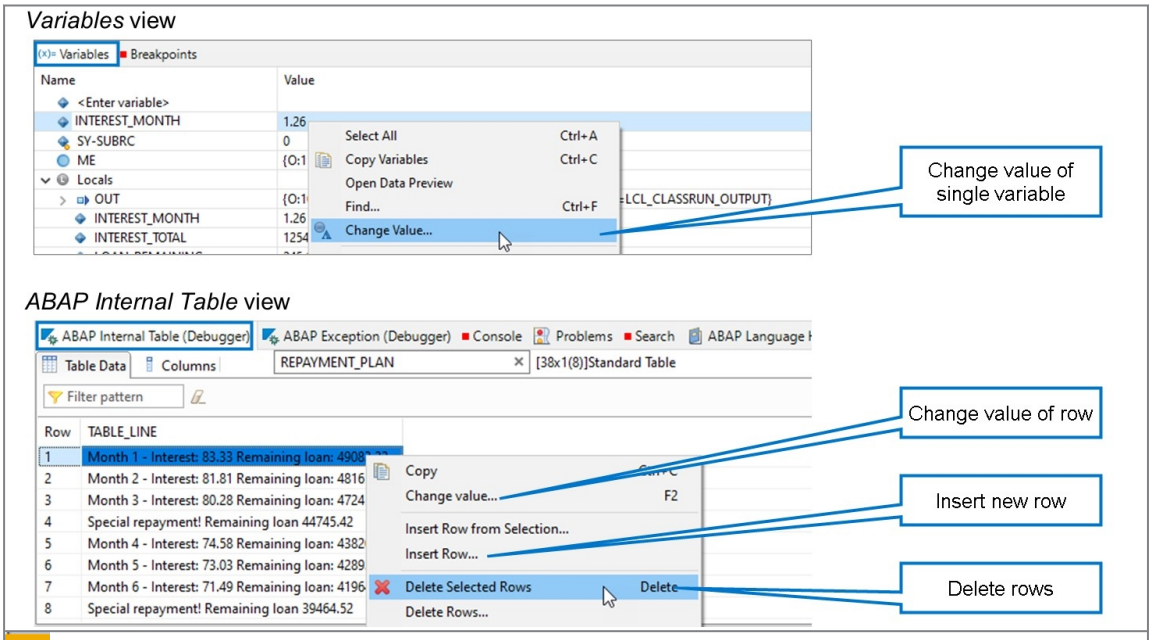


Figure 73: Changing the values of variables

For simple variables, locate the variable in the *Variables* view, right-click on it and choose *Change Value...*.

To change the content of an internal table, we have to distinguish between changing the value of an existing row and adding or deleting of rows.

As shown in the figure, the following functions are available when you right-click in the *ABAP Internal Table* view:

Change value...

Choose *Change value...* to change the content of an existing row.

Insert Row...

Choose *Insert Row...* to add a new row. You can decide whether you want to append the new row or insert it at the chosen position.

Delete Selected Rows

Choose *Delete Selected Rows* to remove the rows you selected before you right-clicked. To select a row, left-click it. To select more than one row, hold down the *Ctrl* key or the *Shift* key when you left-click additional rows.

Delete Rows...

Choose *Delete Rows...* to remove a larger range of rows, or even all rows. You are asked for the number of the start row and the end row you want to delete.

© Copyright. All rights reserved.

Unit 2
Exercise 7

Debug ABAP Code

In this exercise, you analyze ABAP code using the ABAP debugger.

Template:

/LRN/CL_S4D400_BTT_DEBUG (global Class)

Solution:

None, the class remains unchanged.

Task 1: Preparation

Copy the class that you want to debug.

1. Open the source code of global class /LRN/CL_S4D400_BTT_DEBUG in the ABAP editor.
2. Link the *Project Explorer* view with the editor.
3. Copy class /LRN/CL_S4D400_BTT_DEBUG to a class in your own package (suggested name: ZCL_##_DEBUG, where ## stands for your group number).
4. Activate and test the class.
5. Check the output in the *Console* view.

Task 2: Analyze the Starting Values

Enter the debugger at the first executable statement in method **`if_oo_adt_classrun~main( )`** and analyze the values of some variable and constant data objects.

1. Set a breakpoint at the first statement that does not define a type or declare a data object.

Hint:
TYPES defines a data type, CONSTANTS declares a constant data object, DATA declares a variable data object.

2. Run the class as a console app and enter the debugger.
3. Display the value of data object **`loan_remaining`** and **`loan_total`** in the *Variables* view.

Task 3: Control Program Execution

Set break points and watch points. Execute single steps or resume execution until the next break point or watch point is reached. Supervise the value changes of the data objects.

1. Execute a single step to debug the value assignment in the current line.
2. Display the content of data object **`spec_repay_mode`**. Then execute another single step to see which WHEN branch of the CASE - control structure is executed.

© Copyright. All rights reserved.

3. Set a watch point for variable **`loan_remaining`** and resume program execution. Where does the program execution stop again?
4. Display the content of data object **`repayment_plan`**. Then execute another single step to see how it is filled with the APPEND statement.

Note:
Because **`repayment_plan`** is an internal table, it not only displays in the *Variables* view but also in the *ABAP Internal Table (Debugger)* view below the editor.

5. Inspect the string template in the APPEND statement and relate it to the resulting first row in internal table **`repayment_plan`**. Display the content of the data objects that appear in the embedded expressions.
6. Set another watch point for variable **`repayment_plan`** and resume program execution for a few times. Whenever execution reaches one of the watch points, analyze the value of **`loan_remaining`** and new rows are added to **`repayment_plan`**.
7. After a while, set a statement break point for all EXIT statements and delete the two watch points.
8. Resume program execution until you reach the first EXIT statement. Deactivate the statement breakpoint for the **`EXIT`** statement. Then execute single steps until you reach the output part of the program.
9. Open the *Console* view. Execute the remaining program and pursue the output on the *console* view.

Note:
Do not press **`F5`** to debug the output. Use **`F6`** instead.

Note:
As you will learn later in the course `out->write( ... )` is not an ABAP statement but a reusable code block that consists of many ABAP statements. By pressing **`F5`** you *Step Into* this code to analyze it in detail. With **`F6`** you *Step Over* the code block, treating it like a single statement.

10. When the application is terminated, do not forget to switch back to the *ABAP perspective*.

© Copyright. All rights reserved.

Unit 2
Solution 7

Debug ABAP Code

In this exercise, you analyze ABAP code using the ABAP debugger.

Template:

/LRN/CL_S4D400_BTT_DEBUG (global Class)

Solution:

None, the class remains unchanged.

Task 1: Preparation

Copy the class that you want to debug.

1. Open the source code of global class /LRN/CL_S4D400_BTT_DEBUG in the ABAP editor.
   a) In the *Eclipse* toolbar, choose *Open ABAP Development Object*. Alternatively, press **`Ctrl + Shift + A`**.
   b) Enter **`/LRN/CL_S4D400_BTT`** as search string.
   c) From the list of development objects choose */LRN/CL_S4D400_BTT_DEBUG*, then choose *Ok*.
2. Link the *Project Explorer* view with the editor.
   a) In the *Project Explorer* toolbar, find the *Link with Editor* button. If it is not yet pressed, choose it. As a result, the development object, that is open in the editor, should be highlighted in the *Project Explorer*.
3. Copy class /LRN/CL_S4D400_BTT_DEBUG to a class in your own package (suggested name: ZCL_##_DEBUG, where ## stands for your group number).
   a) In the *Project Explorer* view, right-click class /LRN/CL_S4D400_BTT_DEBUG to open the content menu.
   b) From the context menu, choose *Duplicate ...*.
   c) Enter the name of your package (**`ZS4D400_##`**, where ## stands for your group number) and the name for the new class (**`ZCL_##_DEBUG`**), then choose *Next*.
   d) Confirm the transport request and choose *Finish*.
4. Activate and test the class.
   a) Press **`Ctrl + F3`** to activate the class.
   b) Press **`F9`** to run the class.
5. Check the output in the *Console* view.
   a) Check the *Console* view that should have opened as a new tab below the editor view.

© Copyright. All rights reserved.

b) If the *Console* view is not visible, open it by choosing *Window* → *Show view* → *Other*. Double-click *Console* in the hit list.

Task 2: Analyze the Starting Values

Enter the debugger at the first executable statement in method **`if_oo_adt_classrun~main( )`** and analyze the values of some variable and constant data objects.

1. Set a breakpoint at the first statement that does not define a type or declare a data object.

Hint:
TYPES defines a data type, CONSTANTS declares a constant data object, DATA declares a variable data object.

a) Double-click the left-hand margin of the editor next to the line `loan_remaining = loan_total.` to set a break point.

2. Run the class as a console app and enter the debugger.
   a) Press **`F9`** to run the class.
   b) If you are asked whether you want to switch to the Debug perspective, mark *Remember my decision* and choose *OK*.
3. Display the value of data object **`loan_remaining`** and **`loan_total`** in the *Variables* view.
   a) In the current code line (the one with a green background) double-click `loan_remaining`.
   b) In the same code line, double-click **`loan_total`**.

Task 3: Control Program Execution

Set break points and watch points. Execute single steps or resume execution until the next break point or watch point is reached. Supervise the value changes of the data objects.

1. Execute a single step to debug the value assignment in the current line.
   a) In the toolbar, choose *Step Into (F5)* or press **`F5`**.
   b) Check that the value of **`loan_remaining`** changed from **`0..00`** to **`5000.00`**.
2. Display the content of data object **`spec_repay_mode`**. Then execute another single step to see which WHEN branch of the CASE - control structure is executed.
   a) In the next code line, double-click **`spec_repay_mode`**.
   b) Press **`F5`** to see that the program jumps to code line `WHEN 'Q'.`.
3. Set a watch point for variable **`loan_remaining`** and resume program execution. Where does the program execution stop again?
   a) In the *Variables* view, right-click on *LOAN_REMAINING* and choose *Set Watchpoint*.
   b) In the toolbar, choose *Resume (F8)* or press **`F8`**.
   c) Program execution stops immediately after code line `loan_remaining = loan_remaining - repayment_month.`, also to be precise, in the next code line with executable code.

© Copyright. All rights reserved.

4. Display the content of data object **`repayment_plan`**. Then execute another single step to see how it is filled with the APPEND statement.

> Note:
> Because **`repayment_plan`** is an internal table, it not only displays in the *Variables* view but also in the *ABAP Internal Table (Debugger)* view below the editor.

   a) Double-click on **`repayment_plan`** at the end of the APPEND statement.

   b) Press **`F5`** to see how **`repayment_plan`** is filled with a first row.

5. Inspect the string template in the APPEND statement and relate it to the resulting first row in internal table **`repayment_plan`**. Display the content of the data objects that appear in the embedded expressions.
   a) Double-click the data objects that appear between the curly brackets to display their contents.

6. Set another watch point for variable **`repayment_plan`** and resume program execution for a few times. Whenever execution reaches one of the watch points, analyze the value of **`loan_remaining`** and new rows are added to **`repayment_plan`**.
   a) In the *Variables* view, right-click on *REPAYMENT_PLAN* and choose *Set Watchpoint*.

   b) Press **`F8`** to resume program execution.

   c) Analyze the data objects in the *Variables* view and the *ABAP Internal Table (Debugger)* view.

7. After a while, set a statement break point for all EXIT statements and delete the two watch points.
   a) Navigate to the *Breakpoints* view.

> Hint:
> You find the *Breakpoints* view next to the *Variables* view.

   b) In the toolbar of the *Breakpoints* view, expand the dropdown button on the very left and choose *Add Statement Breakpoint ...*.

   c) On the dialog window that appears, enter **`EXIT`** as search string, click on the value *EXIT* in the hitlist, and choose *OK*.

   d) In the *Breakpoints* view, right-click the watch point *LOAN_REMAINING* and choose *Remove*.

   e) In the *Breakpoints* view, right-click the watch point *REPAYMENT_PLAN* and choose *Remove*.

8. Resume program execution until you reach the first EXIT statement. Deactivate the statement breakpoint for the **`EXIT`** statement. Then execute single steps until you reach the output part of the program.
   a) Press **`F8`** to resume program execution.

© Copyright. All rights reserved.

b) Switch to the *Breakpoints* view and deselect the line that says **`EXIT [Statement]`**.

c) Press **F5** until you reach the first code line that starts with `out->write(`.

9. Open the *Console* view. Execute the remaining program and pursue the output on the *console* view.

Note:
Do not press **F5** to debug the output. Use **F6** instead.

Note:
As you will learn later in the course `out->write( ... )` is not an ABAP statement but a reusable code block that consists of many ABAP statements. By pressing **F5** you *Step Into* this code to analyze it in detail. With **F6** you *Step Over* the code block, treating it like a single statement.

a) Press **F6** several times until you reach the end of the application.

b) Analyze the addition output on the *Console* view and compare it to the string template in the previous code line.

10. When the application is terminated, do not forget to switch back to the *ABAP perspective*.

a) Choose *ABAP* on the very right of the Eclipse toolbar.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Enter debugging mode.
- Control the execution of code.
- Analyze the content of data objects.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Learning Assessment

1. Which of the following can you use to denote a comment in ABAP?

   Choose the correct answers.

   - ☐ A // at the beginning of a line
   - ☐ B * at the beginning of a line
   - ☐ C -- anywhere in the line
   - ☐ D " anywhere in the line

2. Which of the following predefined ABAP types are complete?

   Choose the correct answers.

   - ☐ A C
   - ☐ B D
   - ☐ C P
   - ☐ D I

3. You declare a variable as follows: DATA var TYPE I VALUE 100. Subsequently, you use the statement CLEAR var. What is the value of var after the CLEAR statement?

   Choose the correct answer.

   - ☐ A 0
   - ☐ B 100

4. The result of the expression result = var MOD 2. is 1. What does this tell you about the value of variable var?

   Choose the correct answer.

   - ☐ A var is an even number.
   - ☐ B var is an odd number.

© Copyright. All rights reserved.

5. You want to concatenate two strings. Which is the correct operator?

   Choose the correct answer.

   ☐ A +

   ☐ B &

   ☐ C ++

   ☐ D &&

6. The IF condition IF a > 10. is followed by the ELSEIF condition ELSEIF a = 25.The variable a has the value 25. Which code branch or branches are executed?

   Choose the correct answer.

   ☐ A The branch introduced by IF a > 10.

   ☐ B The branch introduced by ELSEIF a = 25.

   ☐ C Both branches.

7. Which of the following statements about exceptions are true?

   Choose the correct answers.

   ☐ A All uncaught exceptions lead to runtime errors.

   ☐ B All execptions are catchable.

   ☐ C Some exceptions are catchable, others are uncatchable.

   ☐ D A TRY...ENDTRY block must contain at least two CATCH statements.

8. How can you exit a loop in ABAP?

   Choose the correct answers.

   ☐ A After a certain number of repetitions

   ☐ B After a certain length of time

   ☐ C When a particular condition is met

9. What is the work area of an internal table?

   Choose the correct answer.

   ☐ A The first row of the internal table

   ☐ B The current row of the internal table

   ☐ C A variable with the same type as the row type of the internal table

© Copyright. All rights reserved.

10. What information do you see when you position the mouse pointer over a variable in the debugger?

    Choose the correct answer.

    - ☐ A The data type of the variable
    - ☐ B The current contents of the variable
    - ☐ C The current memory consumption of the variable

11. When do you choose *Resume* (F8) in the debugger?

    Choose the correct answer.

    - ☐ A To execute the program up to the next breakpoint
    - ☐ B To execute a single step
    - ☐ C To terminate the debug session
    - ☐ D To execute the program up to the current cursor position

© Copyright. All rights reserved.

Learning Assessment - Answers

1. Which of the following can you use to denote a comment in ABAP?

   Choose the correct answers.

   - [ ] A // at the beginning of a line
   - [x] B * at the beginning of a line
   - [ ] C -- anywhere in the line
   - [x] D " anywhere in the line

   That is correct

2. Which of the following predefined ABAP types are complete?

   Choose the correct answers.

   - [ ] A C
   - [x] B D
   - [ ] C P
   - [x] D I

   That is correct.

3. You declare a variable as follows: DATA var TYPE I VALUE 100. Subsequently, you use the statement CLEAR var. What is the value of var after the CLEAR statement?

   Choose the correct answer.

   - [x] A 0
   - [ ] B 100

   Yes, the CLEAR statement ignores the VALUE addition in the DATA statement and returns the variable to the correct initial value for its type.

© Copyright. All rights reserved.

4. The result of the expression result = var MOD 2. is 1. What does this tell you about the value of variable var?

   Choose the correct answer.

   - [ ] A var is an even number.
   - [X] B var is an odd number.

   That is correct.

5. You want to concatenate two strings. Which is the correct operator?

   Choose the correct answer.

   - [ ] A +
   - [ ] B &
   - [ ] C ++
   - [X] D &&

   That is correct.

6. The IF condition IF a > 10. is followed by the ELSEIF condition ELSEIF a = 25.The variable a has the value 25. Which code branch or branches are executed?

   Choose the correct answer.

   - [X] A The branch introduced by IF a > 10.
   - [ ] B The branch introduced by ELSEIF a = 25.
   - [ ] C Both branches.

   That is correct.

7. Which of the following statements about exceptions are true?

   Choose the correct answers.

   - [X] A All uncaught exceptions lead to runtime errors.
   - [ ] B All execptions are catchable.
   - [X] C Some exceptions are catchable, others are uncatchable.
   - [ ] D A TRY...ENDTRY block must contain at least two CATCH statements.

   That is correct.

© Copyright. All rights reserved.

8. How can you exit a loop in ABAP?

   Choose the correct answers.

   - [X] A After a certain number of repetitions
   - [ ] B After a certain length of time
   - [X] C When a particular condition is met

   That is correct.

9. What is the work area of an internal table?

   Choose the correct answer.

   - [ ] A The first row of the internal table
   - [ ] B The current row of the internal table
   - [X] C A variable with the same type as the row type of the internal table

   That is correct.

10. What information do you see when you position the mouse pointer over a variable in the debugger?

    Choose the correct answer.

    - [ ] A The data type of the variable
    - [X] B The current contents of the variable
    - [ ] C The current memory consumption of the variable

    That is correct.

11. When do you choose *Resume* (F8) in the debugger?

    Choose the correct answer.

    - [X] A To execute the program up to the next breakpoint
    - [ ] B To execute a single step
    - [ ] C To terminate the debug session
    - [ ] D To execute the program up to the current cursor position

    That is correct.

© Copyright. All rights reserved.

UNIT 3 Working with Local Classes

Lesson 1

Defining a Local Class 128
Exercise 8: Define a Local Class 133

Lesson 2

Creating Instances of a Class 137
Exercise 9: Create and Manage Instances 143

Lesson 3

Defining and Calling Methods 150
Exercise 10: Define and Call Methods 159

Lesson 4

Using Encapsulation to Ensure Consistency 168
Exercise 11: Use Private Attributes and a Constructor 175

UNIT OBJECTIVES

- Define a local class inside a global class.
- Create Instances of an ABAP Class.
- Define and call methods.
- Explain encapsulation.
- Define and use constructors.

© Copyright. All rights reserved.

Unit 3
Lesson 1

Defining a Local Class

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Define a local class inside a global class.

Local Classes

Local And Global Classes

Classes in ABAP can be either local or global.

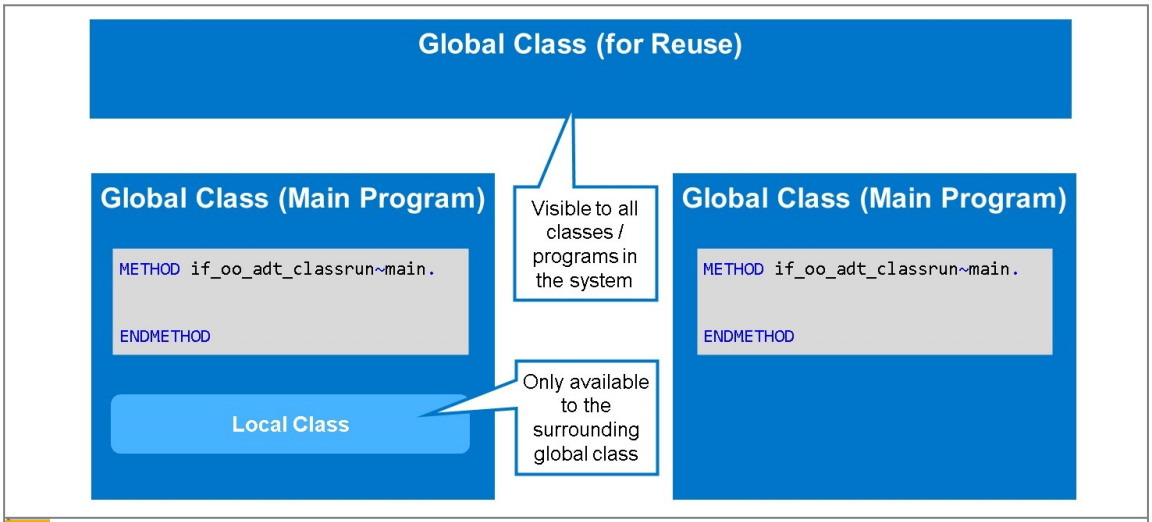


Figure 74: Local And Global Classes

Global classes are stored centrally and are contained in their own repository object called a class pool. A global class can serve as main program. Global classes can also contain logic to be reused by other ABAP programs, including other global classes.

Local classes are defined as part of an ABAP program, for example a global class. You can use them only in the program or class in which they are defined. Local classes are useful for entities or functions that you only need in a single program.

The ABAP syntax of both local and global classes is almost identical. In this course, you will be working with local classes in your global class. The global class with method if_oo_adt_classrun~main will only serve as a kind of main program.

Local Classes in Global Classes

As shown in the figure below, when you open a global class in ADT, the focus is on tab Global Class, first. Here you find the source code of the global class itself. To see or enter the source code of local classes, you have to navigate to tab Local Types.

© Copyright. All rights reserved.

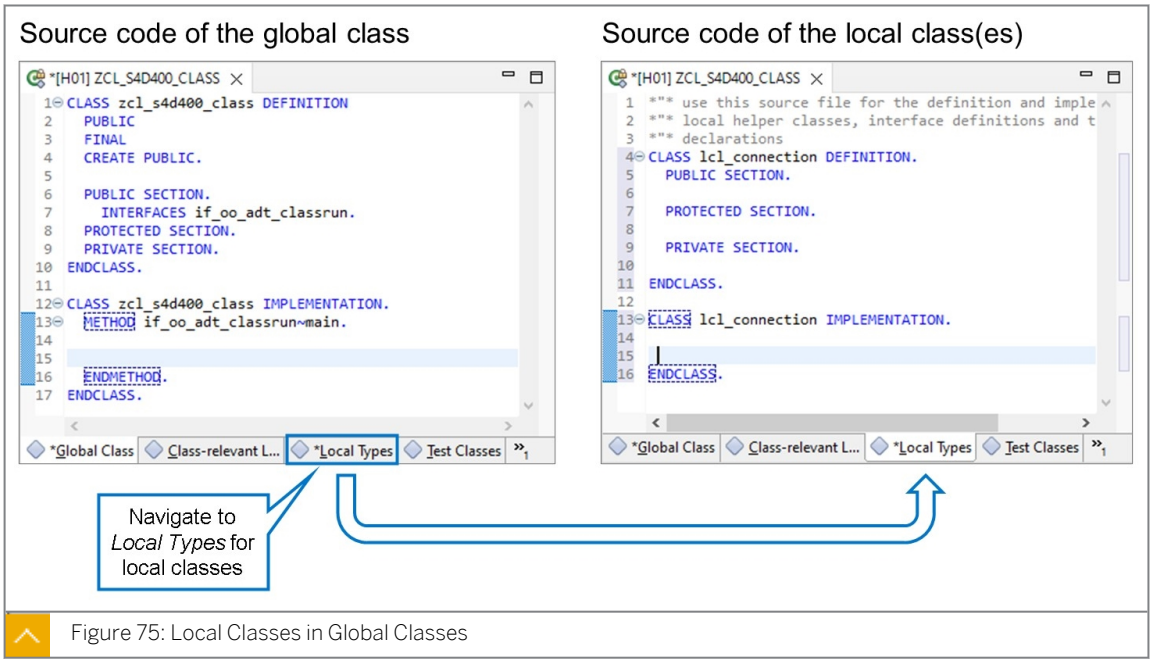


Figure 75: Local Classes in Global Classes

ADT provides a source code template for local class. To use this template proceed as follows:

1. In the ABAP editor, type **`lcl`** and press **`Ctrl + Space`**.
2. From the list that displays choose *lcl - Local class* and press **`Enter`**.
3. Adjust the name of the new local class.

For the classes in this course you have to remove the create private addition from the CLASS ... DEFINITION statement.

Attributes

Source Code of a Class in ABAP

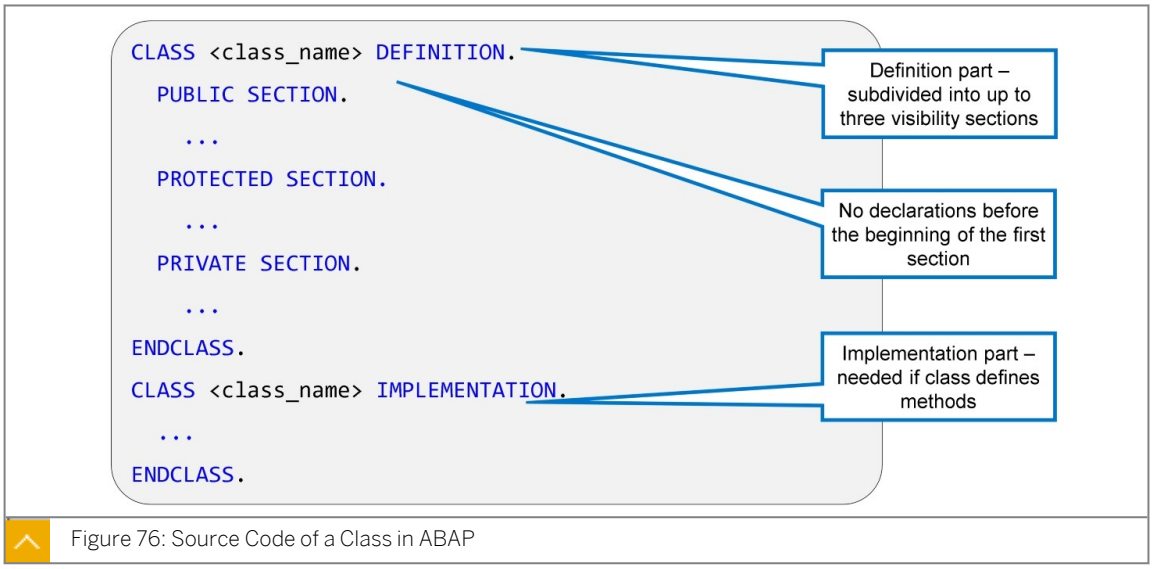


Figure 76: Source Code of a Class in ABAP

In ABAP. the source code of a class has two parts - the definition and the implementation. The definition part of a class is subdivided into up to three sections, called the visibility sections of the class.

© Copyright. All rights reserved.

Definition

The definition part of a class contains the definition and declaration of all of the elements in the class, that is, the types, the constants, the attributes, and the methods. It begins with CLASS <class_name> DEFINITION. and ends with ENDCLASS.

Implementation

The implementation part of a class contains the executable code of the class, namely the implementation of its methods. It begins with CLASS <class_name> IMPLEMENTATION. and ends with ENDCLASS. The implementation part of a class is optional. It becomes mandatory as soon as the class definition contains executable methods.

Visibility sections

Each visibility section of a class starts with one of the statements PUBLIC SECTION, PROTECTED SECTION, PRIVATE SECTION and ends implicitly when the next section begins. The last section ends with statement ENDCLASS. All declarations of a class have to be inside one of the sections. In other words: No declarations are allowed between the beginning of the class definition and the beginning of the first section.

The section in which a declaration is located defines the visibility of the declared element of the class.

The three visibility sections of a class are optional; if you do not need a particular section, you do not have to declare it. But if a class definition consists of more than one section, they must follow the order PUBLIC SECTION - PROTECTED SECTION - PRIVATE SECTION.

Definition Part of a Class

```
CLASS <class_name> DEFINITION.
  <any> SECTION.

    TYPES         <type>               TYPE <type>.
    DATA          <instance_attribute> TYPE <type> [VALUE <v> ].
    CLASS-DATA    <static_attribute>   TYPE <type> [VALUE <v> ].
    CONSTANTS     <constant>           TYPE <type>  VALUE <v>.

    METHODS       <instance_method>   <signature>.
    CLASS-METHODS <static_method>     <signature>.

ENDCLASS.
```

Figure 77: Definition Part of a Class

The figure shows the different kinds of components you can define in a class. You can define any of these components in any visibility section of the class.

TYPES

The TYPES statement allows you to define types within your class. You do this in exactly the same way as you would outside a class. If types are public, a program that uses the class can use the type (or types) to declare ist own variables.

DATA and CLASS-DATA

You use DATA and CLASS-DATA to declare the attributes of the class. DATA declares an instance attribute, while CLASS-DATA declares a static attribute.

CONSTANTS

© Copyright. All rights reserved.

Classes may also contain constants which are declared with CONSTANTS. Constants and types are static components of the class.

METHODS and CLASS-METHODS

You use METHODS and CLASS-METHODS to define the methods of a class. METHODS defines an **instance method**, while CLASS-METHODS defines a **static method**. The name of the method is followed by the method's signature; that is, the set of values that the method exchanges with its caller and the exceptions that may arise during the method.

Example: Attributes of the Flight Connection Class

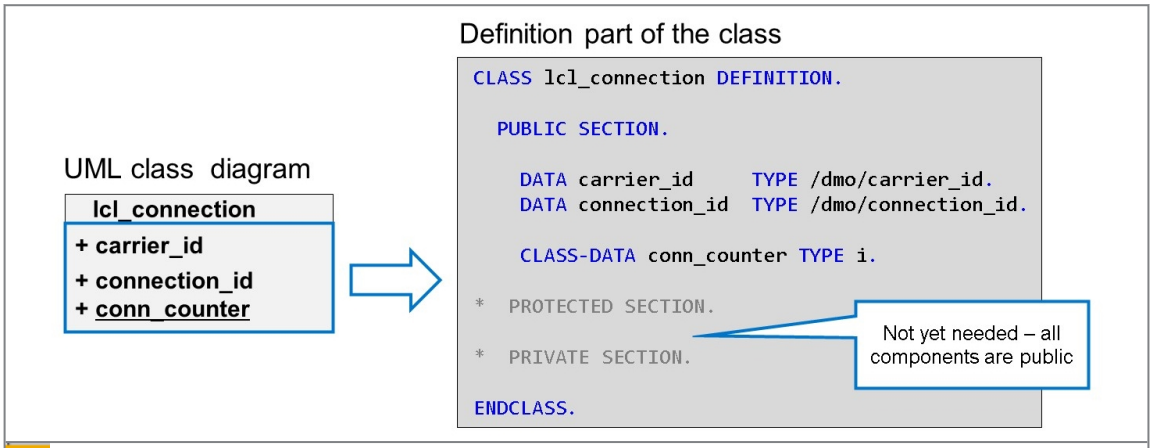


Figure 78: Example: Attributes of the Flight Connection Class

Let us start implementing our UML model in ABAP, and take the attributes of the connection class to begin with. To declare an attribute, use the DATA statement within the appropriate visibility section.

The attribute conn_counter is underlined, which denotes a static attribute. You declare static attributes using the CLASS-DATA statement. The syntax of CLASS-DATA is identical to that of DATA.

Note:
We begin with public attributes at this point. Later in the course, when we discuss encapsulation, we will turn them into private attributes.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 3
Exercise 8

Define a Local Class

In this exercise, you define a local class inside of a global class.

Template:

none

Solution:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (global Class)

Task 1: Create a Global Class

In your own package, create a new ABAP class.

1. Create a new global class that implements interface *IF_OO_ADT_CLASSRUN* (suggested name: **`ZCL_##_LOCAL_CLASS`**, where ## stands for your group number).

Task 2: Define a Local Class

Define a local class **`lcl_connection`** and declare some public attributes.

1. Inside the global class, create a new local class **`lcl_connection`**. Use code completion to generate the code, but make sure you remove the `create private` addition from the generated code.
2. In local class **`lcl_connection`**, declare the following **public** attributes:

   Table 1: Attributes

Attribute Name	Scope	Data Type
`carrier_id`	instance	/DMO/CARRIER_ID
`connection_id`	instance	/DMO/CONNECTION_ID
`conn_counter`	static	I

3. Activate the class.

Note:
Because the **`if_oo_adt_classrun~main`** method does not contain executable code, yet, there is nothing to test or debug at this point.

© Copyright. All rights reserved.

Unit 3
Solution 8

Define a Local Class

In this exercise, you define a local class inside of a global class.

Template:

none

Solution:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (global Class)

Task 1: Create a Global Class
In your own package, create a new ABAP class.

1. Create a new global class that implements interface *IF_OO_ADT_CLASSRUN* (suggested name: **`ZCL_##_LOCAL_CLASS`**, where ## stands for your group number).
   a) Choose *File* → *New* → *ABAP Class*.
   b) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_LOCAL_CLASS`**, where **`##`** is your group number. Enter a description.
   c) In the *Interfaces* group box, choose *Add*.
   d) Enter **`IF_OO_ADT_CLASSRUN`**. When the interface appears in the hit list, double-click it to add it to the class definition.
   e) Choose *Next*.
   f) Select your transport request and choose *Finish*.

Task 2: Define a Local Class
Define a local class **`lcl_connection`** and declare some public attributes.

1. Inside the global class, create a new local class **`lcl_connection`**. Use code completion to generate the code, but make sure you remove the `create private` addition from the generated code.
   a) Switch to the *Local Types* tab.
   b) Type **`lcl`** into the editor and press **`Ctrl + Space`**.
   c) Double-click *lcl - class* in the pop-up.
   d) While there is still a frame visible around **`lcl`** in the line `class lcl definition create private.`, complete the name of the class to **`lcl_connection`**.
   e) Delete the words `create private` at the end of the code row `class lcl_connection definition.`.
2. In local class **`lcl_connection`**, declare the following **public** attributes:

© Copyright. All rights reserved.

Table 1: Attributes

Attribute Name	Scope	Data Type
carrier_id	instance	/DMO/CARRIER_ID
connection_id	instance	/DMO/CONNECTION_ID
conn_counter	static	I

a) After line `PUBLIC SECTION.` and before line `PROTECTED SECTION.`, add the following code:

```
DATA carrier_id    TYPE /dmo/carrier_id.
DATA connection_id TYPE /dmo/connection_id.

CLASS-DATA conn_counter TYPE i.
```

3. Activate the class.

Note:
Because the **if_oo_adt_classrun~main** method does not contain executable code, yet, there is nothing to test or debug at this point.

a) Press **Ctrl + F3** to activate the class.

b) Compare your code on tab *Local Types* to the following extract from the model solution:

```
*"* use this source file for the definition and implementation of
*"* local helper classes, interface definitions and type
*"* declarations

CLASS lcl_connection DEFINITION.

  PUBLIC SECTION.

    DATA carrier_id    TYPE /dmo/carrier_id.
    DATA connection_id TYPE /dmo/connection_id.

    CLASS-DATA conn_counter TYPE i.

  PROTECTED SECTION.
  PRIVATE SECTION.

ENDCLASS.

CLASS lcl_connection IMPLEMENTATION.

ENDCLASS.
```

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Define a local class inside a global class.

© Copyright. All rights reserved.

Unit 3
Lesson 2

Creating Instances of a Class

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Create Instances of an ABAP Class.

Instance Creation

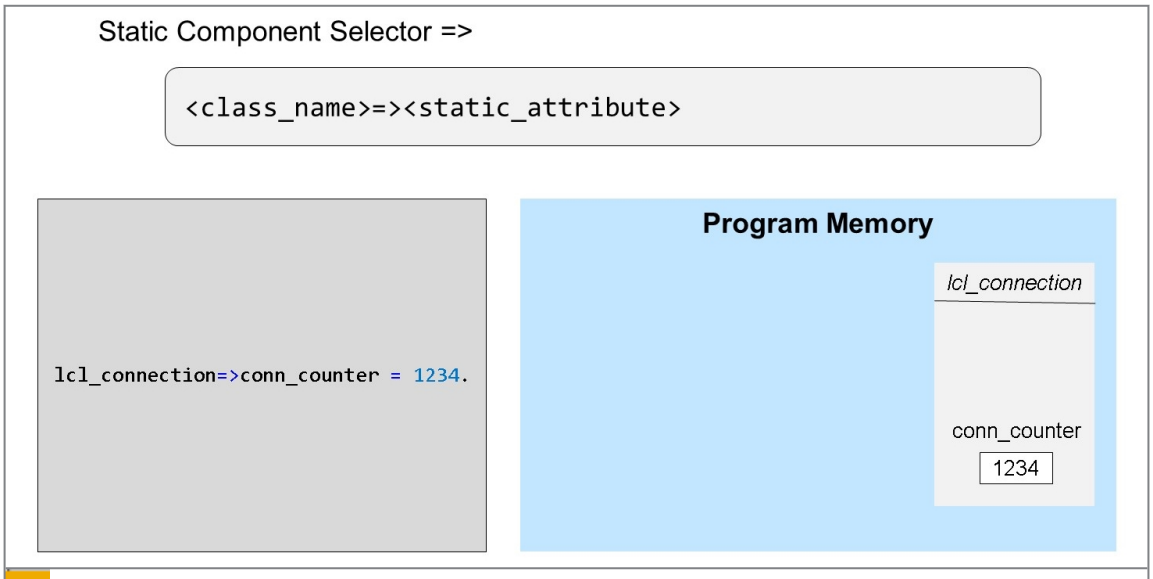


Figure 79: Access to Static Attributes

You work with attributes like normal variables of the same type. Outside the class, however, the attribute name is not sufficient to identify the attribute unambiguously. To address a static attribute outside the class, first type the class name, then the static component selector (=>), and only then the attribute name. The static component selector is a double arrow made up of an equals sign and the greater than sign.

Hint:
No blanks are allowed before or after the component selector.

© Copyright. All rights reserved.

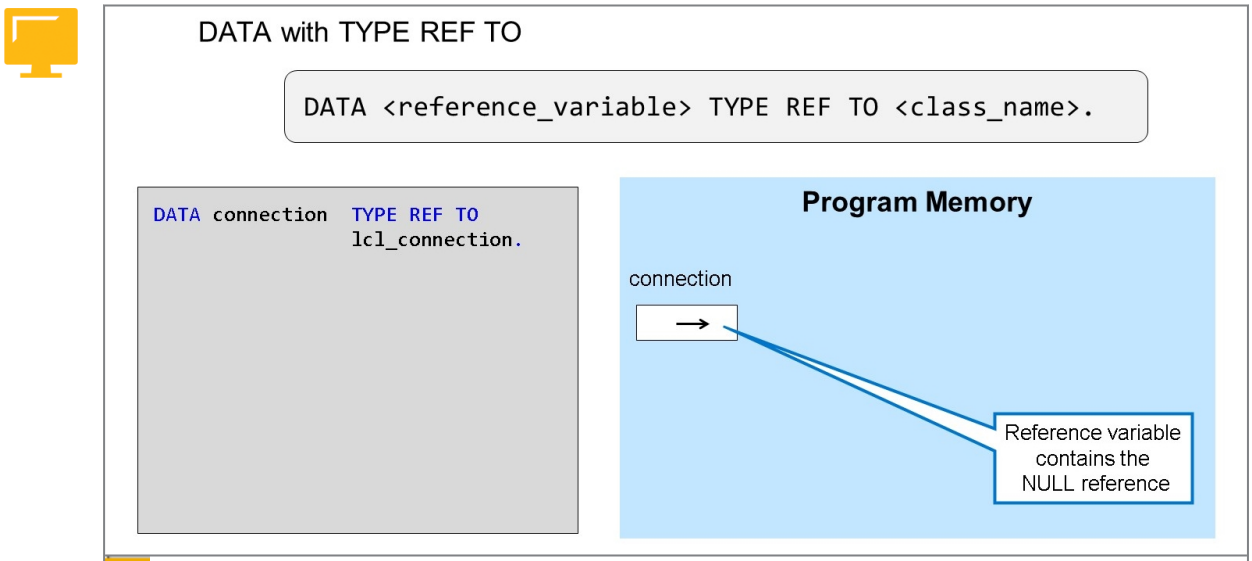


Figure 80: Reference Variables

For instance attributes the situation is even more complicated: In order to access an instance component, you need a reference variable.

A reference variable is a special kind of variable that you use to create, address, and manage an object. A reference variable is used to point at the instance of a class in the program memory. You declare reference variables using the DATA statement with the addition TYPE REF TO followed by the name of a class.

The initial value of a reference variable is called the NULL reference; the reference does not yet point anywhere.

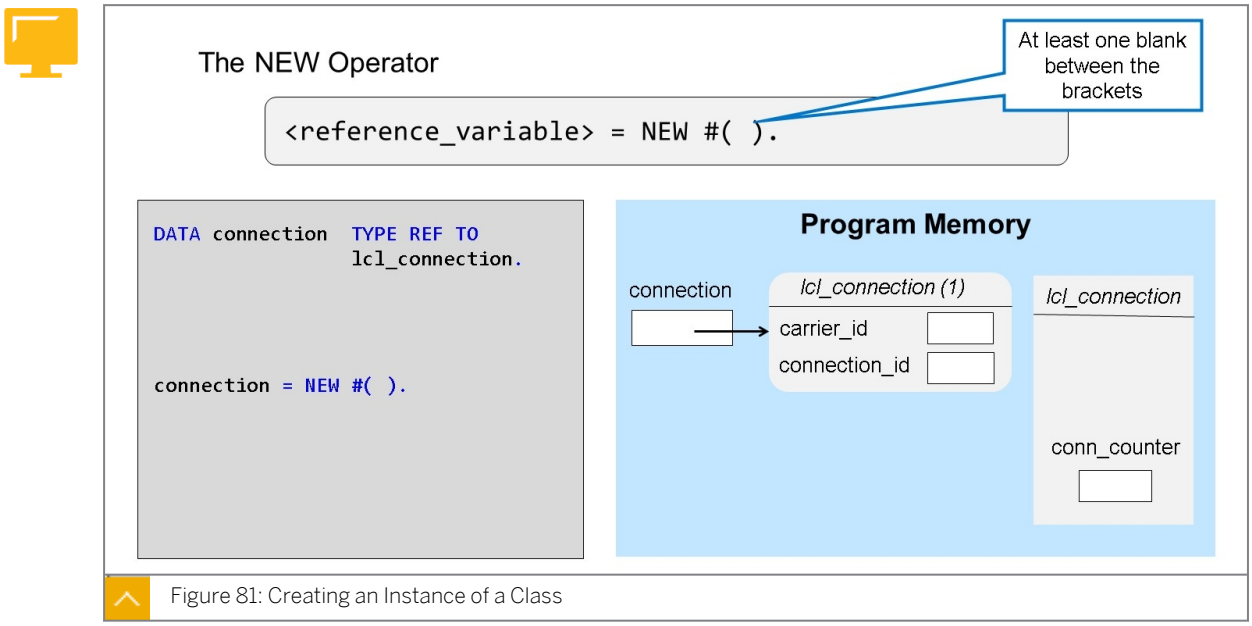


Figure 81: Creating an Instance of a Class

To create a new instance of a class, you use the NEW operator. The example above, uses a NEW #( ) expression on the right hand side of a value assignment. The result of the expression is the memory address of the newly created instance. This reference is then stored in the reference variable on the left-hand side of the assignment.

You may have noticed that the name of the class that you want to instantiate does not appear anywhere in the expression. However, from the location of the NEW #( ) expression, the

© Copyright. All rights reserved.

system already knows that the target variable connection has the type REF TO lcl_connection, and consequently it knows that it should create an instance of the class lcl_connection. The pound sign after the NEW operator means "use the type of the variable before the equals sign". (In more advanced scenarios, you can actually specify the name of the class in place of the pound sign).

Hint:
There must be at least one blank between the brackets.

When you address a class for the first time (which could be accessing a static component or creating an instance of the class), the runtime system also loads the class definition into the program memory. This class definition contains all of the static attributes, which only exist once in the class instead of once for each instance.

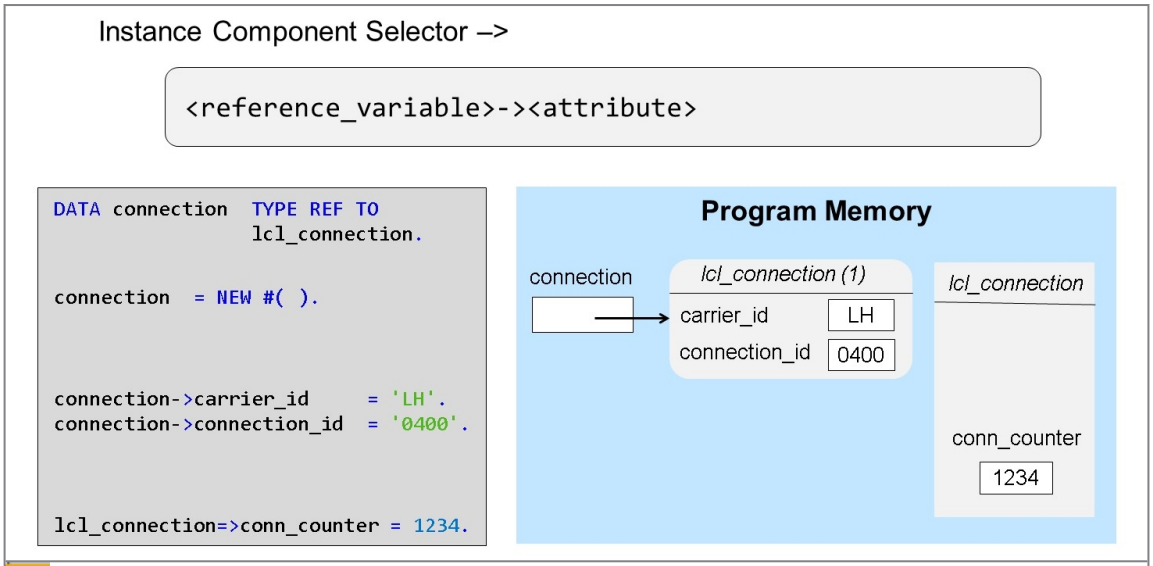


Figure 82: Access to Instance Attributes of a Class

You address static components using the class name and the static component selector. This does not work for instance components because you have to specify the instance you want to access.

To address an instance attribute outside the class, first type the reference variable, then the instance component selector (->), and only then the attribute name. The instance component selector is an arrow made up of a dash and the greater than sign.

Note:
Unlike many other programming languages, ABAP uses different characters for instance component selector and static component selector.

© Copyright. All rights reserved.

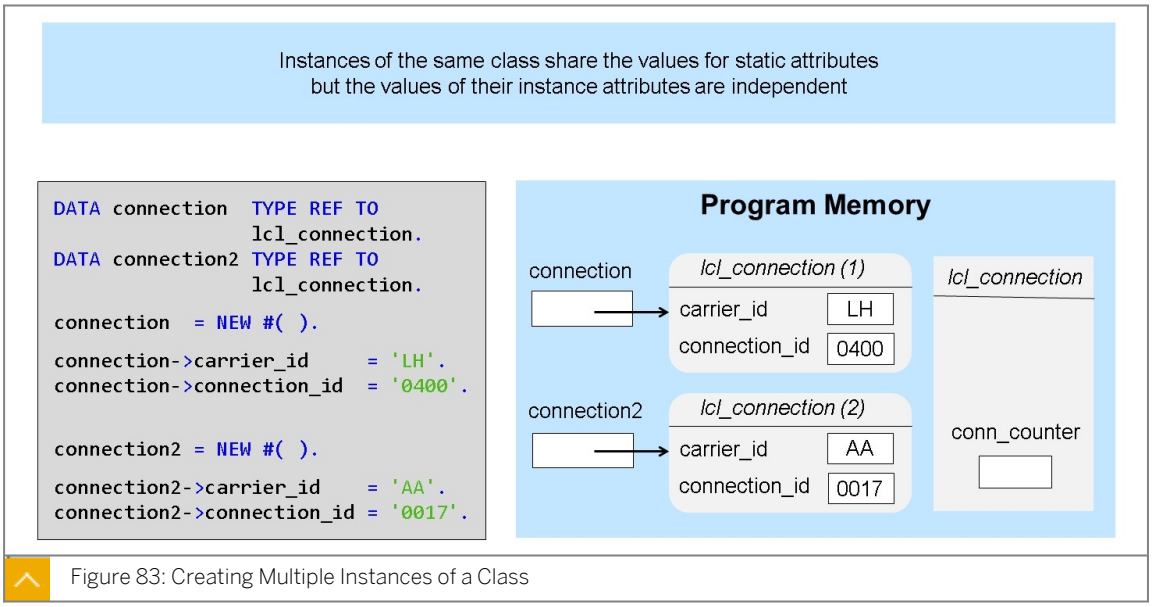


Figure 83: Creating Multiple Instances of a Class

One of the main characteristics of object oriented programming is the fact that you can create multiple instances of the same class. Each instance is created in a different place in the program memory and the values of instance attributes in one instance are independent from the values in other instances. But as the graphic illustrates, instances of the same class share the value for the static attributes.

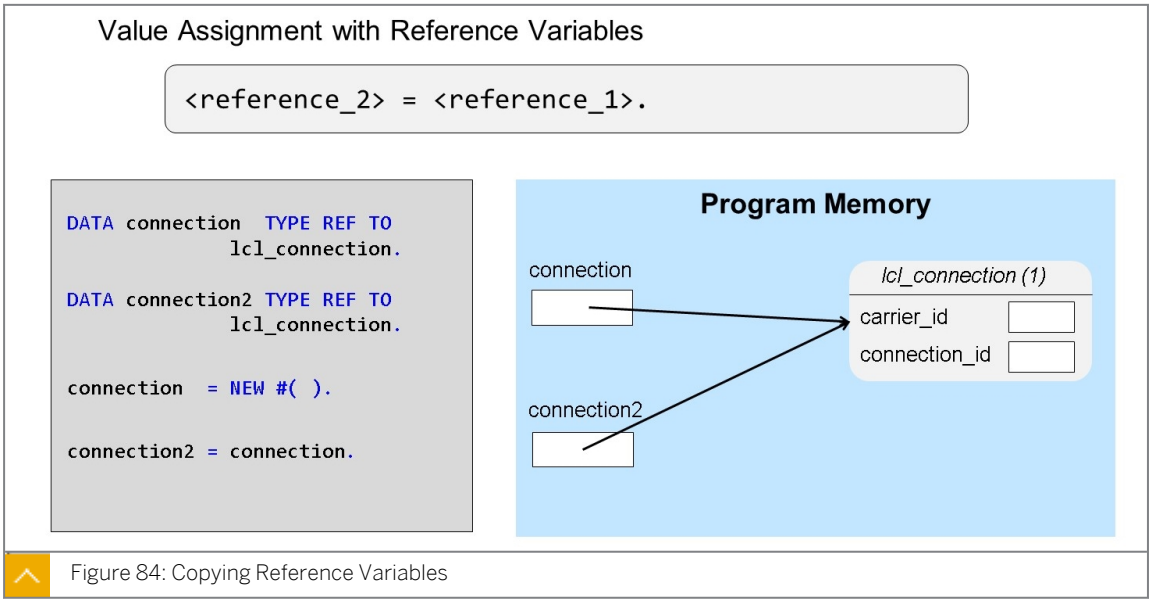


Figure 84: Copying Reference Variables

If you assign one reference variable to another, the system copies the address of the object to which the first variable is pointing into the second reference variable. The result of this is that you have two reference variables that point to the same object.

© Copyright. All rights reserved.

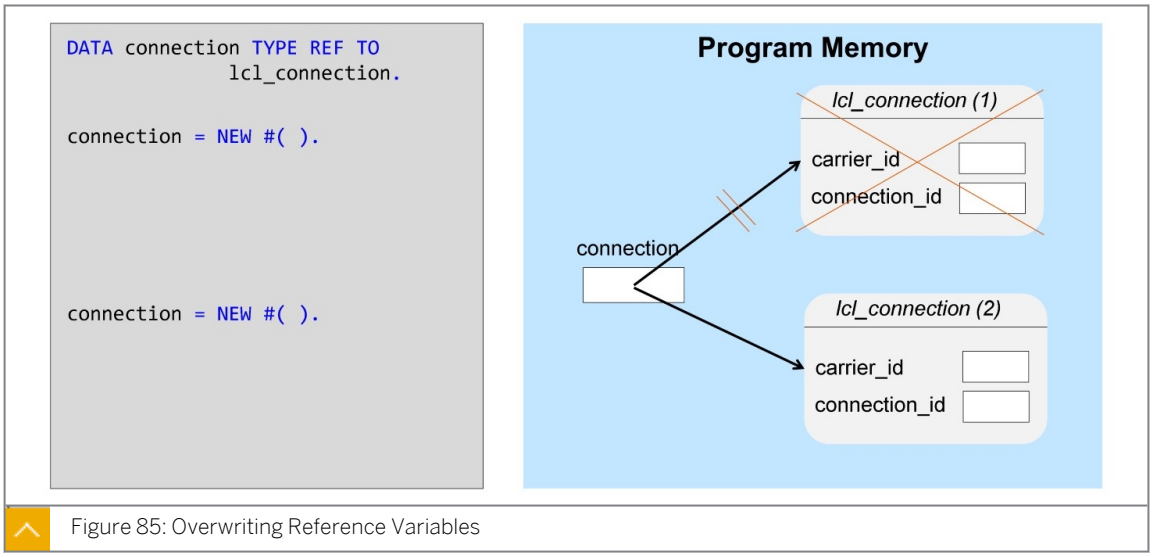


Figure 85: Overwriting Reference Variables

You can use the same reference variable to create more than one instance of a class. Each time you use the NEW #( ) expression, the system creates a new instance of the class and places the address of the new instance into the reference variable. However, the address of the new instance overwrites the address of the previous instance.

In the example above, the address of lcl_connection (2) overwrites the address of lcl_connection (1). Consequently, there is no longer a reference variable in the program pointing to lcl_connection (1). When this happens to an instance, it can no longer be addressed from the program.

To prevent the program memory from becoming filled with objects that can no longer be addressed and eventually overflowing, the runtime system has a component called the garbage collector. The garbage collector is a program that runs periodically to look for and destroy objects to which no more references point. If during a program you delete the last reference to an object by overwriting it or using the CLEAR statement, the garbage collector will destroy the object on its next pass.

Note:
At the end of a program, when all of the reference variables are freed, the garbage collector will destroy all of the instances to which they had pointed. You do not have to worry about resource management in the program yourself.

© Copyright. All rights reserved.

Instance Management in an Internal Table

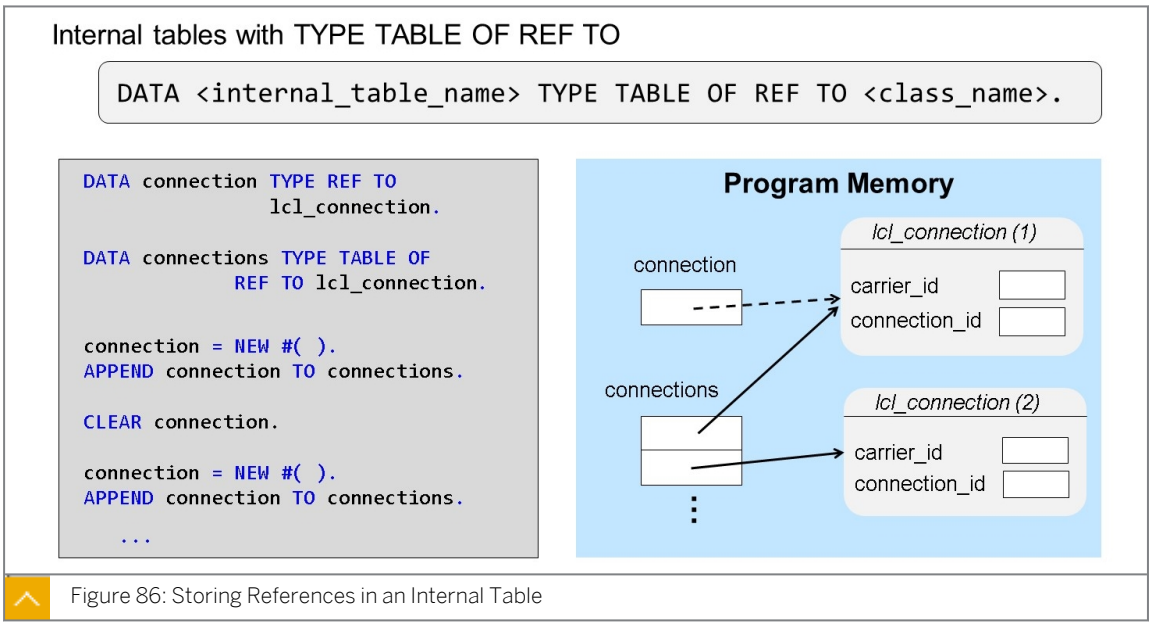


Figure 86: Storing References in an Internal Table

One way in which you can keep objects alive is to place the references into an internal table. This is a technique that you may well want to use if you are creating a whole series of objects. It enables you to use a single reference variable to create lots of objects. Although the reference variable is overwritten with the address of the next object, the existing objects are safe because the internal table contains a reference to them. You therefore never delete the "last" reference to the objects.

© Copyright. All rights reserved.

Unit 3
Exercise 9

Create and Manage Instances

In this exercise, you create and manage instances of your local class.

Template:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (global Class)

Solution:

/LRN/CL_S4D400_CLS_INSTANCES (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise you can skip this task and continue editing your class **`ZCL_##_LOCAL_CLASS`**.

1. Open the source code of global class **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** in the ABAP editor.
2. Link the *Project Explorer* view with the editor.
3. Copy class **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** to a class in your own package (suggested name: **`ZCL_##_INSTANCES`**, where ## stands for your group number).

Task 2: Create an Instance
In the **`main`** method of your global class, declare a reference variable and create an instance of your local class.

1. In method **`if_oo_adt_classrun~main`** of your global class, declare a reference variable (suggested name: **`connection`**) and type it with your local class **`lcl_connection`**.
2. Create an instance of the class and set the attributes **`carrier_id`** and **`connection_id`** (suggested values: *LH* for attribute **`carrier_id`** and *0400* for attribute **`connection_id`**).
3. Activate the class and use the debugger to analyze step by step what happens.

Task 3: Manage Several Instances
Create more instances of your local class and store the references to these instances in an internal table.

1. In method **`if_oo_adt_classrun~main( )`** of your global class, declare an internal table with the line type `REF TO lcl_connection` (suggested name: **`connections`**).
2. After instantiating the class and setting the attributes, append the object reference to the internal table.
3. Create two more instances of class **`lcl_connection`**, set their instance attributes to different values than in the first instance and append the references to the new instances to internal table **`connections`**. Use the same reference variable **`connection`** for all three instances.

© Copyright. All rights reserved.

4. Activate the class and use the debugger to analyze step by step what happens.

© Copyright. All rights reserved.

Unit 3
Solution 9

Create and Manage Instances

In this exercise, you create and manage instances of your local class.

Template:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (global Class)

Solution:

/LRN/CL_S4D400_CLS_INSTANCES (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise you can skip this task and continue editing your class **`ZCL_##_LOCAL_CLASS`**.

1. Open the source code of global class **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** in the ABAP editor.
   a) In the *Eclipse* toolbar, choose *Open ABAP Development Object*. Alternatively, press **`Ctrl + Shift + A`**.
   b) Enter **`/LRN/CL_S4D400_CLS`** as search string.
   c) From the list of development objects choose */LRN/CL_S4D400_CLS_LOCAL_CLASS*, then choose *OK*.
2. Link the *Project Explorer* view with the editor.
   a) In the *Project Explorer* toolbar, find the *Link with Editor* button. If it is not yet pressed, choose it. As a result, the development object, that is open in the editor, should be highlighted in the *Project Explorer*.
3. Copy class **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** to a class in your own package (suggested name: **`ZCL_##_INSTANCES`**, where ## stands for your group number).
   a) In the *Project Explorer* view, right-click class **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** to open the content menu.
   b) From the context menu, choose *Duplicate ...*.
   c) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_INSTANCES`**, where ## stands for your group number.
   d) Adjust the description and choose *Next*.
   e) Confirm the transport request and choose *Finish*.

Task 2: Create an Instance
In the **`main`** method of your global class, declare a reference variable and create an instance of your local class.

© Copyright. All rights reserved.

1. In method **if_oo_adt_classrun~main** of your global class, declare a reference variable (suggested name: **connection**) and type it with your local class **lcl_connection**.
   a) Switch to the *Global Class* tab.
   b) Adjust the code as follows:

```
METHOD if_oo_adt_classrun~main.

  DATA connection TYPE REF TO lcl_connection.

ENDMETHOD.
```

2. Create an instance of the class and set the attributes **carrier_id** and **connection_id** (suggested values: *LH* for attribute **carrier_id** and *0400* for attribute **connection_id**).
   a) Adjust the code as follows:

```
DATA connection TYPE REF TO lcl_connection.

connection = new #( ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.
```

Hint:
After typing the component selector (->), press **Strg + Space** to choose the attribute names from a suggestion list.

3. Activate the class and use the debugger to analyze step by step what happens.
   a) Press **Ctrl + F3** to activate the class.
   b) Double-click the left-hand margin of the editor next to the line `connection = new #( ).` to set a break point.
   c) Press **F9** to run the class.
   d) Double-click the word **connection** to display the content of reference variable **connection**.
   e) Press **F5** to go one step further in the debugger. Check that the value of **connection** has changed.
   f) In the *Variables* view, expand the branch **connection** to display the initial attributes of the class.
   g) Press **F5** to go one step further in the debugger. Check that the value of **carrier_id** has changed.
   h) Press *F5* again. Check that the value of **Connection_ID** has changed.

Task 3: Manage Several Instances

Create more instances of your local class and store the references to these instances in an internal table.

© Copyright. All rights reserved.

1. In method **`if_oo_adt_classrun~main( )`** of your global class, declare an internal table with the line type `REF TO lcl_connection` (suggested name: **`connections`**).
   a) Adjust the code as follows:

```
DATA connection TYPE REF TO lcl_connection.
DATA connections TYPE TABLE OF REF TO lcl_connection.

connection = new #( ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.
```

2. After instantiating the class and setting the attributes, append the object reference to the internal table.
   a) Adjust the code as follows:

```
connection = NEW #( ).

connection->carrier_id = 'LH'.
connection->connection_id = '0400'.

APPEND connection TO connections.
```

3. Create two more instances of class **`lcl_connection`**, set their instance attributes to different values than in the first instance and append the references to the new instances to internal table **`connections`**. Use the same reference variable **`connection`** for all three instances.
   a) Adjust the code as follows:

```
connection = NEW #(  ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.

APPEND connection TO connections.

connection = NEW #(  ).

connection->carrier_id    = 'AA'.
connection->connection_id = '0017'.

APPEND connection TO connections.

connection = NEW #(  ).

connection->carrier_id    = 'SQ'.
connection->connection_id = '0001'.

APPEND connection TO connections.
```

4. Activate the class and use the debugger to analyze step by step what happens.
   a) Press **`Ctrl + F3`** to activate the class.
   b) Double-click the left-hand margin of the editor next to the first line `connection = new #( )`. to remove the break point.

© Copyright. All rights reserved.

c) Double-click the left-hand margin of the editor next to the first line `APPEND connection TO connections.` to set a new break point.

d) Press **F9** to run the class.

e) Double-click the word **connection** to display the content of reference variable **connection**.

f) Double-click the word **connections** to display the content of internal table **connections**.

g) Press **F5** to go one step further in the debugger. Check that the content of **connections** has changed.

h) In the *Variables* view, expand the branch **connections** to display content of the internal table.

i) Press **F5** several times until you reach the end of the program. While you do so, check the content of the internal table.

j) Compare your code on tab *Global Class* to the following extract from the model solution:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************

    connection = NEW #(  ).

    connection->carrier_id    = 'LH'.
    connection->connection_id = '0400'.

    APPEND connection TO connections.

* Second Instance
**********************************************************************
    connection = NEW #(  ).

    connection->carrier_id    = 'AA'.
    connection->connection_id = '0017'.

    APPEND connection TO connections.

* Third Instance
**********************************************************************
    connection = NEW #(  ).

    connection->carrier_id    = 'SQ'.
    connection->connection_id = '0001'.

    APPEND connection TO connections.

  ENDMETHOD.
```


© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Create Instances of an ABAP Class.

© Copyright. All rights reserved.

Unit 3
Lesson 3

Defining and Calling Methods

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Define and call methods.

Method Definition

```
CLASS <class_name> DEFINITION.
   <any> SECTION.

     METHODS <method_name>
         IMPORTING       <input_1>   TYPE <type>
                         <input_2>   TYPE <type> DEFAULT <val>
                          ...
         EXPORTING       <output_1>  TYPE <type>
                          ...
         CHANGING        <inout_1>   TYPE <type> OPTIONAL
...
          RETURNING VALUE(<result>)   TYPE <type>
          RAISING        <exception1>
                         <exception2>
                           ...     .
ENDCLASS.
```

Figure 87: Method Definition

In the definition part of a class you use METHODS to define an instance method and CLASS-METHODS to define a static method. The name of the method is followed by the method's signature; that is, the set of values that the method exchanges with its caller and the exceptions that may arise during the method.

The signature of a method consists of parameters and exceptions. Each parameter has a name and a type.

ABAP knows the following kinds of parameters:

Importing Parameters

Importing parameters are values that the method receives from the caller. A method can have any number of importing parameters.

By default, importing parameters are mandatory, but there are two ways to make them optional:

© Copyright. All rights reserved.

- Using the OPTIONAL addition. The parameter is optional and its default value is the initial value appropriate to the type of the parameter
- Using the DEFAULT <val> addition. The parameter is optional and its default value is the value that you specified as <val>.

Inside of a method, you may not change importing parameters. If you attempt to do so, you will cause a syntax error.

Exporting Parameters

Exporting parameters are results that are returned by the method. A method can have any number of exporting parameters. All exporting parameters are optional - a calling program only uses the values that it actually needs.

Changing Parameters

Changing parameters are values that the method receives from the caller. Unlike importing parameters, the method can change the values of these parameters. They are then returned to the caller under the same name. A method can have any number of changing parameters. Changing parameters are mandatory by default; you can make them optional in the same way as importing parameters.

Returning Parameters

A returning parameter is a method result that can be used directly in an expression. A method can only have one returning parameter. Returning parameters have to use a special form of parameter passing which is called pass-by-value. This form of parameter passing is defined by surrounding the parameter name in brackets (no blanks!) and preceding it with keyword VALUE.

Keyword RAISING is used to list the exceptions the method might raise to indicate an error situation. The calling program can then react to the error.

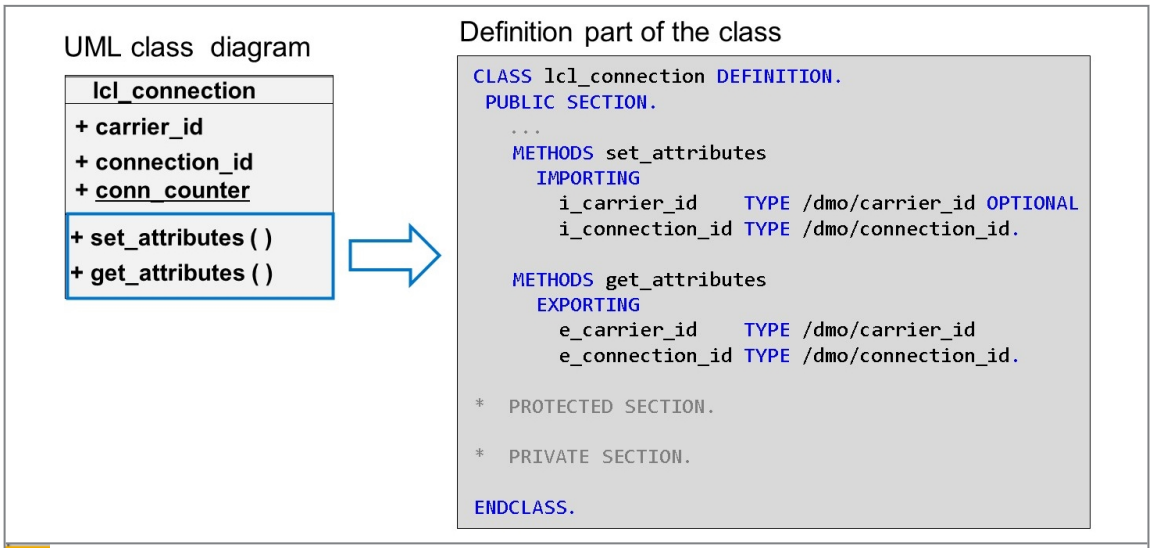


Figure 88: Example: Methods of the Flight Connection Class

As an example, add two methods to our class lcl_connection.

A `set_attributes( )` method to set the values for attribute `carrier_id` and `connection_id` and a `get_attributes( )` method to return the values of these attributes. Where `set_attributes( )` needs two importing parameters, one for each attribute, the `get_attributes( )` method needs two exporting parameters.

© Copyright. All rights reserved.

For every method that you define, you must also create an implementation in the implementation part of the class. Until you do so, you see syntax errors in your class; the syntax check tells you that the implementation of the method is missing.

Hint:
ADT offers a Quick Fix to add the missing implementation. To use this Quick Fix, proceed as follows:

1. Position the cursor in a METHODS statement with an error and press *Ctrl + 1*.
2. From the list of possible Quick Fixes choose *Add implementation for ...* and press *Enter* . If the implementation is missing for several methods, the Quick Fix title reads *Add ... unimplemented methods*.

Method Implementation

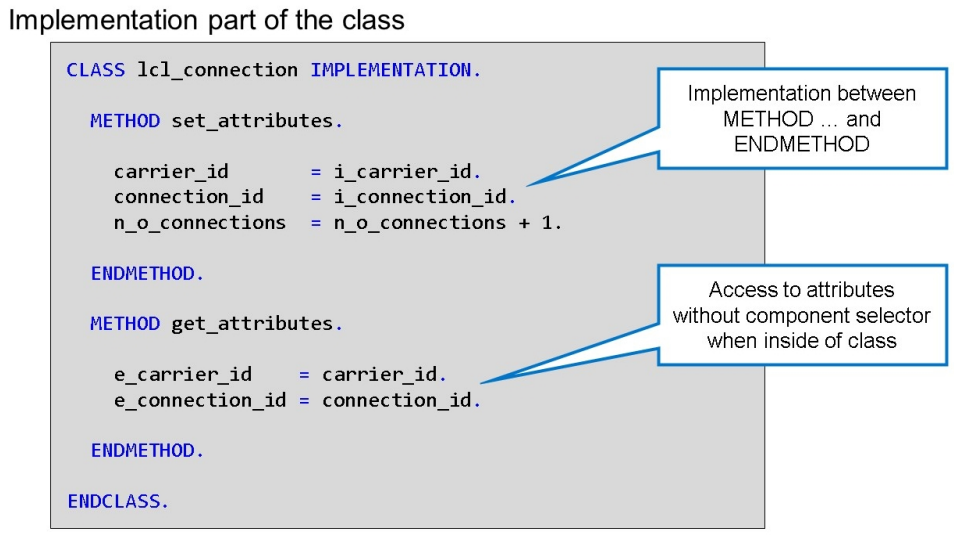


Figure 89: Implementing Methods

You must implement every method that you define. You do this in the implementation part of the class, by adding a pair of statements METHOD <method_name> and ENDMETHOD.

The method implementation contains ABAP statements that can access the parameters of the method (you are not allowed to change importing parameters) and all types, attributes, and constants that you declared in the class, independent from their visibility. Instance methods may access both instance attributes and static attributes. Static methods may only access static components.

Inside the implementation part of a class, you can access the attributes of that class without a reference variable and '->' (or the class name and '=>' , in case of static attributes).

© Copyright. All rights reserved.

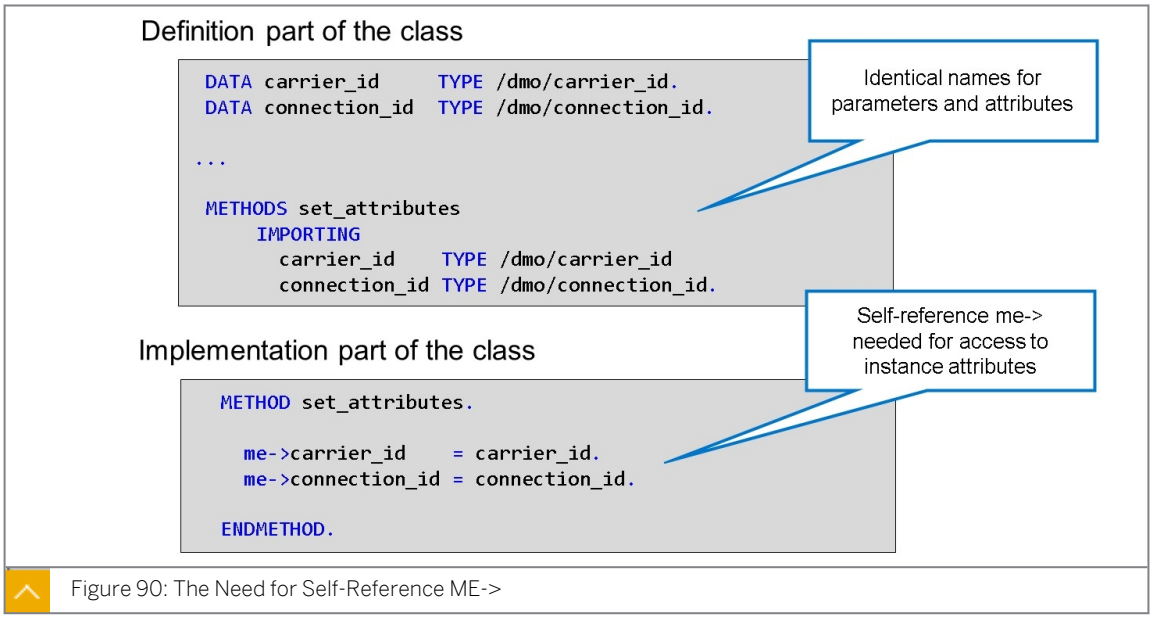


Figure 90: The Need for Self-Reference ME->

Only in the implementation of instance methods, ABAP offers built-in variable ME. ME is a reference variable, typed with the current class and filled at runtime with the address of the current instance. The use of ME is optional and should be avoided, unless the name of an attribute collides with the name of another data object, for example a parameter name.

In the example, the import parameters of method set_attributes( ) have the same names as the corresponding attributes. In such a situation, the name by itself denotes the parameter. Only by putting me-> in front it becomes clear that an attribute of the current instance is to be accessed.

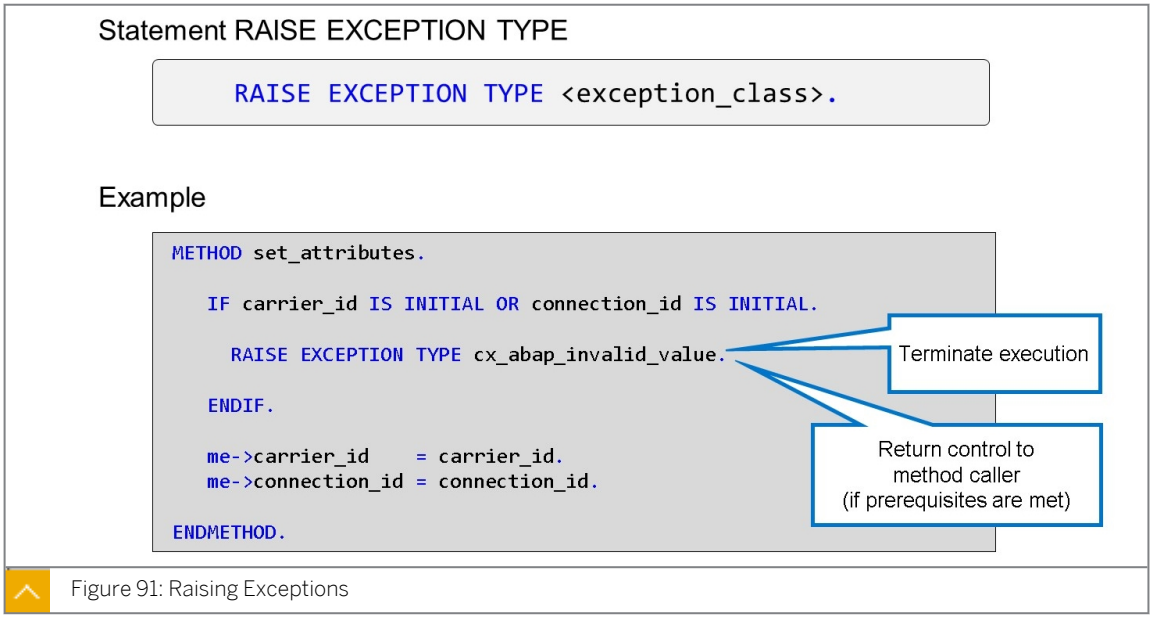


Figure 91: Raising Exceptions

If an error occurs during the execution of a method, the method can trigger an exception using statement RAISE EXCEPTION TYPE, followed by the name of the exception.

© Copyright. All rights reserved.

Note:
Technically, exception names are the names of special ABAP classes and statement RAISE EXCEPTION TYPE creates an instance of the referenced class. The instance is referred to as exception object and the special kind of ABAP classes are called exception classes.

The moment an exception is raised, the method execution is terminated.

Control is returned to the calling program if all prerequisites are met. Otherwise the entire program terminates with a runtime error.

The prerequisites for continuing the program are as follows:

- The raised exception is declared in the RAISING clause of the method signature
- The method call is surrounded by a TRY ... ENDTRY control structure
- The TRY ... ENDTRY control structure contains a CATCH block for the exception

Note:
For some exceptions, the ABAP editor displays a syntax warning with a quick fix when you raise an exception that is not yet declared in the RAISING clause of the method.

Method Calls

Instance Methods

```
<reference_variable>->method_name>(
          EXPORTING    <input_1>   = <expression>
                       <input_2>   = <expression>
                       ...
          IMPORTING    <output_1> = <variable>
                       ...
          CHANGING     <inout_1>  = <variable>
                       ...
                                  ).
```

Static Methods

```
<class_name>=><static_method>(        ...          ).
```

Figure 92: Calling Methods

You call an instance method using a reference variable and the instance component selector (->). The component selector is followed by the name of the method you want to call. For static methods you use the class name and the static component selector (=>). On both cases, the parameter passing takes place in a pair of brackets. The brackets are needed in any case. They remain empty if no parameters are to be passed.

© Copyright. All rights reserved.

Note:
No blank is allowed between the method name and the opening bracket. On the other hand you need at least one blank after the opening bracket and before the closing bracket.

Importing parameters are listed after keyword EXPORTING because the values that are imported by the called method are exported by the calling program. The parameter names are listed on the left-hand side of an equals sign (=) and an expression on the right-hand side.. This expression can be as simple as a literal, constant, or variable but any other expressions are also allowed as long as the type of the expression matches the type of the parameter.

Exporting parameters are listed after keyword IMPORTING. Note, that for exporting parameter the parameter name is also listed on the left-hand side of the equals sign. The variable on the right-hand side has to have the same type as the parameter.

Changing parameters are listed after keyword CHANGING. The type of the variable on the right-hand side has to match the type of the parameter on the left hand side.

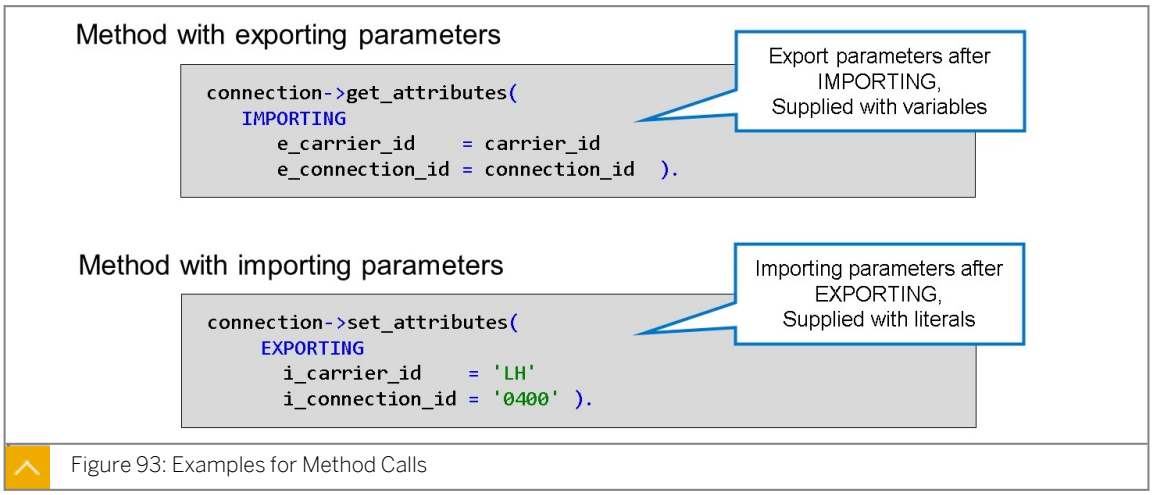


Figure 93: Examples for Method Calls

The example illustrates the call of instance methods `set_attributes( )` and `get_attributes( )`. The parameter names are always on the left-hand side of the equals sign (=).

Hint:
You can use code completion in ADT to generate the method call including the parameter passing. To do so, proceed as follows:

1. Type in the reference variable (or class name) and the component selector.
2. Once you typed the component selector press **`Ctrl + Space`** to display a list of components you can address.
3. Choose a method, and press **`Shift + Enter`** to insert the name of the method and its full signature into your code. Optional parameters are listed in comment lines.

© Copyright. All rights reserved.

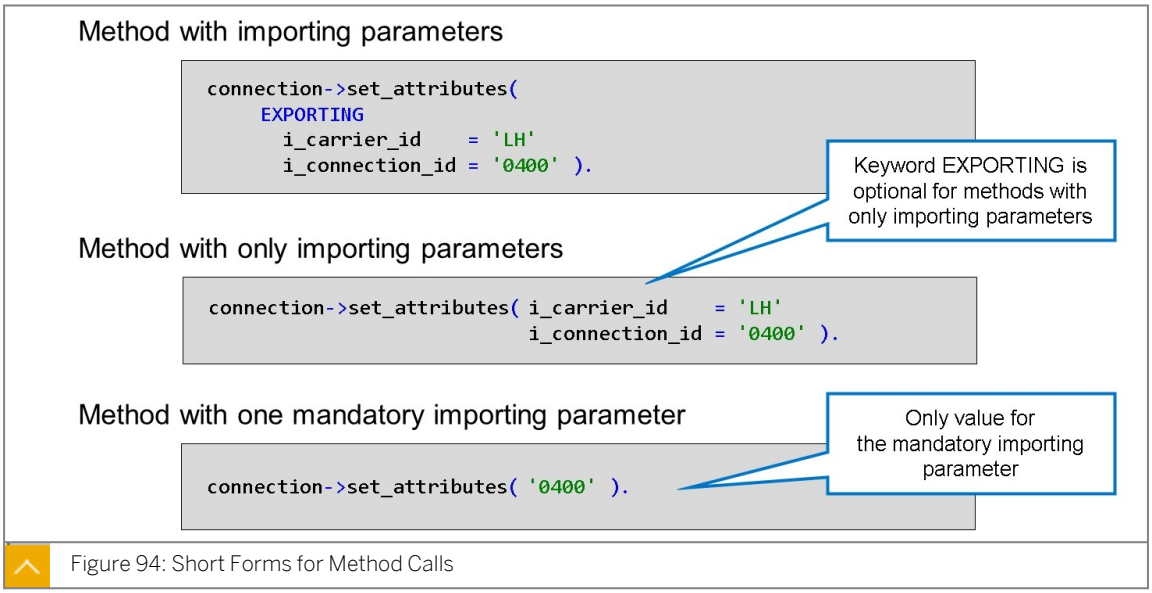


Figure 94: Short Forms for Method Calls

When calling a method, keyword IMPORTING is always needed to receive the value of exporting parameters. But keyword EXPORTING becomes optional, if you only want to supply importing parameters.

For methods with only one mandatory importing parameter, you can even omit the explicit parameter assignment. In the example, parameter i_carrier_id is optional with default value 'LH'. Therefore the value between the brackets is assigned to the remaining mandatory importing parameter i_connection_id.

Exception Handling

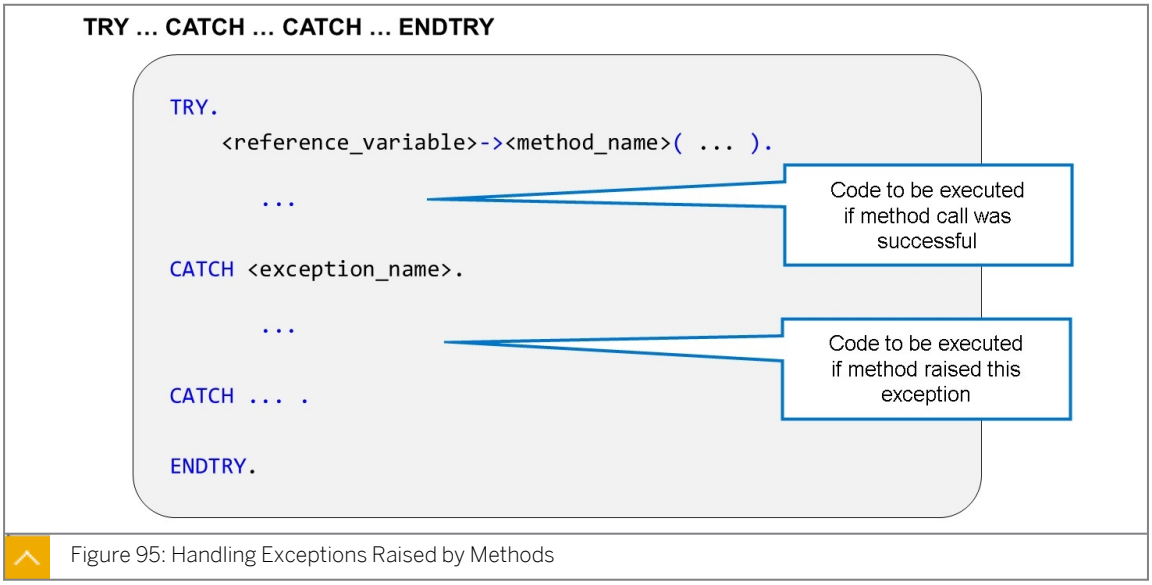


Figure 95: Handling Exceptions Raised by Methods

In a previous section of this course, you learned how to handle runtime errors with TRY ... ENDTRY control structures.

Exceptions declared in the RAISING clause of a method definition are handled in exactly the same way:

- Place the method call in the TRY block of the TRY ... ENDTRY control structure.

© Copyright. All rights reserved.

- Define a CATCH block with the name of the exception you want to handle.

> Note:
> If there is code that you want to skip in case the method call failed, you can place this code inside the TRY block, after the method call.

Functional Methods

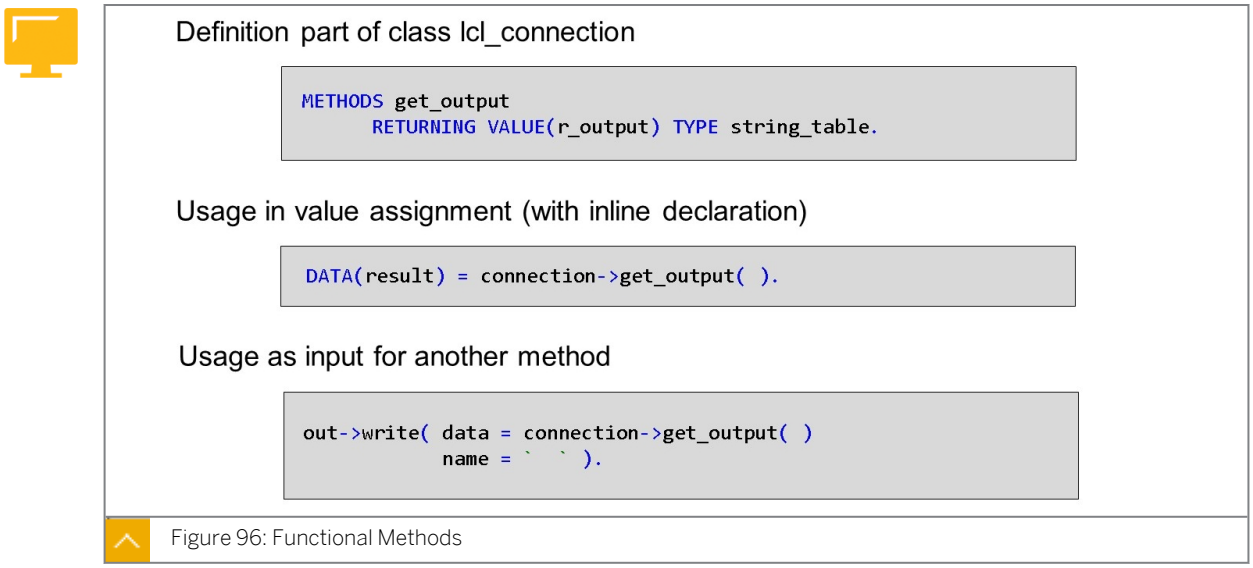


Figure 96: Functional Methods

Methods that have a returning parameter are called functional methods. It is not possible to define more than one returning parameter for the same method. It is mandatory that you define the returning parameter in the form VALUE(<parameter_name>). Besides the returning parameter a functional method can have any combination of other parameters. It is most common however, to add importing parameters only. The reason is, that with additional exporting or changing parameters you loose the biggest advantage of functional methods, namely, that you can use the result of a functional method directly in other ABAP expressions.

The figure shows two examples of how you can use functional methods in expressions; the first is a simple example in which the returning parameter is assigned directly to a variable. The second example shows how the call of the functional method is used as input for another method. The system executes the functional method, and then uses the value of the returning parameter as input for method `write( )`.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 3
Exercise 10

Define and Call Methods

In this exercise, you will define and implement methods in your local class lcl_connection. You will then call these methods from the `main( )` method of your global class.

Template:

/LRN/CL_S4D400_CLS_INSTANCES (global Class)

Solution:

/LRN/CL_S4D400_CLS_METHODS (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_LOCAL_CLASS`** or your class **`ZCL_##_INSTANCES`**.

1. Open the source code of global class **`/LRN/CL_S4D400_CLS_INSTANCES`** in the ABAP editor.
2. Link the *Project Explorer* view with the editor.
3. Copy class **`/LRN/CL_S4D400_CLS_INSTANCES`** to a class in your own package (suggested name: **`ZCL_##_METHODS`**, where ## stands for your group number).

Task 2: Define Methods
In your local class **`lcl_connection`**, define instance methods **`get_output`** and **`set_attributes`**.

1. Navigate to the definition of the local class **`lcl_connection`**.
2. Add a method **`get_output`** to the public section of the class. It should have one returning parameter **`r_output`** of global table type **`STRING_TABLE`**.

Note:
Remember to surround the name of the returning parameter with `VALUE( )`.

3. Add a method **`set_attributes`** to the public section of the class. It should have one importing parameter for each instance attribute of the class. Use the same types for the parameters that you used to type the attributes. To distinguish the parameters from the attributes, add prefix **`i_`** to the parameter names. In addition, the method should raise exception **`CX_ABAP_INVALID_VALUE`**.

Task 3: Implement Methods

1. Use a quick fix to add the method implementations to the class.

© Copyright. All rights reserved.

2. Implement the method **get_output**. Append some string templates to returning parameter **r_output**. In the string templates, use embedded expressions to add the values of attributes **carrier_id** and **connection_id**.
3. Implement the method **set_attributes**. If either of the two importing parameters is empty (`IS INITIAL`), raise exception **CX_ABAP_INVALID_VALUE**. Otherwise, fill the two instance attributes with the values of the importing parameters.
4. Activate your class.

Task 4: Call a Functional Method

In the **main** method of your global class, call the functional method **get_output**.

1. Switch to the implementation of method **if_oo_adt_classrun~main** in the global class.
2. At the end of the method, add a loop over the internal table **connections** with reference variable **connection** as a work area.
3. In the loop, call functional method **get_output** for each instance in turn and write the result to the console.

Hint:
You can use the call of method **get_output** directly as input for **out->write( ).**

4. Activate the class. Execute it as a console app and analyze the console output.

Task 5: Use Code Completion and Handle Exceptions

Use code completion to call the method **set_attributes** and handle the exception the method raises.

1. For the first instance of class **lcl_connection**, replace the direct access to attributes **carrier_id** and **connection_id** with a call of the instance method **set_attributes( )**. Use code completion to insert the full signature of the method.

Hint:
Press **Ctrl + Space** after you typed the component select (->) to display a list of available attributes and methods. Choose the method and then press **Shift + Enter** to insert the full signature of the method.

2. Handle the exception. Surround the method call with `TRY.` and `ENDTRY.`. Add a CATCH-Block to handle exception **CX_ABAP_INVALID_VALUE**. Make sure the new instance is only added to internal table **connections** if the method call was successful.
3. Repeat the previous step for the other two instances of class **lcl_connection**.
4. Activate the class. Execute it and debug the method calls.

© Copyright. All rights reserved.

Unit 3
Solution 10

Define and Call Methods

In this exercise, you will define and implement methods in your local class lcl_connection. You will then call these methods from the `main( )` method of your global class.

Template:

/LRN/CL_S4D400_CLS_INSTANCES (global Class)

Solution:

/LRN/CL_S4D400_CLS_METHODS (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_LOCAL_CLASS`** or your class **`ZCL_##_INSTANCES`**.

1. Open the source code of global class **`/LRN/CL_S4D400_CLS_INSTANCES`** in the ABAP editor.
   a) In the *Eclipse* toolbar, choose *Open ABAP Development Object*. Alternatively, press **`Ctrl + Shift + A`**.
   b) Enter **`/LRN/CL_S4D400_CLS`** as search string.
   c) From the list of development objects choose */LRN/CL_S4D400_CLS_INSTANCES*, then choose *OK*.
2. Link the *Project Explorer* view with the editor.
   a) In the *Project Explorer* toolbar, find the *Link with Editor* button. If it is not yet pressed, choose it. As a result, the development object, that is open in the editor, should be highlighted in the *Project Explorer*.
3. Copy class **`/LRN/CL_S4D400_CLS_INSTANCES`** to a class in your own package (suggested name: **`ZCL_##_METHODS`**, where ## stands for your group number).
   a) In the *Project Explorer* view, right-click class **`/LRN/CL_S4D400_CLS_INSTANCES`** to open the content menu.
   b) From the context menu, choose *Duplicate ...*.
   c) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_METHODS`**, where ## stands for your group number.
   d) Adjust the description and choose *Next*.
   e) Confirm the transport request and choose *Finish*.

Task 2: Define Methods
In your local class **`lcl_connection`**, define instance methods **`get_output`** and **`set_attributes`**.

© Copyright. All rights reserved.

1. Navigate to the definition of the local class **`lcl_connection`**.
   a) In the global class, choose *Local Types*.
2. Add a method **`get_output`** to the public section of the class. It should have one returning parameter **`r_output`** of global table type **`STRING_TABLE`**.

Note:
Remember to surround the name of the returning parameter with `VALUE( )`.

   a) Adjust the code as follows:

```
PUBLIC SECTION.

   DATA carrier_id    TYPE /dmo/carrier_id.
   DATA connection_id TYPE /DMO/Connection_id.

   CLASS-DATA conn_counter TYPE i.

   METHODS get_output
     returning
       value(r_output) type string_table.

 PROTECTED SECTION.
```

Note:
For the moment, ignore the syntax error *Implementation missing for method "GET_OUTPUT"*.

3. Add a method **`set_attributes`** to the public section of the class. It should have one importing parameter for each instance attribute of the class. Use the same types for the parameters that you used to type the attributes. To distinguish the parameters from the attributes, add prefix **`i_`** to the parameter names. In addition, the method should raise exception **`CX_ABAP_INVALID_VALUE`**.
   a) Adjust the code as follows:

```
PUBLIC SECTION.

   DATA carrier_id    TYPE /dmo/carrier_id.
   DATA connection_id TYPE /DMO/Connection_id.

   CLASS-DATA conn_counter TYPE i.

   METHODS set_attributes
     IMPORTING
       i_carrier_id    TYPE /dmo/carrier_id
       i_connection_id TYPE /dmo/connection_id
     RAISING
       cx_abap_invalid_value.

   METHODS get_output
     returning
       value(r_output) type string_table.
```


© Copyright. All rights reserved.

```
  PROTECTED SECTION.
```

Note:
For the moment, ignore the syntax error *Implementation missing for method "SET_ATTRIBUTES"*.

Task 3: Implement Methods

1. Use a quick fix to add the method implementations to the class.
   a) Position the cursor on the name of one of the methods in the editor and press **Ctrl + 1**.
   b) Double-click the suggestion *Add 2 unimplemented methods*.
2. Implement the method **get_output**. Append some string templates to returning parameter **r_output**. In the string templates, use embedded expressions to add the values of attributes **carrier_id** and **connection_id**.
   a) In the local class add the following code between the statements `METHOD get_output.` and `ENDMETHOD.`

```
    APPEND |------------------------------| TO r_output.
    APPEND |Carrier:     { carrier_id    }| TO r_output.
    APPEND |Connection:  { connection_id }| TO r_output.
```

3. Implement the method **set_attributes**. If either of the two importing parameters is empty (`IS INITIAL`), raise exception **CX_ABAP_INVALID_VALUE**. Otherwise, fill the two instance attributes with the values of the importing parameters.
   a) Adjust the code as follows:

```
METHOD set_attributes.

  IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  carrier_id    = i_carrier_id.
  connection_id = i_connection_id.

ENDMETHOD.
```

4. Activate your class.
   a) Choose **Ctrl + F3** to activate the class.

Task 4: Call a Functional Method

In the **main** method of your global class, call the functional method **get_output**.

1. Switch to the implementation of method **if_oo_adt_classrun~main** in the global class.
   a) In the global class, choose *Global Class* scroll down to the implementation of method **if_oo_adt_classrun~main**.
2. At the end of the method, add a loop over the internal table **connections** with reference variable **connection** as a work area.

© Copyright. All rights reserved.

a) Add the following code before `ENDMETHOD.`.

```
LOOP AT connections INTO connection.

ENDLOOP.
```

3. In the loop, call functional method **`get_output`** for each instance in turn and write the result to the console.

Hint:
You can use the call of method **`get_output`** directly as input for **`out->write( ).`**

a) Adjust the code as follows:

```
LOOP AT connections INTO connection.

   out->write( connection->get_output( ) ).

ENDLOOP.
```

4. Activate the class. Execute it as a console app and analyze the console output.
   a) Press **`Ctrl + F3`** to activate the class.
   b) Press **`F9`** to run the class.
   c) Analyze the output on the *Console* window.

Task 5: Use Code Completion and Handle Exceptions

Use code completion to call the method **`set_attributes`** and handle the exception the method raises.

1. For the first instance of class **`lcl_connection`**, replace the direct access to attributes **`carrier_id`** and **`connection_id`** with a call of the instance method **`set_attributes( )`**. Use code completion to insert the full signature of the method.

Hint:
Press **`Ctrl + Space`** after you typed the component select (->) to display a list of available attributes and methods. Choose the method and then press **`Shift + Enter`** to insert the full signature of the method.

   a) Place the cursor in the next line after the first `connection = NEW #( ).`.
   b) Type `connection->` and press **`Ctrl + Space`**.
   c) Choose *set_attributes* and press **`Shift + Enter`**.
   d) Pass values to the importing parameters. Use the same literals that you used previously to set the attributes of this instance.
   e) Remove or comment the direct access to the instance attributes for this instance.

2. Handle the exception. Surround the method call with `TRY.` and `ENDTRY.`. Add a CATCH-Block to handle exception **`CX_ABAP_INVALID_VALUE`**. Make sure the new instance is only added to internal table **`connections`** if the method call was successful.

© Copyright. All rights reserved.

a) Uncomment the generated code line `CATCH cx_abap_invalid_value..`

b) Add `TRY.` before the method call .

c) Add `ENDTRY.` after the CATCH statement.

d) Between the CATCH statement and the ENDTRY statement add `out->write( `Method call failed` ).`

e) Move the APPEND statement up to between the method call and CATCH statement.

f) The complete method call should now look like this:

```
    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'LH'
            i_connection_id = '0400'
        ).

*       connection->carrier_id    = 'LH'.
*       connection->connection_id = '0400'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.
```

3. Repeat the previous step for the other two instances of class **`lcl_connection`**.

a) After this step, the implementation of method **`if_oo_adt_classrun~main`** should look similar to this:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************
    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'LH'
            i_connection_id = '0400'
        ).

*        connection->carrier_id    = 'LH'.
*        connection->connection_id = '0400'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Second instance
**********************************************************************
```

© Copyright. All rights reserved.

```
    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'AA'
            i_connection_id = '0017'
        ).

*        connection->carrier_id    = 'AA'.
*        connection->connection_id = '0017'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Third instance
**********************************************************************
    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'SQ'
            i_connection_id = '0001'
        ).

*        connection->carrier_id    = 'SQ'.
*        connection->connection_id = '0001'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.


* Output
**********************************************************************

    LOOP AT connections INTO connection.

      out->write( connection->get_output( ) ).

    ENDLOOP.

  ENDMETHOD.
```

4. Activate the class. Execute it and debug the method calls.
   a) Press **Ctrl + F3** to activate the class.
   b) Press **F9** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Define and call methods.

© Copyright. All rights reserved.

Unit 3
Lesson 4

Using Encapsulation to Ensure Consistency

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Explain encapsulation.
- Define and use constructors.

Data Encapsulation

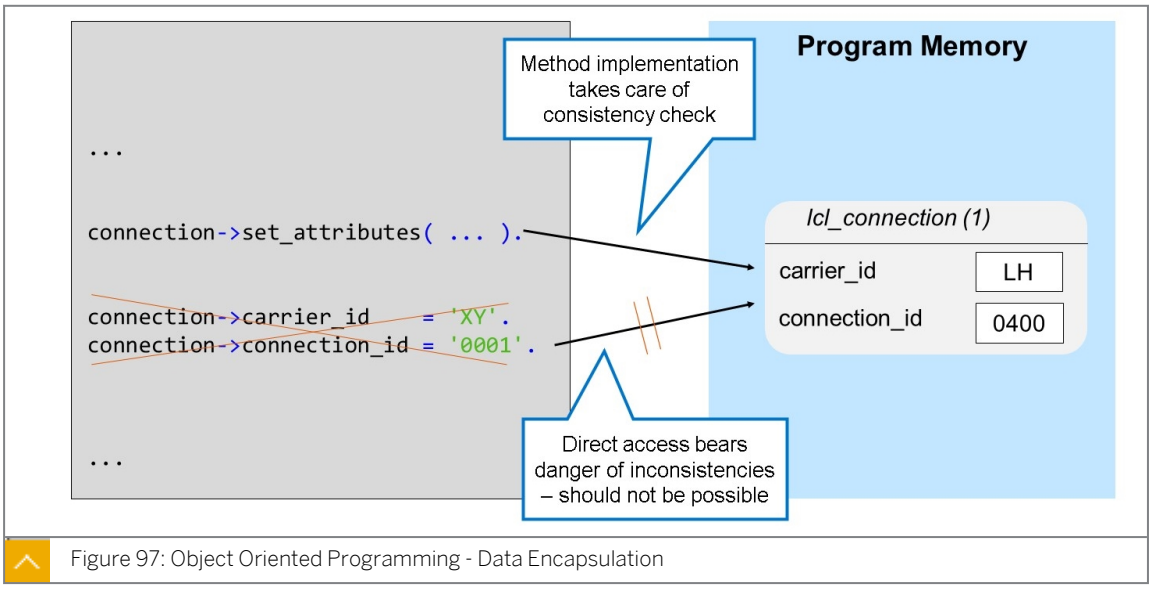


Figure 97: Object Oriented Programming - Data Encapsulation

In object-oriented programming, an object corresponds to a real-life object such as an employee, a vehicle, or, in our case, a flight connection. It has attributes that describe it, in the case of a flight connection, those are the carrier id and the flight number.

Now let us consider what happens when a program that is using this object wants to provide values for these attributes. It is obvious that only those combinations of carrier id and flight number should be accepted that correspond to a flight connection in the real world.

In object orientation, the client program should not be able to change the attribute values directly. Instead, it should need to call a method to perform this task. The method, which comes with the object, can then check whether the combination of carrier id and flight number is valid and, if this is not the case - reject the change.

As developers of class lcl_connection, we can ensure the use of method set_attributes( ) by making attributes carrier_id and connection_id private, or at least read only.

This concept is called data encapsulation; The information about the flight connection is managed by the connection object itself and cannot be manipulated by anyone else. This

© Copyright. All rights reserved.

ensures that the check of the consistency cannot be bypassed - whether knowingly or unknowingly. This is one of the major advantages of object-orientation.

> Note:
> In object oriented programming, it is recommended to use data encapsulation as much as possible!

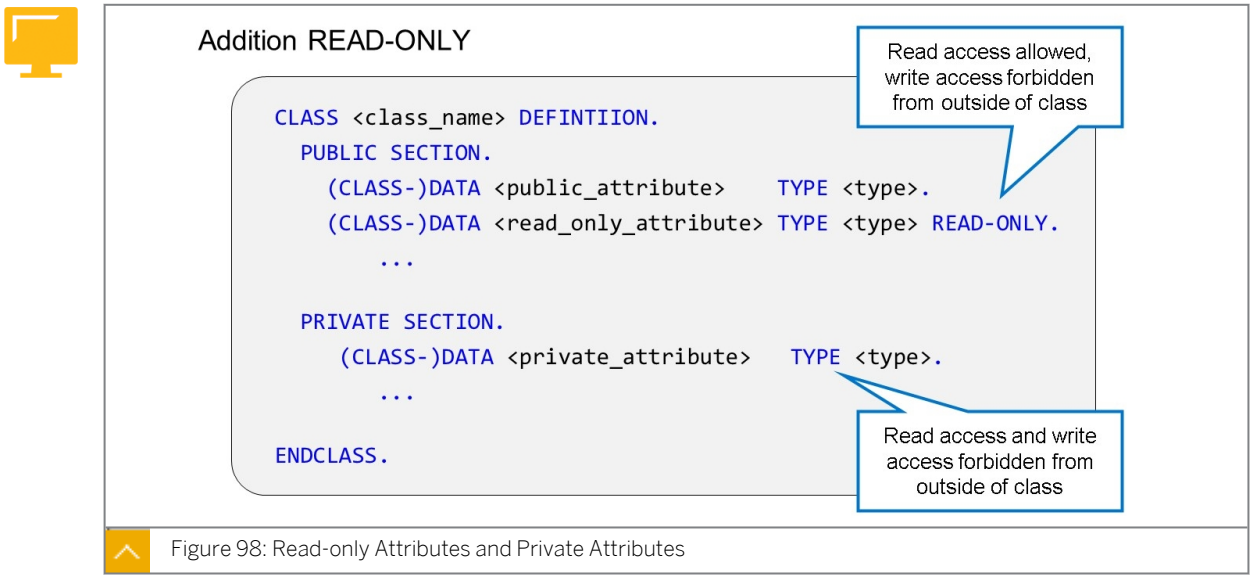


Figure 98: Read-only Attributes and Private Attributes

With the public attributes we defined by now, it was possible to read and change the values of the attributes anywhere inside and outside the class.

In order to restrict access to the attributes you have two options:

1. Keep the DATA statement or CLASS-DATA statement in the public section and add addition READ-ONLY. In doing so, write access to the attribute is forbidden outside of the class, but read access is still possible.

   > Note:
   > Addition READ-ONLY is only allowed in the public section of a class.

2. Move the DATA statement or CLASS-DATA statement from the public section to one of the other visibility sections, for example the private section. In doing so, read access and write access to the attribute is forbidden outside of the class.

   > Hint:
   > ADT offers a quick fix to change the visibility of an attribute. To use it place the cursor in the attribute name, press **`Ctrl + 1`**, and choose Make <attribute> private.

© Copyright. All rights reserved.

Instance Constructor

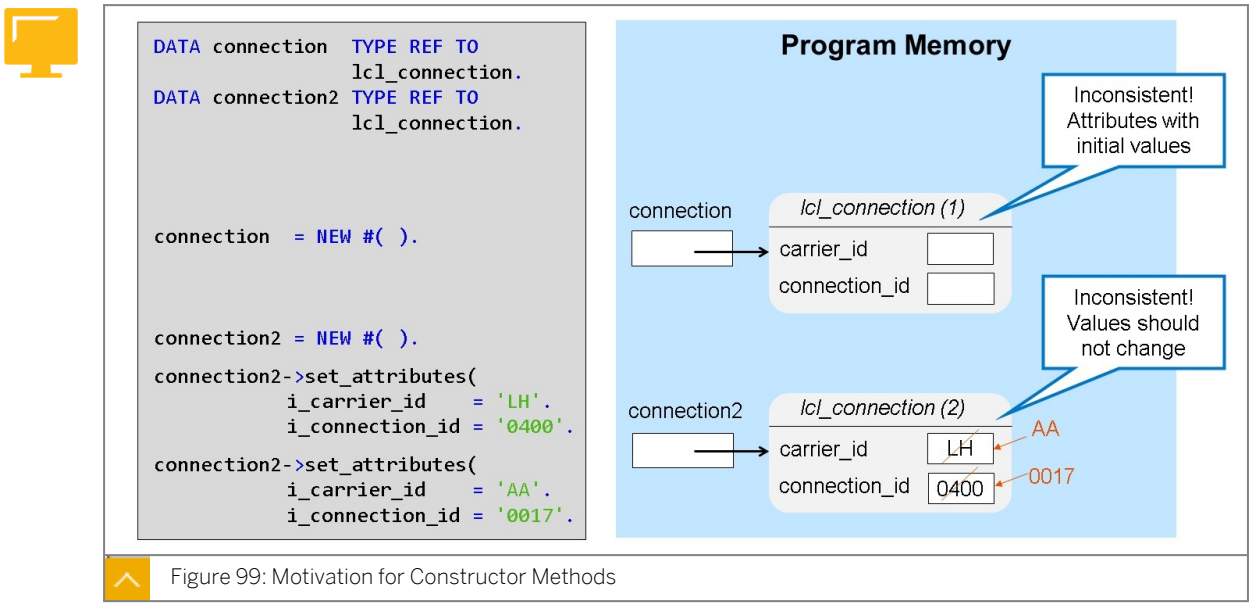


Figure 99: Motivation for Constructor Methods

By making your attributes private - or read-only, at least - you can ensure that the client program uses the available set_attributes( ) method to set the values for attributes carrier_id and connection_id.

But there is still potential for inconsistencies:

- You cannot force the program to call set_attributes( ) for each new instance. As a result, there can be instances with initial values for carrier_id and connection_id.
- The client program can call set_attributes( ) several times for the same instance. This should also not be possible.

What you need is a technique to enforce non-initial values during instantiation and to prevent later changes.

To solve these problems, object-oriented programming languages use constructor methods.

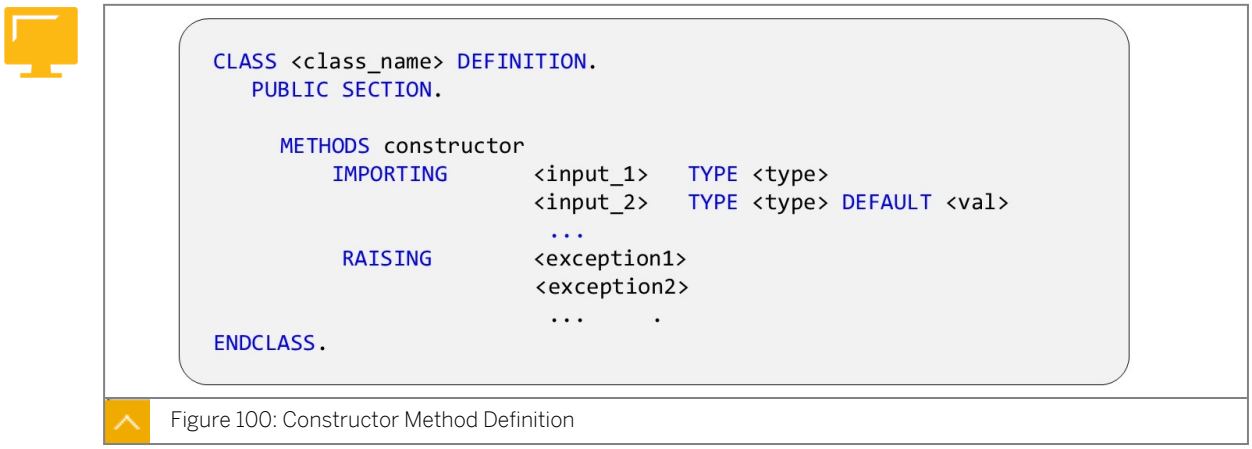


Figure 100: Constructor Method Definition

The runtime system calls the constructor automatically when you create a new instance of the class, but you may not call it explicitly. Thus the constructor is guaranteed to run once and once only for each instance that you create.

© Copyright. All rights reserved.

From a syntax perspective a constructor method has following properties:

- a public instance method with the reserved name constructor
- may have importing parameters, for example to obtain starting values for the attributes of the new instance
- may raise exceptions

Note:
In ABAP, it is not possible to define more than one constructor method in the same class.

Hint:
In ADT, you can use a quick fix to generate a constructor method. To use this quick fix proceed as follows:

1. Either in the definition or implementation part, place the cursor in the class name and press Ctrl + 1.
2. From the list of available quick fixes, choose Generate constructor
3. On the dialog window that appears, select the attributes you want to initialize with the constructor and choose Finish

Once you generated the constructor you can adjust its definition and implementation to your needs.

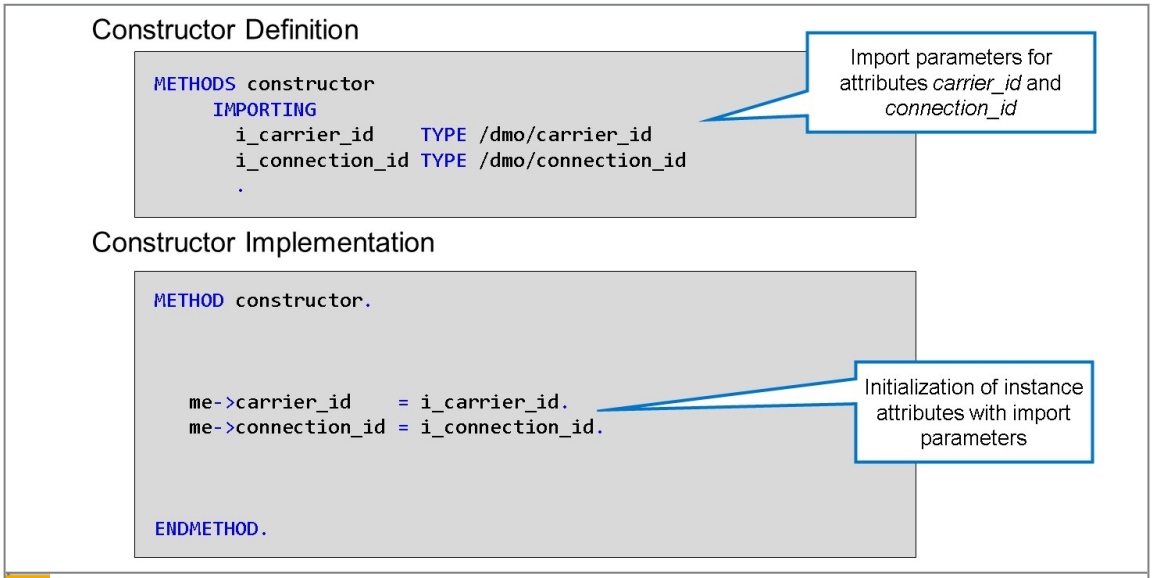


Figure 101: Example: Constructor Definition and Implementation (Generated)

The example shows the constructor of class lcl_connection. The generated definition contains an importing parameter for each of the attributes `carrier_id` and `connection_id`.

The generated part of the implementation contains a value assignment for each of the attributes with the related importing parameter on the right.

© Copyright. All rights reserved.

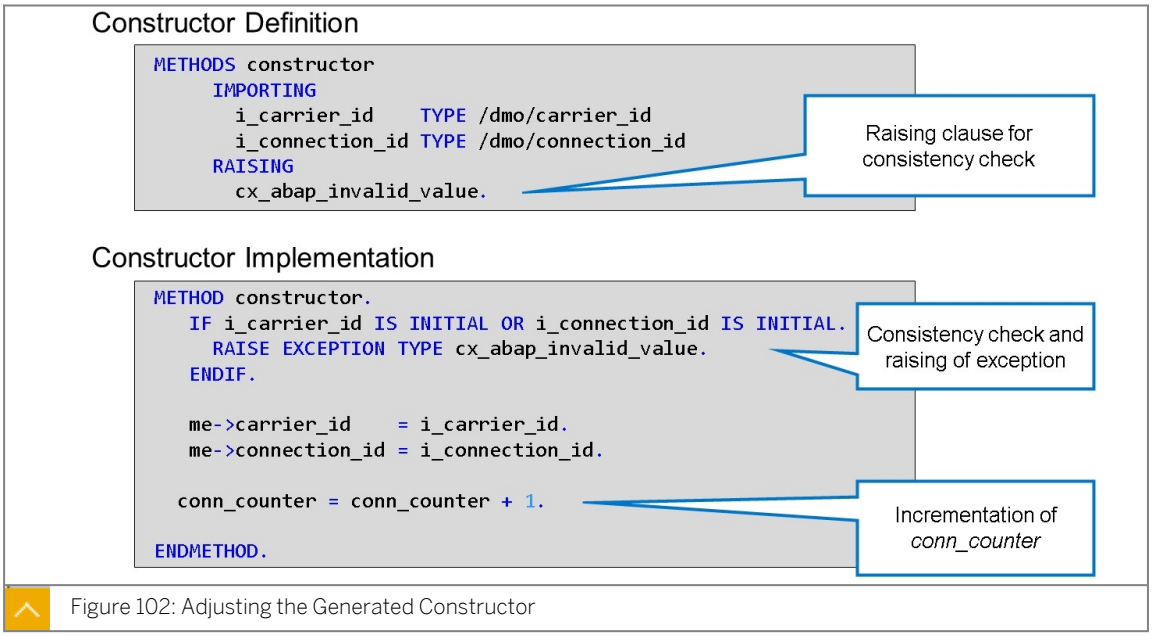


Figure 102: Adjusting the Generated Constructor

The example shows some manual additions to the generated constructor of class lcl_connection.

In order to reject the creation of instances with initial values, a consistency check was added to the implementation and a RAISING clause to the definition of the constructor.

Because the constructor is executed once and only once for each new instance of class lcl_connection, the constructor implementation is the perfect place to increment static attribute conn_counter.

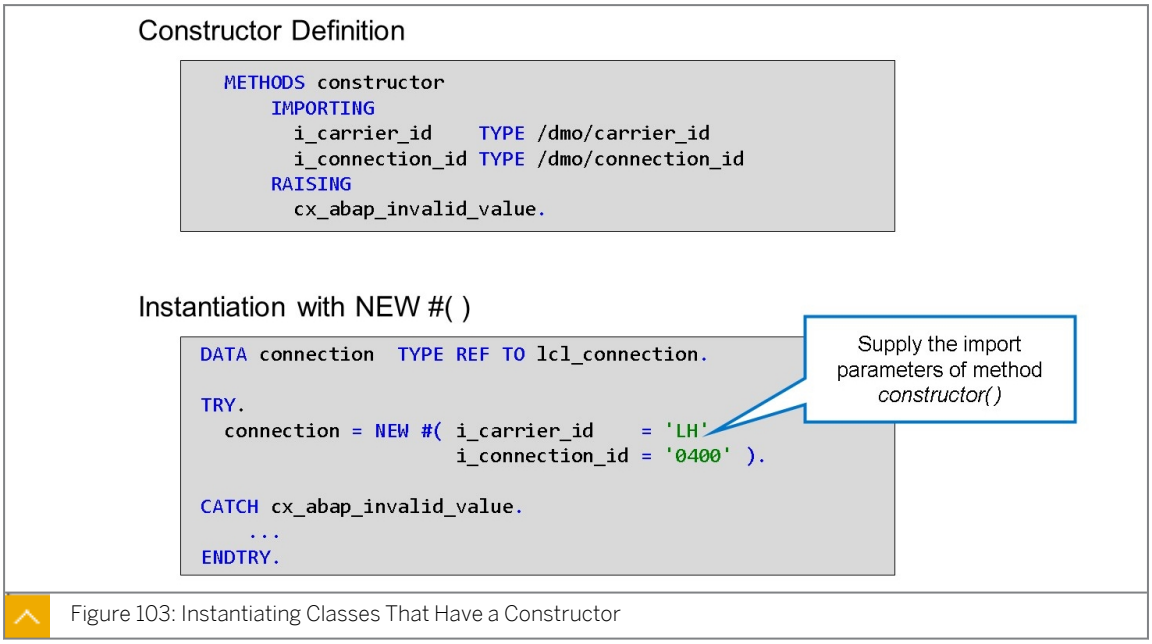


Figure 103: Instantiating Classes That Have a Constructor

When you instantiate a class that has a constructor, the system calls the constructor method automatically. If the constructor has importing parameters, you pass them in the NEW expression exactly as you would pass parameters to a normal method.

© Copyright. All rights reserved.

Note:
A constructor can have only importing parameters. Keyword EXPORTING is neither needed nor allowed in the NEW expression.

Note:
If the constructor has exceptions, you must ensure that you catch them by enclosing the instantiation in a TRY... CATCH...ENDTRY block. If an constructor raises an exception, control returns immediately to the program containing the NEW expression. In this case, you do not receive a new instance of the class.

Static Constructor

```
CLASS <class_name> DEFINITION.
   PUBLIC SECTION.

     CLASS-METHODS class_constructor.
ENDCLASS.
```

Figure 104: Static Constructor Definition

While the instance constructor is called once when each instance of a class is created, you will sometimes need to perform actions once only for the entire class. For this purpose ABAP allows you to define a static constructor, also known as class constructor.

The runtime system calls the static constructor once only when the class is addressed for the first time during the execution of a program.

The first addressing of a class could be one of the following:

- First instantiation of the class
- First call of a static method
- First access to a public static attribute

Note:
This list is not complete. There are other actions (related to inheritance) that can cause a class to be addressed for the first time.

A typical use case for the static constructor is the dynamic initialization of static attributes with non-initial values. Therefore it is important that the runtime calls the static constructor before creating the instance, calling the static method, or addressing the static attribute.

From a syntax perspective a constructor method has following properties:

- A public static method with the reserved name class_constructor
- Without parameters or exceptions

© Copyright. All rights reserved.

Note:
A static constructor must not have a signature because it is impossible to tell exactly when a class will be addressed for the first time.

Hint:
In ADT, you can use a quick fix to generate a class constructor method. To use this quick fix proceed as follows:

1. Either in the definition or implementation part, place the cursor in the class name and press **`Ctrl + 1`**.
2. From the list of available quick fixes, choose Generate class constructor

© Copyright. All rights reserved.

Unit 3
Exercise 11

Use Private Attributes and a Constructor

In this exercise, you make the static attribute of your class read-only and the instance attributes private. You use a constructor to set their values.

Template:

/LRN/CL_S4D400_CLS_METHODS (global Class)

Solution:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class (**`ZCL_##_METHODS`**, **`ZCL_##_LOCAL_CLASS`** or **`ZCL_##_INSTANCES`**).

1. Copy class **`/LRN/CL_S4D400_CLS_METHODS`** to a class in your own package (suggested name: **`ZCL_##_CONSTRUCTOR`**, where ## stands for your group number).

Task 2: Make Attributes Private
Change the visibility of the attributes **`carrier_id`** and **`connection_id`** to enforce the use of the methods **`set_attributes( )`** and **`get_output( )`**.

1. Set the visibility of the attribute **`carrier_id`** to private using a quick fix.
2. Set the visibility of the attribute **`connection_id`** to private using a quick fix.

Task 3: Create Instance Constructor
Replace the public method **`set_attributes`** with an instance constructor to ensure that the attributes are set during the creation of a new instance and that they are not changed afterward.

1. Comment out the definition and implementation of the method **`set_attributes`**.
2. Add an instance constructor to the local class **`lcl_connection`** using a quick fix. Ensure that the constructor has importing parameters corresponding to attributes **`carrier_id`** and **`connection_id`**.
3. Extend the generated constructor definition. Add exception **`CX_ABAP_INVALID_VALUE`** to the definition of the method **`constructor`**.
4. Extend the generated constructor implementation. If either of the importing parameters is initial, raise exception **`CX_ABAP_INVALID_VALUE`**.
5. In the constructor implementation, add a statement to increase the value of the static attribute **`conn_counter`** by one. Make sure the statement is only executed if no exception was raised.

© Copyright. All rights reserved.

6. Make sure that it is not possible to change the value of **`conn_counter`** from outside the class.

Task 4: Use the Constructor

Adjust the instantiation of class **`lcl_connection`** to supply the parameters of the instance constructor and handle the exception.

1. In method **`if_oo_adt_classrun~main`**, go to the NEW #( ) expression for the first instance of **`lcl_connection`**.
2. In the NEW expression, supply the import parameters of **`constructor`**.

Hint:
You can copy the parameter passing from the call of method **`set_attributes`** for this instance.

3. Remove or comment out all calls of the method **`set_attributes`**.
4. Move the instance creation into the TRY block of the exception handling.
5. Repeat the previous steps for the other instances of your local class.
6. Activate the class. Execute it and debug the instantiation.

© Copyright. All rights reserved.

Unit 3
Solution 11

Use Private Attributes and a Constructor

In this exercise, you make the static attribute of your class read-only and the instance attributes private. You use a constructor to set their values.

Template:

/LRN/CL_S4D400_CLS_METHODS (global Class)

Solution:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class (**`ZCL_##_METHODS`**, **`ZCL_##_LOCAL_CLASS`** or **`ZCL_##_INSTANCES`**).

1. Copy class **`/LRN/CL_S4D400_CLS_METHODS`** to a class in your own package (suggested name: **`ZCL_##_CONSTRUCTOR`**, where ## stands for your group number).
   a) Open the source code of the global class **`/LRN/CL_S4D400_CLS_METHODS`**.
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click the class **`/LRN/CL_S4D400_CLS_METHODS`** to open the context menu.
   d) From the context menu, choose *Duplicate ...*.
   e) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_CONSTRUCTOR`**, where ## stands for your group number.
   f) Adjust the description and choose *Next*.
   g) Confirm the transport request and choose *Finish*.

Task 2: Make Attributes Private
Change the visibility of the attributes **`carrier_id`** and **`connection_id`** to enforce the use of the methods **`set_attributes( )`** and **`get_output( )`**.

1. Set the visibility of the attribute **`carrier_id`** to private using a quick fix.
   a) Switch to the *Local Types* tab.
   b) Go to the declaration of the attribute **`carrier_id`** in local class **`lcl_connection`**.
   c) Place the cursor on **`carrier_id`** and press **`Ctrl + 1`**.
   d) Double-click the suggestion *Make carrier_id private* .
   e) Check that the declaration of the attribute **`carrier_id`** has moved to the private section.

© Copyright. All rights reserved.

2. Set the visibility of the attribute **connection_id** to private using a quick fix.
   a) Go to the declaration of the attribute **connection_id** in the local class **lcl_connection**.
   b) Place the cursor on **connection_id** and press **Ctrl + 1**.
   c) Double-click the suggestion *Make connection_id private* .
   d) Check that the declaration of attribute **connection_id** has moved to the private section.

Task 3: Create Instance Constructor

Replace the public method **set_attributes** with an instance constructor to ensure that the attributes are set during the creation of a new instance and that they are not changed afterward.

1. Comment out the definition and implementation of the method **set_attributes**.
   a) Select all lines that belong to the `METHODS set_attributes` statement, including the line with the period sign (.).
   b) Press **Ctrl + <** to add a star sign (*) in front of each selected line.
   c) Select all lines that belong to the implementation of the method **set_attributes** including the corresponding `ENDMETHOD.` statement and press **Ctrl + <** again.
2. Add an instance constructor to the local class **lcl_connection** using a quick fix. Ensure that the constructor has importing parameters corresponding to attributes **carrier_id** and **connection_id**.
   a) Place the cursor on the name of the class and press **Ctrl + 1**.
   b) Double-click on *Generate constructor*.
   c) In the dialog box, ensure that **carrier_id** and **connection_id** are selected and choose *Finish*.
3. Extend the generated constructor definition. Add exception **CX_ABAP_INVALID_VALUE** to the definition of the method **constructor**.
   a) Navigate to the generated definition of the method **constructor**.
   b) Adjust the code as follows:

```
METHODS constructor
      IMPORTING
        i_connection_id TYPE /dmo/connection_id
        i_carrier_id    TYPE /dmo/carrier_id
      RAISING
        cx_ABAP_INVALID_VALUE.
```

4. Extend the generated constructor implementation. If either of the importing parameters is initial, raise exception **CX_ABAP_INVALID_VALUE**.
   a) Navigate from the definition to the implementation of the method **constructor**, for example, by placing the cursor on **constructor** and pressing **F3**.

© Copyright. All rights reserved.

b) Adjust the code as follows:

```
METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->connection_id = i_connection_id.
    me->carrier_id    = i_carrier_id.

  ENDMETHOD.
```

5. In the constructor implementation, add a statement to increase the value of the static attribute **conn_counter** by one. Make sure the statement is only executed if no exception was raised.

   a) Adjust the code as follows:

```
    me->connection_id = i_connection_id.
    me->carrier_id    = i_carrier_id.

    conn_counter = conn_counter + 1.

  ENDMETHOD.
```

6. Make sure that it is not possible to change the value of **conn_counter** from outside the class.

   a) Add `READ-ONLY` to the declaration of the static attribute **conn_counter**.

   b) Adjust the code as follows:

```
  CLASS-DATA conn_counter TYPE i READ-ONLY.
```

Task 4: Use the Constructor

Adjust the instantiation of class **lcl_connection** to supply the parameters of the instance constructor and handle the exception.

1. In method **if_oo_adt_classrun~main**, go to the NEW #( ) expression for the first instance of **lcl_connection**.

   a) Switch to the *Global Class* tab.

   b) Find the first line with `connection = NEW #( ).`.

2. In the NEW expression, supply the import parameters of **constructor**.

Hint:
You can copy the parameter passing from the call of method **set_attributes** for this instance.

   a) Adjust the NEW expression as follows:

```
  connection = NEW #(
                     i_carrier_id    = 'LH'
                     i_connection_id = '0400'
                   ).
```

3. Remove or comment out all calls of the method **set_attributes**.

© Copyright. All rights reserved.

a) Select all lines that belong to the `connection->set_attributes( ... ).` statement, including the line with the closing bracket and the period sign (.).

b) Press **Ctrl + <** to add a star sign (*) in front of each selected line.

4. Move the instance creation into the TRY block of the exception handling.

a) Move the `TRY.` statement up, to before the instance creation.

b) Your first instance creation should now look like this:

```
TRY.
    connection = NEW #(
                         i_carrier_id    = 'LH'
                         i_connection_id = '0400'
                       ).

*        connection->set_attributes(
*          EXPORTING
*            i_carrier_id    = 'LH'
*            i_connection_id = '0400'
*        ).

         APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.
```

5. Repeat the previous steps for the other instances of your local class.

a) After this step, the implementation of the method **if_oo_adt_classrun~main** should look similar to this:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************
    TRY.
    connection = NEW #(
                         i_carrier_id    = 'LH'
                         i_connection_id = '0400'
                       ).

*        connection->set_attributes(
*          EXPORTING
*            i_carrier_id    = 'LH'
*            i_connection_id = '0400'
*        ).

         APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Second instance
**********************************************************************
    TRY.
```


© Copyright. All rights reserved.

```
    connection = NEW #(
                         i_carrier_id    = 'AA'
                         i_connection_id = '0017'
                       ).
        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Third instance
**********************************************************************
    TRY.
    connection = NEW #(
                         i_carrier_id    = 'SQ'
                         i_connection_id = '0001'
                       ).
        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Output
**********************************************************************
    LOOP AT connections INTO connection.
      out->write( connection->get_output( ) ).
    ENDLOOP.
  ENDMETHOD.
```

6. Activate the class. Execute it and debug the instantiation.
   a) Press **Ctrl + F3** to activate the class.
   b) Press **F9** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Explain encapsulation.
- Define and use constructors.

© Copyright. All rights reserved.

Learning Assessment

1. In which order do you define the visibility sections of a class?

   Choose the correct answer.

   - ☐ A PUBLIC SECTION. PROTECTED SECTION. PRIVATE SECTION.
   - ☐ B PRIVATE SECTION. PROTECTED SECTION. PUBLIC SECTION.
   - ☐ C It doesn't matter.

2. You have defined a class containing instance attributes and static attributes. You have also declared a reference variable but not yet created an instance of the class. Which components of the class can you access at this point, and how?

   Choose the correct answer.

   - ☐ A Instance components using the reference variable.
   - ☐ B Instance components using the name of the class.
   - ☐ C Static components using the reference variable.
   - ☐ D Static components using the name of the class.

3. You have declared two reference variables, ref1 and ref2. ref1 points to an object. You now execute the statement ref2 = ref1. What happens?

   Choose the correct answer.

   - ☐ A The ABAP system creates a copy of the object to which ref1 is pointing and assigns its address to ref2. There are now two objects.
   - ☐ B The ABAP system assigns the address of the object to which ref1is pointing to reference variable ref2. There is only one object.

© Copyright. All rights reserved.

4. A class my_class contains the public static method my_method. What is the correct code to call this method?

   Choose the correct answer.

   - ☐ A my_class->my_method().
   - ☐ B my_class=>my_method().
   - ☐ C my_class->my_method( ).
   - ☐ D my_class=>my_method( ).

5. A functional method must have

   Choose the correct answer.

   - ☐ A Exactly one importing parameter
   - ☐ B Exactly one exporting parameter
   - ☐ C No changing parameters
   - ☐ D Exactly one returning parameter

6. Your class contains a public instance attribute attr. How could you ensure that its value can only be changed within the class?

   Choose the correct answers.

   - ☐ A Make it a private attribute.
   - ☐ B Convert it into a constant.
   - ☐ C Leave it in the public section but use the READ-ONLY addition.
   - ☐ D Move it to a different local class.

7. Which of the following signature elements may an instance constructor have?

   Choose the correct answers.

   - ☐ A Importing parameters
   - ☐ B Exporting parameters
   - ☐ C Changing parameters
   - ☐ D Exceptions

© Copyright. All rights reserved.

8. The static constructor is executed when the class is addressed for the first time. When might this be?

   Choose the correct answers.

   - ☐ A At the beginning of the program.
   - ☐ B When you call a static method of the class.
   - ☐ C When you instantiate the class.

© Copyright. All rights reserved.

Learning Assessment - Answers

1. In which order do you define the visibility sections of a class?

   Choose the correct answer.

   [X] A PUBLIC SECTION. PROTECTED SECTION. PRIVATE SECTION.

   [ ] B PRIVATE SECTION. PROTECTED SECTION. PUBLIC SECTION.

   [ ] C It doesn't matter.

   That's right.

2. You have defined a class containing instance attributes and static attributes. You have also declared a reference variable but not yet created an instance of the class. Which components of the class can you access at this point, and how?

   Choose the correct answer.

   [ ] A Instance components using the reference variable.

   [ ] B Instance components using the name of the class.

   [ ] C Static components using the reference variable.

   [X] D Static components using the name of the class.

   That is correct.

3. You have declared two reference variables, ref1 and ref2. ref1 points to an object. You now execute the statement ref2 = ref1. What happens?

   Choose the correct answer.

   [ ] A The ABAP system creates a copy of the object to which ref1 is pointing and assigns its address to ref2. There are now two objects.

   [X] B The ABAP system assigns the address of the object to which ref1is pointing to reference variable ref2. There is only one object.

   That is correct.

© Copyright. All rights reserved.

4. A class my_class contains the public static method my_method. What is the correct code to call this method?

   Choose the correct answer.

   - [ ] A my_class->my_method().
   - [ ] B my_class=>my_method().
   - [ ] C my_class->my_method( ).
   - [x] D my_class=>my_method( ).

   Correct!

5. A functional method must have

   Choose the correct answer.

   - [ ] A Exactly one importing parameter
   - [ ] B Exactly one exporting parameter
   - [ ] C No changing parameters
   - [x] D Exactly one returning parameter

   That's right.

6. Your class contains a public instance attribute attr. How could you ensure that its value can only be changed within the class?

   Choose the correct answers.

   - [x] A Make it a private attribute.
   - [ ] B Convert it into a constant.
   - [x] C Leave it in the public section but use the READ-ONLY addition.
   - [ ] D Move it to a different local class.

   That is correct.

© Copyright. All rights reserved.

7. Which of the following signature elements may an instance constructor have?

   Choose the correct answers.

   - [X] A Importing parameters
   - [ ] B Exporting parameters
   - [ ] C Changing parameters
   - [X] D Exceptions

   That's right.

8. The static constructor is executed when the class is addressed for the first time. When might this be?

   Choose the correct answers.

   - [ ] A At the beginning of the program.
   - [X] B When you call a static method of the class.
   - [X] C When you instantiate the class.

   That's right.

© Copyright. All rights reserved.

UNIT 4 Reading Data from the Database

Lesson 1

Investigating a Table Definition 191

Lesson 2

Implementing Basic SELECT Statements 194

Exercise 12: Read Data from a Database Table 201

Lesson 3

Working with CDS Views 209

Exercise 13: Analyze and Use a CDS View Entity 215

UNIT OBJECTIVES

- Investigate a table definition.
- Describe basic features of ABAP SQL.
- Read single values from the database.
- Analyze a CDS view definition.
- Read data using a CDS view.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 4
Lesson 1

Investigating a Table Definition

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Investigate a table definition.

Database Table Definitions

Every ABAP system runs on a relational database management system (DBMS). Originally, ABAP supported different database management systems from different vendors. Therefore the definition of database objects like tables and views is done in a database independent way.

> Note:
> Newer ABAP releases only support SAP HANA as database. The ABAP Environment on SAP BTP always runs on SAP HANA.

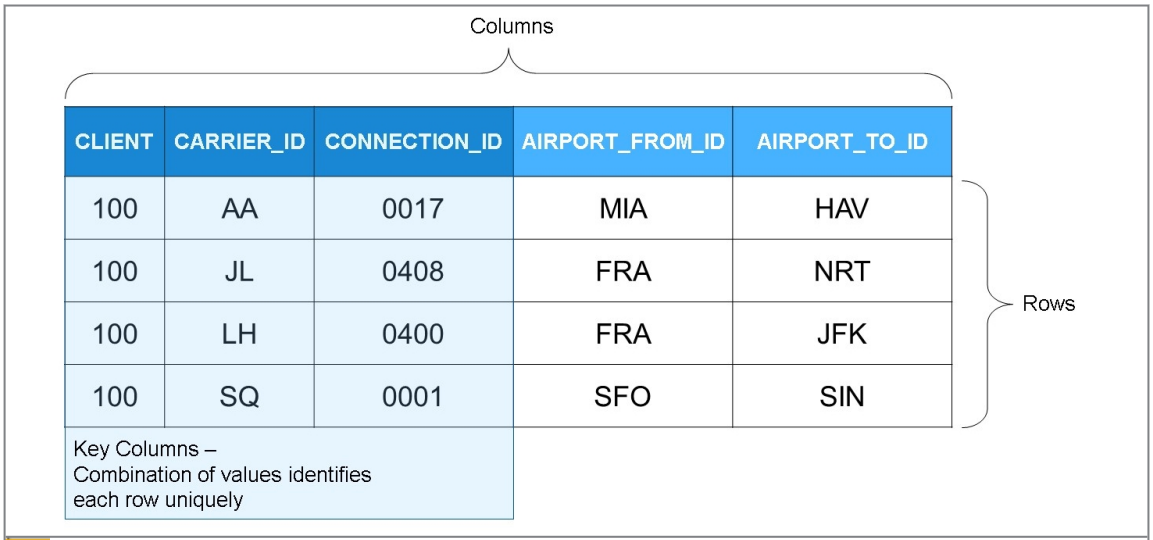


Figure 105: Database Table Definition

In a relational database, information is stored in two-dimensional tables, in which each row represents one data record whose contents are split up into columns. The database is called a relational database, as there are also relations between tables. For example, the table in the figure contains an airline code. In the database table, there would be a relationship to a further table containing the information that AA stands for American Airlines, JL for Japanese Airline, LH for Lufthansa, and so on.

© Copyright. All rights reserved.

A sequence of columns at the beginning of each database table forms its key. The key is a combination of values that ensures that each row in the table can be identified uniquely.

In SAP systems, database table definitions are development objects, and as such, are cross-client. However, the vast majority of tables contain business data, which is client-specific. To keep the data separate, client-specific tables have a client field (often named CLIENT or MANDT) as their first key field. The database of SAP accesses statements using ABAP SQL to ensure that a statement only manipulates data from the current client.

Database Table Definitions

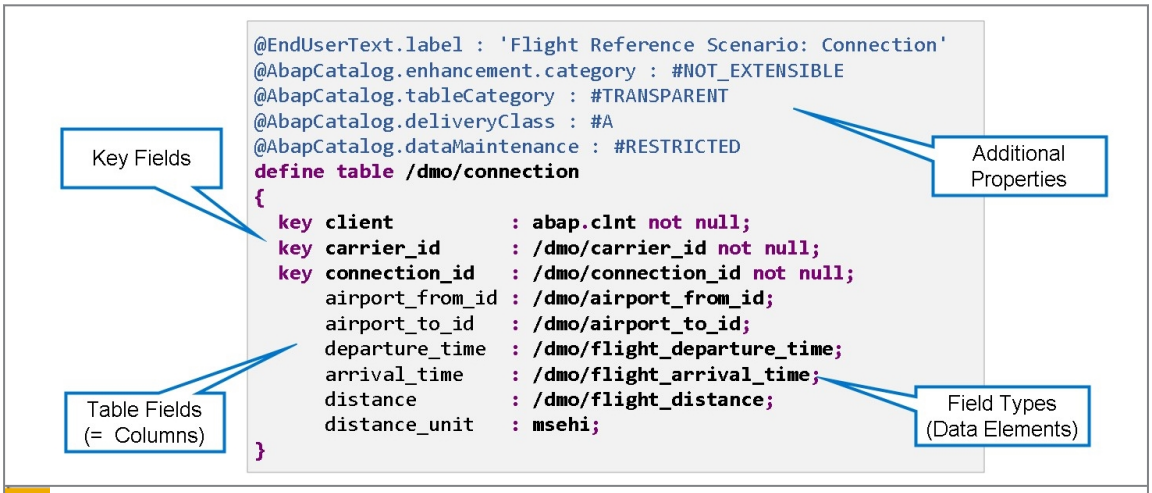


Figure 106: Database Tables in ABAP Development Tools

ADT provides a dedicated editor for database table definitions.

Let's look at the different parts of the table definition.

The main part of the definition consists of a DEFINE TABLE statement with the name of the table. This is followed by the list of table fields in a pair of curly brackets ({ , }); each field with a type that is described by a data element. A subset of fields at the beginning of the list defines the key, which identifies each record uniquely.

Several additional code lines before the DEFINE TABLE statement specify additional properties of the database table, among them a label.

© Copyright. All rights reserved.

Data Preview for Database Tables

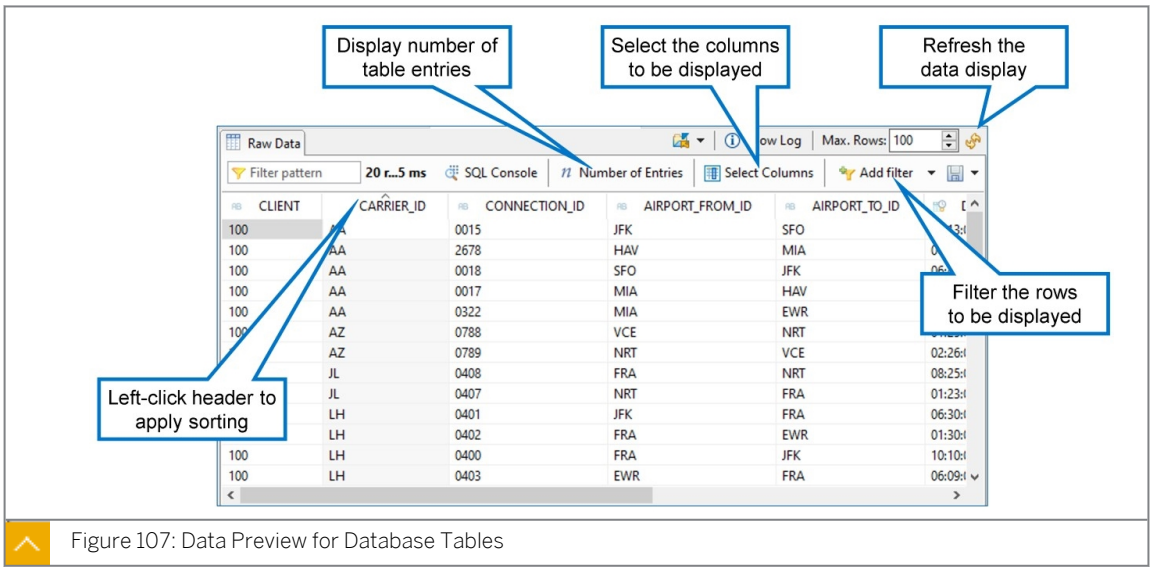


Figure 107: Data Preview for Database Tables

You can use the Data Preview tool to display and analyze the content of a database table. To open the Data Preview for a given table, right-click anywhere in the table definition and choose *Open With → Data Preview*. Alternatively, place the cursor anywhere in the database table definition and press `F8`.

The tool displays the data stored in the database table. Some of the most important functions are the following:

Select Columns

For tables with many columns, you can adjust the columns you want to display.

Change Column Sequence

You can change the column sequence by dragging and dropping a column header to another position.

Apply Sorting

You can apply a sorting by left-clicking on a column header.

Add Filter

For tables with many rows you can filter the data by choosing *Add Filter*.

Refresh Display

Choose the *Refresh* button to reload the data.

Number of Entries

Choose *Number of Entries* if you are only interested in the number of rows that meet your filter conditions. Without filters, the total number of entries is displayed.

LESSON SUMMARY

You should now be able to:

- Investigate a table definition.

© Copyright. All rights reserved.

Unit 4
Lesson 2

Implementing Basic SELECT Statements

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Describe basic features of ABAP SQL.
- Read single values from the database.

Basic ABAP SQL Syntax

Structured Query Language (SQL)

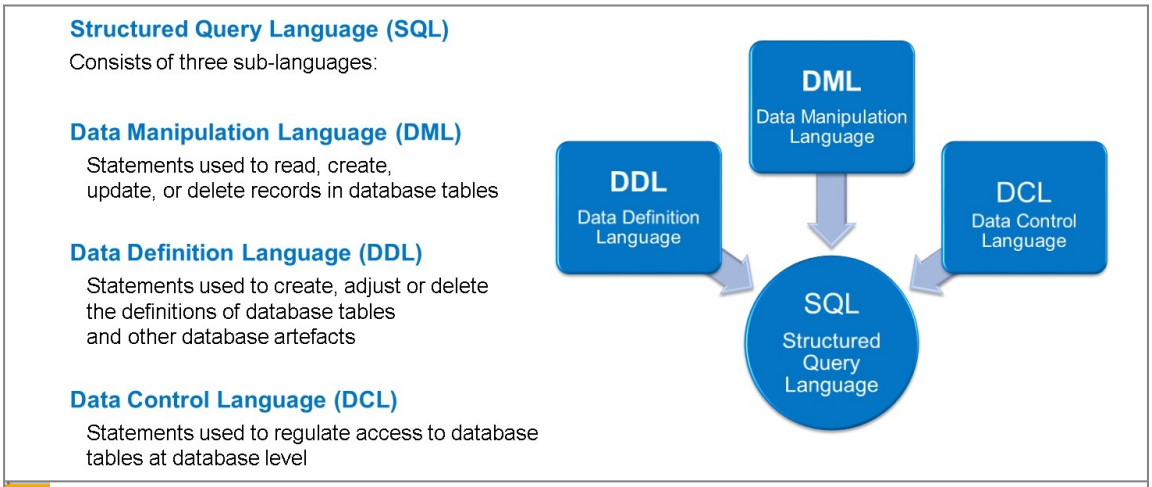


Figure 108: Structured Query Language (SQL)

All relational database systems use a variant of Structured Query Language (SQL) to allow you to work with them. Standard SQL consists of three main components:

Data Manipulation Language (DML)

DML comprises of the statements that you need to work with information in database tables. These are INSERT, for adding new records, SELECT, for reading data, UPDATE, to change existing records, and DELETE, to delete table contents. These statements are reflected in ABAP in the form of ABAP SQL.

Data Definition Language (DDL)

Data Definition Language provides you with commands that enable you to create and delete database tables and other database artifacts such as schemata, table indexes, and views. In an SAP system, you perform these tasks using dedicated editors of ADT, not with ABAP programs. One example is the editor for database table definitions we saw earlier.

Data Control Language (DCL)

© Copyright. All rights reserved.

Data Control Language is used in SQL to restrict access to data in the database for a particular user. It is not used in its classic form in ABAP, since the users at database level do not correspond one-to-one with the end users. Consequently, ABAP has its own authorization concept.

ABAP SQL Architecture

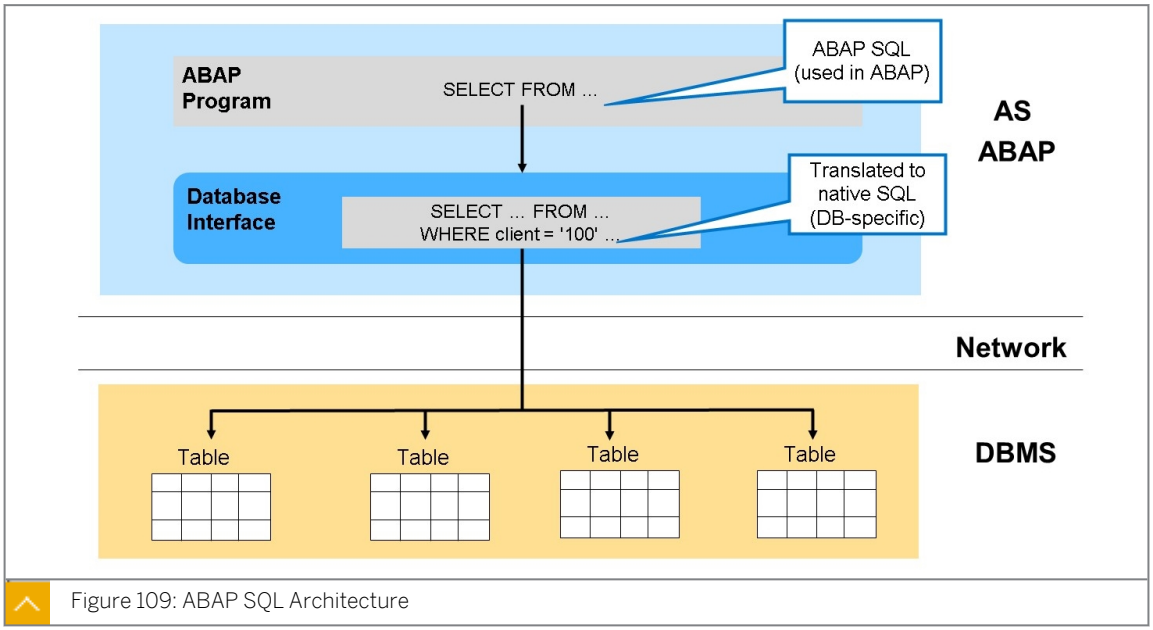


Figure 109: ABAP SQL Architecture

In the past, SAP systems had to support a range of database platforms, and each platform had a slightly different implementation of the SQL standard. This meant that each platform needed slightly different commands to achieve a particular task. To avoid the ABAP code being database-specific, SAP invented ABAP SQL - or Open SQL as it was called originally.

Note:
The name change from Open SQL and ABAP SQL also illustrates that as off release 7.53 ABAP only supports SAP HANA as DBMS.

ABAP SQL is an abstract set of SQL commands implemented at ABAP level and integrated into the ABAP language. At runtime, ABAP SQL is translated into a variant of SQL that the database understands. This variant is called Native SQL to distinguish it from ABAP SQL, the SQL variant that is integrated into ABAP. The translation from ABAP SQL to Native SQL takes place in the database interface, a component of the ABAP system that consists of a general part and a database-specific library.

Even though newer ABAP releases only support SAP HANA as DBMS, SAP has still retained the concept of ABAP SQL and the database interface. This is because of the following reasons:

Architecture compatibility

ABAP SQL and the database interface are an integral part of the system architecture.

Code compatibility

ABAP SQL coding from previous SAP products (including customer-specific development) should run free of side-effects in the modern, SAP-HANA-only ABAP environments.

© Copyright. All rights reserved.

Tasks of the Database Interface

The database interface does not just translate statements; it is also responsible for ABAP specific tasks like, for example, automatic client-handling.

Single Record Reading from the Database

The SELECT Statement in ABAP

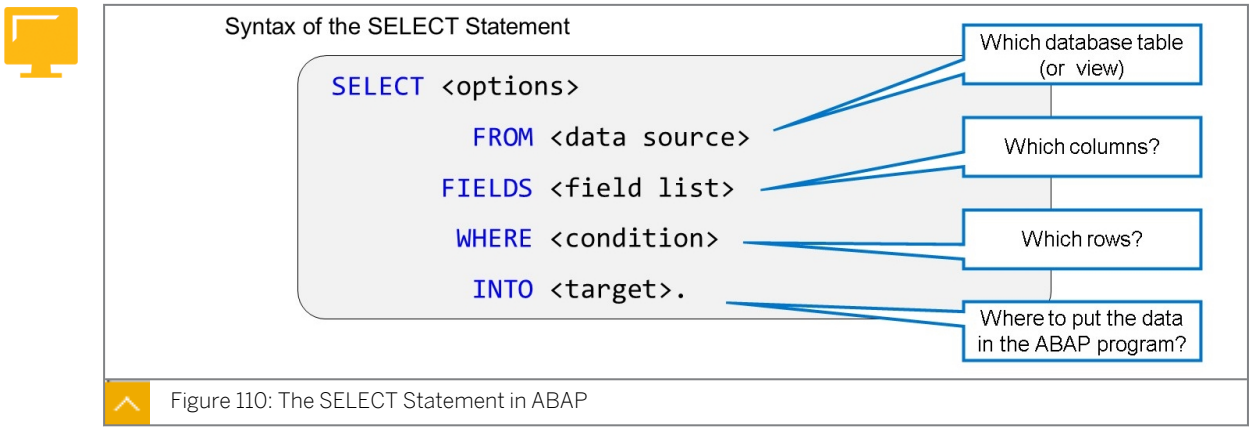


Figure 110: The SELECT Statement in ABAP

To read data from the database, you use the SELECT statement.

When you write a SELECT statement in ABAP SQL, the syntax check compares what you have written with the definition of the tables and views. If you try to address tables, views, or fields that do not exist, a syntax error occurs.

The basic syntax of the SELECT statement contains several sections, called clauses, and always follows the pattern in the figure, The SELECT Statement in ABAP. The most important clauses of a SELECT statement are as follows:

FROM

In the FROM clause of the SELECT statement, you specify the data source from which you want to read. This can be either be a database table or a view. Special SQL techniques allow you to combine data from multiple sources in the same SELECT statement.

FIELDS

In the FIELDS clause of the SELECT statement, you list the columns of the database table that you want to read. The columns in the list must be separated by commas. If you want to read the entire table line, you can specify FIELDS * instead of a column list. Be aware, however, that this can cause the database considerably more work than just reading the columns you need.

WHERE

In the WHERE clause, you can specify a condition that describes which rows of the table will be read. For example, the condition WHERE carrier_id = 'LH' means that only those rows will be read (in which the column CARRIER_D contains the value LH).

The WHERE clause can contain multiple conditions linked with the AND and OR operators. For example, WHERE carrier_id = 'LH' and connection_id = '0400' would return the data of flight connection LH 0400. You can also negate conditions using NOT. The WHERE clause is the only clause that is optional. Be aware, however, that without a WHERE clause you read all data from the table, or if the table has a client field - all data

© Copyright. All rights reserved.

that belongs to the logon client of the user. SELECTs without a WHERE clause can cause serious performance problems and should be avoided.

INTO

The INTO clause specifies the variable or variables in the ABAP program into which the data should be placed. This is normally a structure, or an internal table, and should ideally have the same sequence of components as the column list in the FIELDS clause.

Note:
You will see other forms of SQL syntax in ABAP. These are older and have been retained to ensure compatibility. You should get used to using the modern syntax, as it provides far more functions and features than the old form.

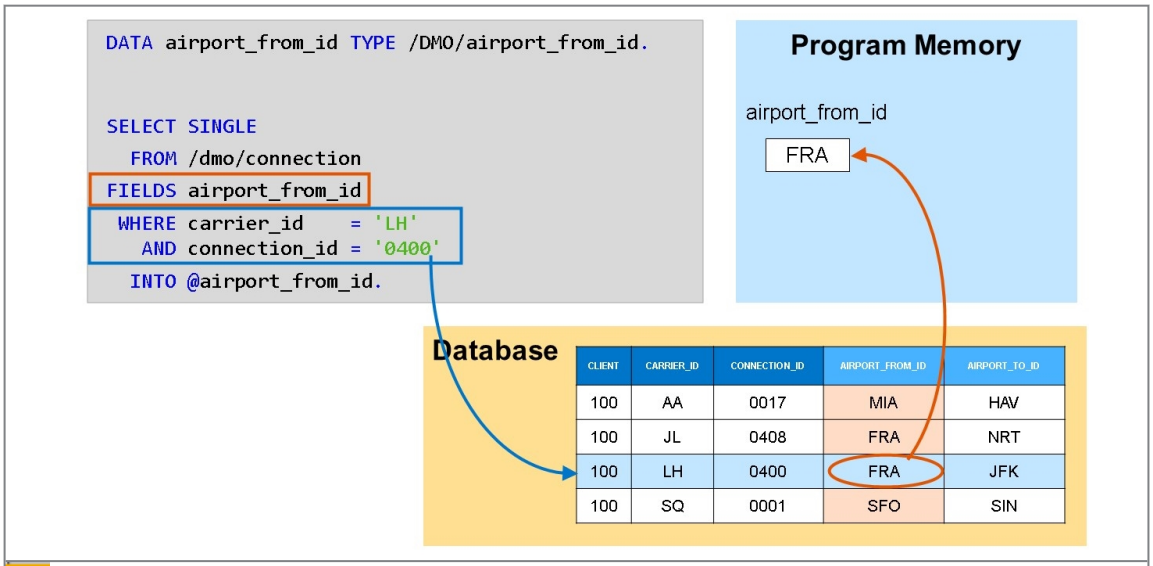


Figure 111: Example 1: Reading Single Field of Single Record

The figure, Example 1: Reading Single Field of Single Record, illustrates a SELECT statement that reads a single value from the database. The FROM clause tells us that the statement reads from database table /DMO/CONNECTION. The option, SINGLE, after the keyword, SELECT, indicates that only one row (a single record) is read. This row is identified in the WHERE clause by providing key filter values for key fields carrier_id and connection_id. Keep in mind, that the database interface will add a filter on the remaining key field client.

The FIELDS clause lists only one column of the table: column AIRPORT_FROM_ID.

The INTO clause has to match the rest of the statement. In our example, this specifies the variable airport_from_id as the target object, a scalar data object of identical type as table field, airport_from_id.

Note:
The at sign (@) identifies airport_from_id as the name of an ABAP data object. It is mandatory for all variables and constants that you use in an ABAP SQL statement. It is needed to avoid ambiguities if, for example, a data object and table field have the same name.

© Copyright. All rights reserved.

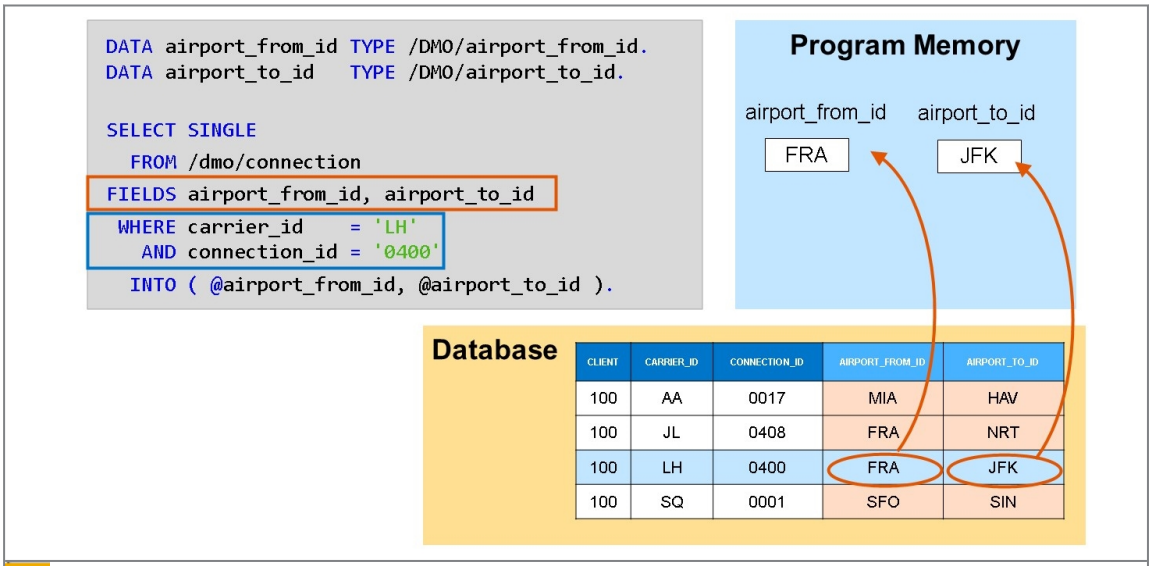


Figure 112: Example 2: Reading Several Fields of Single Record

The figure, Example 2: Reading Several Fields of Single Record, illustrates a SELECT statement that reads two values from the same record of the database.

This time, the FIELDS clause lists two columns of the table; column AIRPORT_FROM_ID and column AIRPORT_TO_ID.

To match this, the INTO clause specifies the variables airport_from_id and airport_to_id as the target objects. They are separated by a comma and surrounded by pair of brackets to make it clear that together they form the target of the SELECT statement.

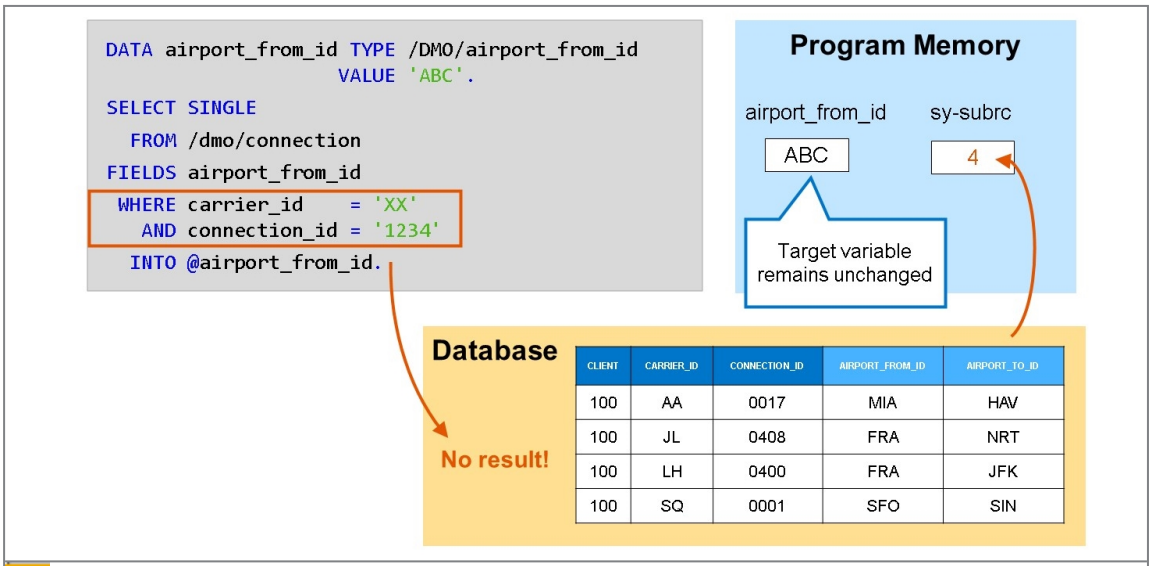


Figure 113: What Happens If There Is No Result?

When you implement a SELECT statement you always have to take into account that there could be no result, either because the database table does not contain any data at all or because it does not contain any rows that fulfill the conditions in the WHERE clause. In the example from the figure above, the database table does not contain a row with carrier_id = 'XX' and connection_id = '1234'.

ABAP SQL uses system field SY-SUBRC to indicate a successful or unsuccessful execution of a statement. System field SY-SUBRC is of type integer. Initial value 0 always indicates a

© Copyright. All rights reserved.

successful execution. If, after a SELECT statement, SY-SUBRC contains the value 4, this indicates that the database returned an empty result.

Note:
If the database returns an empty result, ABAP SQL does NOT touch the target variable after INTO!

In particular, the target variable is not initialized in case of an error.

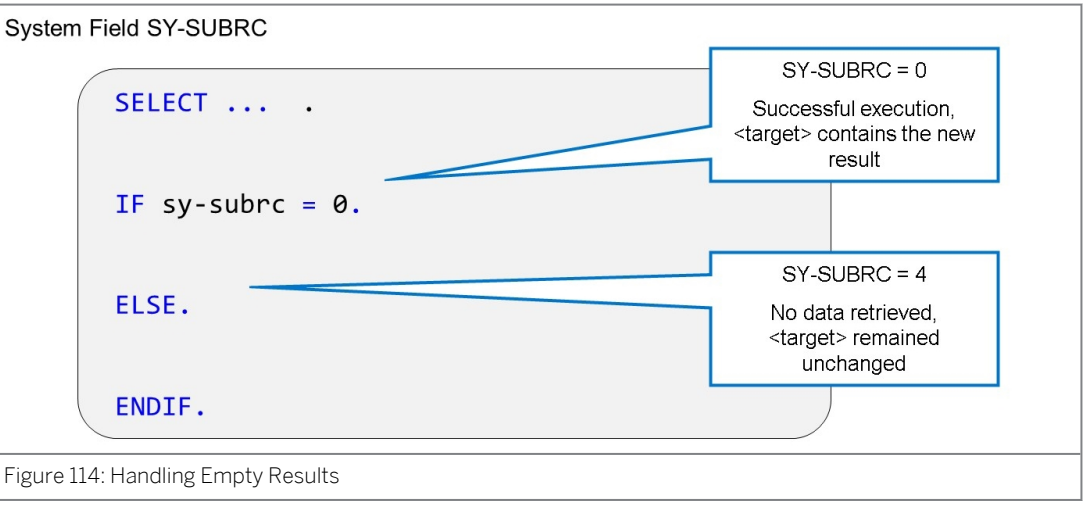


Figure 114: Handling Empty Results

It is recommended that you evaluate the content of system field sy-subrc immediately after each SELECT statement.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 4
Exercise 12

Read Data from a Database Table

In this exercise, you extend your local class with attributes for the departure airport and the destination airport, and read the values for these attributes from a database table.

Template:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (global Class)

Solution:

/LRN/CL_S4D400_DBS_SELECT (global Class)

Task 1: Copy Template
Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`** to a class in your own package (suggested name: **`ZCL_##_SELECT`**, where ## stands for your group number).

Task 2: Declare Additional Attributes
Extend the local class **`lcl_connection`** with the private instance attributes **`airport_from_id`** and **`airport_to_id`**. Add some output for the new attributes to the implementation of the method **`get_output`**.

Note:
The new attributes are not filled yet. In order to fill them we will add a SELECT statement to the **`constructor`** in one of the next tasks of this exercise.

1. Switch to the local class **`lcl_connection`**.
2. Add the following private attributes to the class definition:

   Table 2: Attributes

Attribute Name	Scope	Data Type
`airport_from_id`	instance	/DMO/AIRPORT_FROM_ID
`airport_to_id`	instance	/DMO/AIRPORT_TO_ID

3. Extend the implementation of the method **`get_output`**. Append more string templates to the returning parameter **`r_output`**. Embed the new attributes as expressions into the string templates.
4. Activate the class. Execute it and analyze the console output.

Task 3: Analyze the Database Table
Analyze the definition of the database table **`/DMO/CONNECTION`**.

© Copyright. All rights reserved.

1. Open the development object that contains the definition of the database table **/DMO/CONNECTION**.
2. Open the *Element Information* (F2) for the database table /DMO/CONNECTION.

   Which fields have the description *Flight Reference Scenario: From Airport* and *Flight Reference Scenario: To Airport*?

Task 4: Read Data from the Database

In the method **constructor** of the local class **lcl_connection**, implement a SELECT statement that reads values for the new attributes from the database table **/DMO/CONNECTION**.

1. Return to the local class **lcl_connection** in your global class.
2. Navigate to the implementation of the method **constructor**.
3. After the `ENDIF.` statement, add a SELECT statement that reads a single record from database table **/DMO/CONNECTION**.
4. Implement the FIELDS clause. Read the table fields **airport_from_id** and **airport_to_id**.

   Hint:
   Use auto-completion (**Ctrl + Space**) to enter the field names.

5. Implement the WHERE condition. Restrict all key fields of the database table (except for the client field) with the values of importing parameters **i_carrier_id** and **i_connection_id**. Do not forget to escape the parameters with prefix @.

   Hint:
   Use auto-completion (**Ctrl + Space**) to enter the element names and parameter names.

   Why is it not necessary to specify the client field in the WHERE condition?

6. Implement the INTO clause. Store the SELECT result in the attributes **airport_from_id**, and **airport_to_id**. Do not forget to escape the attributes with prefix @.

© Copyright. All rights reserved.

Hint:
Use auto-completion (**`Ctrl + Space`**) to enter the attribute names.

7. Implement error handling after the SELECT statement. Check the content of the system field **`sy-subrc`**. If it does not equal zero, raise exception **`CX_ABAP_INVALID_VALUE`**.
8. Activate the class. Execute it and analyze the console output. Check that the output for the new attributes displays data.

© Copyright. All rights reserved.

Unit 4
Solution 12

Read Data from a Database Table

In this exercise, you extend your local class with attributes for the departure airport and the destination airport, and read the values for these attributes from a database table.

Template:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (global Class)

Solution:

/LRN/CL_S4D400_DBS_SELECT (global Class)

Task 1: Copy Template
Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`** to a class in your own package (suggested name: **`ZCL_##_SELECT`**, where ## stands for your group number).
   a) Open the source code of the global class **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`**.
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click class **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`** to open the context menu.
   d) From the context menu, choose *Duplicate ...*.
   e) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_SELECT`**, where ## stands for your group number.
   f) Adjust the description and choose *Next*.
   g) Confirm the transport request and choose *Finish*.

Task 2: Declare Additional Attributes
Extend the local class **`lcl_connection`** with the private instance attributes **`airport_from_id`** and **`airport_to_id`**. Add some output for the new attributes to the implementation of the method **`get_output`**.

Note:
The new attributes are not filled yet. In order to fill them we will add a SELECT statement to the **`constructor`** in one of the next tasks of this exercise.

1. Switch to the local class **`lcl_connection`**.
   a) In the global class, choose *Local Types*.
2. Add the following **private** attributes to the class definition:

© Copyright. All rights reserved.

Table 2: Attributes

Attribute Name	Scope	Data Type
airport_from_id	instance	/DMO/AIRPORT_FROM_ID
airport_to_id	instance	/DMO/AIRPORT_TO_ID

a) Adjust the code as follows:

```
  PRIVATE SECTION.
    DATA carrier_id    TYPE /dmo/carrier_id.
    DATA connection_id TYPE /dmo/connection_id.

    DATA airport_from_id TYPE /dmo/airport_from_id.
    DATA airport_to_id   TYPE /dmo/airport_to_id.

ENDCLASS.
```

3. Extend the implementation of the method **get_output**. Append more string templates to the returning parameter **r_output**. Embed the new attributes as expressions into the string templates.
   a) Navigate to the implementation of the method **get_output**.
   b) Adjust the code as follows:

```
    APPEND |--------------------------------|            TO r_output.
    APPEND |Carrier:     { carrier_id      }|            TO r_output.
    APPEND |Connection:  { connection_id   }|            TO r_output.
    APPEND |Departure:   { airport_from_id }|            TO r_output.
    APPEND |Destination: { airport_to_id   }|            TO r_output.
```

4. Activate the class. Execute it and analyze the console output.
   a) Press **Ctrl + F3** to activate the class.
   b) Press **F9** to run the class.

Task 3: Analyze the Database Table

Analyze the definition of the database table **/DMO/CONNECTION**.

1. Open the development object that contains the definition of the database table **/DMO/CONNECTION**.
   a) From the eclipse toolbar, choose *Open ABAP Development Object* or press **Ctrl + Shift + A**.
   b) In the input field, enter **/dmo/con** as a search string.
   c) In the list of matching items, click on */DMO/CONNECTION (Database Table)* and choose *OK*.
2. Open the *Element Information* (F2) for the database table /DMO/CONNECTION.
   a) Click on `/dmo/connection` after the keyword `define table` and press **F2** to show the element information.

© Copyright. All rights reserved.

Which fields have the description *Flight Reference Scenario: From Airport* and *Flight Reference Scenario: To Airport*?

The field names are **airport_from_id** and **airport_to_id**.

Task 4: Read Data from the Database

In the method **constructor** of the local class **lcl_connection**, implement a SELECT statement that reads values for the new attributes from the database table **/DMO/CONNECTION**.

1. Return to the local class **lcl_connection** in your global class.
   a) In the editor view of Eclipse, open tab *ZCL_##_SELECT*.
   b) In the global class, choose *Local Types*.
2. Navigate to the implementation of the method **constructor**.
   a) Search for the code line `METHOD constructor.`.
3. After the `ENDIF.` statement, add a SELECT statement that reads a single record from database table **/DMO/CONNECTION**.
   a) Adjust the code as follows:

```
IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.

SELECT SINGLE
  FROM /dmo/connection
```

4. Implement the FIELDS clause. Read the table fields **airport_from_id** and **airport_to_id**.

Hint:
Use auto-completion (**Ctrl + Space**) to enter the field names.

   a) After `FROM /DMO/CONNECTION` enter `FIELDS`.
   b) After a blank, press **Ctrl + Space** and choose *airport_from_id*.
   c) After a comma and a blank press **Ctrl + Space** again and choose *airport_to_id*.
   d) The complete FIELDS clause should look like this:

```
    FIELDS airport_from_id, airport_to_id
```

5. Implement the WHERE condition. Restrict all key fields of the database table (except for the client field) with the values of importing parameters **i_carrier_id** and **i_connection_id**. Do not forget to escape the parameters with prefix @.

© Copyright. All rights reserved.

Hint:
Use auto-completion (**Ctrl + Space**) to enter the element names and parameter names.

a) Add the following code after the FIELDS clause:

```
  WHERE carrier_id    = @i_carrier_id
    AND connection_id = @i_connection_id
```

Why is it not necessary to specify the client field in the WHERE condition?

Client handling is performed by the compiler.

6. Implement the INTO clause. Store the SELECT result in the attributes **airport_from_id**, and **airport_to_id**. Do not forget to escape the attributes with prefix @.

Hint:
Use auto-completion (**Ctrl + Space**) to enter the attribute names.

a) Adjust the code as follows:

```
  WHERE carrier_id    = @i_carrier_id
    AND connection_id = @i_connection_id
  INTO ( @airport_from_id, @airport_to_id ).
```

b) The complete SELECT statement should look like this:

```
SELECT SINGLE
      FROM /dmo/connection
    FIELDS airport_from_id, airport_to_id
     WHERE carrier_id    = @i_carrier_id
       AND connection_id = @i_connection_id
      INTO ( @airport_from_id, @airport_to_id ).
```

7. Implement error handling after the SELECT statement. Check the content of the system field **sy-subrc**. If it does not equal zero, raise exception **CX_ABAP_INVALID_VALUE**.

a) Add the following code after the SELECT statement:

```
  IF sy-subrc <> 0.
     RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.
```

8. Activate the class. Execute it and analyze the console output. Check that the output for the new attributes displays data.

a) Press **Ctrl + F3** to activate the class.

b) Press **F9** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Describe basic features of ABAP SQL.
- Read single values from the database.

© Copyright. All rights reserved.

Unit 4
Lesson 3

Working with CDS Views

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Analyze a CDS view definition.
- Read data using a CDS view.

CDS Views

View Definitions in ABAP Core Data Services (ABAP CDS)

ABAP Core Data Services, or ABAP CDS for short, is an infrastructure for defining and consuming semantically rich data models in ABAP. They use a CDS DDL to define CDS entities that implement a semantic data model. The most important CDS modeling entities are CDS View entities.

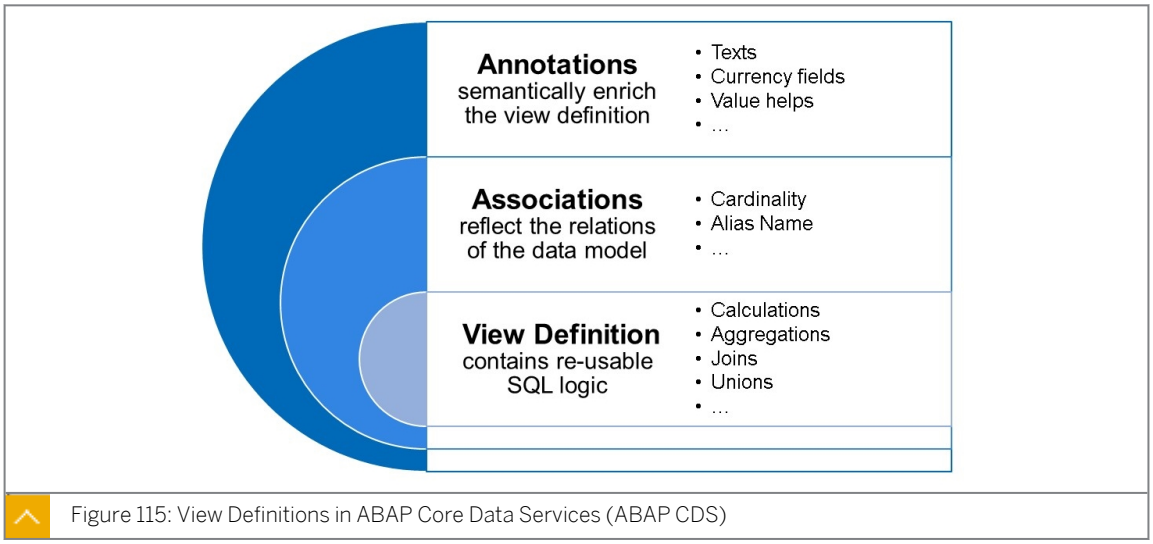


Figure 115: View Definitions in ABAP Core Data Services (ABAP CDS)

The CDS View definition contains re-usable SQL logic; sometimes as simple as a projection of table fields and sometimes more sophisticated with calculations, aggregations, joins, unions, and so on.

A CDS View definition can contain associations to reflect the relations of the data model. Consumers of the view can use the associations to retrieve related data.

Finally, annotations are used to semantically enrich the view definition. This metadata is evaluated by frameworks that build on top of CDS View definitions. One such framework is the ABAP RESTful application programming model , for which we will discuss an example later in this course.

Let's watch examples of CDS view definitions.

© Copyright. All rights reserved.

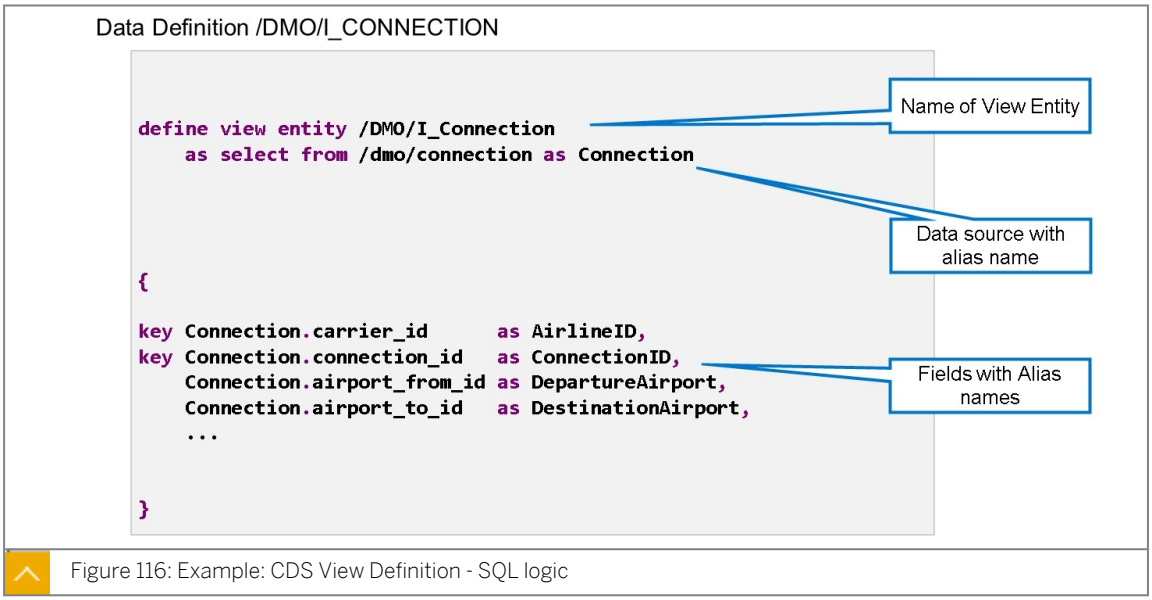


Figure 116: Example: CDS View Definition - SQL logic

CDS View definitions are contained in repository objects of type Data Definition. Let us now have a look at the source code of data definition /DMO/I_CONNECTION.

The main part is the DEFINE VIEW ENTITY statement. It contains the name of the CDS View entity and, after keyword FROM, the data source. In our example, the name of the view entity is /DMO/I_Connection and the data source is database table /dmo/connection. Optional addition AS, defines an alias name Connection to address the data source inside the View definition.

Note:
The source of a CDS View entity could also be another CDS View.

A pair of curly brackets contains the list of view elements. In our example, the view elements are fields of database table /dmo/connection. Keyword Key in front of the first two elements defines them as key fields of the CDS View entity. Optional addition AS defines an alias name for each view element.

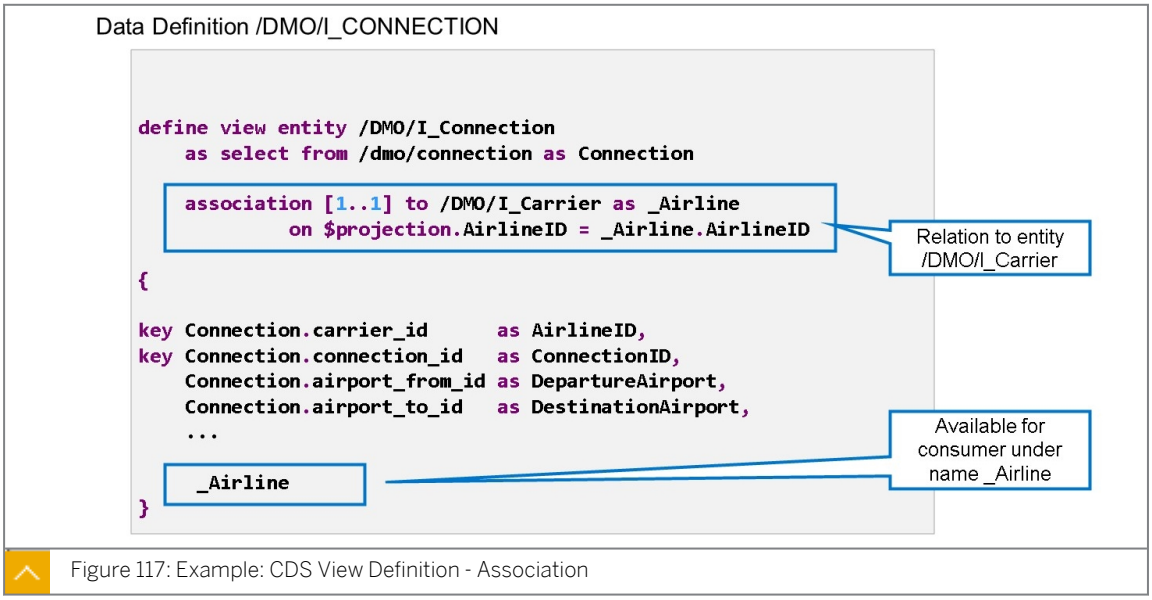


Figure 117: Example: CDS View Definition - Association

© Copyright. All rights reserved.

Addition `association` defines a relation to another CDS view entity. In our example, the related view is CDS view entity /DMO/I_Carrier and the name of the association is _Airline.

This association becomes available for consumers of the view by adding it to the element list. This is referred to as exposing the association.

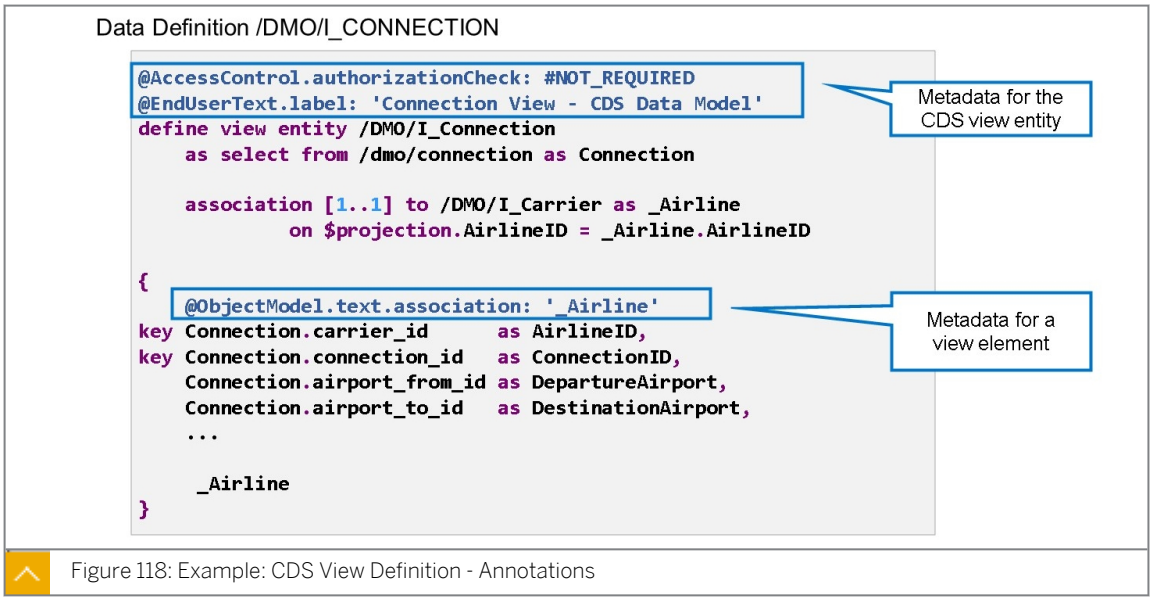


Figure 118: Example: CDS View Definition - Annotations

Annotations start with the at sign (@) and they are used to semantically enrich the view definition for consumers. Annotations before the view definition are called entity annotations. Entity annotations are used to define metadata for the view entity as a whole. Annotations between the curly brackets are called element annotations. Element annotations are used to define metadata for the different elements of the view.

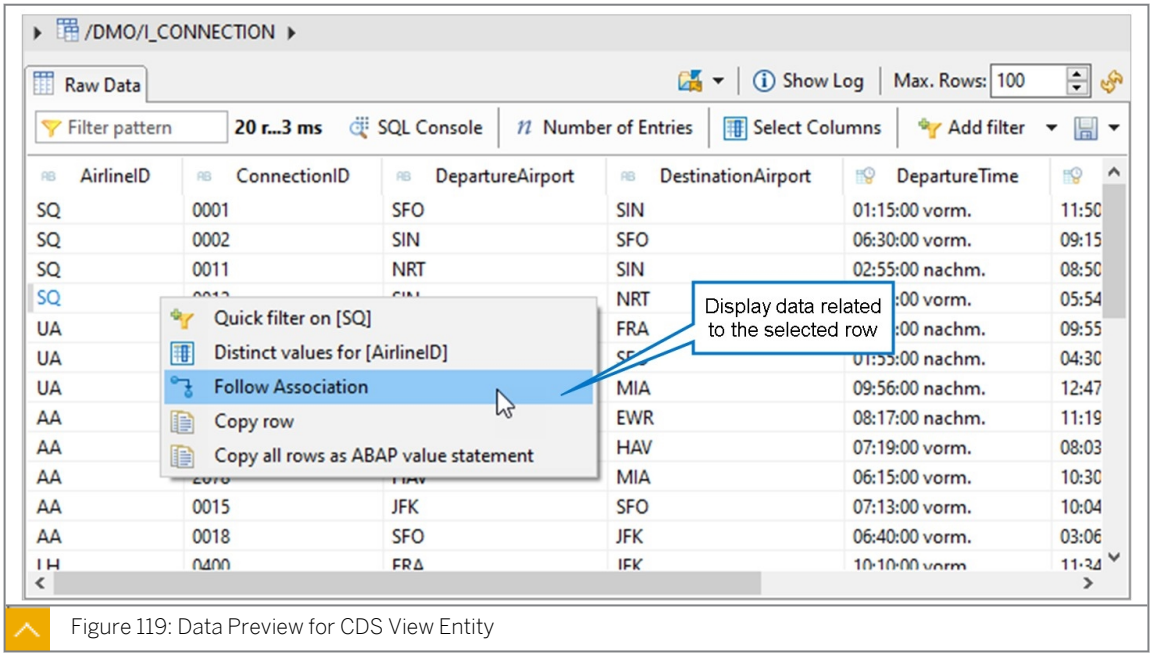


Figure 119: Data Preview for CDS View Entity

You already learned how to use the *Data Preview* tool to display and analyze the content of a database table. This tool is also available for CDS view entities.

© Copyright. All rights reserved.

To open the *Data Preview* for a given CDS entity, right-click anywhere in the data definition and choose *Open With → Data Preview*. Alternatively, place the cursor anywhere in the data definition and press *F8*.

The tool displays the data returned by the CDS entity. The same functions are available to sort or filter the data and adjust the display.

If the view definition contains one or more associations, you can use them to display related data. To do so, proceed in the following manner:

1. Right-click on a row in the display.
2. From the context menu, choose *Follow Association*.
3. From the list of available associations, choose the one in which you are interested.

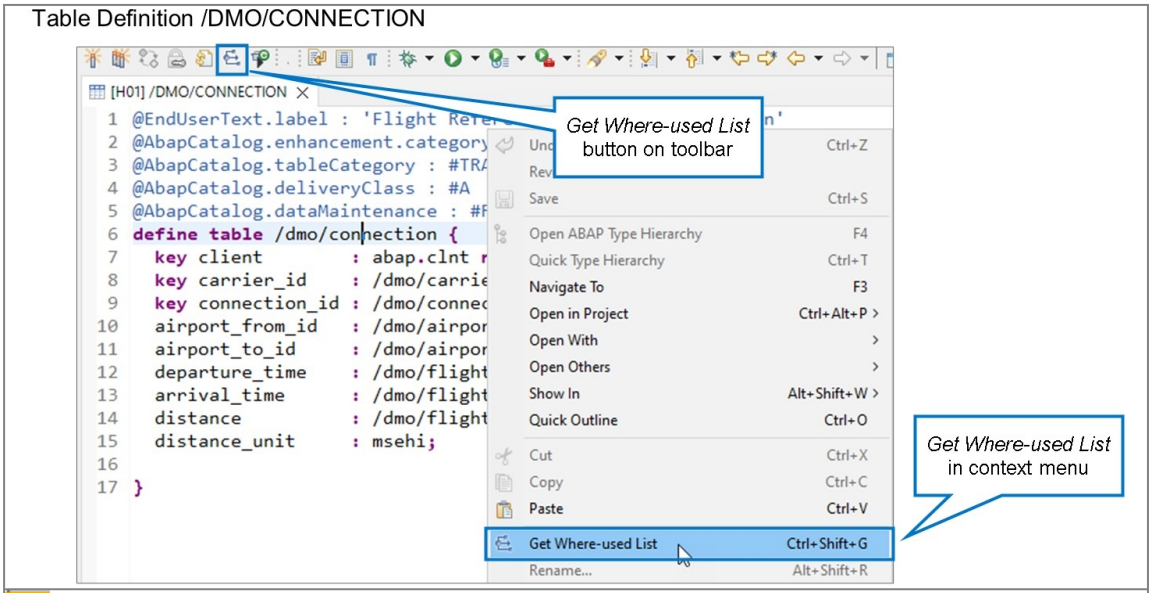


Figure 120: Finding CDS Views Based on a Database Table

If you want to find all CDS Views with a certain database table as a source, you can utilize the Where-used List tool of ADT. To use this tool, proceed as follows:

1. Open the definition of the database table.
2. Right-click anywhere in the source code and choose *Get Where-used List* from the context menu. Alternatively, you can press Ctrl + Shift + G, or choose the button from the toolbar with the same symbol.
3. The *Search* view displays a list of all development objects that directly use the database table.

© Copyright. All rights reserved.

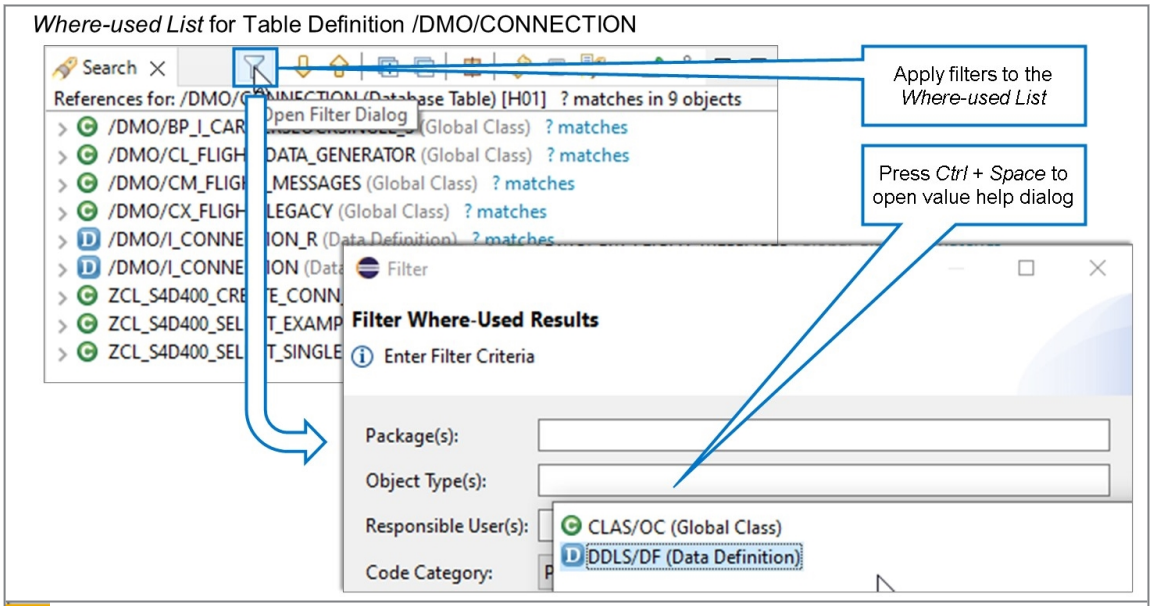


Figure 121: Applying Filters to the Where-used List

You can apply filters to the Where-used List if, for example, you are only interested in objects from certain packages or objects of the specific object type. The example illustrates how to filter for CDS views that use the table.

CDS Views in ABAP SQL

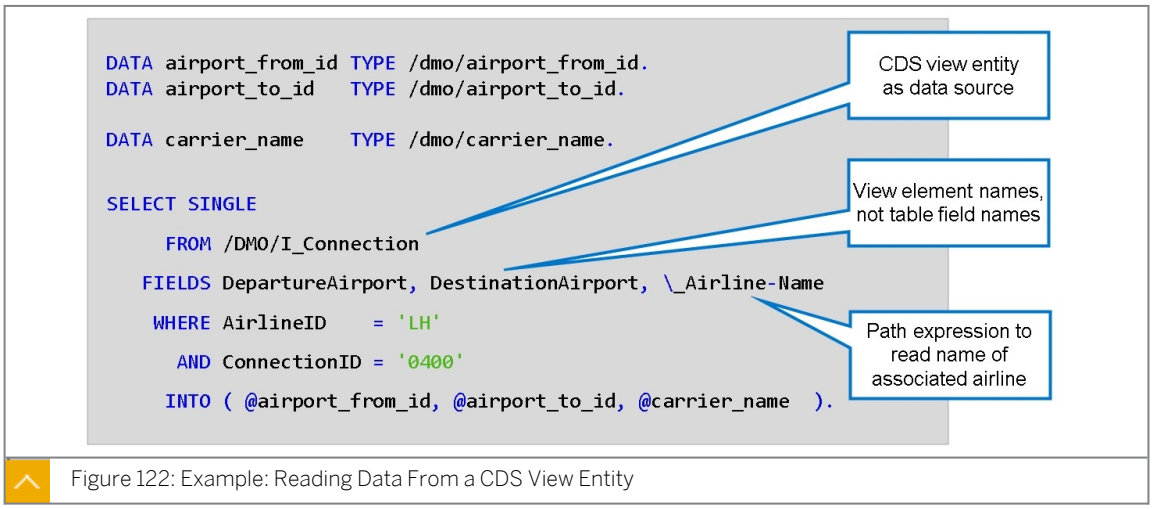


Figure 122: Example: Reading Data From a CDS View Entity

When you implement a SELECT statement in ABAP, you can use a CDS view entity as the data source instead of reading from the database table directly. This has several advantages.

- Re-use of the SQL logic contained in the CDS view
- Concise, easy-to-read reading of related data using the associations
- Sometimes the names of views and view elements are better readable than the more technical names of database tables and table fields

The SELECT statement in the example uses CDS view entity /DMO/I_Connection as a data source.

© Copyright. All rights reserved.

Note:
Note that the names used in the FIELDS clause and WHERE conditions are the alias names of the view elements.

The third element in the FIELDS clause makes use of exposed association _Airline. It reads element `Name` from the associated CDS view entity /DMO/I_Carrier. This kind of element is called a path expression. The backslash (\) is a mandatory prefix for association names.

© Copyright. All rights reserved.

Unit 4
Exercise 13

Analyze and Use a CDS View Entity

In this exercise, you read from a CDS view entity rather than from the database table directly. This provides you with comfortable access to the name of the airline as well.

Template:

/LRN/CL_S4D400_DBS_SELECT (global Class)

Solution:

/LRN/CL_S4D400_DBS_CDS (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_SELECT`**.

1. Copy the class **`/LRN/CL_S4D400_DBS_SELECT`** to a class in your own package (suggested name: **`ZCL_##_CDS`**, where ## stands for your group number).

Task 2: Analyze CDS View Entity
Analyze the definition of the CDS view entity **`/DMO/I_Connection`**.

1. Open the development object that contains the definition of the CDS view entity **`/DMO/I_Connection`**.
2. Analyze the element list of the CDS view entity.

   Which are the alias names for fields **`airport_from_id`** and **`airport_to_id`**?

3. Open the *Element Information* (F2) for the target of association **`_Airline`**.

   Which element is described as *Flight Reference Scenario: Carrier Name*? What is the element type?

Task 3: Declare Additional Attribute
Extend the local class **`lcl_connection`** with a private instance attribute **`carrier_name`**. Add some output for the new attribute to the implementation of method **`get_output( )`**.

© Copyright. All rights reserved.

Note:
The new attribute is not filled, yet. In order to fill it we will adjust the SELECT statement in the next task of this exercise.

1. Return to the local class **lcl_connection** in your global class.
2. Add the following **private** attribute to the class definition:

   Table 3: Attributes

Attribute Name	Scope	Data Type
carrier_name	instance	/DMO/CARRIER_NAME

3. Extend the implementation of method **get_output**. Add the carrier name to the string template with the carrier identification.
4. Activate the class. Execute it and analyze the console output.

Task 4: Use CDS View Entity

In the implementation of method **constructor**, replace the SELECT statement that reads from database table **/dmo/connection** with a SELECT statement that reads from CDS view entity **/DMO/I_Connection**.

1. In the implementation of method **constructor**, comment the SELECT statement.
2. After the commented code, add a SELECT statement that reads a single record from CDS view **/DMO/I_Connection**.
3. Implement the FIELDS clause. Read view elements **DepartureAirport** and **DestinationAirport**. In addition, read view element **Name** from the target of association **_Airline**.

Hint:
Use auto-completion (**Ctrl + Space**) to enter the names.

4. Implement the WHERE condition. Restrict the key elements of CDS view entity with the values of importing parameters **i_carrier_id** and **i_connection_id**. Do not forget to escape the parameters with prefix @.

Hint:
Use auto-completion (**Ctrl + Space**) to enter the element names and parameter names.

© Copyright. All rights reserved.

Is it possible to add the client field to the WHERE condition?

5. Implement the INTO clause. Store the SELECT result in attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`**. Do not forget to escape the attributes with prefix @.

Hint:
Use auto-completion (**`Ctrl + Space`**) to enter the attribute names.

6. Activate the class. Execute it and analyze the console output. Check that the output for the new attribute displays data.

© Copyright. All rights reserved.

Unit 4
Solution 13

Analyze and Use a CDS View Entity

In this exercise, you read from a CDS view entity rather than from the database table directly. This provides you with comfortable access to the name of the airline as well.

Template:

/LRN/CL_S4D400_DBS_SELECT (global Class)

Solution:

/LRN/CL_S4D400_DBS_CDS (global Class)

Task 1: Copy Template (Optional)
Copy the template class. If you finished the previous exercise, you can skip this task and continue editing your class **`ZCL_##_SELECT`**.

1. Copy the class **`/LRN/CL_S4D400_DBS_SELECT`** to a class in your own package (suggested name: **`ZCL_##_CDS`**, where ## stands for your group number).
   a) Open the source code of the global class **`/LRN/CL_S4D400_DBS_SELECT`**.
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click the class **`/LRN/CL_S4D400_DBS_SELECT`** to open the context menu.
   d) From the context menu, choose *Duplicate ...*.
   e) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_CDS`**, where ## stands for your group number.
   f) Adjust the description and choose *Next*.
   g) Confirm the transport request and choose *Finish*.

Task 2: Analyze CDS View Entity
Analyze the definition of the CDS view entity **`/DMO/I_Connection`**.

1. Open the development object that contains the definition of the CDS view entity **`/DMO/I_Connection`**.
   a) From the eclipse toolbar, choose *Open ABAP Development Object* or press **`Ctrl + Shift + A`**.
   b) In the input field, enter **`/DMO/I_Con`** as a search string.
   c) In the list of matching items, click on */DMO/I_CONNECTION (Data Definition)* and choose *OK*.
2. Analyze the element list of the CDS view entity.

© Copyright. All rights reserved.

Which are the alias names for fields **airport_from_id** and **airport_to_id**?

The alias names are **DepartureAirport** and **DestinationAirport**.

a) Navigate to the comma-separated list between the pair of curly brackets ({ }).

b) You find the alias names after keyword `AS`.

3. Open the *Element Information* (F2) for the target of association **_Airline**.
   a) Click on `/DMO/I_Carrier` before `as _Airline` and press **F2** to show the element information.

   Which element is described as *Flight Reference Scenario: Carrier Name*? What is the element type?

   The element name is **Name** and the element type is **/dmo/carrier_name**.

Task 3: Declare Additional Attribute

Extend the local class **lcl_connection** with a private instance attribute **carrier_name**. Add some output for the new attribute to the implementation of method **get_output( )**.

Note:
The new attribute is not filled, yet. In order to fill it we will adjust the SELECT statement in the next task of this exercise.

1. Return to the local class **lcl_connection** in your global class.
   a) In the global class, choose *Local Types*.
2. Add the following **private** attribute to the class definition:

   Table 3: Attributes

Attribute Name	Scope	Data Type
carrier_name	instance	/DMO/CARRIER_NAME

   a) Adjust the code as follows:

```
  PRIVATE SECTION.
    DATA carrier_id      TYPE /dmo/carrier_id.
    DATA connection_id   TYPE /dmo/connection_id.

    DATA airport_from_id TYPE /dmo/airport_from_id.
    DATA airport_to_id   TYPE /dmo/airport_to_id.

    DATA carrier_name    TYPE /dmo/carrier_name.
ENDCLASS.
```

3. Extend the implementation of method **get_output**. Add the carrier name to the string template with the carrier identification.
   a) Navigate to the implementation of method **get_output**.

© Copyright. All rights reserved.

b) Replace statement `APPEND |Carrier: { carrier_id }| TO r_output.` with the following statement:

```
APPEND |Carrier: { carrier_id } { carrier_name }| TO r_output.
```

4. Activate the class. Execute it and analyze the console output.
   a) Press **Ctrl + F3** to activate the class.
   b) Press **F9** to run the class.

Task 4: Use CDS View Entity

In the implementation of method **constructor**, replace the SELECT statement that reads from database table **/dmo/connection** with a SELECT statement that reads from CDS view entity **/DMO/I_Connection**.

1. In the implementation of method **constructor**, comment the SELECT statement.
   a) Select all lines that belong to the `SELECT SINGLE ... .`.
   b) Press **Ctrl + <** to add a star sign (*) in front of each selected line.
2. After the commented code, add a SELECT statement that reads a single record from CDS view **/DMO/I_Connection**.
   a) Adjust the code as follows:

```
*    SELECT SINGLE
*      FROM /dmo/connection
*    FIELDS airport_from_id, airport_to_id
*     WHERE carrier_id    = @i_carrier_id
*       AND connection_id = @i_connection_id
*      INTO ( @airport_from_id, @airport_to_id ).

  SELECT SINGLE
    FROM /DMO/I_Connection
```

3. Implement the FIELDS clause. Read view elements **DepartureAirport** and **DestinationAirport**. In addition, read view element **Name** from the target of association **_Airline**.

> Hint:
> Use auto-completion (**Ctrl + Space**) to enter the names.

   a) After `FROM /DMO/I_Connection` enter `FIELDS`.
   b) After a blank, press **Ctrl + Space** and choose *DepartureAirport*.
   c) After a comma and a blank press **Ctrl + Space** again and choose **DestinationAirport**.
   d) After a comma and a blank, type in a backslash (\) , press **Ctrl + Space** and choose *_Airline*.
   e) Immediately after `_Airline`, type in a dash sign (-) , press **Ctrl + Space** and choose *Name*.

© Copyright. All rights reserved.

f) The complete FIELDS clause should look like this:

```
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
```

4. Implement the WHERE condition. Restrict the key elements of CDS view entity with the values of importing parameters **`i_carrier_id`** and **`i_connection_id`**. Do not forget to escape the parameters with prefix @.

Hint:
Use auto-completion (**`Ctrl + Space`**) to enter the element names and parameter names.

a) Add the following code after the FIELDS clause:

```
  WHERE AirlineID    = @i_carrier_id
    AND ConnectionID = @i_connection_id
```

Is it possible to add the client field to the WHERE condition?

No, the element list of the CDS view entity does not contain the client field.

5. Implement the INTO clause. Store the SELECT result in attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`**. Do not forget to escape the attributes with prefix @.

Hint:
Use auto-completion (**`Ctrl + Space`**) to enter the attribute names.

a) Adjust the code as follows:

```
  WHERE AirlineID    = @i_carrier_id
    AND ConnectionID = @i_connection_id
   INTO ( @airport_from_id, @airport_to_id, @carrier_name ).
```

b) The complete SELECT statement should look like this:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
     WHERE AirlineID    = @i_carrier_id
       AND ConnectionID = @i_connection_id
      INTO ( @airport_from_id, @airport_to_id, @carrier_name ).
```

6. Activate the class. Execute it and analyze the console output. Check that the output for the new attribute displays data.

a) Press **`Ctrl + F3`** to activate the class.

b) Press **`F9`** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Analyze a CDS view definition.
- Read data using a CDS view.

© Copyright. All rights reserved.

Learning Assessment

1. What is the shortcut to preview the content of a database table?

   Choose the correct answer.

   - [ ] **A** F1
   - [ ] **B** F2
   - [ ] **C** F3
   - [ ] **D** F8

2. Which part of SQL is represented by ABAP SQL?

   Choose the correct answer.

   - [ ] **A** Data Control Language
   - [ ] **B** Data Definition Language
   - [ ] **C** Data Manipulation Language

3. What is the correct sequence of clauses in the SELECT SINGLE statement?

   Choose the correct answer.

   - [ ] **A** SELECT SINGLE FIELDS... FROM... INTO... WHERE...
   - [ ] **B** SELECT SINGLE FROM... FIELDS... WHERE... INTO
   - [ ] **C** SELECT SINGLE FROM... FIELDS... INTO... WHERE...
   - [ ] **D** You can write the clauses in any sequence.

4. Which of the following are valid data sources for a CDS view entity?

   Choose the correct answers.

   - [ ] **A** An internal table
   - [ ] **B** A database table
   - [ ] **C** Another CDS view entity
   - [ ] **D** A structure

© Copyright. All rights reserved.

5. You are writing a SELECT statement that reads data using a CDS view entity. In the field list, you want to read a field from an associated table. What do you use?

   Choose the correct answer.

   - ☐ A A logical expression
   - ☐ B A path expression
   - ☐ C A regular expression

© Copyright. All rights reserved.

Learning Assessment - Answers

1. What is the shortcut to preview the content of a database table?

   Choose the correct answer.

   - [ ] A F1
   - [ ] B F2
   - [ ] C F3
   - [x] D F8

   Correct! The shortcut to preview the content of a database table is F8.

2. Which part of SQL is represented by ABAP SQL?

   Choose the correct answer.

   - [ ] A Data Control Language
   - [ ] B Data Definition Language
   - [x] C Data Manipulation Language

   That's right.

3. What is the correct sequence of clauses in the SELECT SINGLE statement?

   Choose the correct answer.

   - [ ] A SELECT SINGLE FIELDS... FROM... INTO... WHERE...
   - [x] B SELECT SINGLE FROM... FIELDS... WHERE... INTO
   - [ ] C SELECT SINGLE FROM... FIELDS... INTO... WHERE...
   - [ ] D You can write the clauses in any sequence.

   That's right!

© Copyright. All rights reserved.

4. Which of the following are valid data sources for a CDS view entity?

   Choose the correct answers.

   - [ ] A An internal table
   - [X] B A database table
   - [X] C Another CDS view entity
   - [ ] D A structure

   That's right.

5. You are writing a SELECT statement that reads data using a CDS view entity. In the field list, you want to read a field from an associated table. What do you use?

   Choose the correct answer.

   - [ ] A A logical expression
   - [X] B A path expression
   - [ ] C A regular expression

   That's right

© Copyright. All rights reserved.

UNIT 5 Working with Structured Data Objects

Lesson 1

Declaring a Structured Data Object 229

Lesson 2

Working with Structured Data Objects 235

Exercise 14: Use a Structured Data Object 245

UNIT OBJECTIVES

- Declare a structured data object.
- Work with structured data objects.
- Use structured data objects in ABAP SQL.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 5
Lesson 1

Declaring a Structured Data Object

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Declare a structured data object.

Structured Data Objects

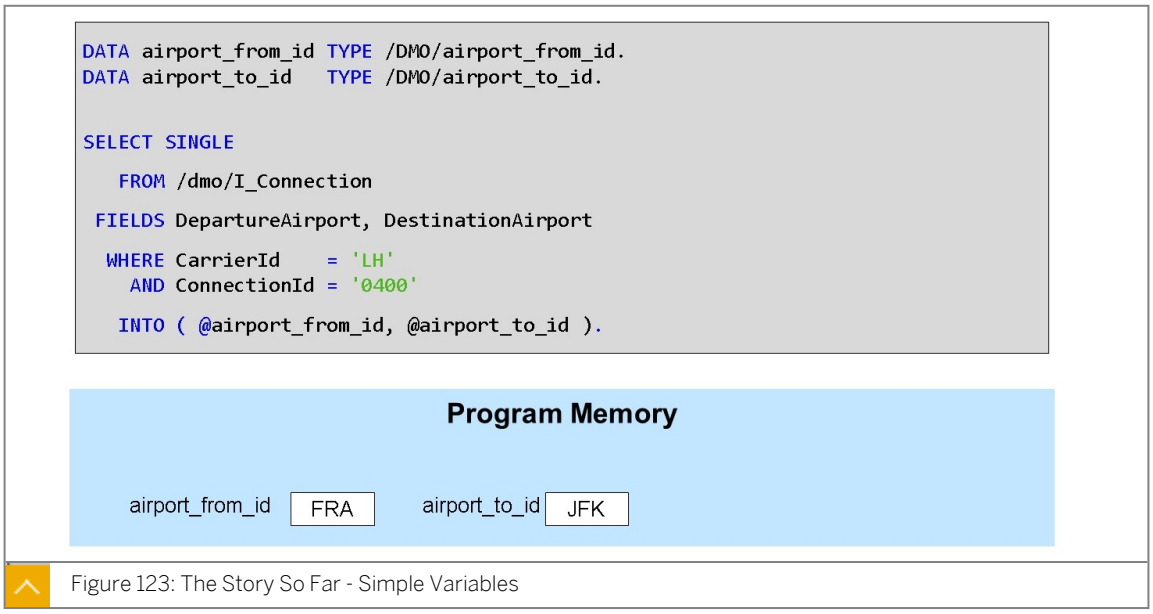


Figure 123: The Story So Far - Simple Variables

Up until now, you have been using simple variables, each of which can store a single piece of information. Here, for example, there are two variables, one for the departure airport, the other for the arrival airport.

When you read a record from the database you need to hold all of this information together. The two variables in the example are completely independent of one another, and are therefore not suitable for storing different pieces of information that belong together.

In ABAP, the solution is to use a structured variable, or a structure, for short.

© Copyright. All rights reserved.

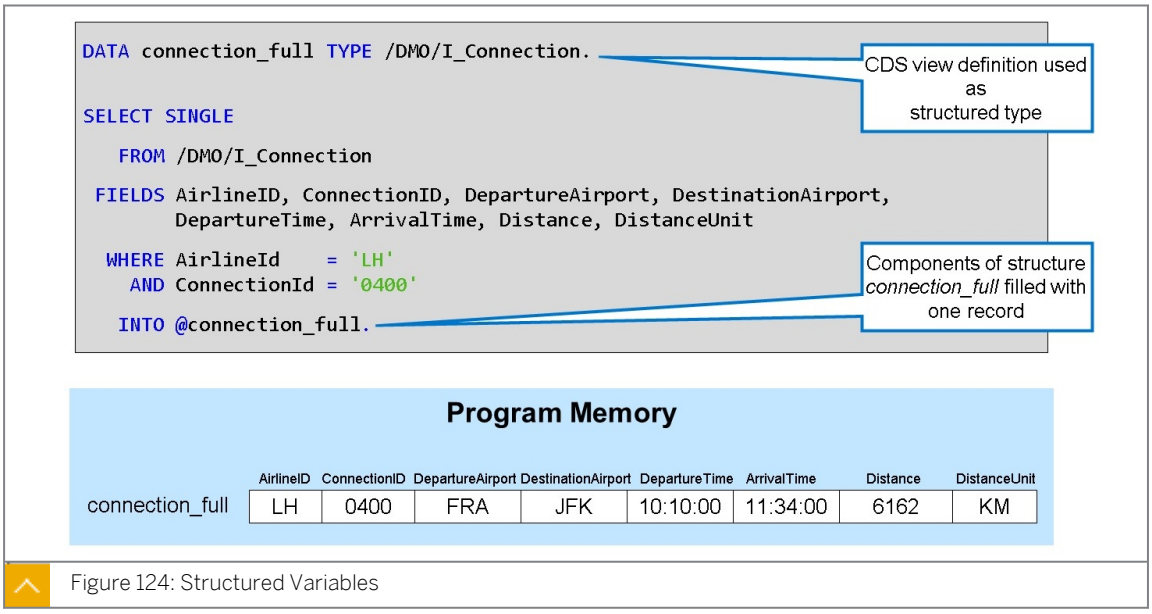


Figure 124: Structured Variables

A structure is a variable ABAP data object with a name and a structured type.

In the example, data object connection_full is such a structure. It is subdivided into eight components, each of which also has a name and a type. You can address both the structure as a whole and the individual components. Importantly, you can use each component in exactly the same way that you would use a standalone simple variable.

There are various possibilities to declare a structure. You can define structured types with keyword TYPES or use a repository object of type Structure. The definitions of views and database tables can also serve as structured types. The example uses CDS View /DMO/I_Connection as a structured type.

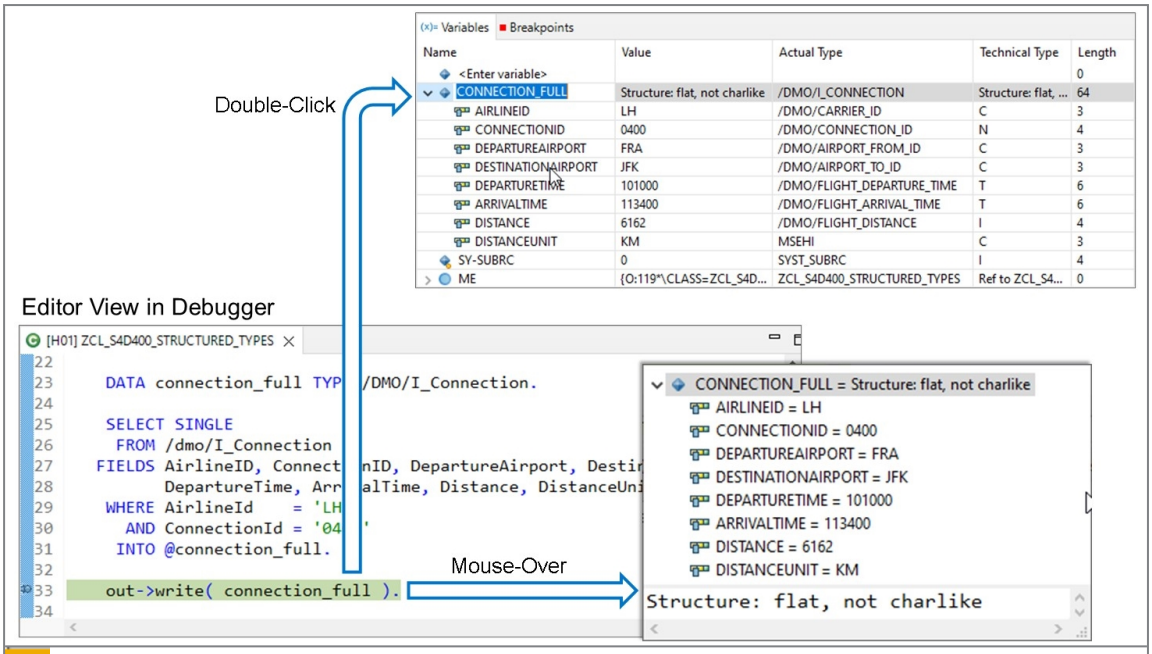


Figure 125: Structured Variables in the Debugger Perspective

In the *Debugger* perspective, there are two ways to analyze the structure and content of a structured variable:

© Copyright. All rights reserved.

Variable Preview (Mouse-Over)

Set the focus on the source code editor and place the pointer on a variable name. After a moment a dialog window opens with details on the structure and content of the data object.

Display in the *Variables* View (Double-Click)

To display the variable in the *Variables* view, either enter the variable name under <*Enter variable*> or double-click the variable name somewhere in the source code editor. Expand the node with the structure name to see the list of components.

Structured Data Types

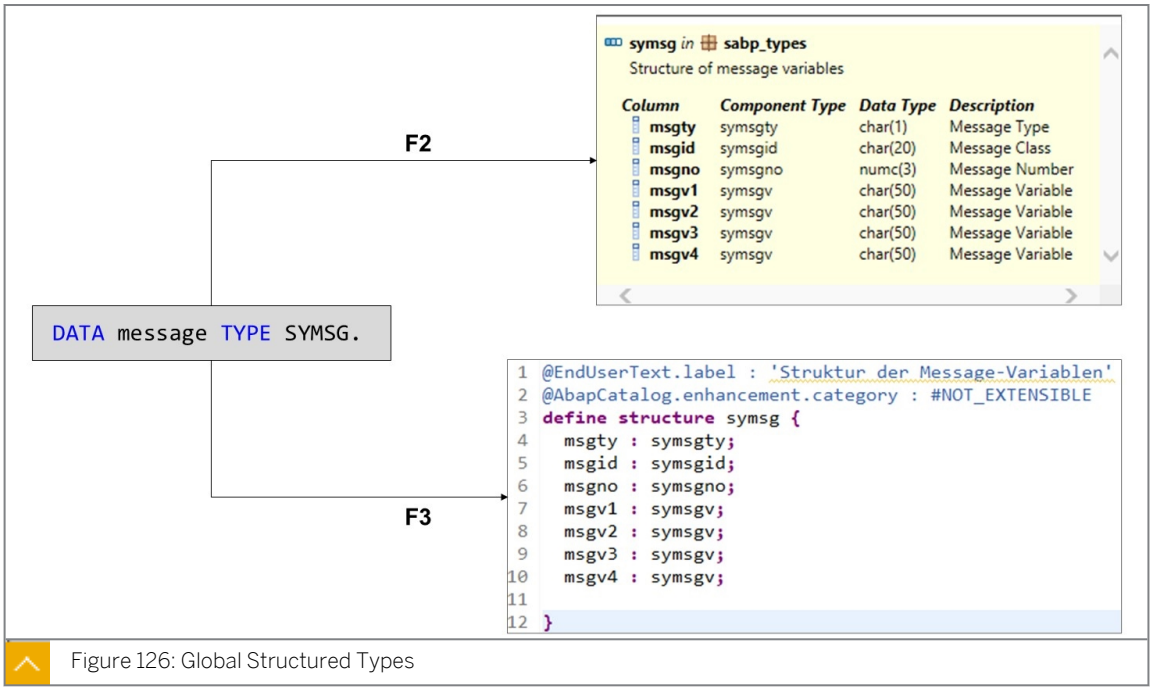


Figure 126: Global Structured Types

A global structure type is a repository object that can be used as data type anywhere in the system. In the example, structured type SYMSG is used to declare variable message.

When you press the **F2** key to display the details of this data type you can see that this is a structure type consisting of seven components. You can also see the names, technical types and descriptions of the components.

When you press the **F3** key to navigate to the definition of the type, a new view opens with the ADT editor for global structured types.

The definition of a global structure type is very similar to the definition of a database table. The main part of the definition consists of a DEFINE STRUCTURE statement with the name of the structure type. This is followed by the list of structure components in a pair of curly brackets ({ , }); each component with a component type. Component types are often described by data elements, but it is also possible to use structure types as component types. Structures with structured components are referred to as Nested Structures.

Additional code lines before the DEFINE STRUCTURE statement specify additional properties of the structure type, among them a label.

© Copyright. All rights reserved.

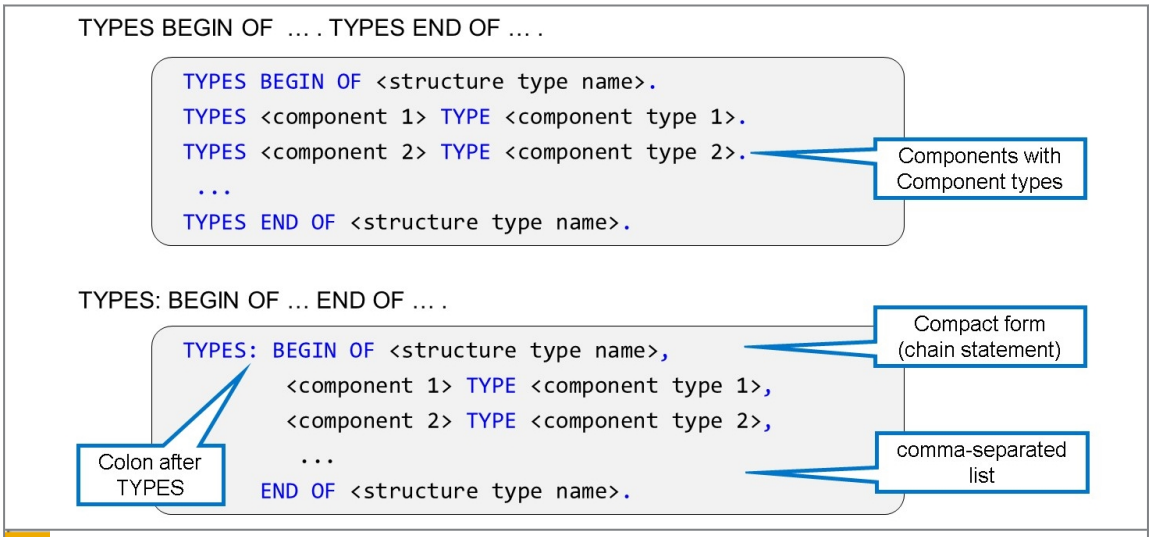


Figure 127: Defining Structured Types Locally

You can also define structured types in an ABAP program using the TYPES statement. The structure definition begins with the statement TYPES BEGIN OF <structure type name> and ends with TYPES END OF <structure type name>. In between, you name each component and specify its type in an additional TYPES statement.

A compact form uses keyword TYPES only once, followed by a colon(:). The BEGIN OF addition, the END OF addition and the component definitions in between are then separated by commas.

This is referred to as chain statement.

Note:
In the past, chain statements were used a lot in ABAP. Nowadays they are only recommended to combine statements that belong closely together.

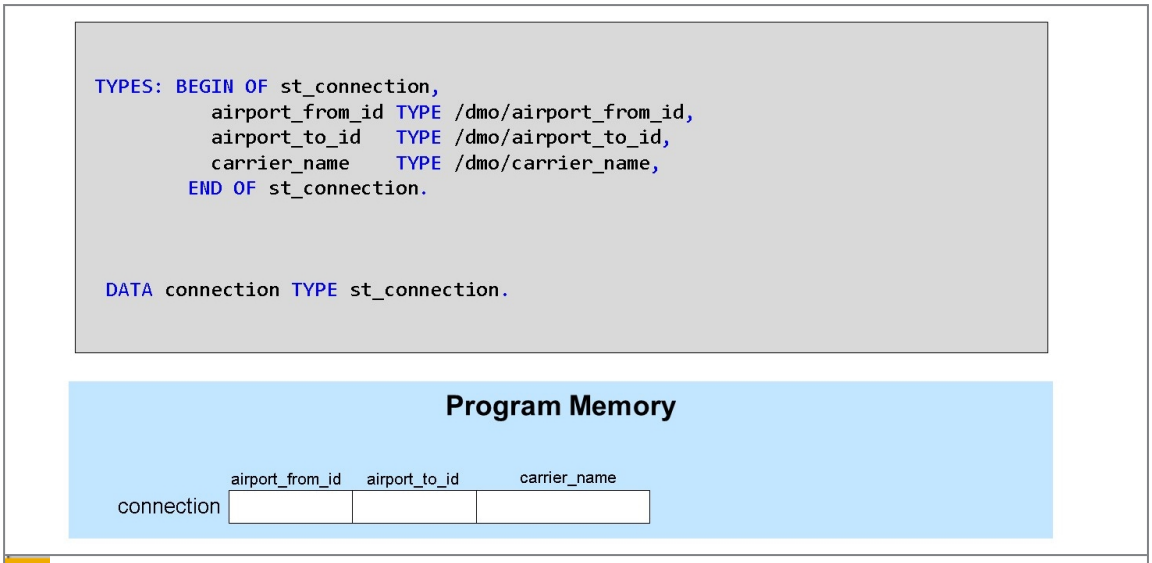


Figure 128: Example: Local Structured Type

© Copyright. All rights reserved.

In this example, a chain statement TYPES: is used to define local structured type st_connection which consists of the three components airport_from_id, airport_to_id, and carrier_name. Each component is typed with a data element beginning with /dmo/.

Local structured type st_connection is then used in a DATA statement to type structured variable connection.

```
TYPES: BEGIN OF st_nested,
         airport_from_id TYPE /dmo/airport_from_id,
         airport_to_id   TYPE /dmo/airport_to_id,
         message         TYPE symsg,
         carrier_name    TYPE /dmo/carrier_name,
       END OF st_nested.


 DATA connection TYPE st_nested.
```

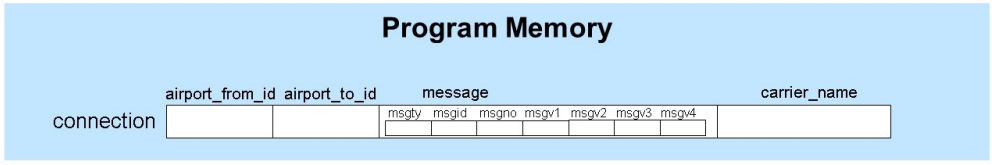


Figure 129: Example: Nested Local Structured Type

In this example, variable `connection` has a nested structured type. Type `st_nested` defines 4 components. Three are typed with data elements, therefore they are simple components. The component `message`, however, is typed with a structured type. Therefore it is a structured component. This makes variable `connection` a nested structure.

Public Section of Class CL_ABAP_BEHV

```
CONSTANTS:
  BEGIN OF co_key_name,
    draft   TYPE seckeyname VALUE 'DRAFT',
    cid     TYPE seckeyname VALUE 'CID',
    pid     TYPE seckeyname VALUE 'PID',
    entity  TYPE seckeyname VALUE 'ENTITY',
    id      TYPE seckeyname VALUE 'ID',
  END OF co_key_name.
```

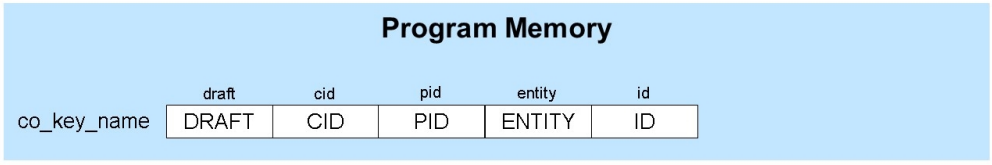


Figure 130: Structured Constants

ABAP supports not only structured variables but also structured constants. To define a structured constant, use BEGIN OF and END OF as part of a CONSTANTS statement. The example shows a structured constant that is defined in the public section of global class

© Copyright. All rights reserved.

CL_ABAP_BEHV. All five components are typed with data element SECKEYNAME. Remember that addition VALUE is mandatory when defining constants.

LESSON SUMMARY

You should now be able to:

- Declare a structured data object.

© Copyright. All rights reserved.

Unit 5
Lesson 2

Working with Structured Data Objects

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Work with structured data objects.
- Use structured data objects in ABAP SQL.

Structured Data Objects

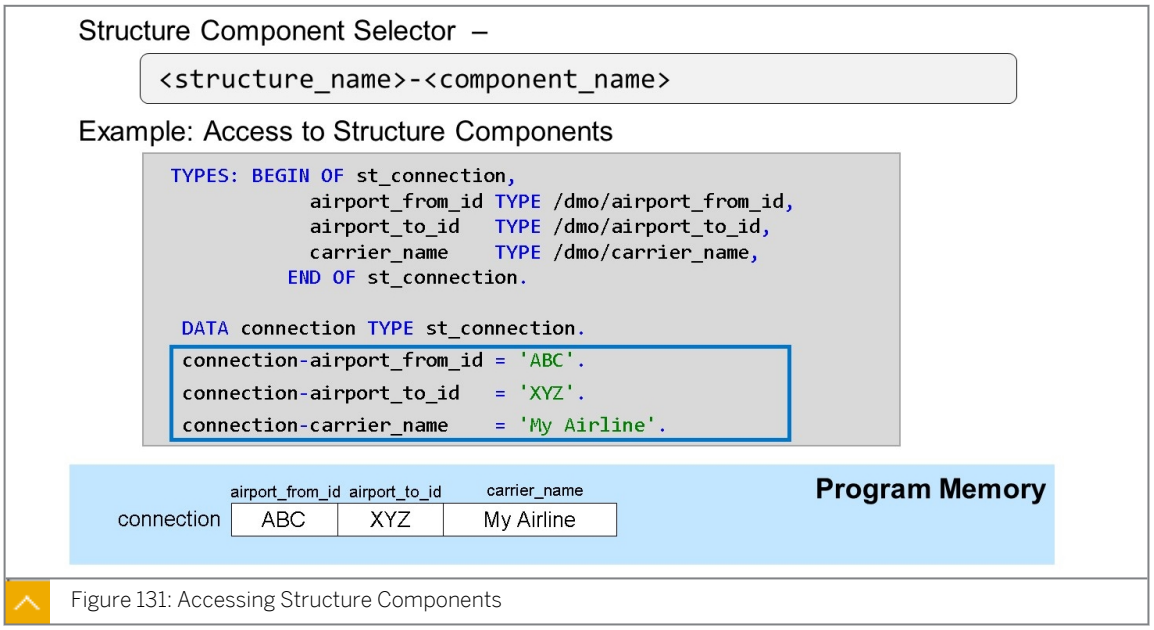


Figure 131: Accessing Structure Components

To access a component of a structure, you have to place a minus-sign (-) between the structure name and the component name.

Note:
No blanks are allowed before or after the component selector.

Accessing a structure component that way, you can use it in any operand position in which you can use a variable of the same type. Component airport_from_id of structure connection in the example above is of type /DMO/AIRPORT_FROM_ID. In consequence, you can use this component in any operand position in which you could use a simple variable of type /DMO/AIRPORT_FROM_ID; not only on the left-hand side of a value assignment as in the example, but also on the right-hand side, in the parameter passing of a method call, in the INTO clause or WHERE clause of a SELECT statement, and so on.

© Copyright. All rights reserved.

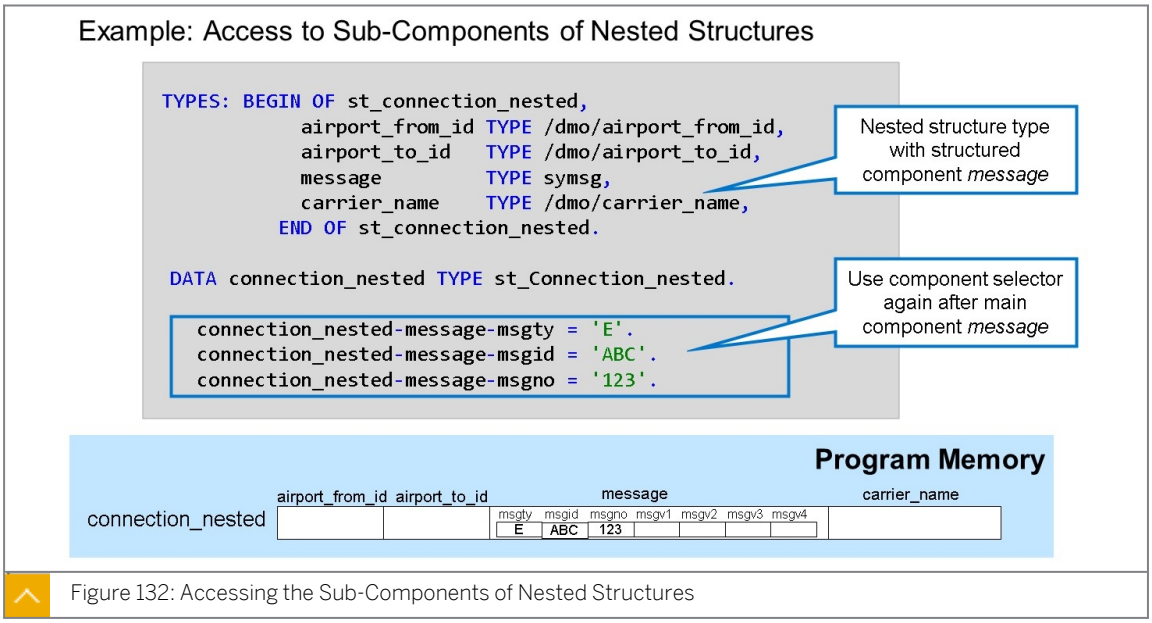


Figure 132: Accessing the Sub-Components of Nested Structures

If the component of a structure is itself a structure you access the sub-components by using the component selector again after the name of the main component. The first value assignment in the example accesses component MSGTY of MESSAGE, which itself is a component of nested structure CONNECTION_NESTED.

Hint:
You can use code-completion to implement access to structure components. Place the cursor immediately after the structure component selector and press **CTRL + SPACE** to see a list of all available structure components.

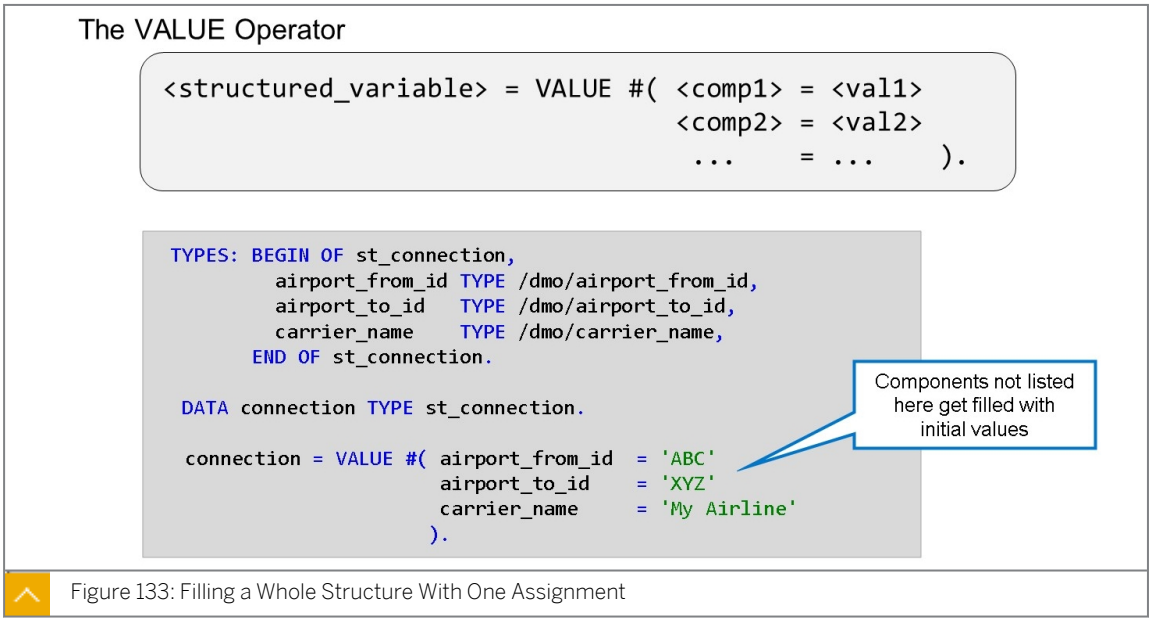


Figure 133: Filling a Whole Structure With One Assignment

The VALUE #( ) expression is an elegant way to assign values to a structured data object.

If you want to fill a whole structure, you can address each component individually as you saw in the previous example.

© Copyright. All rights reserved.

However, you can also use a VALUE #( ) expression to fill the structure. The expression constructs a structure, fills it with values and assigns the filled structure to a variable, in this case `connection`. The pound sign (#) tells the ABAP runtime environment to construct a structure with the same type as the target variable `connection`. In the brackets, you list the components of the structure that you want to fill (it does not have to be all of them) and assign a value to them.

When you fill a structure in this way, the runtime system deletes all existing values from the structure before refilling it with the values from your expression.

Note:
An assignment in the form `connection` = VALUE #( ). with just a blank between the brackets, fills all components of the structure with the type-specific initial value. This has the same effect as statement CLEAR `connection`.

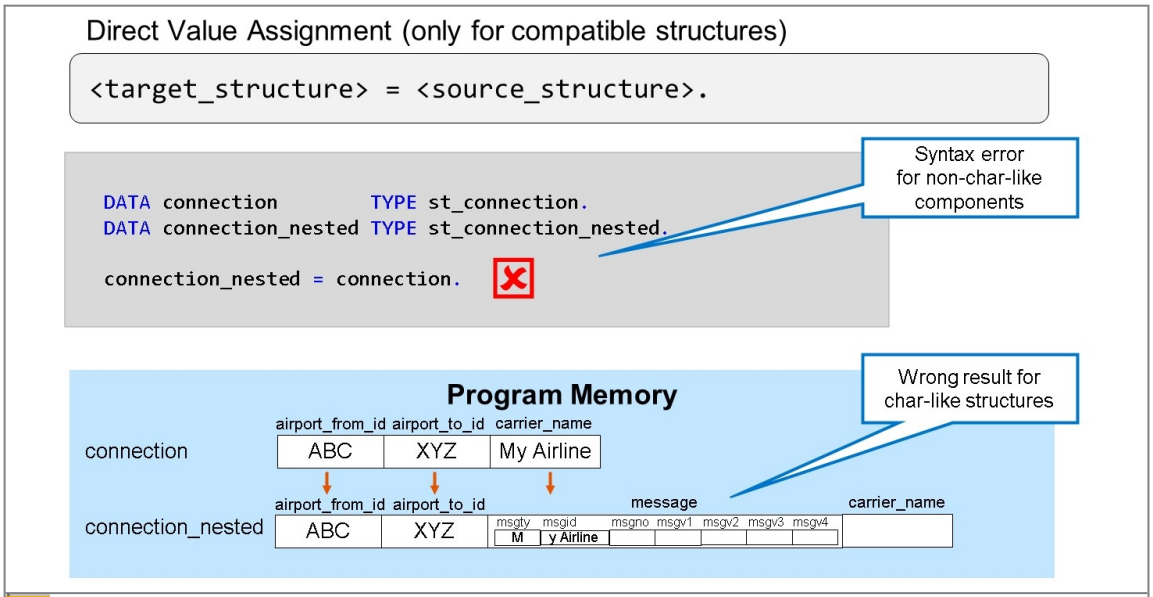


Figure 134: Value Assignment Between Structures

In ABAP, you may only copy the contents of one structure directly into another structure using the notation `<target structure> = <source structure>` if the two structure types are compatible. This is generally only the case if both structures have the same type. If the structures have different types, two things can happen:

- If one of the structures has a non-char-like component at a position where the other structure has a char-like component, direct assignment leads to a syntax error.
- If both structures are char-like, or, in other words, both structures consist of char-like components only, direct assignment is technically possible. But usually, the result will be wrong.

In the example, source structure and target structure are char-like. Therefore, direct assignment is technically possible. But because they are not compatible the result is wrong: The content of component `carrier_name` is copied to component `message` in the target structure.

© Copyright. All rights reserved.

Note:
Because there is no syntax error, you have to be extra careful when working with non-compatible char-like structures.

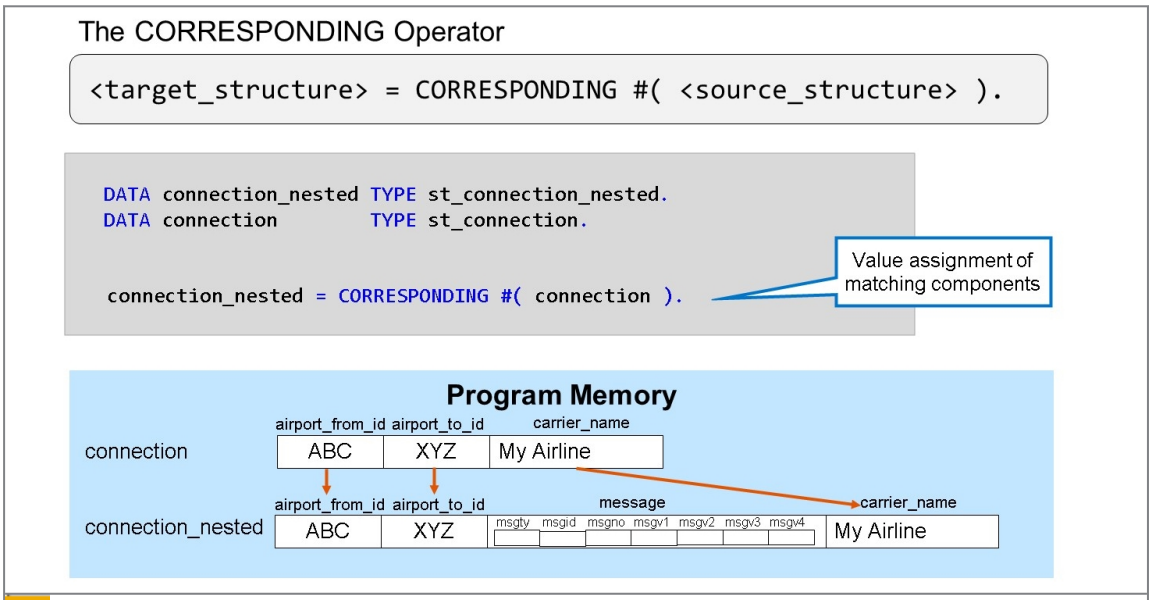


Figure 135: Component-Based Value Assignment

When you copy data between structures, you usually want to copy information from one field into the corresponding field of the target structure - `airport_from_id` to `airport_from_id`, `airport_to_id` to `airport_to_id`, and so on. To achieve this in ABAP, use the CORRESPONDING expression. This assigns values from `<source_structure>` to the corresponding, that is, identically-named components of `<target_structure>`. You must remember the following points:

- The fields must have identical names.
- The components do not have to be in the same position or sequence in the two structures.
- If the fields have different types, ABAP attempts a type conversion according to the predefined set of rules.

The CORRESPONDING expression in the example creates a structure of type `st_connection_nested`. The values of the components with the same name from the `connection` structure are then assigned to this structure. All other components remain initial. The structure thus constructed is then assigned to the `connection_nested` structure. This assigns initial values to the `message` component.

© Copyright. All rights reserved.

Structured Data Objects in ABAP SQL

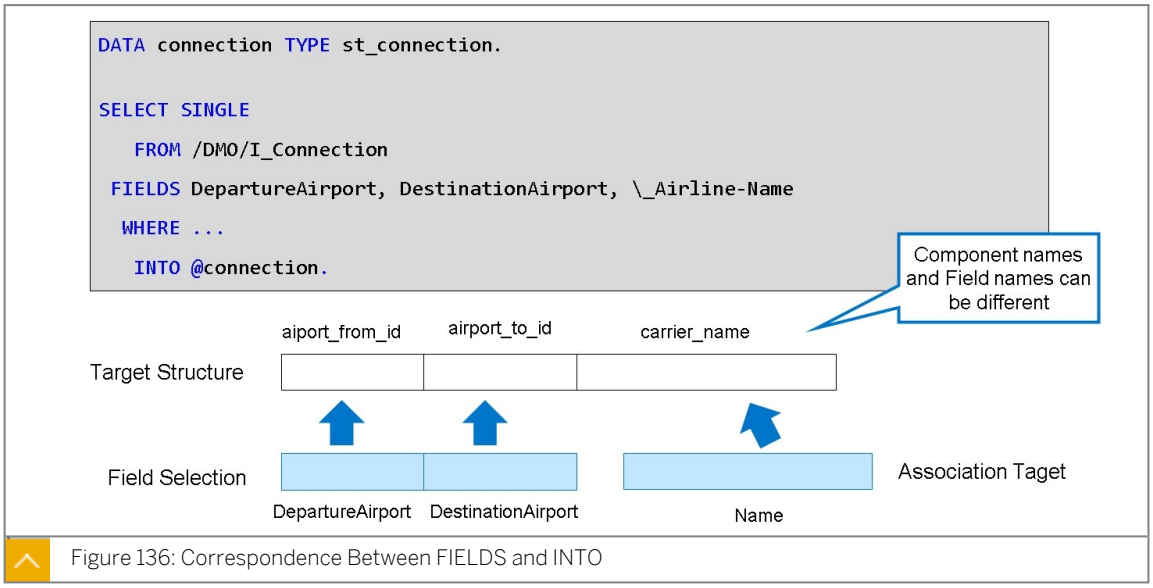


Figure 136: Correspondence Between FIELDS and INTO

The INTO clause of the SELECT statement will only work correctly if the number and types of the components of the structure correspond to the number and types of the columns specified in the FIELDS clause. In the above example, the statement can only work if the target structure connection has three components with the same type and length as the columns DepartureAirport, DestinationAirport, and _Airline-Name listed in the FIELDS clause. Note that, in this case, the names do not have to be identical - the system fills the target structure from left to right.

Note:
If the field list in the FIELDS clause does not match the structure or table line type in the INTO clause, a runtime error will occur.

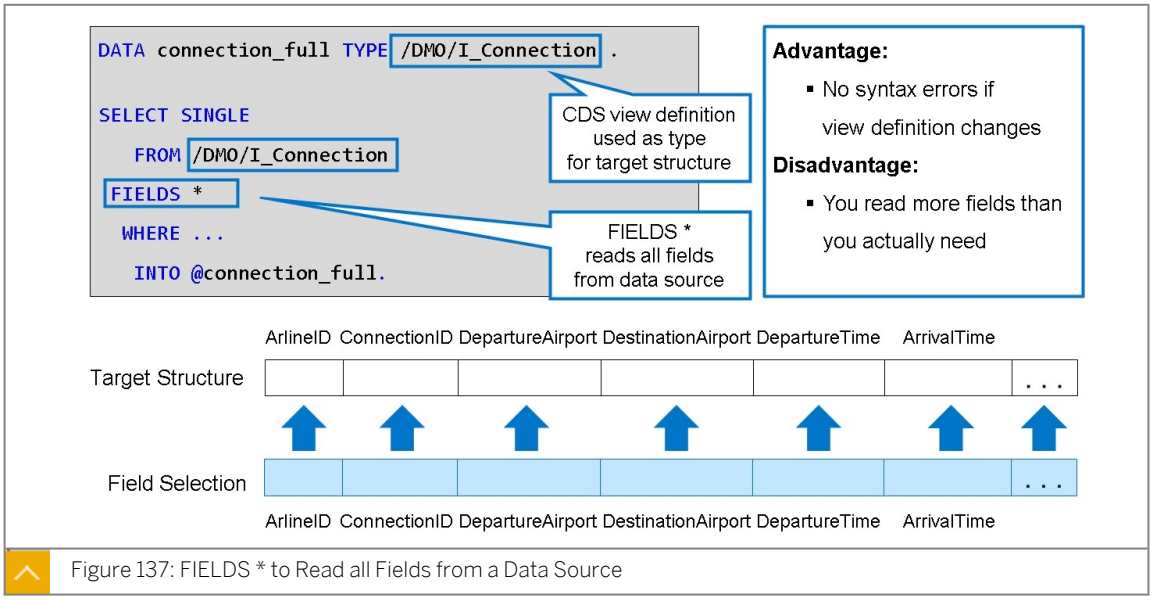


Figure 137: FIELDS * to Read all Fields from a Data Source

© Copyright. All rights reserved.

The example shows an easy technique to ensure that the target structure matches the field selection:

- The target structure is typed with CDS view /DMO/I_Connection, which is the data source in the FROM clause.
- The asterisk sign (*) after keyword FIELDS is a short notation to make sure, that all fields of the view are part of the field selection. Exposed associations are ignored.

Note:
This technique is also available when you read directly from a database table. Just like CDS view definitions, database table definitions can also serve as global structure types in ABAP.

The main advantage of this technique is, that the SELECT statement stays syntactically intact even if you or someone else makes changes to CDS view or database table. The most important drawback is, that you always read all fields from the database, whether you actually need them or not.

Note:
Only use this technique for views and tables with a small number of fields and if you actually need all the fields. Unnecessary reading of data from the database is a major cause for performance problems.

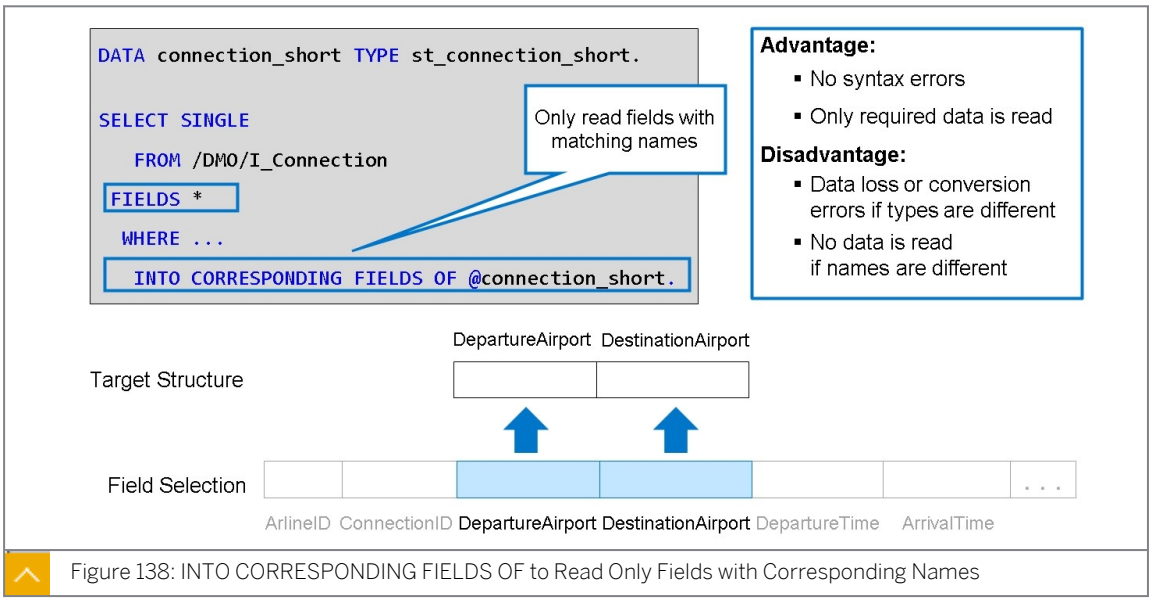


Figure 138: INTO CORRESPONDING FIELDS OF to Read Only Fields with Corresponding Names

Another way to avoid syntax errors is variant INTO CORRESPONDING FIELDS. This variant has the same effect as the CORRESPONDING #( ) operator that you learned about earlier. It ensures that data is copied between identically-named components. By defining the structure type according to your needs you can ensure that only the required data is read.

Once again, only the names must be identical. But to avoid problems you should make sure that identically-named components have compatible types. Otherwise the system attempts to convert the contents of the source field into the type of the target field. This can lead to data loss or (catchable) runtime errors.

© Copyright. All rights reserved.

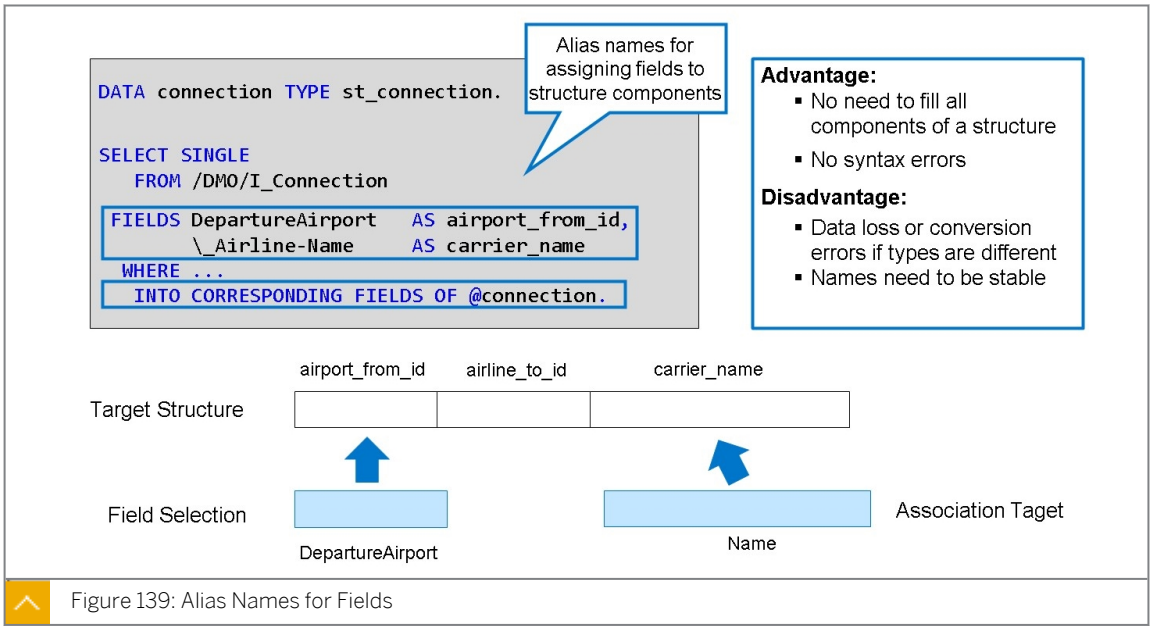


Figure 139: Alias Names for Fields

If the field names in the data source and the component names in the target structure do not match, the combination of FIELDS * and INTO CORRESPONDING FIELDS OF does not work.

If you want to keep variant INTO CORRESPONDING FIELDS OF, you can define alias names for the selected fields in the field list. For this, add addition AS after the field name, followed by the alias name. In the example, the alias name for view field DepartureAirport is airport_from_id and the alias name for path expression _Airline-Name is carrier_name. Based on this alias names, INTO CORRESPONDING FIELDS OF correctly identifies the structure component in which to store the retrieved data.

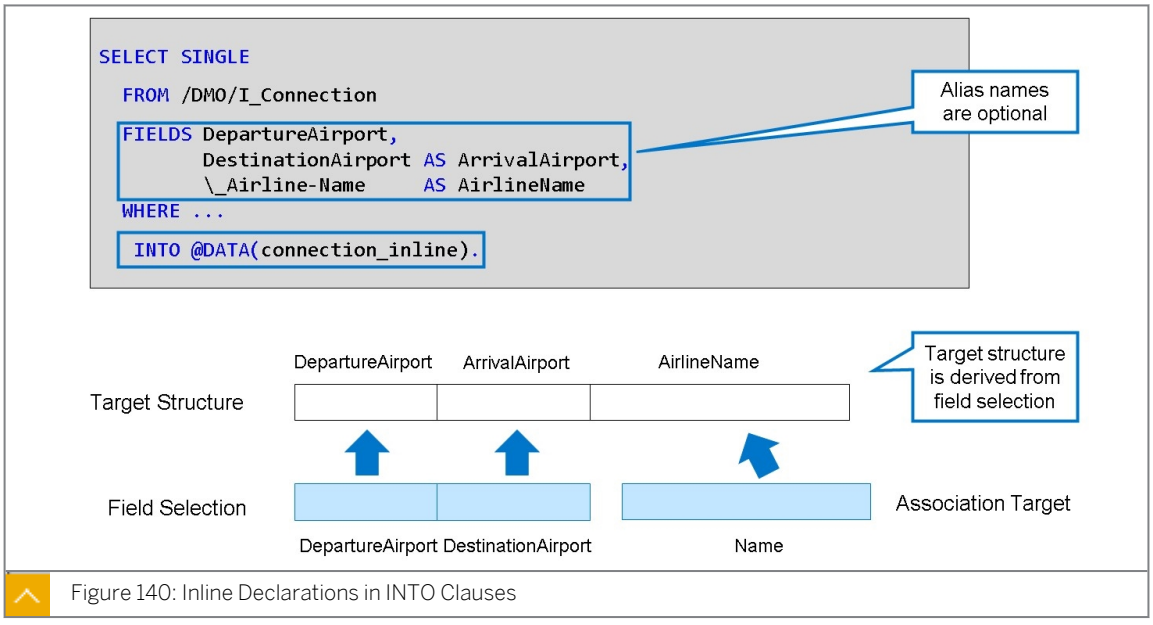


Figure 140: Inline Declarations in INTO Clauses

The simplest technique to avoid conflicts between the field selection and the target structure is an inline declaration in the INTO clause. The sequence, type and name of the inline declared structure is derived from the FIELDS clause. Therefore the target structure always fits the field selection.

© Copyright. All rights reserved.

> Note:
> Inline declarations are only supported after INTO. You cannot use inline declarations after INTO CORRESPONDING FIELDS OF.

In the example, there is no alias for field `DepartureAirport`. The name of the field is used as component name in structure `connection_inline`. Field `DestinationAirport`as well as the path expression `\_Airline-Name` have optional alias names (`ArrivalAirport` and `AirlineName`). Here the alias names are used as component names.

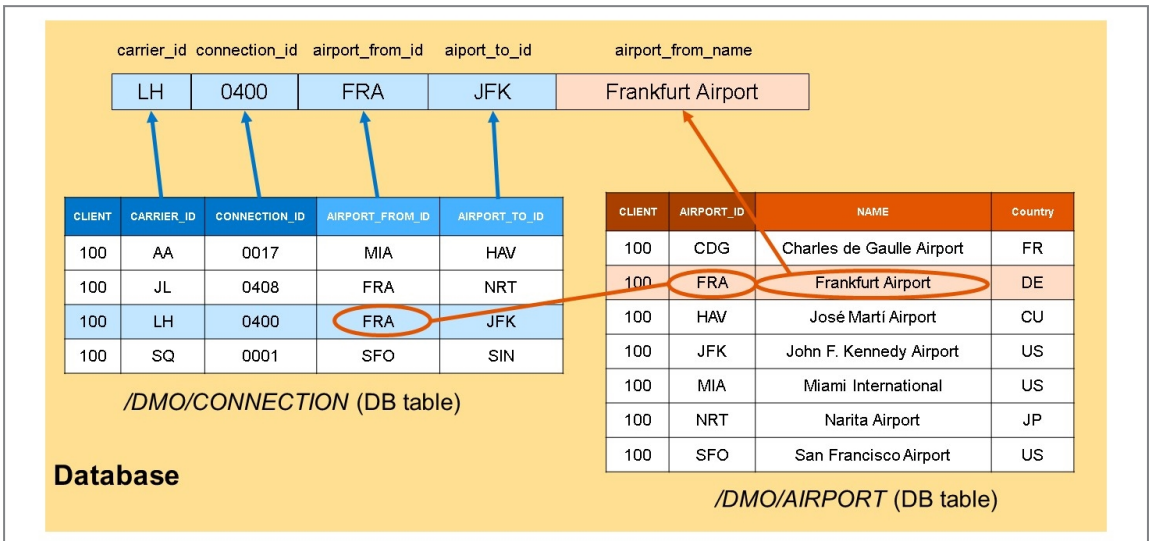


Figure 141: Combining Data From Different Sources (Joins)

When working with a relational database you often face the problem that you have to read related data from different database tables. We already learned that associations in CDS views are an elegant way to perform this task.

If no CDS View with suitable associations exist you can implement SQL joins, instead. The example above illustrates the principle of joins:

We are interested in flight connections and the airports they connect with each other. We find the 3-letter IDs of the airports in database table /DMO/CONNECTION. The full names of the airports are stored in database table /DMO/AIRPORT.

To retrieve a connection with the departure airport name, in one SELECT statement, we read connection data from DB table /DMO/CONNECTION and join it with DB table /DMO/AIRPORT.

A join consists of the following building blocks:

Data Sources

The Database tables and views to join with each other. A single join always combines a left-hand data source with a right-hand data source. In the example above, table /DMO/CONNECTION is the left-hand data source and table /DMO/AIRPORT the right-hand data source. ABAP SQL also supports joins of joins (nested joins)

Join Condition

The join condition specifies which records of the right-hand data source belong to a record from the left-hand data source. In the example above, the related departure

© Copyright. All rights reserved.

airport is identified by the value in columns CLIENT and AIRPORT_ID. The join condition reads:

/DMO/CONNECTION~CLIENT = /DMO/AIRPORT~CLIENT

AND

/DMO/CONNECTION~AIRPORT_FROM_ID = /DMO/AIRPORT~AIRPORT_ID

Join Type

The join type has an influence on the result if one of the data sources does not contain matching records. ABAP SQL currently supports INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, and CROSS JOIN. The most common join type is a LEFT OUTER JOIN.

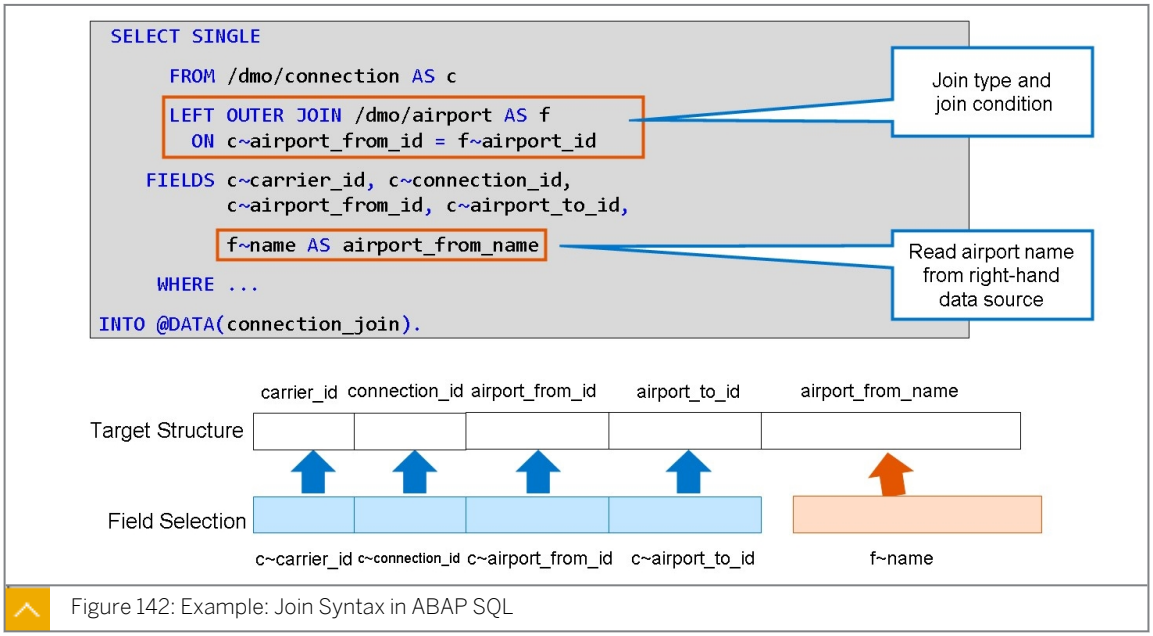


Figure 142: Example: Join Syntax in ABAP SQL

The figure shows the ABAP SQL syntax for a join. In the FROM clause, the join type is specified by keywords LEFT OUTER JOIN between the left-hand data source /dmo/connection and the right-hand data source /dmo/airport. The syntax introduces alias names c and f for the data sources. Alias names for data sources are optional, unless a data source appears more than once in the join.

The join condition follows keyword ON. The separator between the data source or its alias and the field is the tilde sign (~).

Note:
In ABAP SQL, it is not necessary to mention the client fields. They are added to the join condition by the database interface before the statement is sent to the database. If the FROM clause defines a join, you can use fields from both data sources in the FIELDS and WHERE clauses.

© Copyright. All rights reserved.

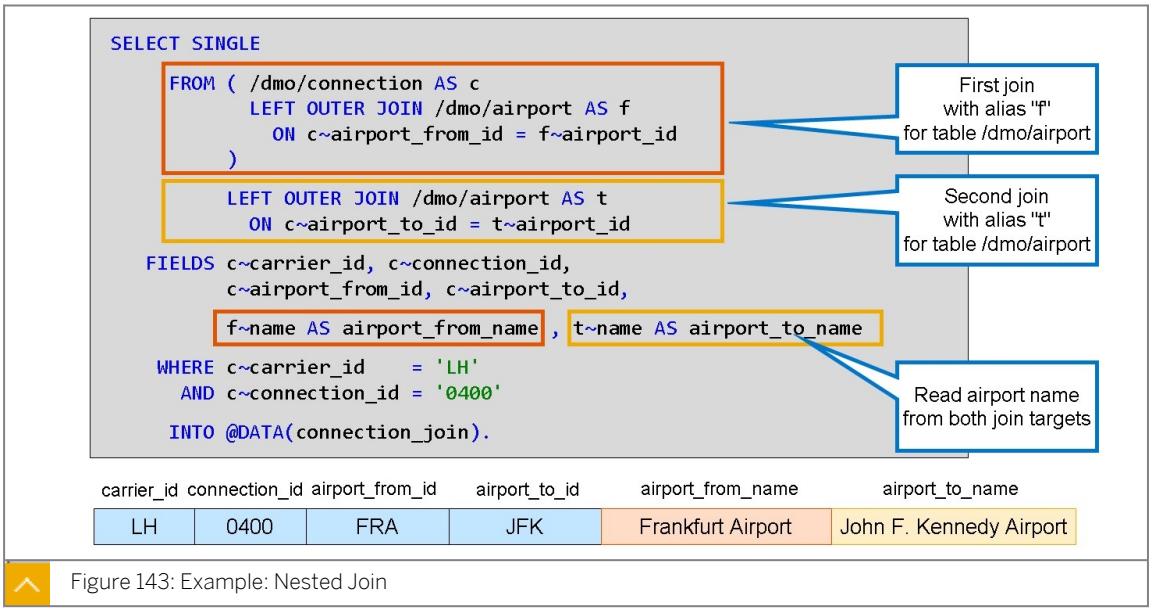


Figure 143: Example: Nested Join

In this example, the SELECT statement not only reads the departure airport name but also the destination airport name. To do so, the FROM clause defines a nested join:

The first join is a left outer join of tables /dmo/connection and /dmo/airport, introducing alias "f" (like "from") for the right-hand data source. This first join is then used as left-hand data source for a second left outer join that has table /dmo/airport as right-hand data source. Note that in this case alias "t" (for "to") is crucial to distinguish this appearance of table /dmo/airport from the previous one.

The FIELDS clause, lists the airport names from both data sources, introducing aliases airport_from_name and airport_to_name to distinguish them from each other.

Hint:
The brackets around the first join are optional. If they are omitted, the joins in the from clause are evaluated from left to right.

© Copyright. All rights reserved.

Unit 5
Exercise 14

Use a Structured Data Object

In this exercise, you declare a structured attribute, fill it using a SELECT statement and access the structure components.

Template:

/LRN/CL_S4D400_DBS_CDS (global Class)

Solution:

/LRN/CL_S4D400_STS_STRUCTURE (global Class)

Task 1: Copy Template
Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_DBS_CDS`** to a class in your own package (suggested name: **`ZCL_##_STRUCTURE`**, where ## stands for your group number).

Task 2: Declare a Structured Data Object
In the local class, declare a structured attribute **`details`** to replace the scalar attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`**. Begin by defining a private structure type **`st_details`** inside the local class.

1. Switch to the local class **`lcl_connection`**.
2. Define a **private** structure type **`st_details`** with the following components:

   Table 4: Components of structure type st_details:

Component Name	Data Type
`DepartureAirport`	**`/dmo/airport_from_id`**
`DestinationAirport`	**`/dmo/airport_to_id`**
`AirlineName`	**`/dmo/carrier_name`**

3. Comment or remove the declaration of attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`**.
4. Declare a new private instance attribute **`details`** and type it with structure type **`st_details`**.

Task 3: Access Structure Components
Use the components of the structured attribute **`details`** in the method **`get_output`**.

1. Adjust the implementation of the method **`get_output`**. Replace any access to the attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`** with the corresponding component of attribute **`details`**.

© Copyright. All rights reserved.

Hint:
Do not type in the component names manually! After typing the structure component selector (-), press **Ctrl + Space** to get a list of all components.

Task 4: Fill the Structured Attribute in the SELECT Statement
Use the structured attribute as the target of the SELECT statement in the **constructor** method.

1. Adjust the SELECT statement in the implementation of the **constructor** method. Replace the list of data objects in the INTO clause with the structured attribute **details**.
2. Optional: Use syntax variant `INTO CORRESPONDING FIELDS OF @details.`.
3. Activate the class. Execute it and analyze the console output. Check that the output displays data for all attributes.

© Copyright. All rights reserved.

Unit 5
Solution 14

Use a Structured Data Object

In this exercise, you declare a structured attribute, fill it using a SELECT statement and access the structure components.

Template:

/LRN/CL_S4D400_DBS_CDS (global Class)

Solution:

/LRN/CL_S4D400_STS_STRUCTURE (global Class)

Task 1: Copy Template
Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_DBS_CDS`** to a class in your own package (suggested name: **`ZCL_##_STRUCTURE`**, where ## stands for your group number).
   a) Open the source code of the global class **`/LRN/CL_S4D400_DBS_CDS`**.
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click the class **`/LRN/CL_S4D400_DBS_CDS`** to open the context menu.
   d) From the context menu, choose *Duplicate ...*.
   e) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_STRUCTURE`**, where ## stands for your group number.
   f) Adjust the description and choose *Next*.
   g) Confirm the transport request and choose *Finish*.

Task 2: Declare a Structured Data Object
In the local class, declare a structured attribute **`details`** to replace the scalar attributes **`airport_from_id`**, **`airport_to_id`**, and **`carrier_name`**. Begin by defining a private structure type **`st_details`** inside the local class.

1. Switch to the local class **`lcl_connection`**.
   a) In the global class, choose *Local Types*.
2. Define a **private** structure type **`st_details`** with the following components:

   Table 4: Components of structure type st_details:

Component Name	Data Type
`DepartureAirport`	**`/dmo/airport_from_id`**
`DestinationAirport`	**`/dmo/airport_to_id`**

© Copyright. All rights reserved.

Component Name	Data Type
AirlineName	**/dmo/carrier_name**

a) After line `PRIVATE SECTION.`, add the following code:

```
  TYPES:
    BEGIN OF st_details,
      DepartureAirport   TYPE /dmo/airport_from_id,
      DestinationAirport TYPE   /dmo/airport_to_id,
      AirlineName        TYPE   /dmo/carrier_name,
    END OF st_details.
```

3. Comment or remove the declaration of attributes **airport_from_id**, **airport_to_id**, and **carrier_name**.
   a) Select the lines with the three DATA statements.
   b) Press **Ctrl + <** to add a star sign (*) in front of each selected line.
4. Declare a new private instance attribute **details** and type it with structure type **st_details**.
   a) Adjust the code as follows:

```
*    DATA airport_from_id TYPE /dmo/airport_from_id.
*    DATA airport_to_id   TYPE /dmo/airport_to_id.
*
*    DATA carrier_name    TYPE /dmo/carrier_name.

    DATA details TYPE st_details.
```

Task 3: Access Structure Components

Use the components of the structured attribute **details** in the method **get_output**.

1. Adjust the implementation of the method **get_output**. Replace any access to the attributes **airport_from_id**, **airport_to_id**, and **carrier_name** with the corresponding component of attribute **details**.

> Hint:
> Do not type in the component names manually! After typing the structure component selector (-), press **Ctrl + Space** to get a list of all components.

   a) Navigate to the implementation of the method **get_output**.
   b) Adjust the APPEND statements as follows:

```
*    APPEND |--------------------------------|             TO r_output.
*    APPEND |Carrier:     { carrier_id } { carrier_name }| TO r_output.
*    APPEND |Connection:  { connection_id   }|             TO r_output.
*    APPEND |Departure:   { airport_from_id }|             TO r_output.
*    APPEND |Destination: { airport_to_id   }|             TO r_output.

    APPEND |--------------------------------|                 TO
r_output.
```


© Copyright. All rights reserved.

```
    APPEND |Carrier:     { carrier_id } { details-airlinename }| TO
r_output.
    APPEND |Connection:  { connection_id   }|                    TO
r_output.
    APPEND |Departure:   { details-departureairport     }|       TO
r_output.
    APPEND |Destination: { details-destinationairport   }|       TO
r_output.
```

Task 4: Fill the Structured Attribute in the SELECT Statement

Use the structured attribute as the target of the SELECT statement in the **`constructor`** method.

1. Adjust the SELECT statement in the implementation of the **`constructor`** method. Replace the list of data objects in the INTO clause with the structured attribute **`details`**.
   a) Navigate to the implementation of method **`constructor`**.
   b) Adjust the SELECT statement as follows:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
     WHERE AirlineID    = @i_carrier_id
       AND ConnectionID = @i_connection_id
*       INTO ( @airport_from_id, @airport_to_id, @carrier_name  ).
      INTO @details.
```

2. Optional: Use syntax variant `INTO CORRESPONDING FIELDS OF @details.`.
   a) In the SELECT statement, replace `INTO @details.` with the following code:

```
  INTO CORRESPONDING FIELDS OF @details.
```

   b) Add alias name **`AirlineName`** for the path expression.
   c) The SELECT statement should now look like this:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name as
AirlineName
     WHERE AirlineID    = @i_carrier_id
       AND ConnectionID = @i_connection_id
      INTO CORRESPONDING FIELDS OF @details.
```

3. Activate the class. Execute it and analyze the console output. Check that the output displays data for all attributes.
   a) Press **`Ctrl + F3`** to activate the class.
   b) Press **`F9`** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Work with structured data objects.
- Use structured data objects in ABAP SQL.

© Copyright. All rights reserved.

Learning Assessment

1. You declare a variable using the statement DATA struct TYPE <type>. Which of the following can you use to declare a structure?

   Choose the correct answers.

   - [ ] **A** A database table
   - [ ] **B** A data element
   - [ ] **C** A CDS view
   - [ ] **D** A predefined ABAP type

2. A structure struct contains a component comp. How do you address the component?

   Choose the correct answer.

   - [ ] **A** struct.comp
   - [ ] **B** struct-comp
   - [ ] **C** struct->comp
   - [ ] **D** struct=>comp

3. What do you have to remember when copying data between structures using the CORRESPONDING expression?

   Choose the correct answer.

   - [ ] **A** Only components with identical names are copied.
   - [ ] **B** Components are copied according to their positions.
   - [ ] **C** Only components with identical types are copied.

© Copyright. All rights reserved.

Learning Assessment - Answers

1. You declare a variable using the statement DATA struct TYPE <type>. Which of the following can you use to declare a structure?

   Choose the correct answers.

   - [X] A A database table
   - [ ] B A data element
   - [X] C A CDS view
   - [ ] D A predefined ABAP type

   That is correct.

2. A structure struct contains a component comp. How do you address the component?

   Choose the correct answer.

   - [ ] A struct.comp
   - [X] B struct-comp
   - [ ] C struct->comp
   - [ ] D struct=>comp

   That's right.

3. What do you have to remember when copying data between structures using the CORRESPONDING expression?

   Choose the correct answer.

   - [X] A Only components with identical names are copied.
   - [ ] B Components are copied according to their positions.
   - [ ] C Only components with identical types are copied.

© Copyright. All rights reserved.

UNIT 6 Working with Complex Internal Tables

Lesson 1

Declaring a Complex Internal Table 255

Lesson 2

Working with Complex Internal Tables 262

Exercise 15: Use a Complex Internal Table 275

UNIT OBJECTIVES

- Declare a complex internal table.
- Fill complex internal tables with data.
- Access the content of complex internal tables.
- Use complex internal tables in ABAP SQL.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Declaring a Complex Internal Table

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Declare a complex internal table.

Properties of Complex Internal Tables

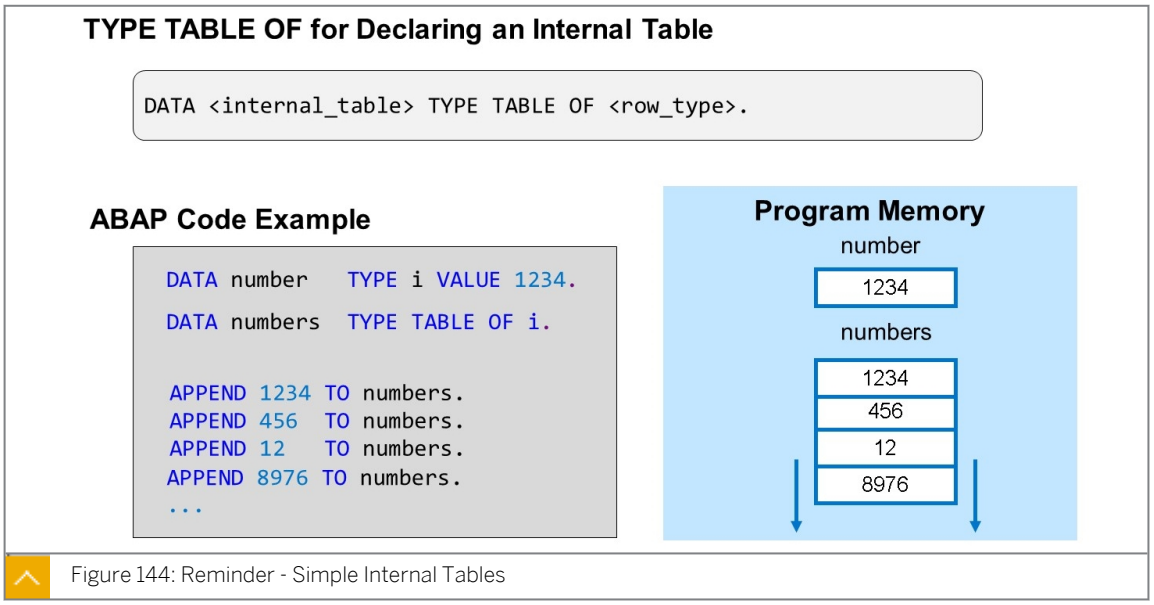


Figure 144: Reminder - Simple Internal Tables

The internal tables we have used so far had a scalar data type as their row type. In the example shown in the figures here, the row type of internal table numbers is the ABAP built-in type I.

We refer to these kinds of internal tables as simple internal tables.

© Copyright. All rights reserved.

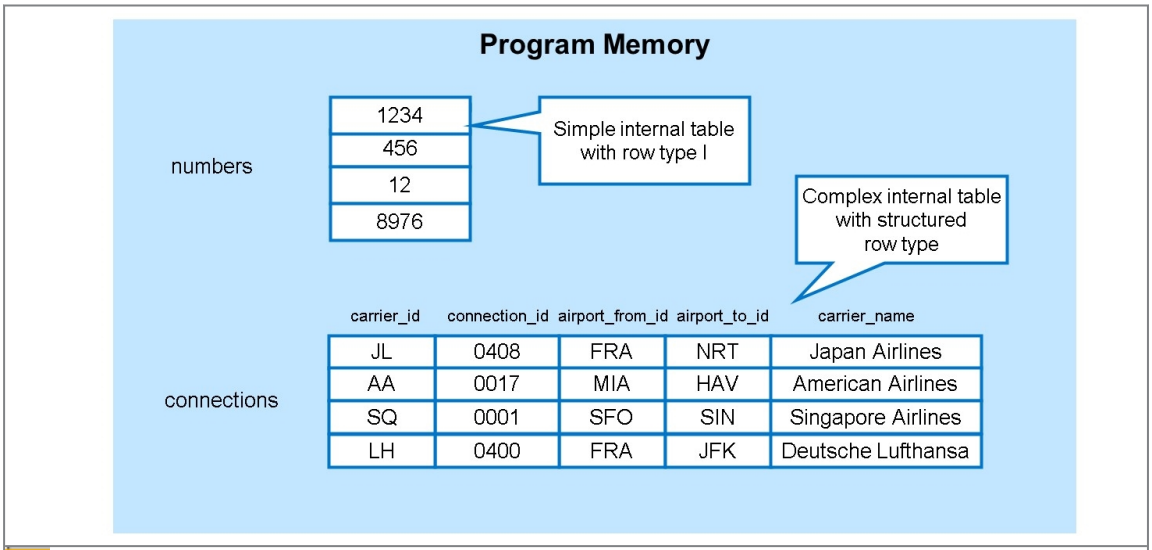


Figure 145: Internal Tables with Structured Row Type

We speak of a complex internal table if the row type is a structured data type.

While a simple internal table has only one nameless column, a complex internal table consists of several columns, each of them with the name and type of the corresponding component of the structured row type. In the example, the row type of internal table `connections` is a structured type with five components: `carrier_id`, `connection_id`, `airport_from_id`, `airport_to_id`, and `carrier_name`. Consequently, internal table `connections` has five columns with those names.

Note:
The columns in the examples in the figures, *Reminder: Simple Internal Tables* and *Internal Tables with Structured Row Type*, all have scalar types. More generally, a column of an internal table could also be of structured type or even have a table type. In the latter case, we talk of a nested internal table.

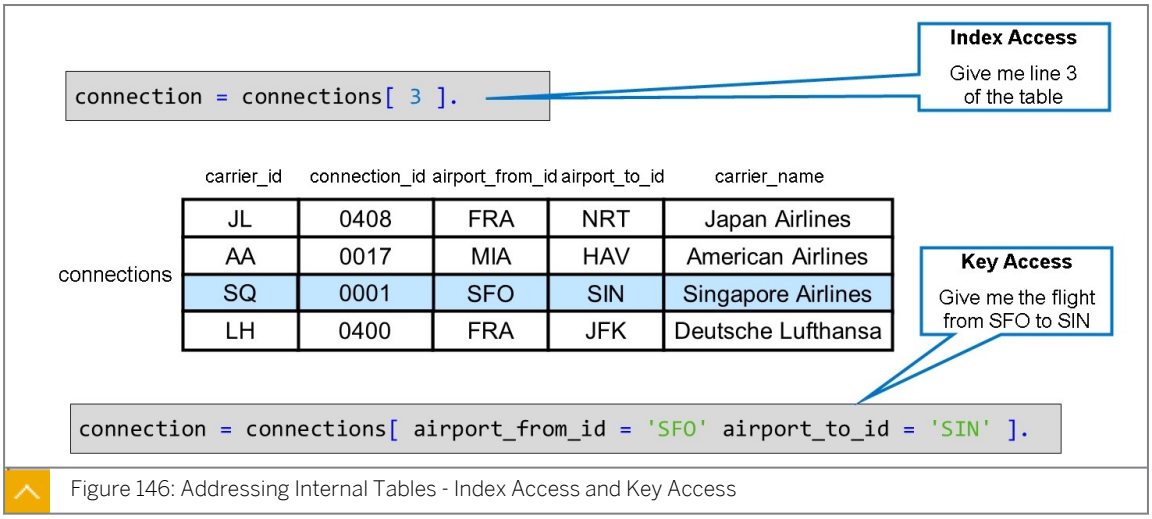


Figure 146: Addressing Internal Tables - Index Access and Key Access

Up to now, we have addressed the rows of an internal table by their position. This is called an index access.

© Copyright. All rights reserved.

With the named columns of a complex internal table, key access becomes more important. Key access means addressing a row of the internal table by looking for particular values in particular columns. The columns in which you search can be any columns of the internal table.

Index access to an internal table is always very fast, even if the internal table contains many rows. Key access, however, can become very slow if the table contains a lot of rows. Choosing the right access type for the internal table can improve the performance of a key access.

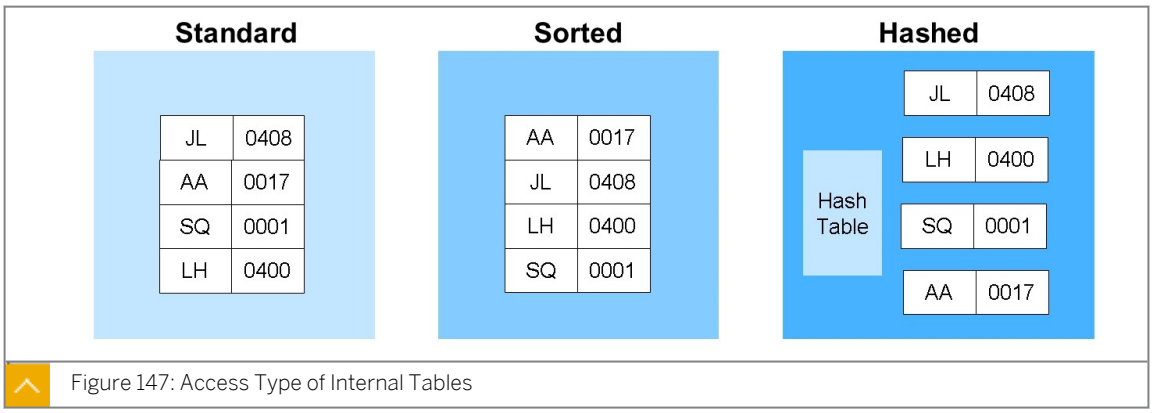


Figure 147: Access Type of Internal Tables

Every internal table has one of three access types. The access type determines how data is stored in the table and, based on that, how the system reads the table to retrieve the data.

The different types of tables are as follows:

Standard Table

In a standard table, the contents are not stored in a particular sort order. By default, new records are appended to the end of the table. In order to retrieve data by key, the system must read it sequentially, which can lead to long retrieval times if the table is very large. The simple internal tables we used so far were standard tables.

Sorted Table

In a sorted table, the contents of the table are always sorted according to the key fields in ascending order. When you insert a new record into the table, the system ensures that it is placed at the correct position. Since the data is always sorted, the system can retrieve records more efficiently than from a standard table (as long as you follow particular rules).

Hashed Table

Hashed tables are managed using a special hash algorithm. This ensures that the system can retrieve records very quickly even if the table is extremely large. However, this performance gain only works in very particular cases.

Note:
In this lesson, we will concentrate on standard tables. More information on sorted and hashed tables can be found in the ABAP documentation.

© Copyright. All rights reserved.

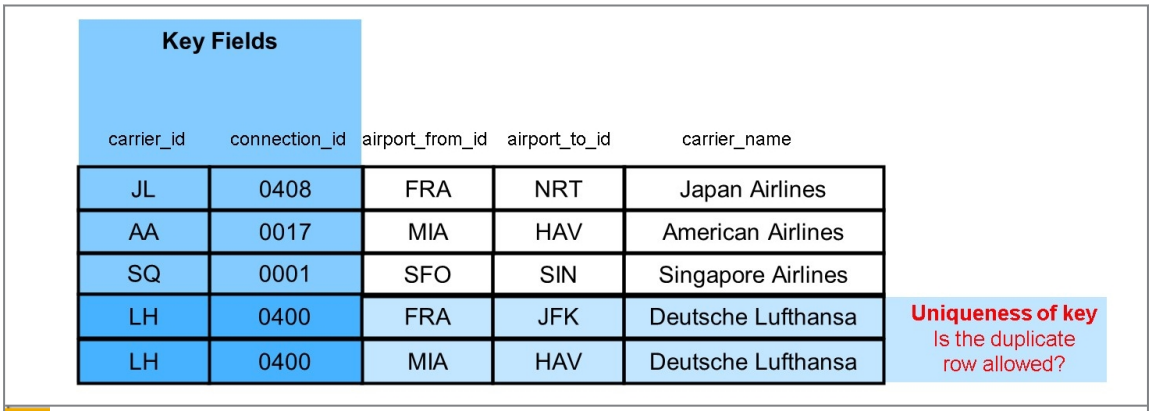


Figure 148: Key Definition of an Internal Table

Every internal table has a key. In standard tables, the key does not play a particularly significant role. For sorted and hashed tables, the key is very important as it determines the way in which the data will be managed in the table. Crucially, sorted and hashed tables are only faster for key access that addresses all or at least a subset of the key fields.

A further attribute of the table key is its uniqueness. You will sometimes want to allow duplicate entries in an internal table, and sometimes you will want to ensure that the key is unique. Here, the following rules apply:

- Duplicates are always allowed in standard tables
- Duplicates are never allowed in hashed tables
- For a sorted table, you choose in the definition whether the key is to be unique or non-unique.

Note:
Internal tables may also have secondary keys. Secondary keys are a way of improving the performance of key accesses to internal tables that use different combinations of fields. You will find more information about secondary keys in the ABAP syntax documentation.

© Copyright. All rights reserved.

Complex Table Types

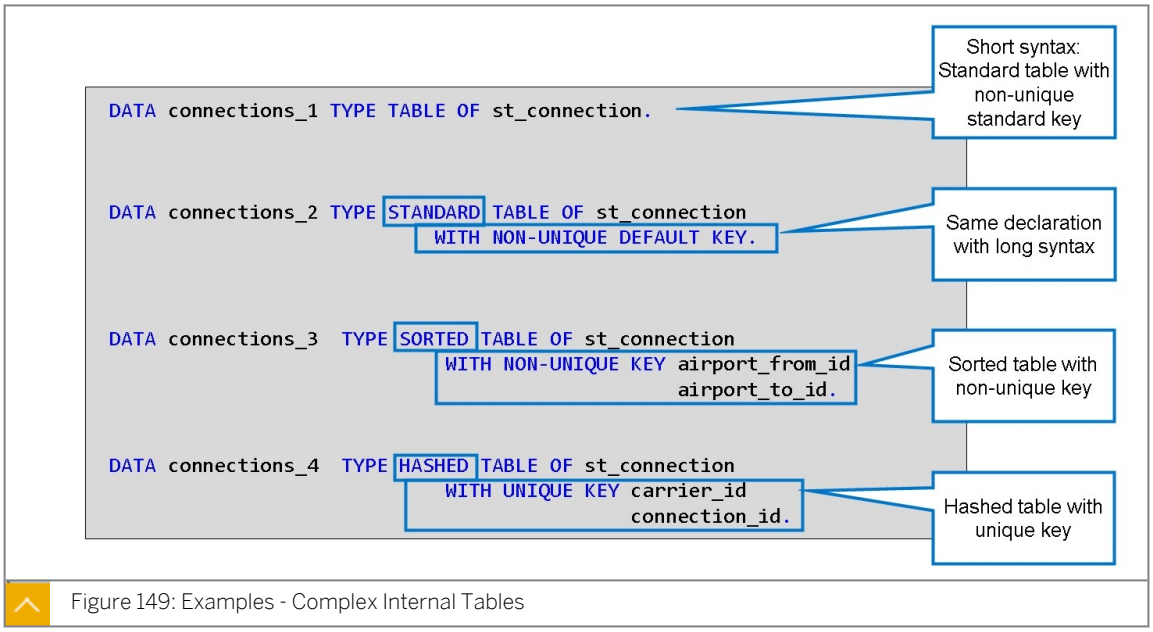


Figure 149: Examples - Complex Internal Tables

The figure, Examples - Complex Internal Tables, shows some examples for the declaration of complex internal tables.

The first example specifies neither the access type nor the key attributes. The ABAP compiler implicitly declares a standard table with a non-unique standard key. The standard key, or default key, consists of all non-numeric columns of the table line type.

For simple internal tables, where the access type and key definition is not important, the short syntax is acceptable. For complex internal tables, you should clearly specify the access type and key. The second example defines an internal table of exactly the same type as the first example, but instead of using the short form it explicitly specifies the access type and the key.

The third and forth internal tables define internal tables of access type sorted and hashed. For these access types the definition of the key is mandatory.

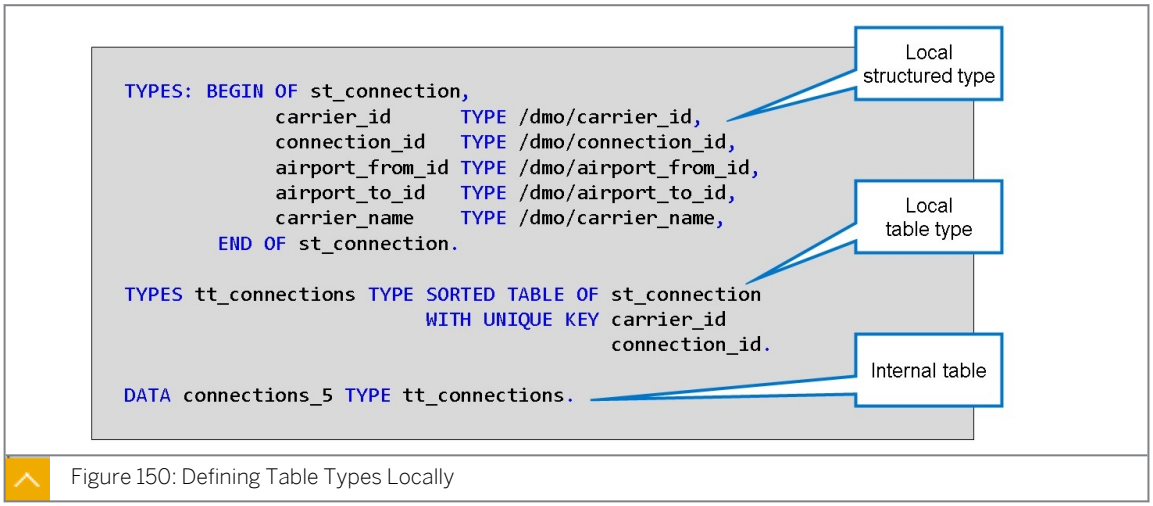


Figure 150: Defining Table Types Locally

Furthermore, it is good programming style to define the data type first and then to create a variable that refers to the type.

© Copyright. All rights reserved.

Instead of specifying the access type and key of an internal table in the DATA statement, you should use a named table type. If you need the table type only locally in a method or in connection with a given class, you can define it using the TYPES statement.

The example defines a structured type st_connection, first. With this structured type as row type it then defines table type tt_connections. Finally, the declaration of internal table connections_5 refers to the table type.

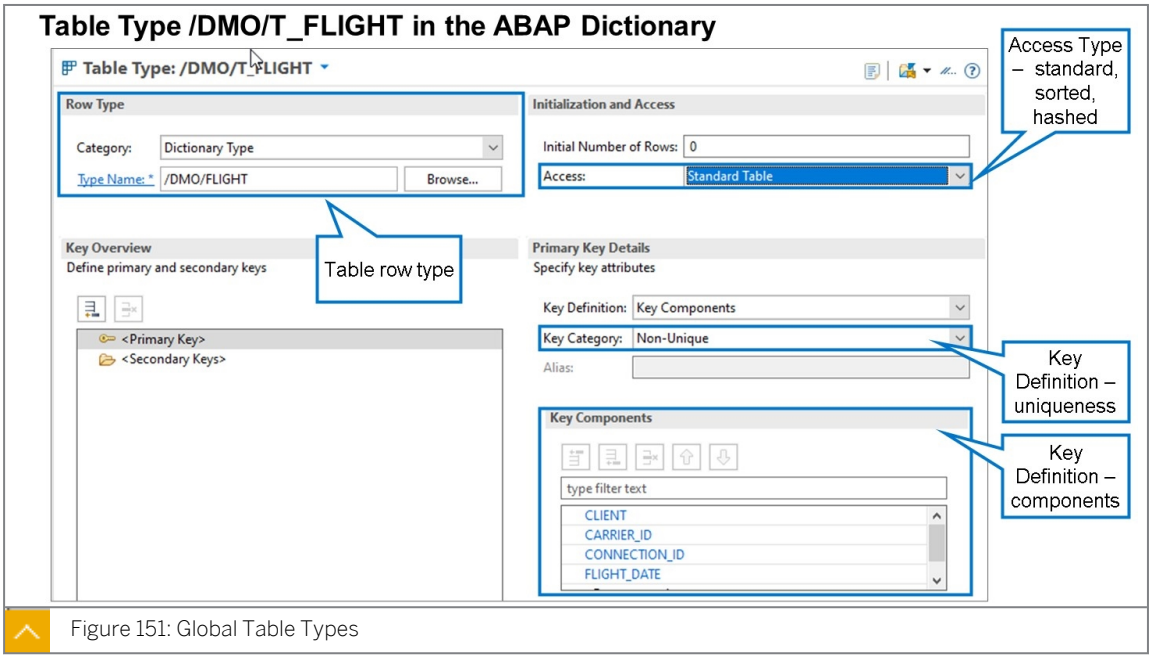


Figure 151: Global Table Types

If you need the table type globally, you can use a global table type.

A global table type is a repository object that can be used as data type anywhere in the system. ADT provides a dedicated editor for this kind of repository object. The tool consists of the following frames:

Row Type

This frame contains the origin and name of the row type. In the example, global type /DMO/FLIGHT is used.

Initialization and Access

Among other things, this frame contains the definition of the access type. In the example, the access type - Standard Table - is selected.

Key Overview

This frame provides an overview of the keys. Choose the key for which you want to see details on the right.

> Note:
> Every table type contains the definition of a primary key. Secondary keys are optional. The table type in the example does not contain any secondary keys.

Key Details

This frame displays details for the selected key: the key category, that is, whether the key is unique or non-unique, and the list of key components. In the example, the primary key

© Copyright. All rights reserved.

is non-unique and consists of components CLIENT, CARRIER_ID, CONNECTION_ID, and FLIGHT_DATE.

LESSON SUMMARY

You should now be able to:

- Declare a complex internal table.

© Copyright. All rights reserved.

Unit 6
Lesson 2

Working with Complex Internal Tables

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Fill complex internal tables with data.
- Access the content of complex internal tables.
- Use complex internal tables in ABAP SQL.

Complex Internal Table Filling

```
DATA connections  TYPE tt_connections.
* DATA connection   TYPE st_connection.
DATA connection   LIKE LINE OF  connections.



APPEND connection TO connections.
```

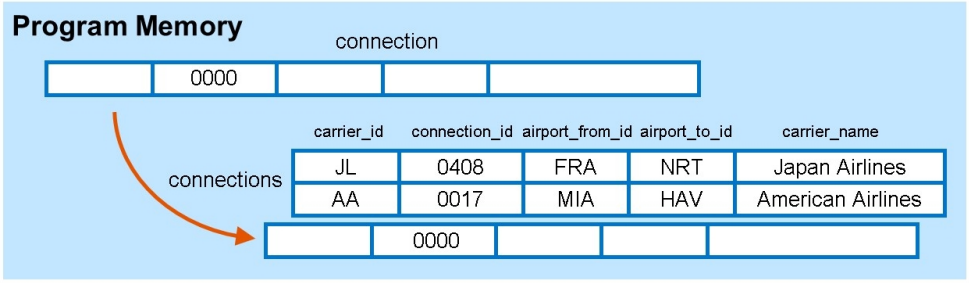


Figure 152: APPEND for Complex Internal Tables - Declaration of a Work Area

As you already learned, the simplest way to add a new row to an internal table is the APPEND statement with a data object whose type corresponds to the row type of the internal table. This data object is sometimes referred to as work area.

For simple internal tables the work area used in APPEND can be a scalar variable, constant, or a literal. For complex internal tables, the work area has to be structured.

In the example, structured variable connection is used to fill internal table connections.

In principle, there are two ways to declare work area connection:

- Reference the row type st_connection directly
- Reference the row type indirectly using LIKE LINE OF <internal_table>.

Defining work areas with LIKE LINE OF has two advantages:

© Copyright. All rights reserved.

- It reveals the purpose of the structured variable as work area for the internal table
- It ensures that the work area fits to the internal table, even if the definition of the internal table changes

```
DATA connections  TYPE tt_connections.
DATA connection   LIKE LINE OF connections.

connection = VALUE #( carrier_id       = 'NN'
                      connection_id    = '1234'
                      airport_from_id  = 'ABC'
                      airport_to_id    = 'XYZ'
                      carrier_name     = 'My Airline' ).
APPEND connection TO connections.
```

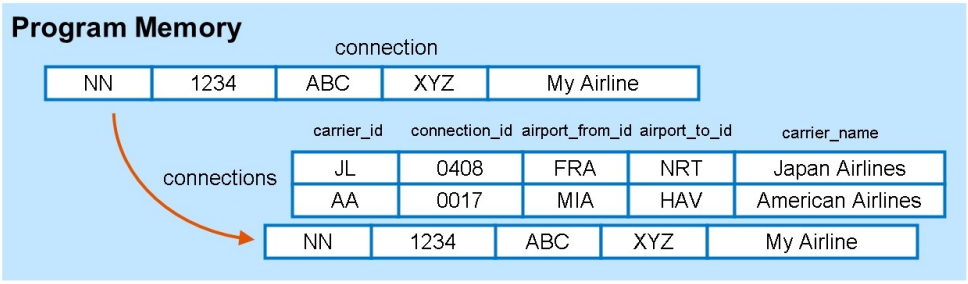


Figure 153: APPEND for Complex Internal Tables - Filling the Work Area

If you do not fill the work area before the APPEND statement, the new row of the internal table will be filled with type-specific initial values.

Hint:
You get the same result with the special variant APPEND INITIAL LINE TO <internal_table>. This variant does not even require a work area.

To fill the structured work area, you can either fill the individual components or, as you can see in the example, use a VALUE #( ) expression.

```
DATA connections  TYPE tt_connections.
APPEND VALUE #( carrier_id       = 'NN'
                connection_id    = '1234'
                airport_from_id  = 'ABC'
                airport_to_id    = 'XYZ'
                carrier_name     = 'My Airline'
              )
    TO connections.
```

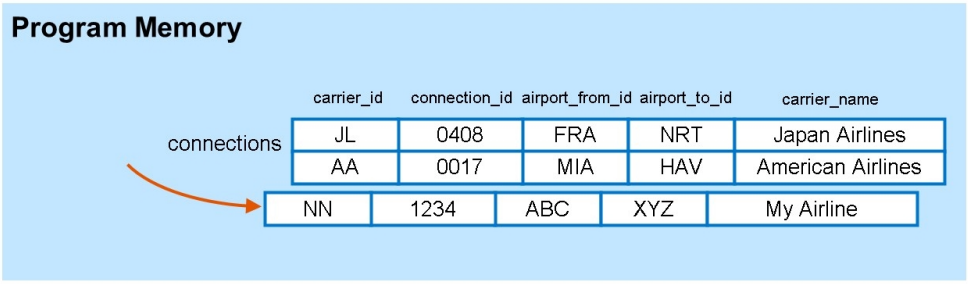


Figure 154: APPEND for Complex Internal Tables - direct use of VALUE #( )

© Copyright. All rights reserved.

As you can see in the example, you can also use a VALUE #( ) expression directly in the APPEND statement. In this case, you do not need a work area.

Note:
This can have a positive effect on the overall memory consumption of your program.

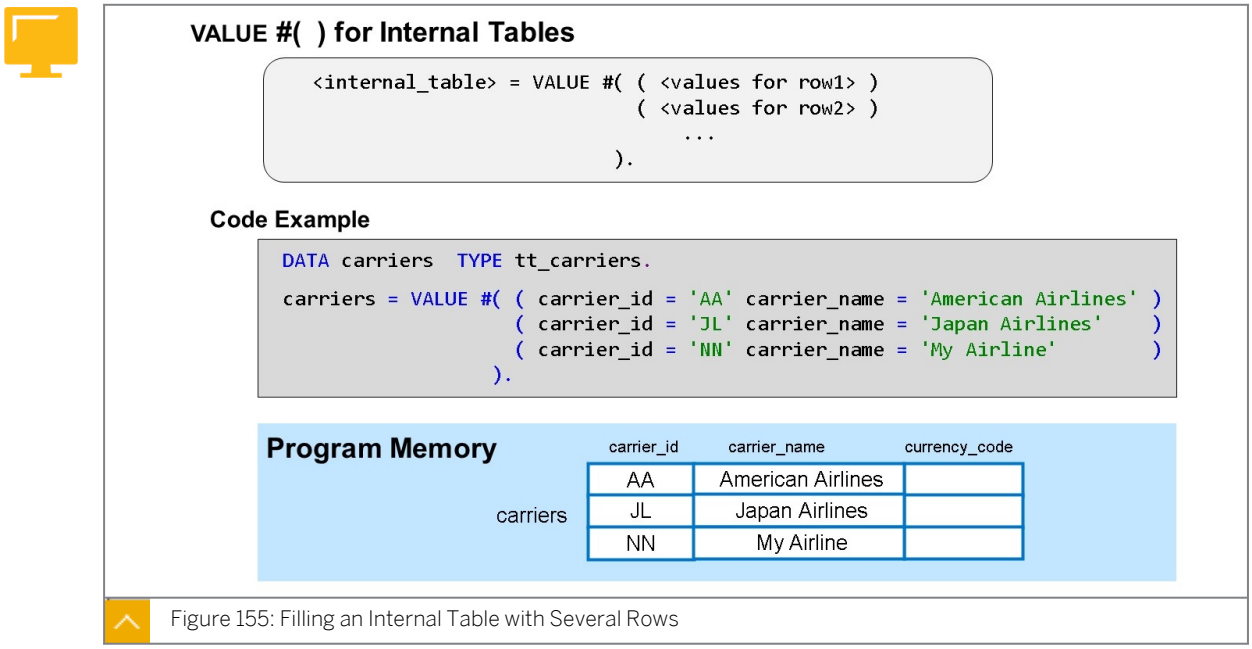


Figure 155: Filling an Internal Table with Several Rows

There is a variant of the VALUE #( ) expression that you can assign directly to an internal table. In this variant of VALUE #( ) additional pairs of brackets are used to separate the table rows from each other.

The code example fills internal table carriers with three rows, each with a different value for carrier_id and carrier_name. As a result of this, column currency_code is not mentioned, it is filled with the type specific initial value.

Note:
With the assignment above, all existing table rows are removed before the table is filled with the new rows.

© Copyright. All rights reserved.

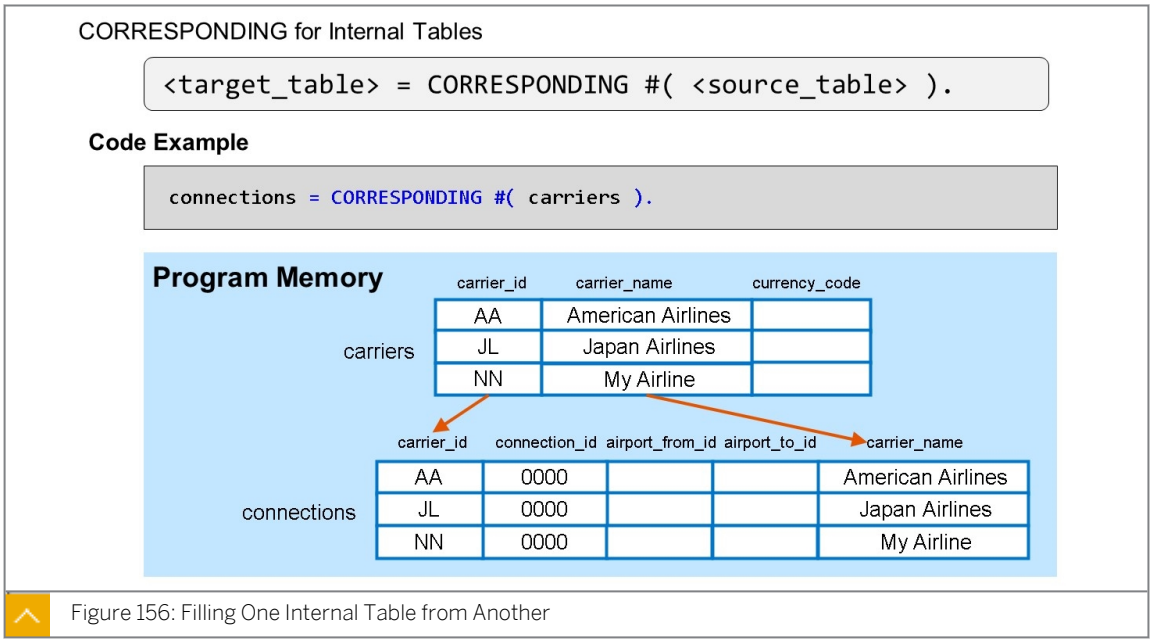


Figure 156: Filling One Internal Table from Another

To copy data between identically-named fields of two internal tables, use the CORRESPONDING operator. This works similarly to CORRESPONDING for structures: for each row of the source internal table, the system creates a new row in the target internal table and copies data between identically-named fields. Source fields for which there is no identically named field in the target are not copied. Target fields for which there is no identically named field in the source are filled with type-specific initial values.

In the example, the source internal table carriers contains three rows. Therefore, after the value assignment, the target internal table connections also contains three rows.

Fields carrier_id and carrier_name exist in both internal tables. They are copied from source to target. Field currency_code only exists in the source. It is not copied. Fields connection_id, airport_from_id, and airport_to_id exist only in the target. They are filled with initial values.

Note:
If the target internal table contains data before the assignment, the system deletes it.

© Copyright. All rights reserved.

Access to Complex Internal Tables

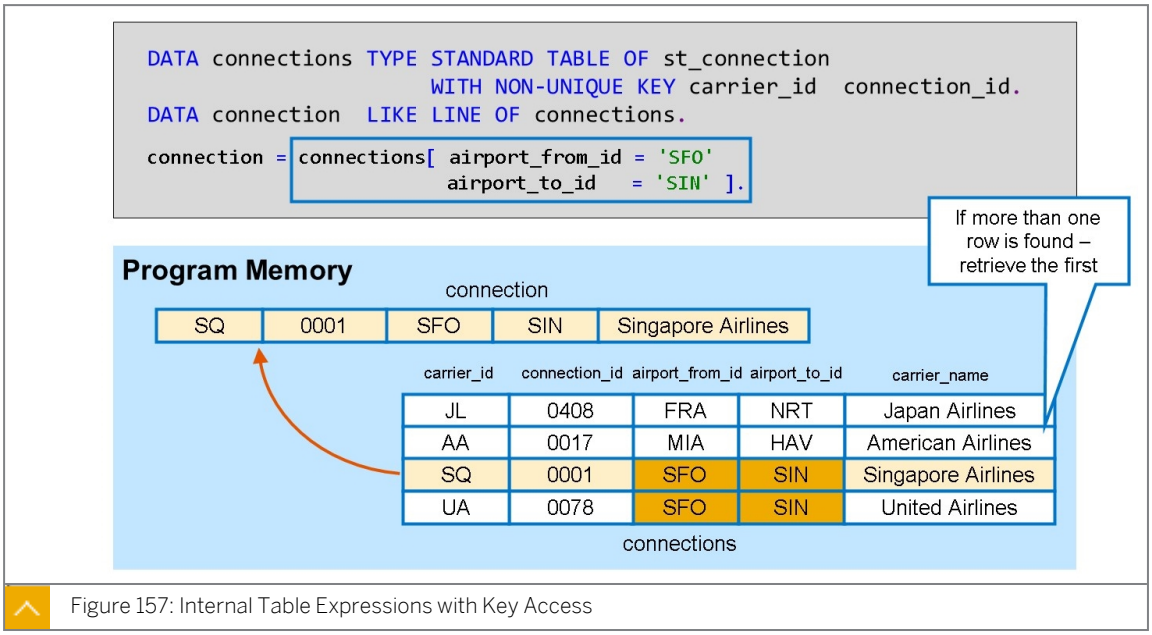


Figure 157: Internal Table Expressions with Key Access

Earlier in this course, you learned how to retrieve a single row from a simple internal table using an internal table expression. Back then we used an index access, that is, we identified the row through its position in the internal table. This index access works for complex internal tables in just the same way. For complex internal tables, however, internal table expressions with key access become important, where you identify the row through its content.

Note:
Even though this is called a key access, you can use any fields for the selection, not only key fields of the internal table. If more than one row fulfills the requirement, STANDARD tables return the first row, that is, the row with the lowest index.

The example reads a single row from internal table connections. The key of this internal table consists of fields, carrier_id and connection_id, but the key access uses airport_from_id and airport_to_id to identify the row. The Internal table contains two connections from airport SFO to SIN, so the first of them is returned.

Note:
Remember that the ABAP runtime raises exception CX_SY_ITAB_LINE_NOT_FOUND if none of the rows fulfills the requirement. Handle this exception in a TRY ... CATCH ... ENDTRY structure to avoid runtime errors

© Copyright. All rights reserved.

```
LOOP AT connections INTO connection
                    WHERE airport_from_id <> 'MIA'.
*   do something with the contents of connection
    ENDLOOP.
```

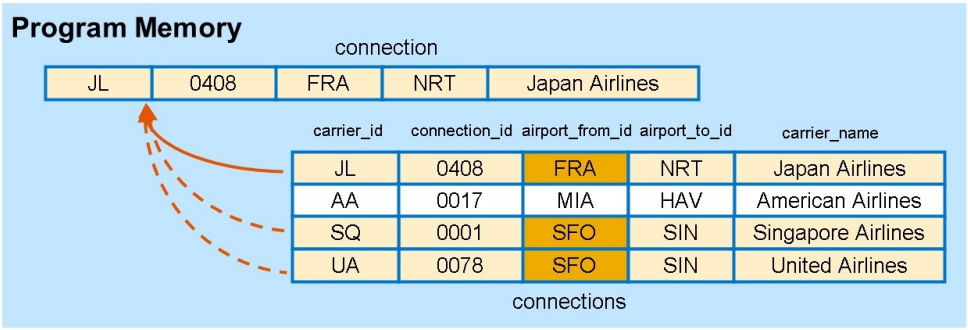


Figure 158: LOOP ... ENDLOOP with Key Access

To process multiple lines of an internal table by specifying fields, you use LOOP AT <internal table> INTO <target> WHERE <condition>. The WHERE condition can contain any number of constituent expressions joined using AND and OR. Within the expressions, you can use not just the equals operator (=) but also operators >, >=, <, <=, <> and BETWEEN.

MODIFY TABLE Statement

```
MODIFY TABLE <internal_table> FROM <work_area>.
```

Example

```
DATA carriers TYPE STANDARD TABLE OF st_carrier
                    WITH NON-UNIQUE KEY carrier_id.
DATA carrier LIKE LINE OF carriers.

 carrier = carriers[  carrier_id = 'JL' ].

 carrier-currency_code = 'JPY'.

MODIFY TABLE carriers FROM carrier.
```

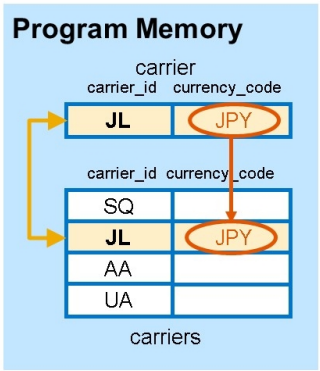


Figure 159: Updating a Table Row with Key Access

After reading the content of a table row into a work area, you sometimes want to write changes from the work area back into the internal table. One way to do this is the MODIFY TABLE statement.

This statement is a key access because the system uses the content of the key fields in the work area to identify the table row that needs to be modified. It then overwrites this table row with the contents of the work area.

In the example, the work area carrier contains value 'JL' in key field, carrier_id. Based on this value, the system identifies the second row to be updated. This row is then updated with the values from the work area.

© Copyright. All rights reserved.

Note:
You can only change non-key fields with MODIFY TABLE. The statement does not support changes to key fields.

MODIFY Statement

```
MODIFY <internal_table> FROM <work_area> [INDEX <number>].
```

Example

```
DATA carriers TYPE STANDARD TABLE OF st_carrier
                  WITH NON-UNIQUE KEY carrier_id.
DATA carrier LIKE LINE OF carriers.

carrier-carrier_id    = 'LH'.
carrier-currency_code = 'EUR'.

MODIFY carriers FROM carrier INDEX 1.
```

Program Memory

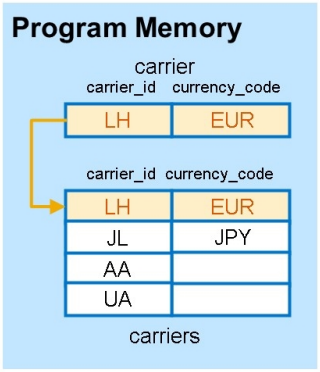


Figure 160: Updating a Table Row with Index Access

The MODIFY statement (without keyword TABLE!) does not distinguish between key fields and non-key fields. It overwrites the entire table row with new values from the work area. This statement is considered an index access because the row to be updated is identified by its position in the internal table. Usually, the index is specified explicitly using addition INDEX followed by an integer argument (literal, constant, variable, expression).

Note:
There is also a special variant without addition INDEX. We will discuss this variant next.

In the example, the MODIFY statement uses the INDEX addition to address the first table row. In this row, all fields are overwritten with the values from the work area, even key field carrier_id.

© Copyright. All rights reserved.

Short Form of MODIFY (without INDEX)

```
MODIFY <internal_table> FROM <work_area>.
```

Example

```
DATA carriers TYPE STANDARD TABLE OF st_carrier
                  WITH NON-UNIQUE KEY carrier_id.
DATA carrier LIKE LINE OF carriers.

 LOOP AT carriers INTO carrier
                  WHERE currency_code IS INITIAL.

     carrier-currency_code = 'USD'.

    MODIFY carriers FROM carrier.

 ENDLOOP.
```

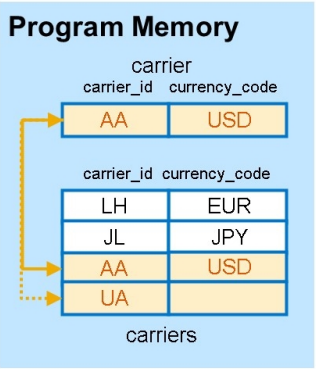


Figure 161: Updating Multiple Table Rows in a LOOP

There will often be times when you need to modify the contents of multiple rows of an internal table, or maybe even all of them. To do this, you implement a loop over the table, which places each row you need to change successively into a work area. Within the loop, you first change the contents of the work area and then write the changes back into the internal table using the MODIFY statement.

Note:
If you do not write your changes back into the table, the changes will be lost when the work area is filled with the data from next row.

In the example, the loop reads all rows of internal table carriers for which field currency_code is not yet filled. This is the case for the last two rows. For each of these rows the program replaces the initial value in field currency_code with the new value 'USD'. Finally, it uses the MODIFY statement to overwrite the current row with the updated values.

Instead of specifying the index explicitly, the code example uses a short form of the MODIFY statement where the INDEX addition is missing. This short form is only allowed between LOOP ... ENDLOOP. Only there the system can implicitly update the row it is currently working on.

Note:
If you use MODIFY without INDEX outside of LOOP...ENDLOOP, the system does not know which row to modify and triggers a non-catchable runtime error. To avoid such runtime errors, make sure not to ignore the related warning from the syntax check!

© Copyright. All rights reserved.

Complex Internal Tables in ABAP SQL

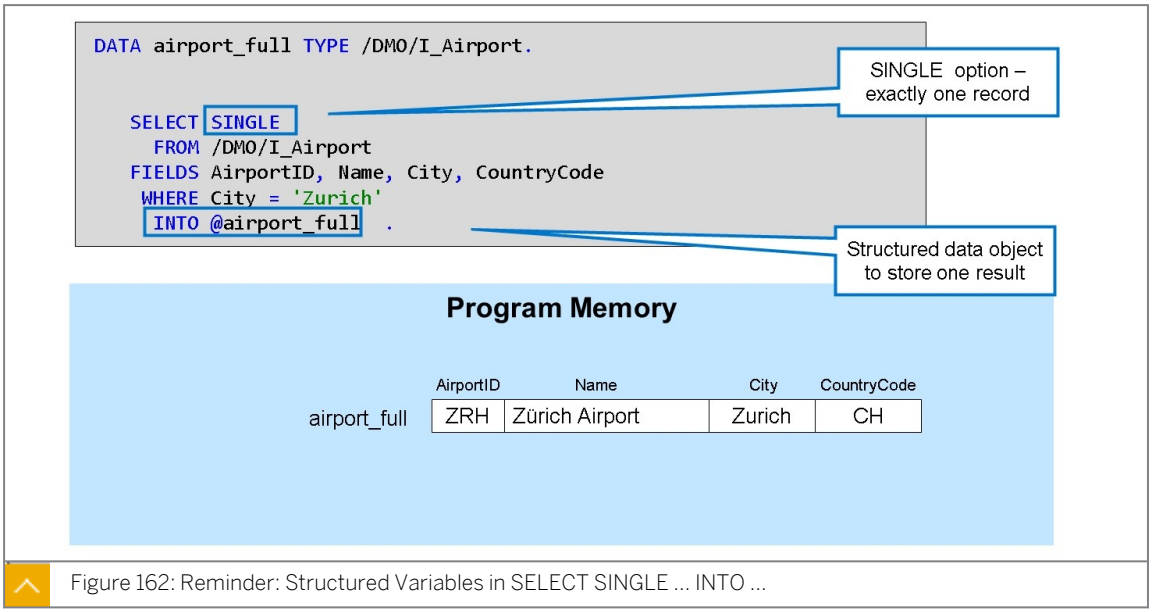


Figure 162: Reminder: Structured Variables in SELECT SINGLE ... INTO ...

The ABAP SQL statement, SELECT, reads data from a database table or a CDS View. When you use the SINGLE option, exactly one record is read from the database, even if more data exist that meets the conditions in the WHERE clause.

As you learned earlier, one way to receive this single record result is structured variable after keyword INTO.

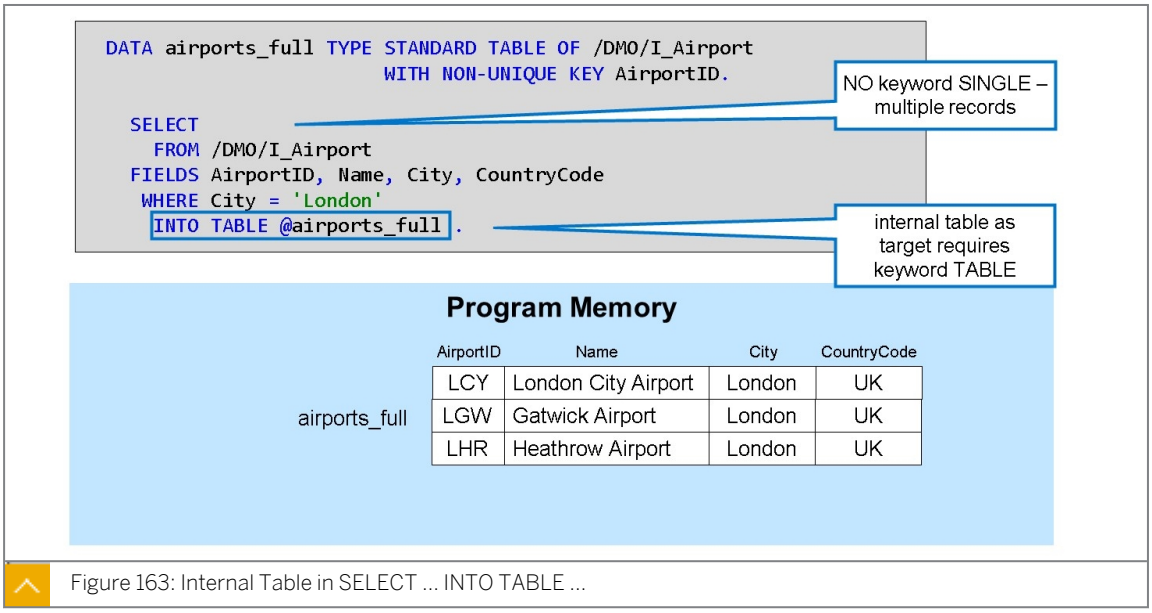


Figure 163: Internal Table in SELECT ... INTO TABLE ...

If you use SELECT without SINGLE, you indicate that you are interested in all records that match the conditions in the WHERE clause. You then have to make sure that you can actually receive and store multiple records. The obvious way to do this is the usage of a complex internal table as target of the SELECT statement. This is possible but it requires addition TABLE between keyword INTO and the name of the internal table.

© Copyright. All rights reserved.

In the example, we want to read all three airports related to London and not just a single one of them. Therefore, we leave out the keyword SINGLE after SELECT, add keyword TABLE after INTO and use internal table airports_full as target of the SELECT statement.

The example uses an explicit field list after FIELDS that matches the columns of internal table airports_full. Of course, you can also use FIELDS *, INTO CORRESPONDING FIELDS OF TABLE, and alias names in the field list.

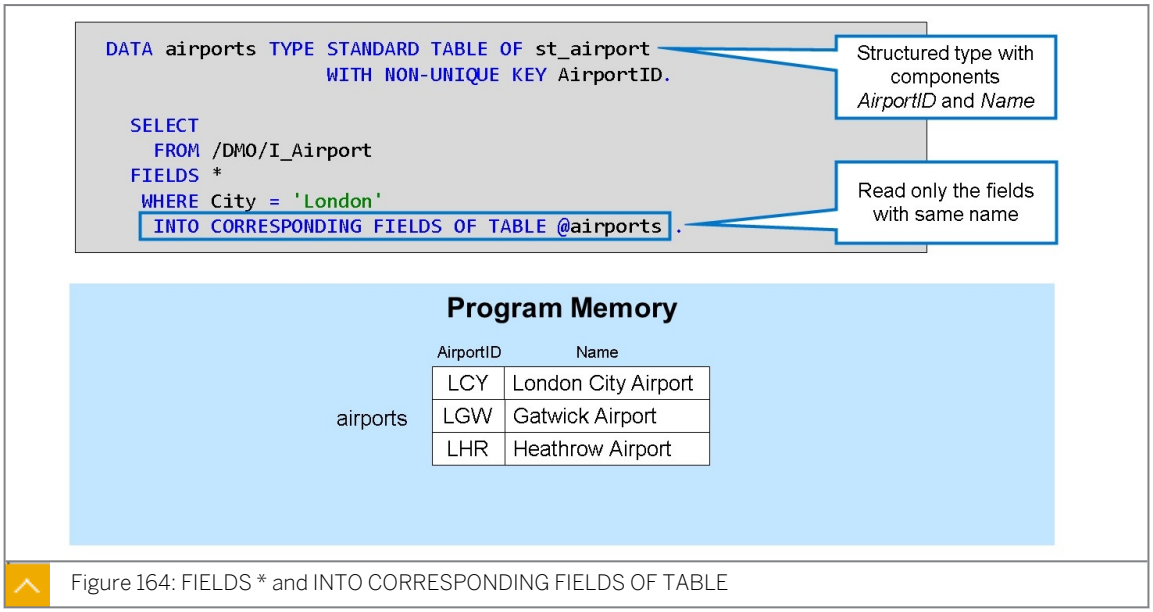


Figure 164: FIELDS * and INTO CORRESPONDING FIELDS OF TABLE

This example uses FIELDS * instead of an explicit field list and INTO CORRESPONDING FIELDS OF TABLE instead of INTO TABLE.

As the row type of internal table airports contains only two components AirportID and Name, only the fields with the same name are read from the database.

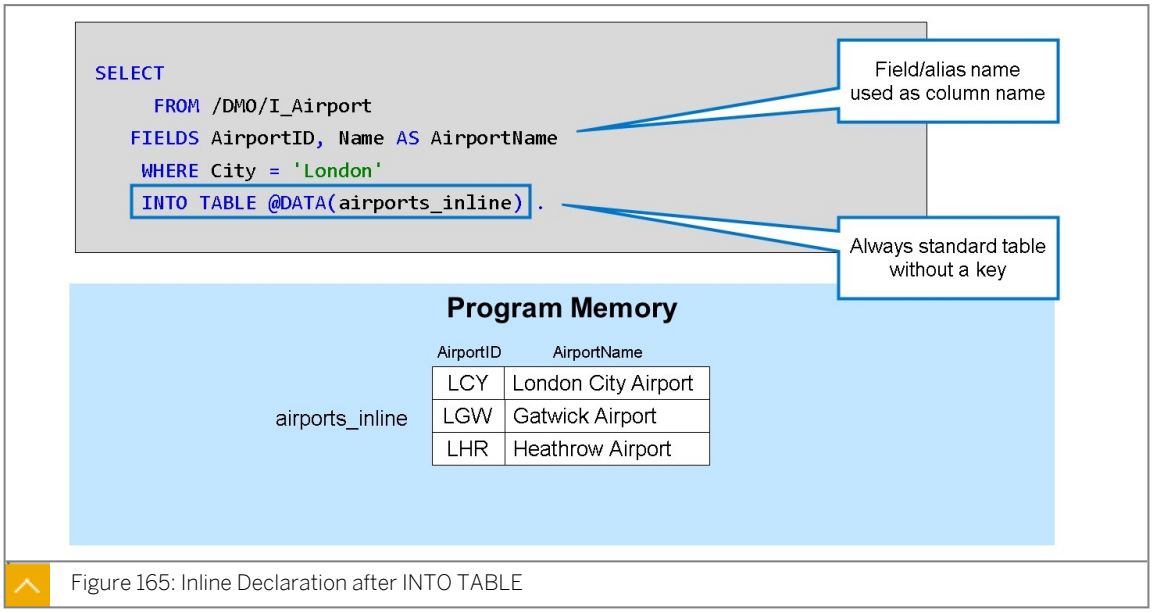


Figure 165: Inline Declaration after INTO TABLE

If you use DATA( ) in a SELECT statement after addition INTO TABLE, you inline declare an internal table. The row type of this internal table is derived from the FIELDS clause. For table

© Copyright. All rights reserved.

fields and view elements an alias name is optional. For expressions in the FIELDS clause, an alias name is mandatory if the INTO clause contains an inline declaration.

> Note:
> Inline declarations of internal tables are only supported after INTO TABLE. You cannot use inline declarations after INTO CORRESPONDING FIELDS OF TABLE.

> Note:
> Inline-declared internal tables are always standard tables without a key. You cannot declare sorted or hashed tables using inline declarations. This can cause performance problems if you fill the internal table with many rows and use key access a lot.

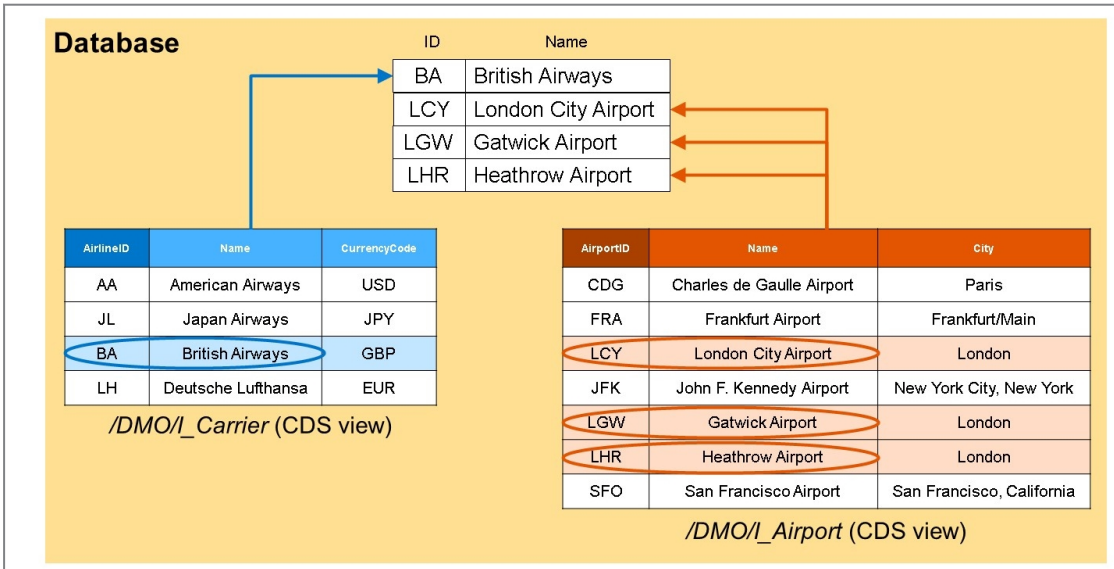


Figure 166: Combining Results from Several SELECTs (Unions)

When you are reading multiple records from the database, some special SQL techniques become particularly interesting. One of these techniques is the UNION directive to combine the results of several SELECT statements.

The figure illustrates the combination of two SELECT results:

The first SELECT result reads ID and NAME of all carriers with `CurrencyCode = 'GBP'`. The second SELECT reads ID and NAME of all airports with `City = 'London'`. The first SELECT returns one record, the second SELECT returns three records. Instead of retrieving these results separately, they are combined into one result with four records. It is important to point out that this happens inside the database.

A prerequisite for this technique is, of course, that the two results are compatible with each other, that is, that they have the same number of fields and the same field names. It is beneficial, though not necessary, that the types of the fields are also the same.

© Copyright. All rights reserved.

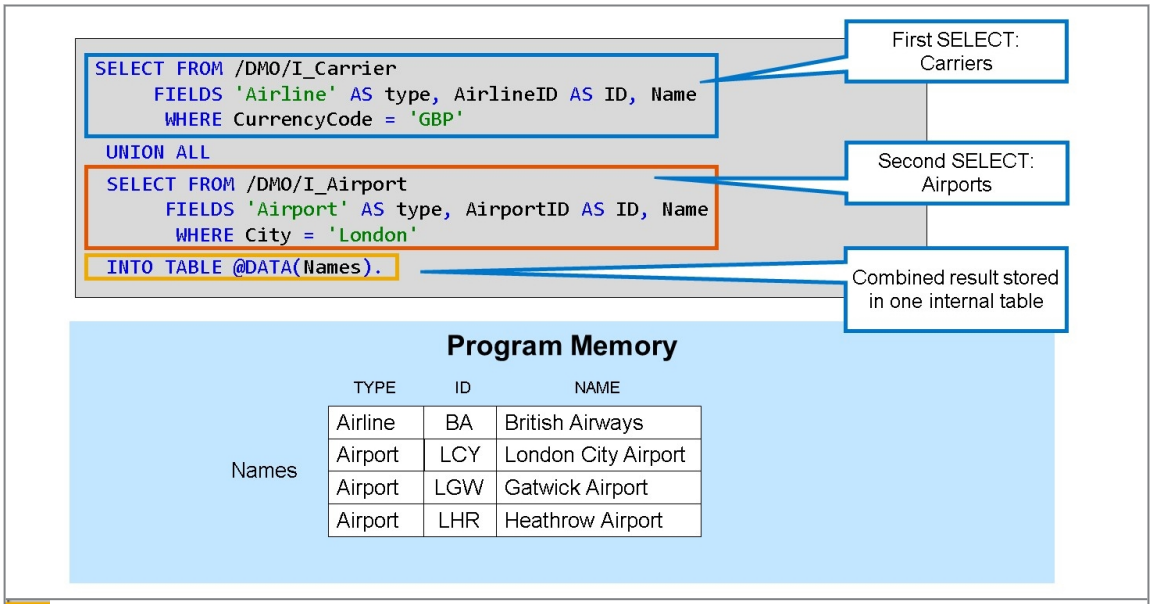


Figure 167: Example: UNION ALL of Two SELECTs

The ABAP SQL syntax for this example consists of two SELECT statements. Each SELECT statement has its own FROM clause, FIELDS clause, and WHERE clause, but there is only one INTO clause at the very end. The two SELECT statements are connected by keywords UNION ALL.

Note:
With UNION instead of UNION ALL, the database would look for and eliminate duplicates before returning the result. We use UNION ALL to avoid this unnecessary additional load on the database.

Both field lists consist of three elements, the first and second element have identical alias names in both FIELDS clauses. The third field does not need an alias because the field name is the same in both CDS views.

Note:
The first element in FIELDS is a literal text that allows us to distinguish between airlines and airports in the combined result.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 6
Exercise 15

Use a Complex Internal Table

In this exercise, you declare a table-like attribute, fill it using a SELECT statement and access the content.

Template:

/LRN/CL_S4D400_STS_STRUCTURE (global Class)

Solution:

/LRN/CL_S4D400_ITS_ITAB (global Class)

Task 1: Copy Template
Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_STS_STRUCTURE`** to a class in your own package (suggested name: **`ZCL_##_ITAB`**, where ## stands for your group number).

Task 2: Declare an Internal Table
In the local class, declare a static, table-like attribute **`airports`** to buffer detail information on all available airports. Begin by defining a private structure type **`st_airport`** and a table type **`tt_airports`** inside the local class.

1. Switch to the local class **`lcl_connection`**.
2. Define a **private** structure type **`st_airport`** with the following components:

   Table 5: Components of structure type st_airport:

Component Name	Data Type
`AirportID`	**`/dmo/airport_id`**
`Name`	**`/dmo/airport_name`**

3. Define a **private** table type **`tt_airports`** with the following properties:

   Table 6: Properties of table type tt_airports:

Property	Value
Line type	**`st_airport`**
Table kind	**`STANDARD TABLE`**
`Key Definition`	**`NON-UNIQUE DEFAULT KEY`**

4. Declare a new private static attribute **`airports`** and type it with table type **`tt_airports`**.

© Copyright. All rights reserved.

Task 3: Fill the Static Attribute in a Class Constructor
Define a class constructor and implement a SELECT statement that reads all available airports from the CDS view entity **`/DMO/I_Airport`** into the static attribute **`airports`**.

1. Add a class constructor to the local class **`lcl_connection`** using a quick fix.
2. In the class constructor, implement a SELECT statement that reads all data sets from the CDS view entity **`/DMO/I_Airport`** into the static attribute **`airports`** .

Task 4: Access the Content of the Internal Table
Use the content of internal table **`airports`** in the method **`get_output`** to add the airport names to the output.

1. Navigate to the implementation of the method **`get_output`**.
2. At the beginning of the method, read the details of the departure airport into a structured data object **`departure`**.

Hint:
Use a table expression `airports[ ... ]` and an inline declaration for the data object **`departure`**.

3. Similarly, read the details of the destination airport into a structured data object **`destination`**.
4. Use the component **`name`** of the two structures to add the airport names to the output.
5. Optional: Omit the structured data object and use the table expressions directly in the string templates.
6. Activate the class. Execute it and analyze the console output. Check that the output displays data for all attributes.

© Copyright. All rights reserved.

Unit 6
Solution 15

Use a Complex Internal Table

In this exercise, you declare a table-like attribute, fill it using a SELECT statement and access the content.

Template:

/LRN/CL_S4D400_STS_STRUCTURE (global Class)

Solution:

/LRN/CL_S4D400_ITS_ITAB (global Class)

Task 1: Copy Template

Copy the template class. Alternatively, copy your solution of the previous exercise.

1. Copy the class **`/LRN/CL_S4D400_STS_STRUCTURE`** to a class in your own package (suggested name: **`ZCL_##_ITAB`**, where ## stands for your group number).
   a) Open the source code of the global class **`/LRN/CL_S4D400_STS_STRUCTURE`**.
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click the class **`/LRN/CL_S4D400_STS_STRUCTURE`** to open the context menu.
   d) From the context menu, choose *Duplicate ...*.
   e) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **`ZCL_##_ITAB`**, where ## stands for your group number.
   f) Adjust the description and choose *Next*.
   g) Confirm the transport request and choose *Finish*.

Task 2: Declare an Internal Table

In the local class, declare a static, table-like attribute **`airports`** to buffer detail information on all available airports. Begin by defining a private structure type **`st_airport`** and a table type **`tt_airports`** inside the local class.

1. Switch to the local class **`lcl_connection`**.
   a) In the global class, choose *Local Types*.
2. Define a **private** structure type **`st_airport`** with the following components:

   Table 5: Components of structure type st_airport:

Component Name	Data Type
`AirportID`	**`/dmo/airport_id`**

© Copyright. All rights reserved.

Component Name	Data Type
Name	**/dmo/airport_name**

a) Adjust the code as follows:

```
TYPES:
   BEGIN OF st_details,
     DepartureAirport   TYPE /dmo/airport_from_id,
     DestinationAirport TYPE /dmo/airport_to_id,
     AirlineName        TYPE /dmo/carrier_name,
   END OF st_details.

 TYPES:
   BEGIN OF st_airport,
     AirportId TYPE /dmo/airport_id,
     Name      TYPE /dmo/airport_name,
   END OF st_airport.
```

3. Define a **private** table type **tt_airports** with the following properties:

Table 6: Properties of table type tt_airports:

Property	Value
Line type	**st_airport**
Table kind	**STANDARD TABLE**
Key Definition	**NON-UNIQUE DEFAULT KEY**

a) Adjust the code as follows:

```
TYPES:
     BEGIN OF st_airport,
       AirportId TYPE /dmo/airport_id,
       Name      TYPE /dmo/airport_name,
     END OF st_airport.

   TYPES tt_airports TYPE STANDARD TABLE OF st_airport
                          WITH NON-UNIQUE DEFAULT KEY.
```

4. Declare a new private static attribute **airports** and type it with table type **tt_airports**.
   a) At the end of the class definition, add the following code:

```
 CLASS-DATA airports TYPE tt_airports.
```

Task 3: Fill the Static Attribute in a Class Constructor

Define a class constructor and implement a SELECT statement that reads all available airports from the CDS view entity **/DMO/I_Airport** into the static attribute **airports**.

1. Add a class constructor to the local class **lcl_connection** using a quick fix.
   a) Place the cursor on the name of the class and press **Ctrl + 1**.
   b) Double-click on *Generate class constructor*.
2. In the class constructor, implement a SELECT statement that reads all data sets from the CDS view entity **/DMO/I_Airport** into the static attribute **airports** .

© Copyright. All rights reserved.

a) Navigate to the implementation of the method **class_constructor**.

b) Inside the method implementation, add the following code:

```
    SELECT FROM /DMO/I_Airport
         FIELDS AirportID, Name
           INTO TABLE @airports.
```

Task 4: Access the Content of the Internal Table

Use the content of internal table **airports** in the method **get_output** to add the airport names to the output.

1. Navigate to the implementation of the method **get_output**.
   a) Proceed as you have done in previous exercises.
2. At the beginning of the method, read the details of the departure airport into a structured data object **departure**.

Hint:
Use a table expression `airports[ ... ]` and an inline declaration for the data object **departure**.

   a) At the beginning of the method, add the following code:

```
  DATA(departure)   = airports[ airportID = details-departureairport ].
```

3. Similarly, read the details of the destination airport into a structured data object **destination**.
   a) Adjust the code as follows:

```
  DATA(departure)   = airports[ airportID = details-
departureairport   ].
  DATA(destination) = airports[ airportID = details-
destinationairport ].
```

4. Use the component **name** of the two structures to add the airport names to the output.
   a) Adjust the code as follows:

```
    APPEND |Departure:   { details-departureairport   } { departure-
name   }| TO r_output.
    APPEND |Destination: { details-destinationairport } { destination-
name }| TO r_output.
```

5. Optional: Omit the structured data object and use the table expressions directly in the string templates.
   a) Comment the code lines where you fill the structures **departure** and **destination**.
   b) In the string template replace `departure-name` with `airports[ airportid = details-departureairport ]-name`.
   c) Similarly replace `destination-name` with `airports[ airportid = details-destinationairport ]-name`.
6. Activate the class. Execute it and analyze the console output. Check that the output displays data for all attributes.

© Copyright. All rights reserved.

a) Press **Ctrl + F3** to activate the class.

b) Press **F9** to run the class.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Fill complex internal tables with data.
- Access the content of complex internal tables.
- Use complex internal tables in ABAP SQL.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Learning Assessment

1. Which of the following statements is true for a standard internal table?

   Choose the correct answer.

   - ☐ A The key is always unique
   - ☐ B The key is always non-unique
   - ☐ C You can choose whether the key should be unique or non-unique

2. The type tt_table is defined as follows: TYPES tt_table TYPE STANDARD TABLE OF st_connection WITH NON-UNIQUE KEY carrier_id connection_id. Which DATA statement would you use to create an internal table with this type?

   Choose the correct answer.

   - ☐ A DATA table TYPE tt_table.
   - ☐ B DATA table TYPE TABLE OF tt_table.
   - ☐ C DATA table TYPE REF TO tt_table.

3. Which of the following can you use to fill an internal table?

   Choose the correct answers.

   - ☐ A READ
   - ☐ B VALUE #( )
   - ☐ C LOOP...ENDLOOP
   - ☐ D APPEND

© Copyright. All rights reserved.

4. You want to read data from two database tables so that the SELECT statement returns a single result set that contains no duplicate entries. Which of the following techniques would you use?

   Choose the correct answer.

   - ☐ A LEFT OUTER JOIN
   - ☐ B UNION
   - ☐ C INNER JOIN
   - ☐ D UNION ALL

© Copyright. All rights reserved.

Learning Assessment - Answers

1. Which of the following statements is true for a standard internal table?

   Choose the correct answer.

   - [ ] A The key is always unique
   - [x] B The key is always non-unique
   - [ ] C You can choose whether the key should be unique or non-unique

   That's right.

2. The type tt_table is defined as follows: TYPES tt_table TYPE STANDARD TABLE OF st_connection WITH NON-UNIQUE KEY carrier_id connection_id. Which DATA statement would you use to create an internal table with this type?

   Choose the correct answer.

   - [x] A DATA table TYPE tt_table.
   - [ ] B DATA table TYPE TABLE OF tt_table.
   - [ ] C DATA table TYPE REF TO tt_table.

   That's right.

3. Which of the following can you use to fill an internal table?

   Choose the correct answers.

   - [ ] A READ
   - [x] B VALUE #( )
   - [ ] C LOOP...ENDLOOP
   - [x] D APPEND

© Copyright. All rights reserved.

4. You want to read data from two database tables so that the SELECT statement returns a single result set that contains no duplicate entries. Which of the following techniques would you use?

   Choose the correct answer.

   - [ ] A LEFT OUTER JOIN
   - [x] B UNION
   - [ ] C INNER JOIN
   - [ ] D UNION ALL

   That's right.

© Copyright. All rights reserved.

UNIT 7 Implementing Database Updates Using Business Objects

Lesson 1

Analyzing a Business Object 288
Exercise 16: Analyze a Business Object 295

Lesson 2

Using the Entity Manipulation Language 304
Exercise 17: Modify Data Using EML 311

UNIT OBJECTIVES

- Analyze a business object.
- Implement an EML statement.

© Copyright. All rights reserved.

Unit 7
Lesson 1

Analyzing a Business Object

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Analyze a business object.

Business Objects

In the **ABAP RESTful application programming model**, a business object defines a particular entity, such as a travel agency. Its definition has two parts; one or more CDS views, which define the structure of the object or, in other words, the fields that it contains, and a behavior definition, which describes what you can do with the business object.

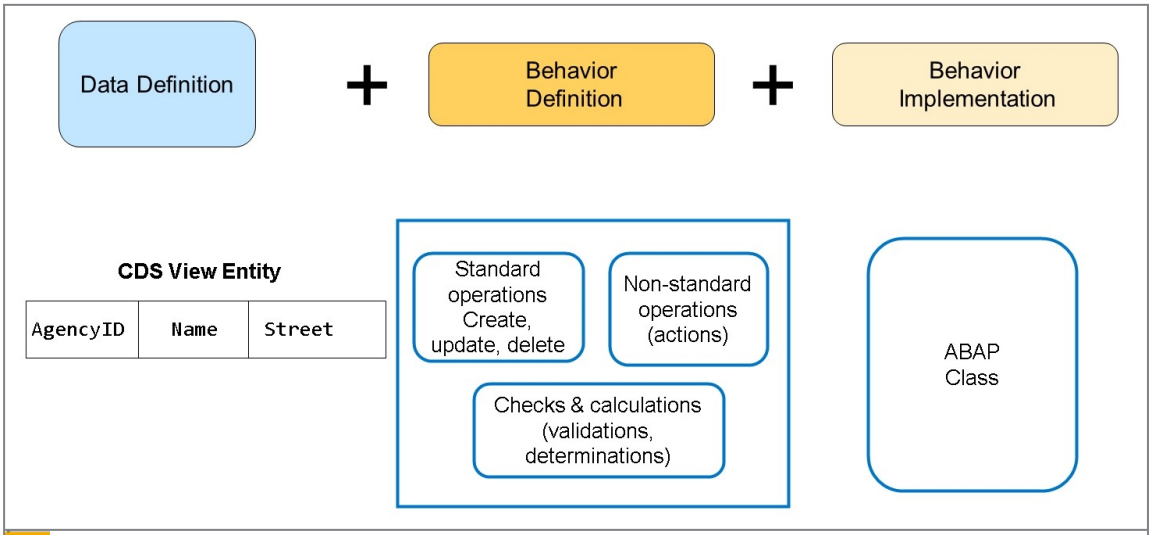


Figure 168: Business Objects

The **behavior definition** specifies which of the standard operations, create, update, delete are allowed. It can also contain the definition of validations, determinations, and actions. Validations check that the data is correct when you create or update a record. Determinations modify instances of business objects based on trigger conditions. Actions are non-standard operations which you use to provide customized, business-logic-specific behavior. Approving a purchase order or canceling a flight are activities that you would implement as an action.

The **behavior implementation** consists of one or more ABAP classes. Here the validations, determinations, and actions are implemented. When it comes to the standard operations, the model distinguishes two implementation scenarios: In the **unmanaged** implementation scenario, create, update, and delete are implemented in the behavior implementation. In the **managed** implementation scenario the runtime takes care of them.

Business objects are commonly used to provide the transactional logic for Fiori Elements apps or Web APIs. However, you can also access them from ABAP coding using the **Entity**

© Copyright. All rights reserved.

Manipulation Language (EML). This is a set of ABAP statements that allows you to create, read, update, and delete data using business objects.

> Note:
> EML is also able to access the application data from inside the behavior implementation of a business object.

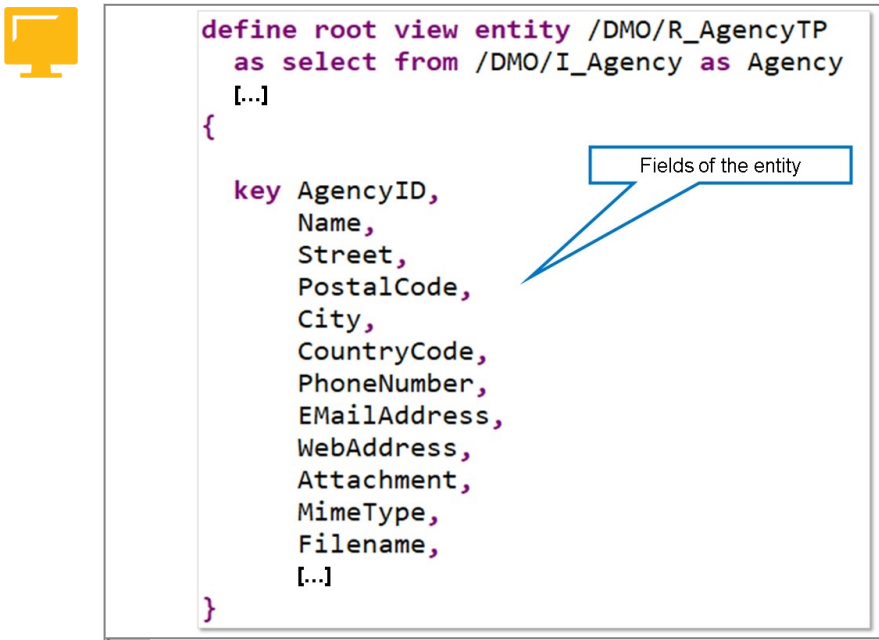


Figure 169: CDS View Entity /DMO/R_AgencyTP

In this unit, you will create a class that uses a business object to modify travel agency data. The view entity contains the key field AgencyID and various other fields containing information about the travel agency.

© Copyright. All rights reserved.

Behavior Definition and Implementation

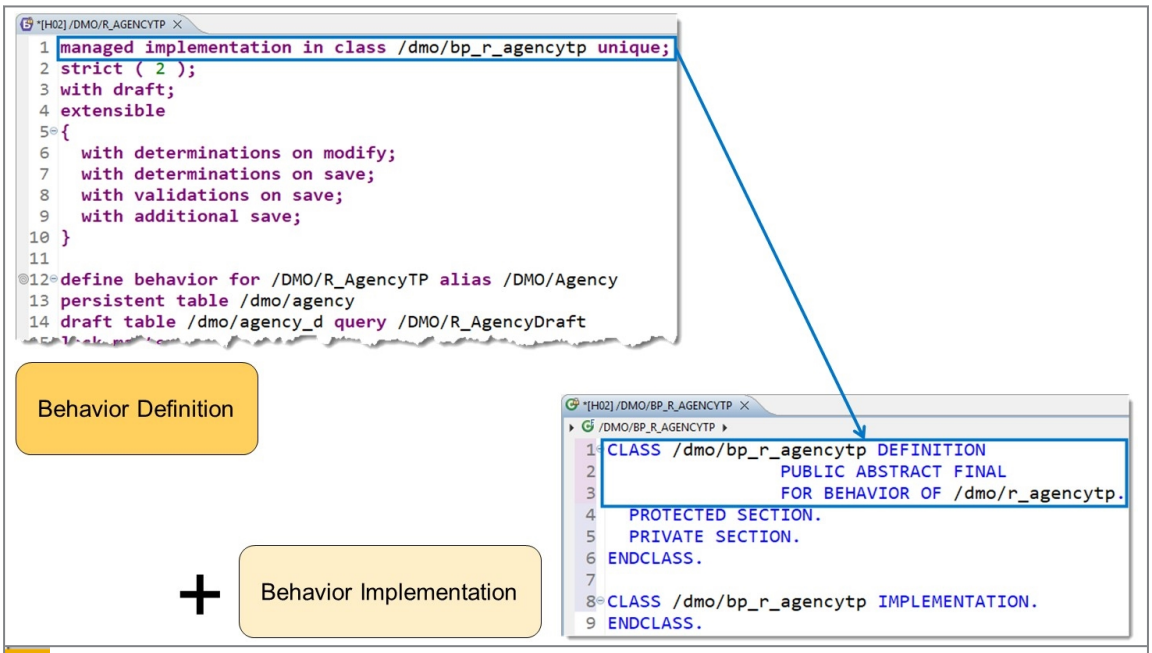


Figure 170: Behavior Definition and Implementation

There are two parts to the behavior of a business object: the behavior definition and the behavior implementation. The behavior definition contains information about what the business object can do, while the behavior implementation contains the actual coding that the system executes.

The behavior implementation is an ABAP class. You declare the class in the behavior definition in the statement `managed implementation in class <class> unique`. The actual coding of the behavior implementation is contained in a local class within the global class that you specify.

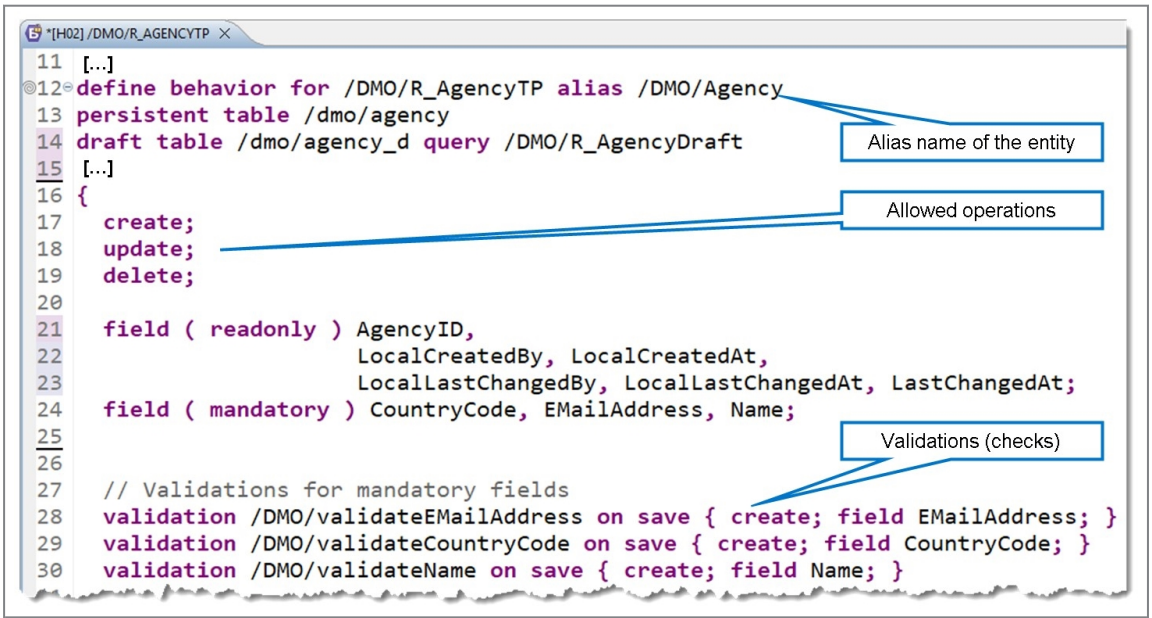


Figure 171: Behavior Definition

© Copyright. All rights reserved.

A behavior definition is an essential component of a business object. It describes which of the standard operations are allowed, for example create, update, delete. It also defines checks (validations) that are performed when you create or change data.

The example here is the behavior definition for the travel agency. At the beginning of the definition, you can see the name of the CDS view we just looked at, and that there is an alias defined for it. This is important, since it is the alias name that is used to address the entity with EML.

The behavior definition also links the CDS entity to the database table in which the data is stored. In this case, the business object uses two tables: one for active data and one for drafts (data that is incomplete and has not been checked). There is also information that is relevant for locking the data, authorization checks, and concurrency control. We are not going to look at this information in depth - you just need to know that the runtime can take care of these issues. You can also generate CDS entities and behavior definitions based on the definition of a database table and, in this case, locking, authorization checks, and concurrency controls are dealt with automatically.

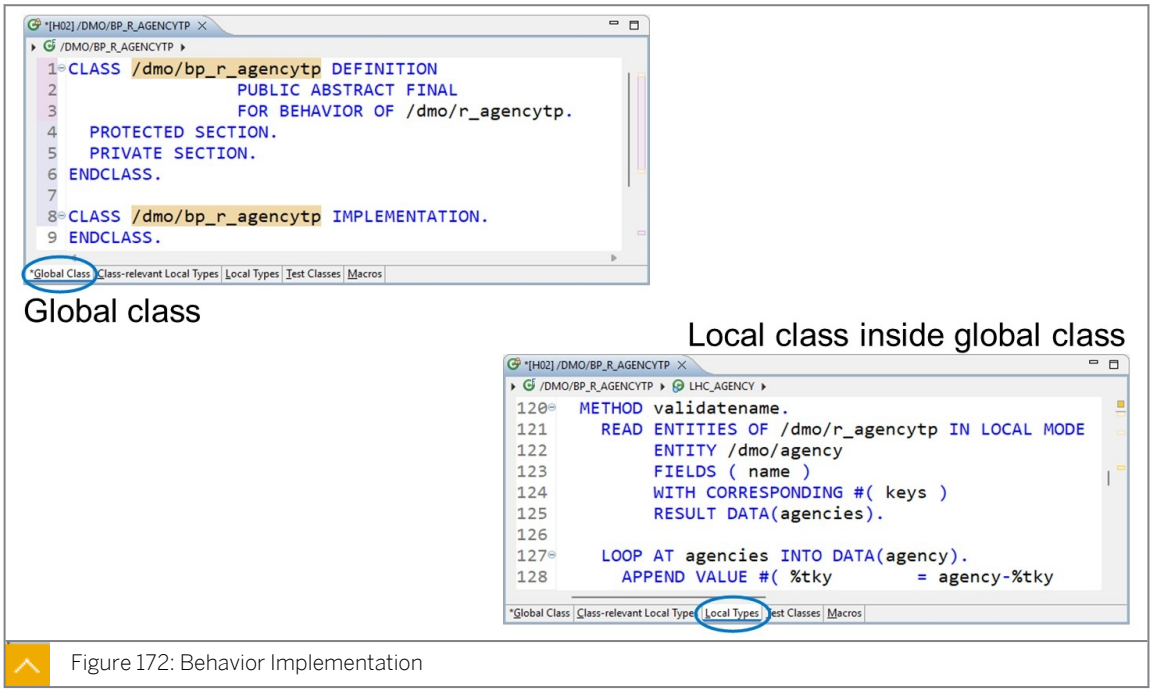


Figure 172: Behavior Implementation

The global class of the behavior implementation (also known as a behavior pool) is just an empty class definition with the special addition FOR BEHAVIOR OF followed by the name of the behavior definition. The actual implementation of the behavior definition is a local class within the global class definition. You access the class by clicking the *Local Types* tab.

The behavior implementation contains code that is specific to the business object, for example, the implementation for validations, determinations, and actions. Whether it also contains code for the standard operations (create, update, delete, and lock) depends on the details of the behavior definition. The behavior implementation for our business object does not contain code for the standard operations. This is because the business object uses the managed implementation type in which the runtime deals with the standard operations.

© Copyright. All rights reserved.

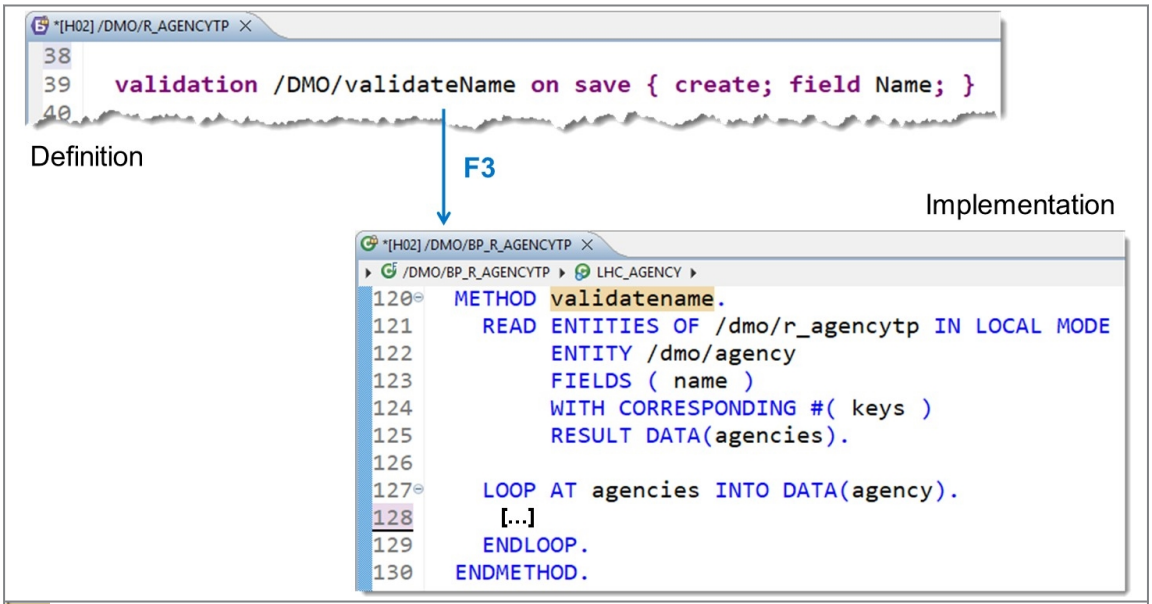


Figure 173: Validations

A validation is a check that the runtime performs when data is changed. Here, the validation is always performed when a new record is created (trigger `create;`) If an existing record is changed, the validation is only performed if the *Name* field has been changed (trigger `field Name;`).

Validations are defined in the behavior definition. For each validation, there is a corresponding method in the behavior implementation.

BO Projections and BO Interfaces

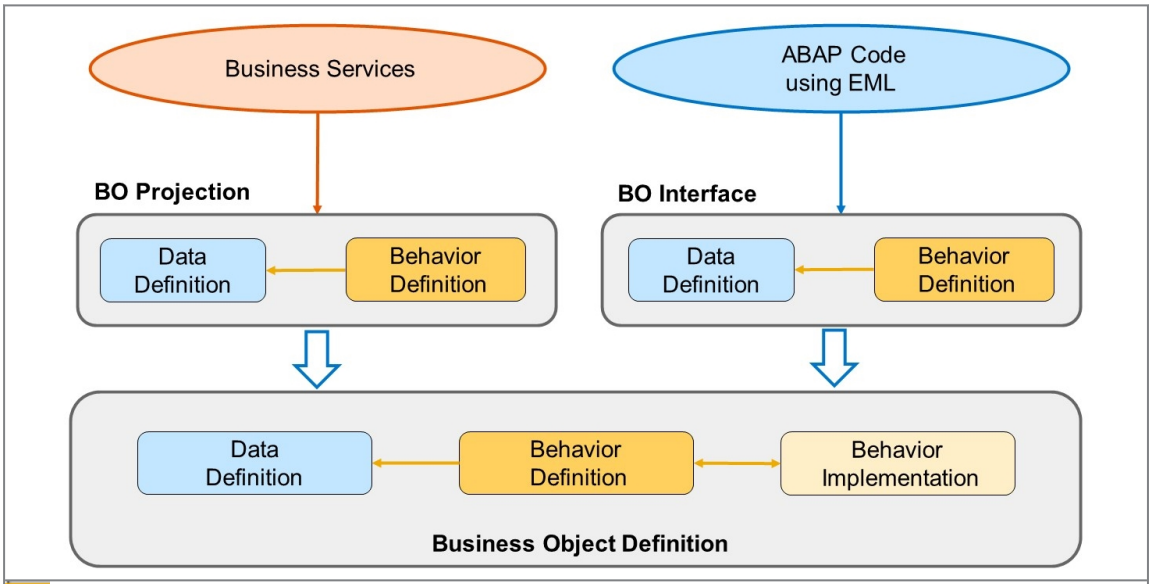


Figure 174: Consumption of Business Objects

In the ABAP RESTful application programming model, there are two important ways to consume a Business Object:

- Through a **Business Service**, for example an OData UI Service for an SAP Fiori Application.
- From inside ABAP Code, using the **Entity Manipulation Language (EML)**.

© Copyright. All rights reserved.

Although technically possible, a Business Object should not be consumed directly. Instead, consumers should access **Business Object Projections** (BO Projections) and **Business Object Interfaces** (BO Interfaces) as follows:

BO Projection

Business Services should always define a service-specific projection of the Business Object. The BO projection specifies which subset of the business object's data and operations are available through this service. A BO Projection can also contain the definition and implementation of service-specific data and behavior.

> Note:
> A BO Projection is either based on the Business Object directly or on a BO Interface.

BO Interface

A Business Object Interface provides stable access to a business object's data and operations. BO Interfaces are typically released for use in other software components. If a BO Interface exists for a given Business Object, ABAP code using EML should always be used to access the BO Interface.

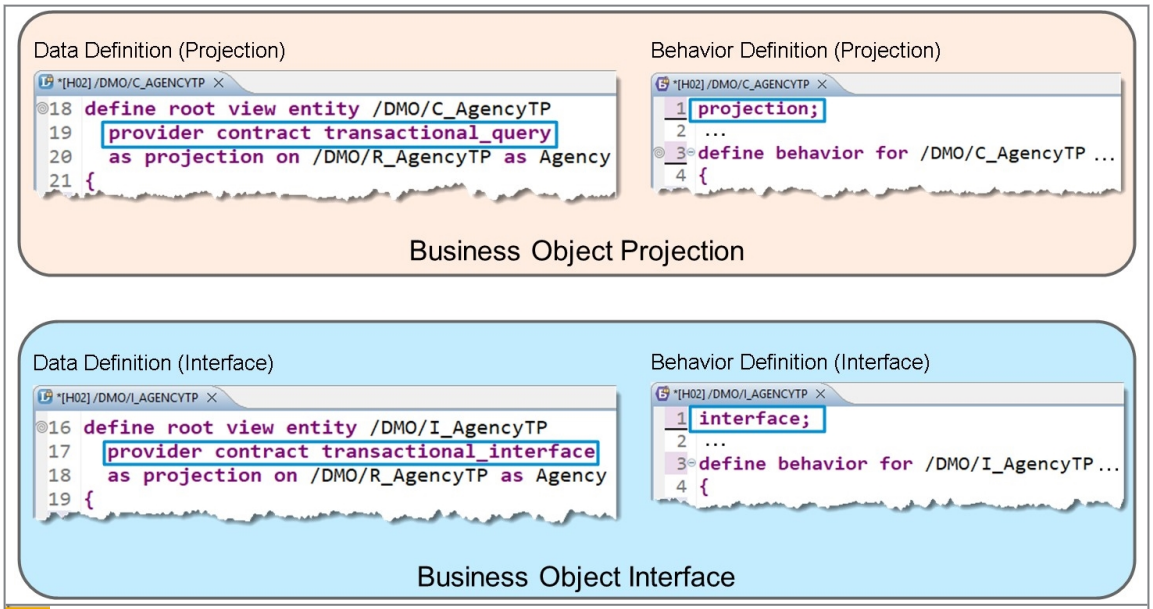


Figure 175: BO Interfaces and BO Projections

Like a Business Object, a BO Projection consists of two parts: one or more CDS view entities (defined in Data Definitions), and a Behavior Definition. The same is true for BO Interfaces. They also consist of one or more CDS views and a Behavior Definition.

The easiest way to identify projections and interfaces is by looking at their behavior definitions: Behavior definitions for projections start with the keyword `projection`, behavior definitions for interfaces start with the keyword `interface`.

The CDS views of BO projections and the CDS views of BO interfaces are always **CDS Projection Views**. This means that their definitions contain the addition `as projection on` where the definitions of ordinary CDS views use `as select from`. In newer releases, the use case of a projection view is specified using the addition `provider contract` after the view entity name as follows:

© Copyright. All rights reserved.

- `provider contract transactional_interface` for BO Interfaces
- `provider contract transactional_query` for BO Projections

Hint:
For SAP developments, the following naming convention applies:

- <namespace>C_<...> for BO Projections
- <namespace>I_<...> for BO Interfaces
- <namespace>R_<...> for BO Definitions

© Copyright. All rights reserved.

Unit 7
Exercise 16

Analyze a Business Object

In this exercise, you analyze the business object interface /DMO/I_AgencyTP to find out about its structure and the data manipulation operations it offers.

Task 1: Analyze the Interface Behavior
Analyze the behavior definition of the business object interface **`/DMO/I_AGENCYTP`**.

1. Open the behavior definition **`/DMO/I_AGENCYTP`** in the editor.
2. Analyze the source code. Open the ABAP language help to find more information.

   What is the purpose of the statements `interface;`, `use draft;`, and `extensible`?

   _____
   _____
   _____

   How many entities does the business object interface consist of? What are the alias names of these entities?

   _____
   _____
   _____

   Which standard operations are accessible through this interface?

   _____
   _____
   _____

   Where do you find the non-standard operations that the interface offers?

   _____
   _____
   _____

Task 2: Analyze the Data Structure
Analyze the CDS view entity that defines the data structure of the business object interface. Navigate to its data source until you reach a CDS view entity that is not a projection.

1. Navigate to the CDS view entity **`/DMO/I_AgencyTP`** and analyze the source code.

© Copyright. All rights reserved.

Which are the key elements of the CDS view entity?

Is the data structure of the business object interface also extensible?

What is the data source of the CDS view entity **/DMO/I_AgencyTP**?

2. Navigate to the data source of the CDS view entity **/DMO/I_AgencyTP** and analyze the source code.

   Is the CDS view entity **/DMO/R_AgencyTP** also extensible?

   What is the data source of the CDS view entity **/DMO/R_AgencyTP**?

   Does CDS view entity **/DMO/R_AgencyTP** belong to a business object interface or a business object projection?

Task 3: Analyze the Business Object Behavior

Analyze the behavior definition of the business object **/DMO/R_AGENCYTP** which lies under business object interface **/DMO/I_AGENCYTP**.

1. Open the behavior definition **/DMO/R_AGENCYTP** in the editor.
2. Analyze the source code. Open the ABAP language help to find more information.

© Copyright. All rights reserved.

What is the purpose of the statement `managed ...;`?

What is the purpose of the addition `... implementation in class /dmo/bp_r_agencytp unique;`

Are there any elements that must not be changed?

What is the purpose of the statement `validation /DMO/validateName on save ... ;`

What is the purpose of the addition `{ create; field Name; }`

Task 4: Analyze the Behavior Implementation

Analyze the behavior implementation of the business object **`/DMO/R_AGENCYTP`**.

1. Navigate to the ABAP class **`/DMO/BP_R_AGENCYTP`** that contains the behavior implementation of the business object **`/DMO/R_AGENCYTP`**.

   Why are the definition and implementation of the global class **`/dmo/bp_r_agencytp`** both empty?

2. Go to the local class **`lhc_agency`** and analyze the definition.

© Copyright. All rights reserved.

How many methods are defined in the private section of local class **`lhc_agency`**? What is their purpose?

© Copyright. All rights reserved.

Unit 7
Solution 16

Analyze a Business Object

In this exercise, you analyze the business object interface /DMO/I_AgencyTP to find out about its structure and the data manipulation operations it offers.

Task 1: Analyze the Interface Behavior
Analyze the behavior definition of the business object interface **`/DMO/I_AGENCYTP`**.

1. Open the behavior definition **`/DMO/I_AGENCYTP`** in the editor.
2. Analyze the source code. Open the ABAP language help to find more information.

   What is the purpose of the statements `interface;`, `use draft;`, and `extensible`?

   `interface;` states that this behavior definition does not belong to the business object itself but rather to a released interface on top of the business object. `use draft;` stipulates that this interface supports the handling of interim versions of data (draft instances). `extensible` indicates that extensions of this interface by partners and customers are supported.

   How many entities does the business object interface consist of? What are the alias names of these entities?

   The business object interface consists of only one entity because the behavior definition contains only one `DEFINE BEHAVIOR FOR ...` statement. The alias name is **`/DMO/Agency`**.

   Which standard operations are accessible through this interface?

   The interface offers operations create, update, delete because the behavior contains the statements `use create;`, `use update;`, and `use delete;`

   Where do you find the non-standard operations that the interface offers?

   Non-standard operations are listed after the keyword `use action`. Note, that all actions in this interface are draft actions. They are related to the handling of interim versions of data.

Task 2: Analyze the Data Structure
Analyze the CDS view entity that defines the data structure of the business object interface. Navigate to its data source until you reach a CDS view entity that is not a projection.

© Copyright. All rights reserved.

1. Navigate to the CDS view entity **/DMO/I_AgencyTP** and analyze the source code.
   a) Go to the statement `define behavior for /DMO/I_AgencyTP`.
   b) Hold down the **Ctrl** key and left-click on `/DMO/I_AgencyTP`. Alternatively, place the cursor on `/DMO/I_AgencyTP` and press **F3.**.

   Which are the key elements of the CDS view entity?

   There is only one element with the prefix `key`. The name of this view element is **AgencyID**.

   Is the data structure of the business object interface also extensible?

   Yes, there is a group of annotations starting with `@AbapCatalog.extensibility` with annotation `extensible: true`.

   What is the data source of the CDS view entity **/DMO/I_AgencyTP**?

   CDS view entity **/DMO/I_AgencyTP** is a projection on CDS view entity **/DMO/R_AgencyTP**.

2. Navigate to the data source of the CDS view entity **/DMO/I_AgencyTP** and analyze the source code.
   a) Go to the code fragment `as projection on /DMO/R_AgencyTP`.
   b) Hold down the **Ctrl** key and left-click on `/DMO/R_AgencyTP`. Alternatively, place the cursor on `/DMO/R_AgencyTP` and press **F3.**.

   Is the CDS view entity **/DMO/R_AgencyTP** also extensible?

   Yes, it also contains annotation `extensible : true`.

   What is the data source of the CDS view entity **/DMO/R_AgencyTP**?

   The CDS view entity **/DMO/R_AgencyTP** reads from the CDS view entity **/DMO/I_Agency**.

   Does CDS view entity **/DMO/R_AgencyTP** belong to a business object interface or a business object projection?

   No. CDS view entity **/DMO/R_AgencyTP** reads its data using the keyword `as select from` and not with `as projection on`. The CDS view entity **/DMO/R_AgencyTP** belongs to the definition of the business object itself.

© Copyright. All rights reserved.

Task 3: Analyze the Business Object Behavior

Analyze the behavior definition of the business object **/DMO/R_AGENCYTP** which lies under business object interface **/DMO/I_AGENCYTP**.

1. Open the behavior definition **/DMO/R_AGENCYTP** in the editor.
2. Analyze the source code. Open the ABAP language help to find more information.

   What is the purpose of the statement `managed ...;`?

   `managed` determines that this is a business object and not an interface or a projection. It also indicates that for this business object the standard operations create, update, delete are handled by the framework.

   What is the purpose of the addition `... implementation in class /dmo/bp_r_agencytp unique;`

   `... implementation in class /dmo/bp_r_agencytp` specifies the ABAP class for the behavior implementation of this business object. Note that the addition `unique` is mandatory and defines that each operation can be implemented exactly once.

   Are there any elements that must not be changed?

   Yes, the statement `field ( readonly ) ...` declares that the elements **AgencyID**, **LocalCreatedBy**, **LocalCreatedAt**, **LocalLastChangedBy**, **LocalLastChangedAt**, and **LastChangedAt** must not be changed.

   What is the purpose of the statement `validation /DMO/validateName on save ... ;`

   This statement defines a consistency check (validation) for the business object. The name indicates that it checks the content of the element **Name** of the CDS view entity **/DMO/R_AgencyTP**.

   What is the purpose of the addition `{ create; field Name; }`

   The addition defines the trigger conditions: When a new data set is created (standard operation create) the check is always performed. When a data set is changed (standard operation update) the check is only performed in case the value of the element **Name** has changed.

Task 4: Analyze the Behavior Implementation

Analyze the behavior implementation of the business object **/DMO/R_AGENCYTP**.

© Copyright. All rights reserved.

1. Navigate to the ABAP class **/DMO/BP_R_AGENCYTP** that contains the behavior implementation of the business object **/DMO/R_AGENCYTP**.
   a) Go to the statement `managed implementation in class /dmo/bp_r_agencytp unique;`.
   b) Hold down the **Ctrl** key and left-click on `/dmo/bp_r_agencytp`. Alternatively, place the cursor on `/dmo/bp_r_agencytp` and press **F3.**.

   Why are the definition and implementation of the global class **/dmo/bp_r_agencytp** both empty?

   The global class **/dmo/bp_r_agencytp** is a behavior pool. It serves as a wrapper for one or more local classes. The behavior implementation itself is located in those local classes.

2. Go to the local class **lhc_agency** and analyze the definition.
   a) Go to the tab *Local Types*. Alternatively, expand root node */DMO/BP_R_AGENCYTP* in the *Outline* view and choose *LHC_AGENCY*.

   How many methods are defined in the private section of local class **lhc_agency**? What is their purpose?

   There are five methods. Four methods implement validations (addition `FOR VALIDATE ON SAVE`). One method implements authorization checks (addition `FOR GLOBAL AUTHORIZATION`).

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Analyze a business object.

© Copyright. All rights reserved.

Unit 7
Lesson 2

Using the Entity Manipulation Language

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Implement an EML statement.

Entity Manipulation Language (EML)

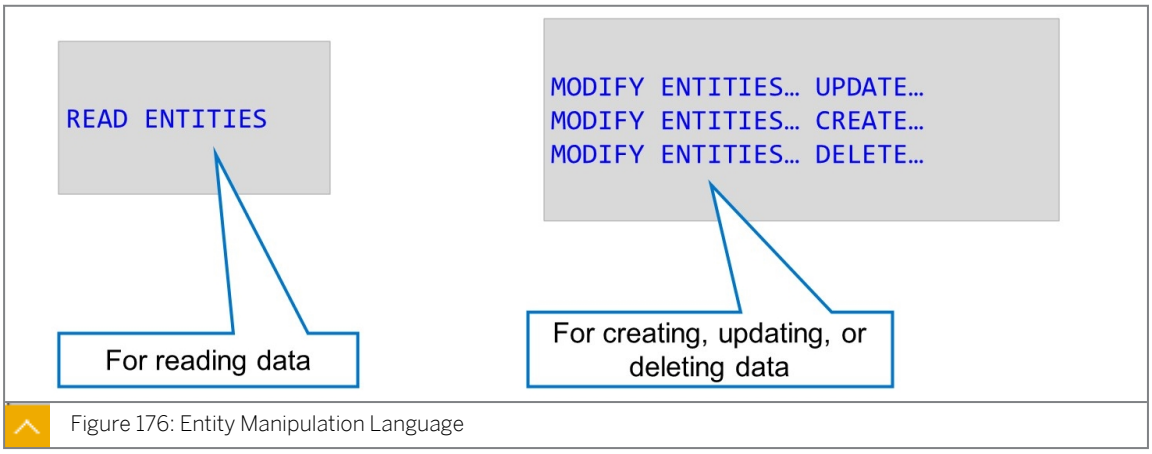


Figure 176: Entity Manipulation Language

EML consists of statements that you can use to manipulate the data of a business object. You use the READ ENTITIES statement to read data; for all other operations, you use the MODIFY ENTITIES statement with the corresponding addition UPDATE, CREATE, or DELETE.

Note:
You can only use the CREATE, UPDATE, and DELETE operations if the behavior definition of the business object interface contains the corresponding `use create`, `use update`, or `use delete` directive. Trying to use a prohibited operation causes a syntax error.

© Copyright. All rights reserved.

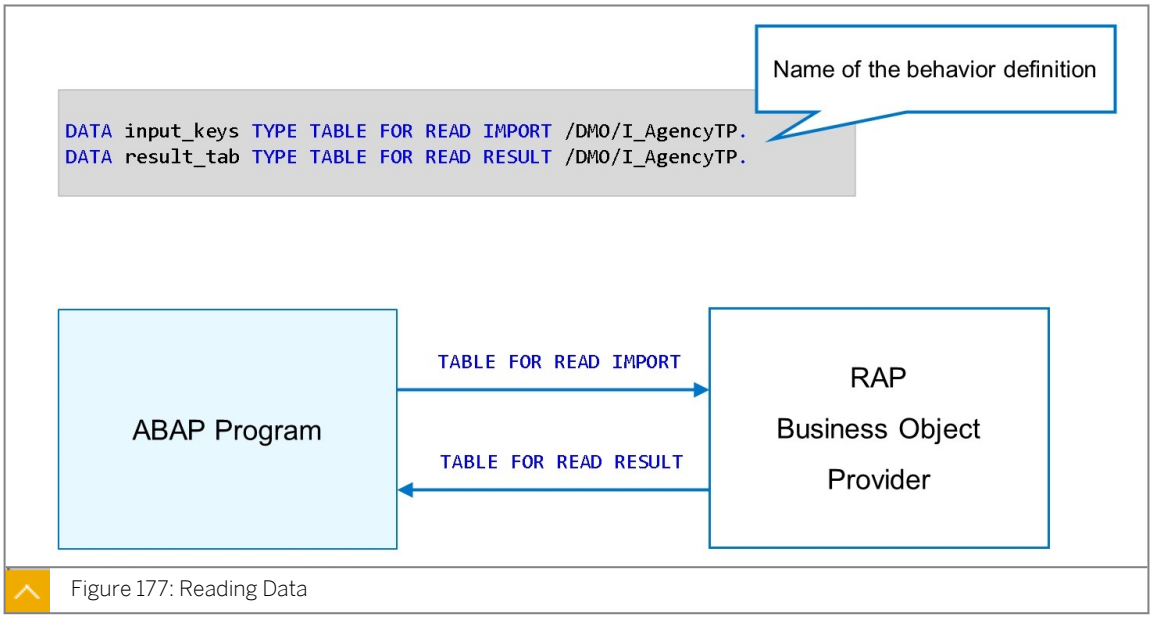


Figure 177: Reading Data

To read data from a business object, you use the READ ENTITIES statement. The statement has two important parameters: one internal table containing the keys of the data that you want to read and another containing the results of the query.

These internal tables have special data types called derived behavior definition types. The system creates them automatically when a developer creates a behavior definition and they contain some or all of the fields of the business object along with further fields that control how the system processes a particular request. You declare the internal tables using the new addition `TYPE TABLE FOR <operation>` in the DATA statement.

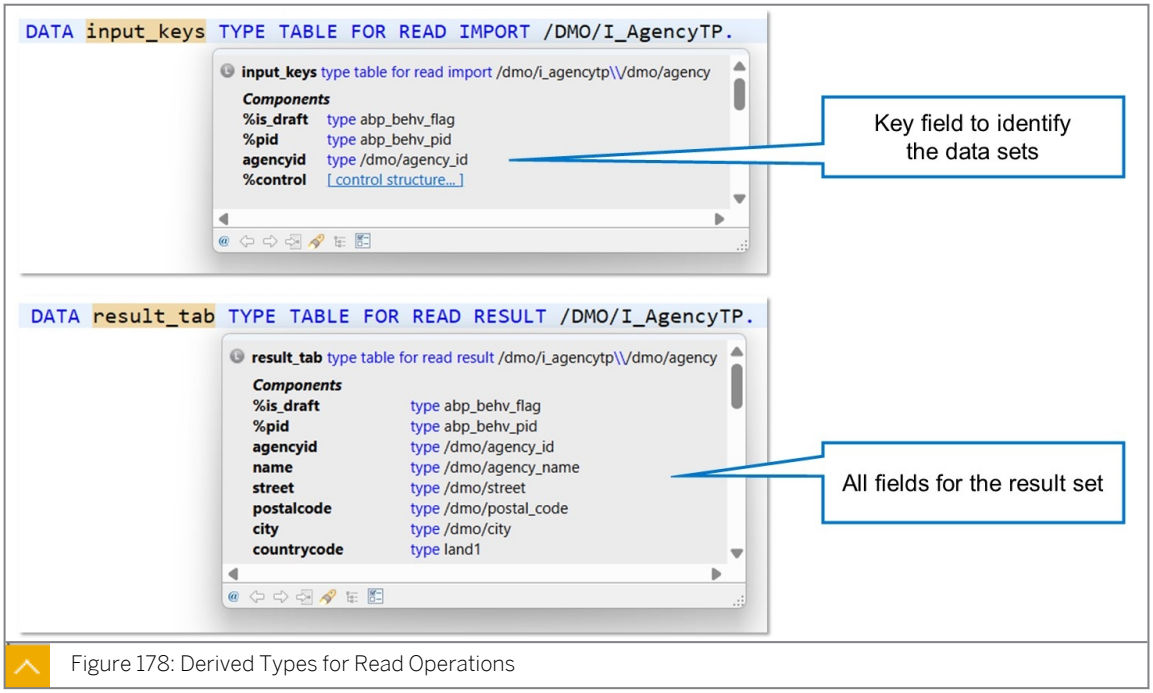


Figure 178: Derived Types for Read Operations

The type `TABLE FOR READ IMPORT` contains the key field or fields of the business object. The *%control* structure is a generated structure that indicates which fields of the business object are actually used in the current operation. In our example, the system fills the structure automatically and you do not have to worry about it.

© Copyright. All rights reserved.

The type `TABLE FOR READ RESULT` contains all of the fields of the business object. This table contains the result set after the read statement has been executed.

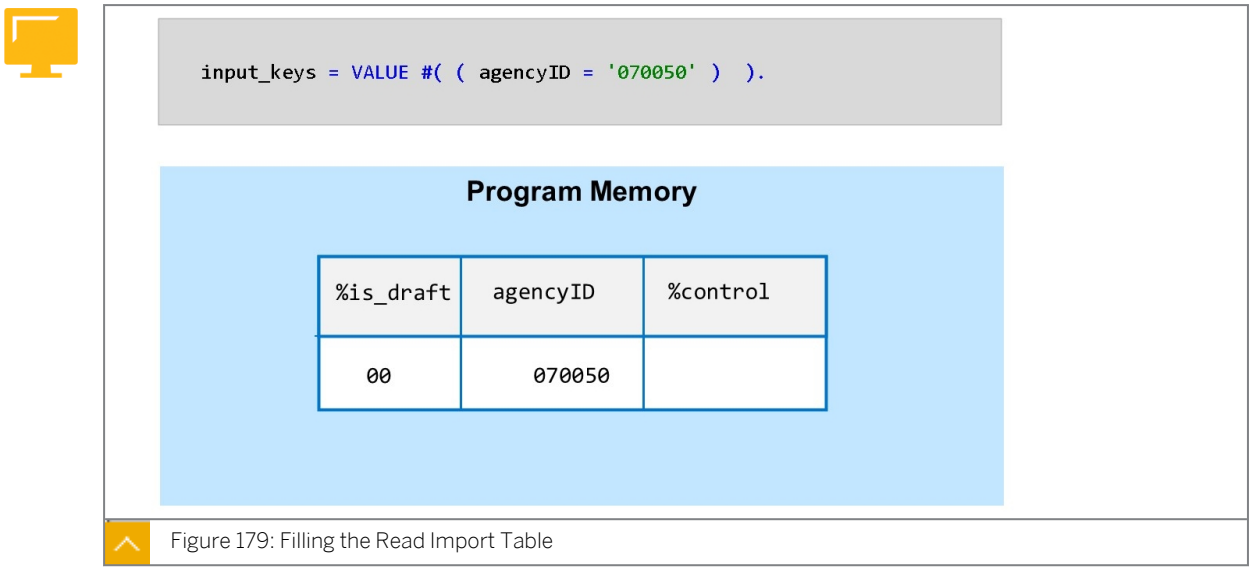


Figure 179: Filling the Read Import Table

The read import table contains a column for each key field of the business object, in this case, *agencyID*. To read a particular agency, you add a row to the internal table containing its key. As well as the key field or fields, the table contains the columns *%is_draft* and *%control*. With *%is_draft* you can specify whether you want to read draft data or active data. The *%control* structure is used to specify which fields are to be read.

Note:
In our example, we only read active data and the *%control* structure is filled by the framework, based on the field list after addition FIELDS.

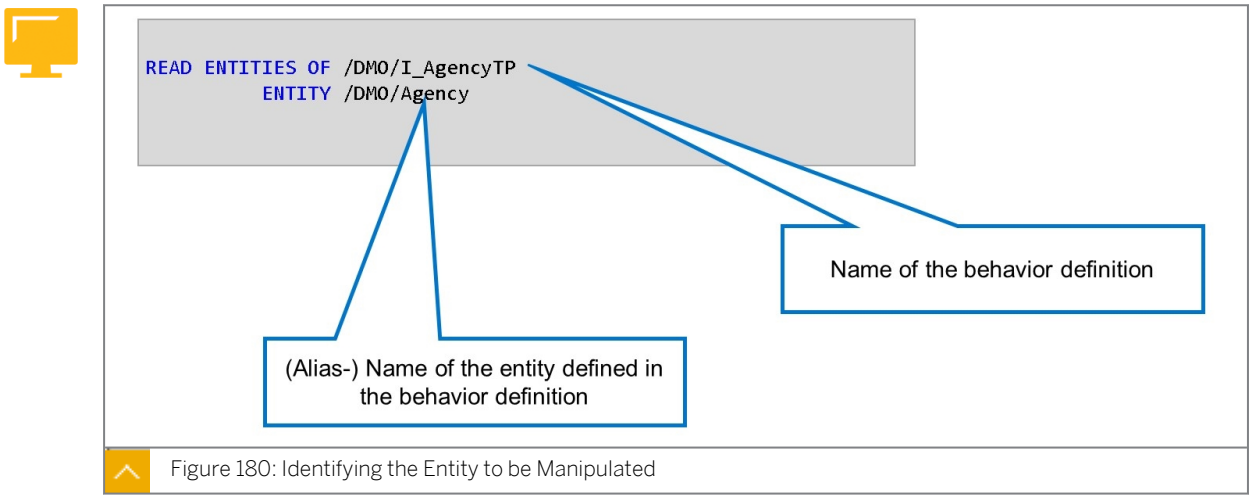


Figure 180: Identifying the Entity to be Manipulated

When you process a business object, using the READ ENTITIES OF or MODIFY ENTITIES OF statement, you must first specify the name of the behavior definition. This is followed by keyword ENTITY and the name of the entity with which you want to work. If the entity has an alias name, you should use it, here.

© Copyright. All rights reserved.

Note:
You cannot use the alias name after addition OF. This is because technically speaking, you specify the name of the behavior definition at this point, not the name of the entity.

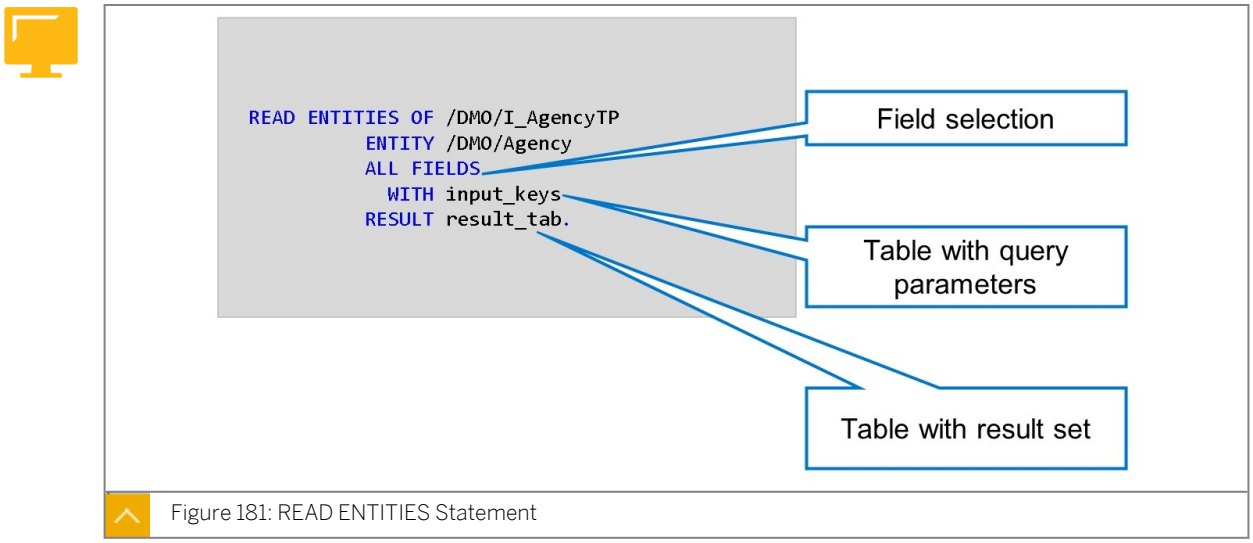


Figure 181: READ ENTITIES Statement

The READ ENTITIES statement reads business object data according to the keys that you pass in the WITH addition. It returns the result in the internal table in the RESULT addition. In the statement, you can also specify which fields of the business object you need. In this example, we have used the ALL FIELDS addition to return all of the fields. However, if you only require a subset of the fields, you can use the variant FIELDS (*field1 field2* ...) to restrict the amount of data that is read.

Note:
Unlike in a SELECT statement, the field list is not comma-separated.

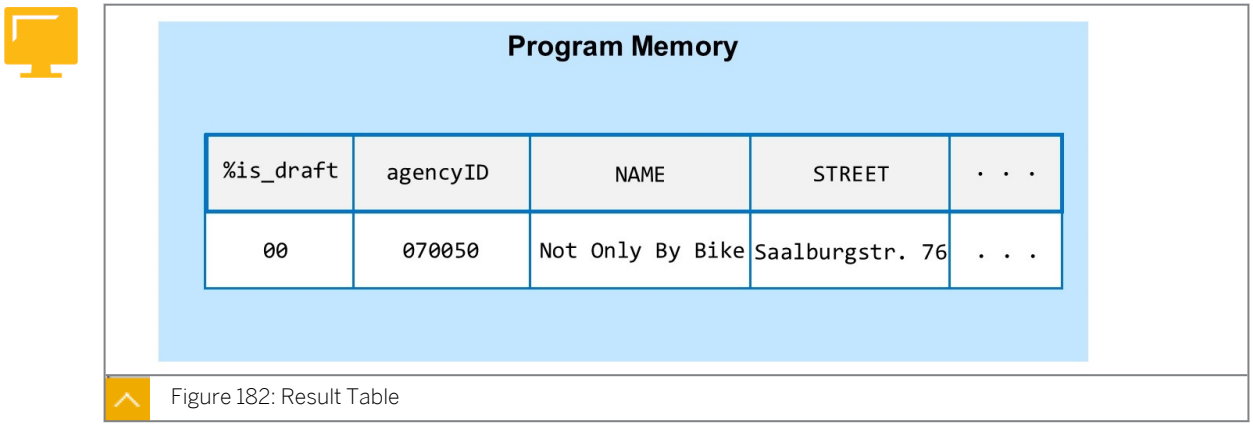


Figure 182: Result Table

The result table contains all of the fields of the business object, along with the control field *%is_draft*. If your READ ENTITIES statement contains the ALL FIELDS variant, the system provides the values of all of the fields. If you use the FIELDS (*f1* ... *fn*) variant, only the fields that you requested will be filled.

© Copyright. All rights reserved.

> Note:
> Key fields are always filled, even if they are not specified in the FIELDS (*f1* ... *fn*) variant.

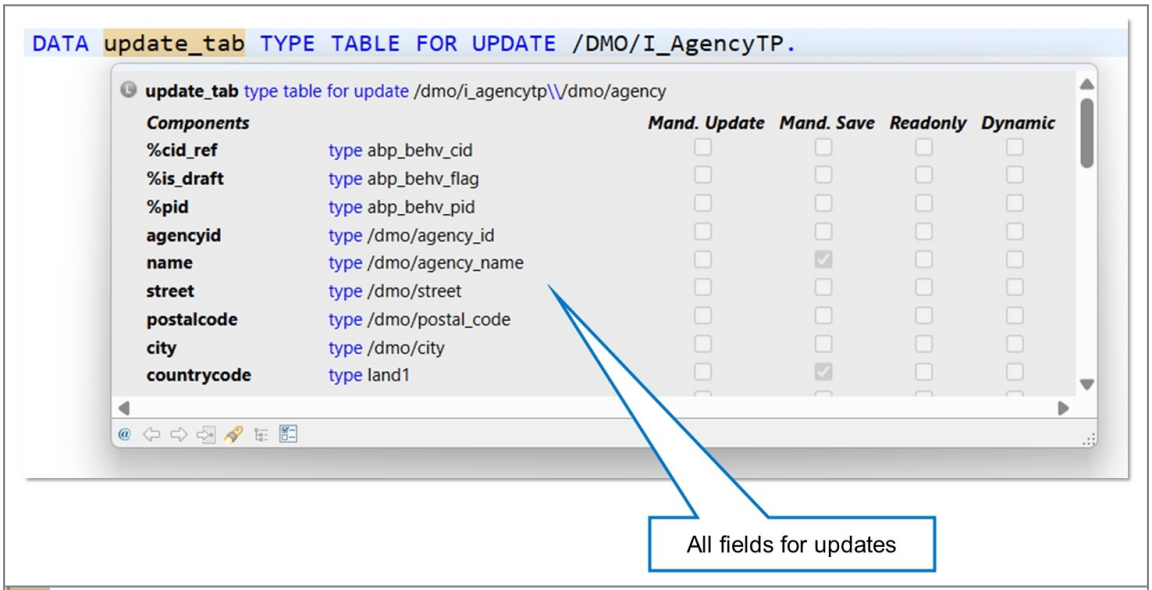


Figure 183: Derived Type for Update

If you want to update data, you declare an internal table with `TYPE TABLE FOR UPDATE`. This contains all of the fields of the business object and also the *%control* structure. In our variant of the MODIFY ENTITIES statement, the system fills the *%control* structure automatically.

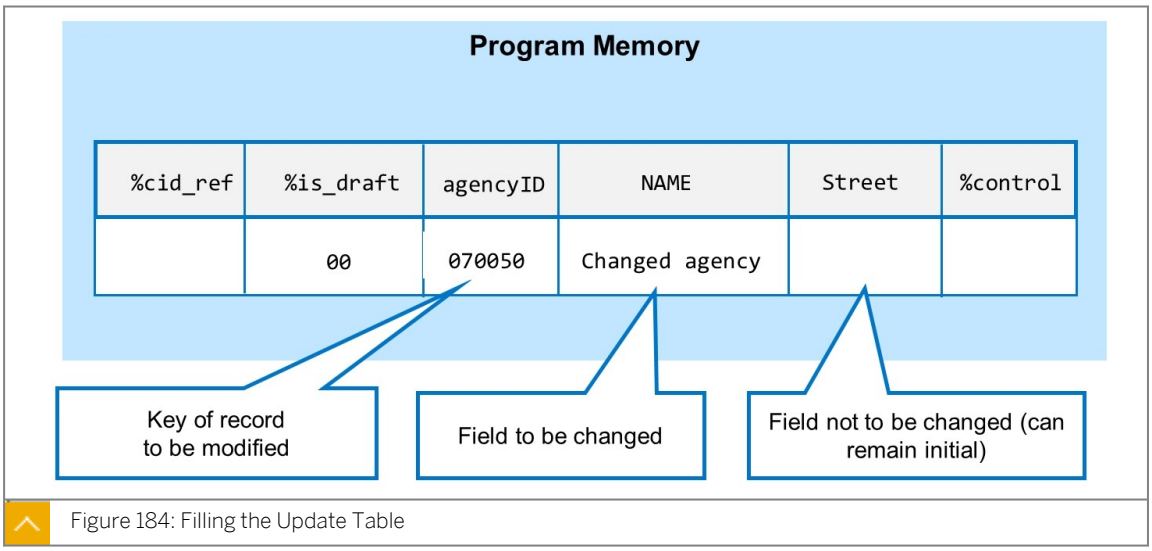


Figure 184: Filling the Update Table

© Copyright. All rights reserved.

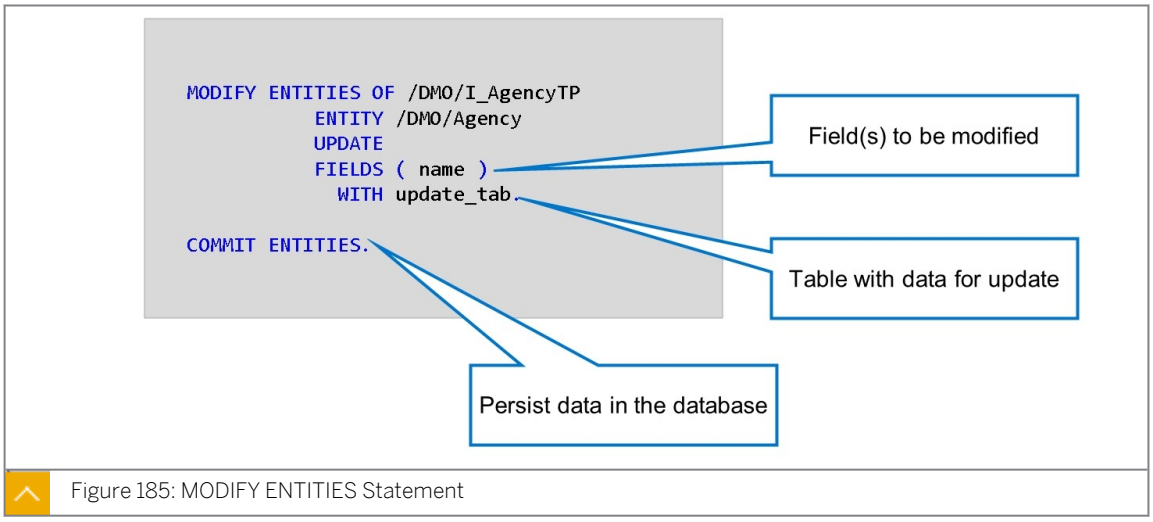


Figure 185: MODIFY ENTITIES Statement

The MODIFY ENTITIES statement updates data in the transactional buffer. In the FIELDS addition, you specify which fields should be changed. In the WITH addition, you pass the internal table containing the data that you want to update.

When you use EML outside the business object, you must use the COMMIT ENTITIES statement to trigger the save sequence and persist the data in the database.

Note:
Later in this course we will use EML inside the behavior implementation of the business object. Inside the behavior implementation it is neither necessary nor allowed to trigger the commit with COMMIT ENTITIES.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 7
Exercise 17

Modify Data Using EML

In this exercise, you use EML to perform the standard operation *update* for the root entity of the business object **/DMO/R_AGENCYTP**. To avoid later inconsistencies, you do not access the business object directly but via its released stable interface **/DMO/I_AGENCYTP.**

Template:

none

Solution:

/LRN/CL_S4D400_BOS_EML (global Class)

Task 1: Analyze Current Data
Use the *Data Preview* tool to check the current name of the travel agency with ID “0700##”, where ## stands for your group number.

1. Open the definition of the CDS view entity **/DMO/I_AgencyTP**.
2. Open the *Data Preview* tool.
3. Check the current name of the travel agency with ID “0700##”, where ## stands for your group number.

Task 2: Create a Global Class
In your own package, create a new ABAP class.

1. Create a new ABAP class called **ZCL_##_EML**, where **##** stands for your group number. Ensure that the class implements the interface **IF_OO_ADT_CLASSRUN**.

Task 3: Update Data
Implement an EML statement to change the name of the travel agency with ID “0700##” (## stands for your group number.).

1. In the method **IF_OO_ADT_CLASSRUN~MAIN**, declare an internal table with the correct derived type for an EML update of entity **/DMO/I_AgencyTP** (suggested name: **agencies_upd)** .
2. Fill the internal table with a single line containing “0700##” as the value for the column **AGENCYID** (where ## stands for your group number) and any new value for the column **NAME**.
3. Implement an EML statement MODIFY ENTITIES for the business object interface **/DMO/I_AgencyTP**. Update the data of the root entity, ensuring that only the field **NAME** is changed.

© Copyright. All rights reserved.

Hint:
Remember that the interface behavior defines alias name **`/DMO/Agency`** for the root entity **`/DMO/I_AgencyTP`**.

4. In order to get some visible result from the application, write a text literal to the console using the method **`out->write`**.
5. Activate and test the class.
6. Check whether the new name has been written to the database.
7. Add the COMMIT ENTITIES statement to your code to commit the changes.
8. Activate and test the class again. Confirm that the changes are now in the database.

© Copyright. All rights reserved.

Unit 7
Solution 17

Modify Data Using EML

In this exercise, you use EML to perform the standard operation *update* for the root entity of the business object **/DMO/R_AGENCYTP**. To avoid later inconsistencies, you do not access the business object directly but via its released stable interface **/DMO/I_AGENCYTP.**

Template:

none

Solution:

/LRN/CL_S4D400_BOS_EML (global Class)

Task 1: Analyze Current Data
Use the *Data Preview* tool to check the current name of the travel agency with ID "0700##", where ## stands for your group number.

1. Open the definition of the CDS view entity **/DMO/I_AgencyTP**.
   a) If you still have the data definition **/DMO/I_AGENCYTP** open from the previous exercise, switch to the corresponding tab. Otherwise, press **Ctrl + Shift + A** to open the ABAP development object.
2. Open the *Data Preview* tool.
   a) Right-click anywhere in the source code to open the context menu.
   b) From the context menu, choose *Open With* → *Data Preview*.
3. Check the current name of the travel agency with ID "0700##", where ## stands for your group number.
   a) Scroll down to the entry with the value "0700##" in the column **AgencyID** and take a note of the value in the column **Name**.

Task 2: Create a Global Class
In your own package, create a new ABAP class.

1. Create a new ABAP class called **ZCL_##_EML**, where **##** stands for your group number. Ensure that the class implements the interface **IF_OO_ADT_CLASSRUN**.
   a) Choose *File* → *New* → *ABAP Class*.
   b) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **ZCL_##_EML**, where **##** is your group number. Enter a description.
   c) In the *Interfaces* group box, choose *Add*.
   d) Enter **IF_OO_ADT_CLASSRUN**. When the interface appears in the hit list, double-click it to add it to the class definition.
   e) Choose *Next*.

© Copyright. All rights reserved.

f) Select your transport request and choose *Finish*.

Task 3: Update Data

Implement an EML statement to change the name of the travel agency with ID “0700##” (## stands for your group number.).

1. In the method **`IF_OO_ADT_CLASSRUN~MAIN`**, declare an internal table with the correct derived type for an EML update of entity **`/DMO/I_AgencyTP`** (suggested name: **`agencies_upd)`**.

   a) Add the following code:

```
    DATA agencies_upd TYPE TABLE FOR UPDATE /DMO/I_AgencyTP.
```

2. Fill the internal table with a single line containing “0700##” as the value for the column **`AGENCYID`** (where ## stands for your group number) and any new value for the column **`NAME`**.

   a) After the data declaration, add some code similar to the following code:

```
   agencies_upd = VALUE #( ( agencyid = '0700##' name = 'Some fancy
new name' ) ).
```

3. Implement an EML statement MODIFY ENTITIES for the business object interface **`/DMO/I_AgencyTP`**. Update the data of the root entity, ensuring that only the field **`NAME`** is changed.

Hint:
Remember that the interface behavior defines alias name **`/DMO/Agency`** for the root entity **`/DMO/I_AgencyTP`**.

   a) At the end of the method, add the following code:

```
    MODIFY ENTITIES OF /dmo/i_agencytp
       ENTITY /dmo/agency
       UPDATE FIELDS ( name )
         WITH agencies_upd.
```

4. In order to get some visible result from the application, write a text literal to the console using the method **`out->write`**.

   a) After the `MODIFY ENTITIES` statement, add the following code:

```
    out->write( `Method execution finished!`  ).
```

5. Activate and test the class.

   a) Press **`Ctrl + F3`** to activate the class.

   b) Press **`F9`** to run the class.

6. Check whether the new name has been written to the database.

   a) Return to the data preview for CDS view entity **`/DMO/I_AgencyTP`**.

   b) From the toolbar of the *Data Preview* tab, choose *Refresh*.

© Copyright. All rights reserved.

Why has your data change not yet arrived in the database?

Your program stores the changes in the application buffer, but they are not written to the database until a COMMIT ENTITIES statement.

7. Add the COMMIT ENTITIES statement to your code to commit the changes.
   a) Return to the method **`IF_OO_ADT_CLASSRUN~MAIN`** in your global class **`ZCL_##_EML`**.
   b) Adjust the code as follows:

```
METHOD if_oo_adt_classrun~main.

    DATA agencies_upd TYPE TABLE FOR UPDATE /dmo/i_agencytp.

    agencies_upd = VALUE #( ( agencyid = '0700##' name = 'Some fancy
new name' ) ).

    MODIFY ENTITIES OF /dmo/i_agencytp
      ENTITY /dmo/agency
      UPDATE FIELDS ( name )
        WITH agencies_upd.

    COMMIT ENTITIES.

    out->write( `Method execution finished!`  ).

  ENDMETHOD.
```

8. Activate and test the class again. Confirm that the changes are now in the database.
   a) Press **`Ctrl + F3`** to activate the class.
   b) Press **`F9`** to run the class.
   c) Return to the data preview for the CDS view entity **`/DMO/I_AgencyTP`**.
   d) From the toolbar of the *Data Preview* tab, choose *Refresh*.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Implement an EML statement.

© Copyright. All rights reserved.

Learning Assessment

1. In our scenario, what is coded in a behavior implementation?

   Choose the correct answers.

   - ☐ A Validations
   - ☐ B Determinations
   - ☐ C Update

2. In the READ ENTITIES statement, you use an internal table with a special type (TYPE TABLE FOR READ RESULT). How is this type created?

   Choose the correct answer.

   - ☐ A The developer of the business object creates it as a global type.
   - ☐ B The developer of the business object creates it in the behavior implementation.
   - ☐ C The system creates it automatically.

© Copyright. All rights reserved.

Learning Assessment - Answers

1. In our scenario, what is coded in a behavior implementation?

   Choose the correct answers.

   - [X] A Validations
   - [X] B Determinations
   - [ ] C Update

   That's right. Validations and determinations are coded in the behavior implementation. But In the managed implementation scenario, the framework takes care of the standard operations create, update and delete.

2. In the READ ENTITIES statement, you use an internal table with a special type (TYPE TABLE FOR READ RESULT). How is this type created?

   Choose the correct answer.

   - [ ] A The developer of the business object creates it as a global type.
   - [ ] B The developer of the business object creates it in the behavior implementation.
   - [X] C The system creates it automatically.

   That's right.

© Copyright. All rights reserved.

UNIT 8 Describing the ABAP RESTful Application Programming Model

Lesson 1

Introducing the Programming Model 321

Lesson 2

Exploring the Architecture 325
Exercise 18: Copy a Database Table 333
Exercise 19: Generate and Preview an OData UI Service 345

Lesson 3

Adding ABAP logic 353
Exercise 20: Validate the Flight Price 367

Lesson 4

Improving the User Experience 375
Exercise 21: Adjust the User Interface 381

UNIT OBJECTIVES

- Describe the process to develop an OData service with the ABAP RESTful application programming model.
- Create a database table.
- Generate the development objects for an OData UI service.
- Implement the behavior of a Business Object.
- Arrange fields in the app.
- Provide input help.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 8
Lesson 1

Introducing the Programming Model

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Describe the process to develop an OData service with the ABAP RESTful application programming model.

Overview

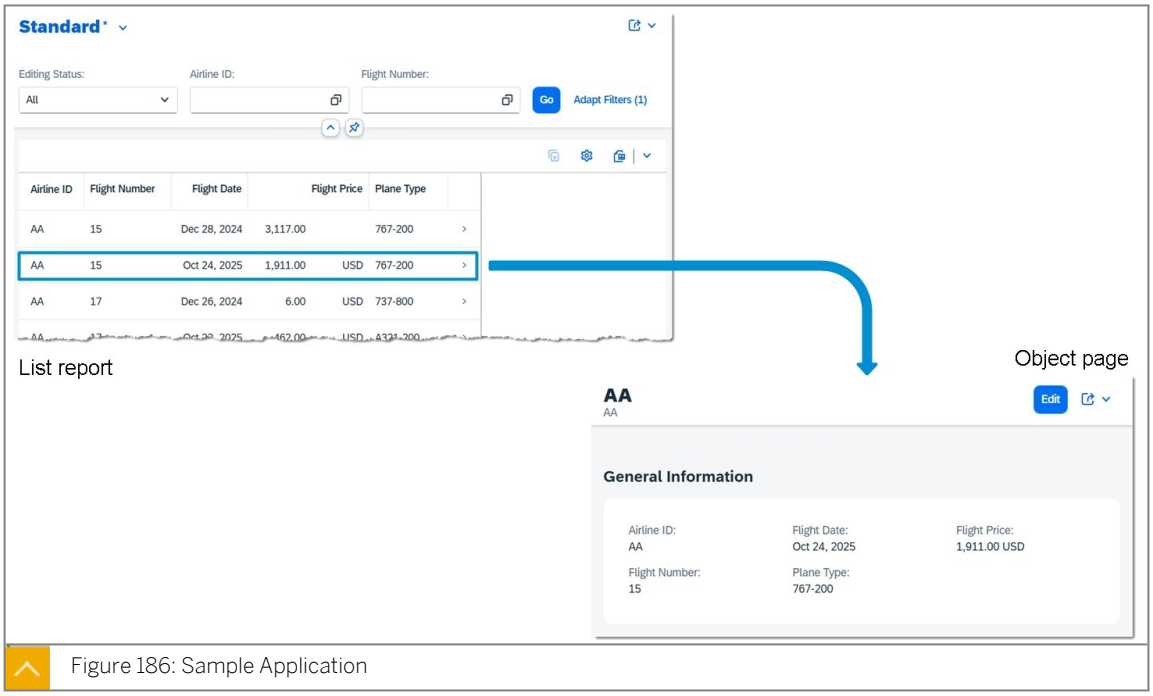


Figure 186: Sample Application

In this unit you will create an application using the ABAP RESTful Application Programming Model. The application displays flight details and allows you to change the price for a particular flight.

© Copyright. All rights reserved.

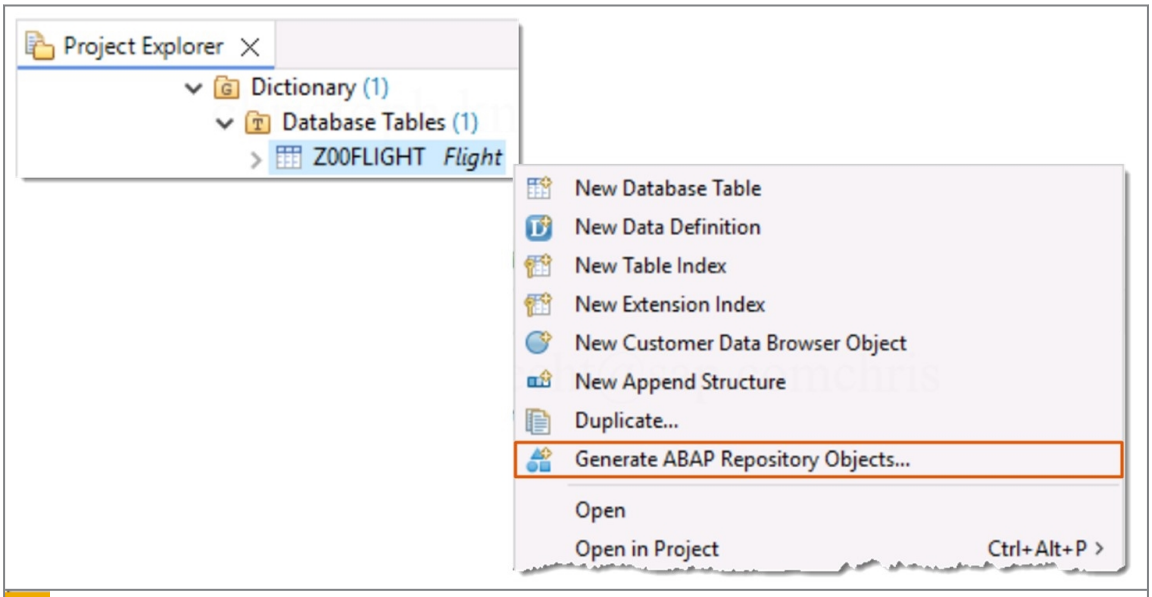


Figure 187: Generating the Required Objects

You will start by creating a database table definition. Based on the definition of your table, you will use the object generator to create all of the further objects that you need.

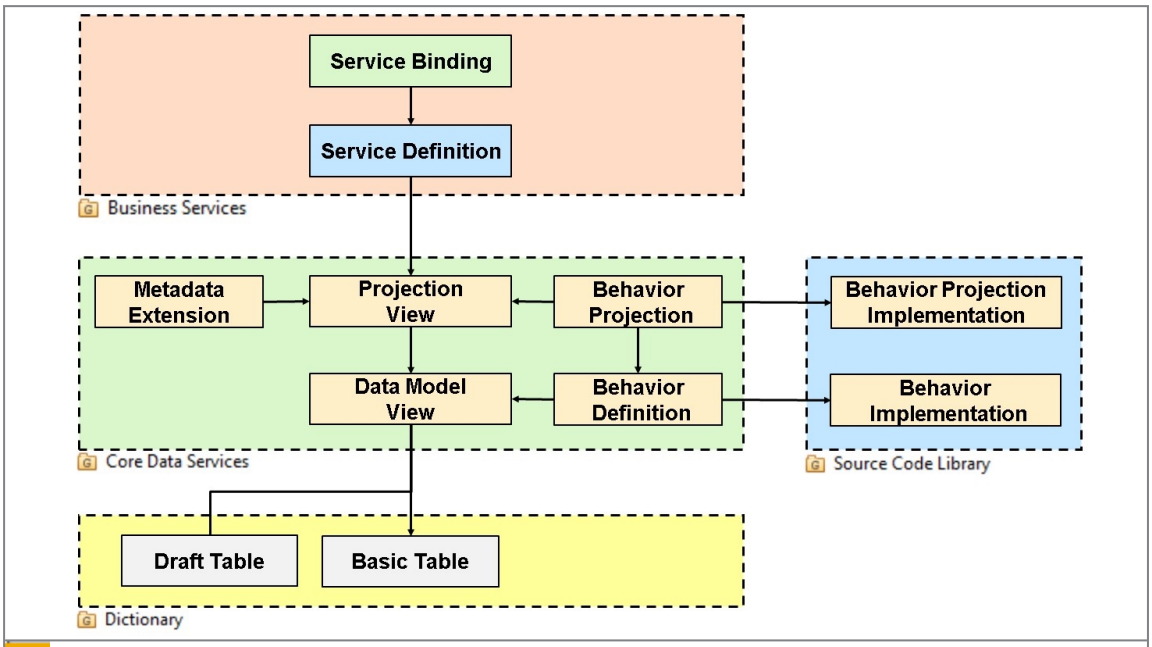


Figure 188: Repository Objects of the Generated OData UI Service

Using the generator, you will create an OData UI service that comprises the following objects:

A Business Object, consisting of:

- A *CDS View Entity*
- A *CDS Behavior Definition*
- An *ABAP class* for the behavior implementation (*Behavior Pool*)
- Two *Database Tables*, one for active data and one for draft data

© Copyright. All rights reserved.

A Business Object Projection, consisting of:

- A *CDS View Entity* (*Projection View*)
- A *CDS Behavior Definition* (*Behavior Projection*)
- An *ABAP class* for the behavior projection implementation (*Behavior Pool*)
- A *CDS Metadata Extension* for the service specific UI metadata

A Business Service, consisting of:

- A *Service Definition*
- A *Service Binding*

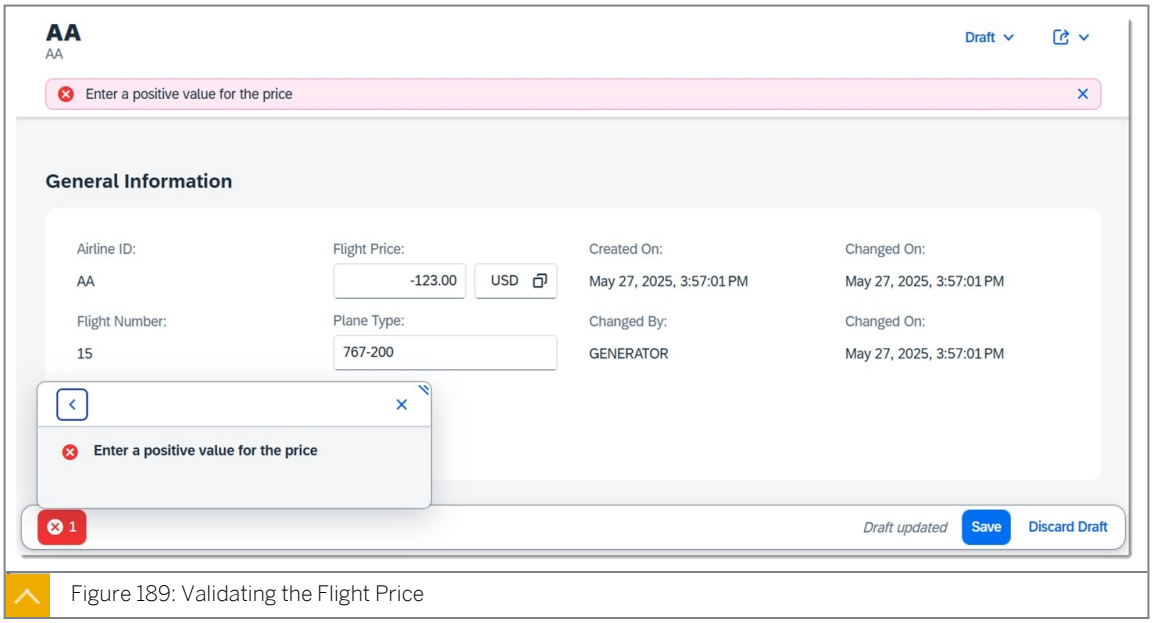


Figure 189: Validating the Flight Price

You will then add a validation to your application to ensure that the price for the flight is not zero or negative.

© Copyright. All rights reserved.

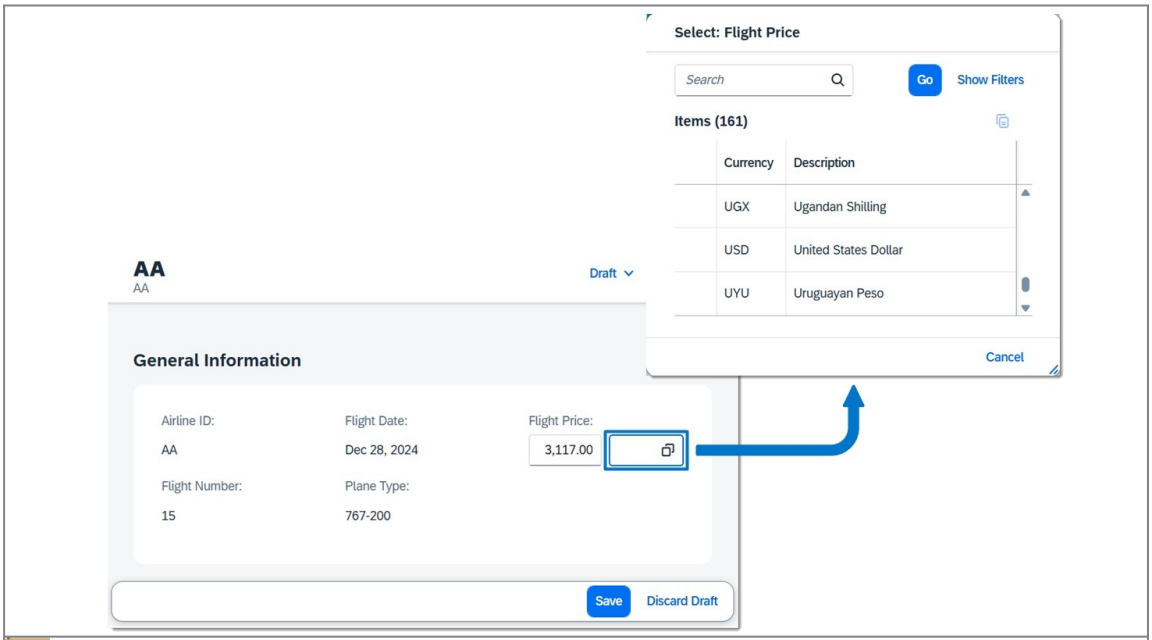


Figure 190: Adjusting the User Interface

Finally, you will explore the value help provided in the application. In addition, you will adjust the user interface of the generated app by editing the UI metadata, and by adjusting the business object behavior.

LESSON SUMMARY

You should now be able to:

- Describe the process to develop an OData service with the ABAP RESTful application programming model.

© Copyright. All rights reserved.

Unit 8
Lesson 2

Exploring the Architecture

LESSON OBJECTIVES
After completing this lesson, you will be able to:

- Create a database table.
- Generate the development objects for an OData UI service.

Creation of the Data Model

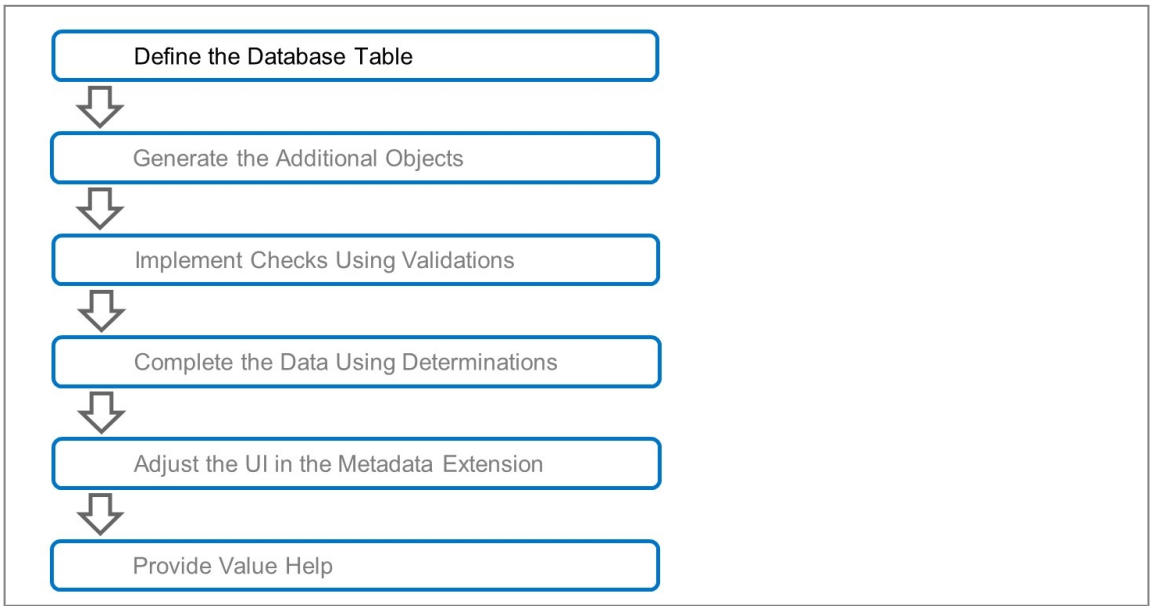


Figure 191: Developing a Simple Application

Example Data Model

client	uuid	carrier_id	connection_id	city_from	airport_from_id	country_from	...
100	<UUID_1>	LH	0400	Frankfurt	FRA	DE	
100	<UUID_2>	LH	0401	New York	JFK	US	
100	<UUID_3>	AA	0017	San Francisco	SFO	US	

Figure 192: Example Data Model

The data model in this example is very simple - there is a single database table that contains details of flight connections - an airline, flight number, details of the departure airport and the destination.

© Copyright. All rights reserved.

> Note:
> Not included in the figure are additional fields that the runtime needs in order to manage concurrency control. We will look at these a little later on.

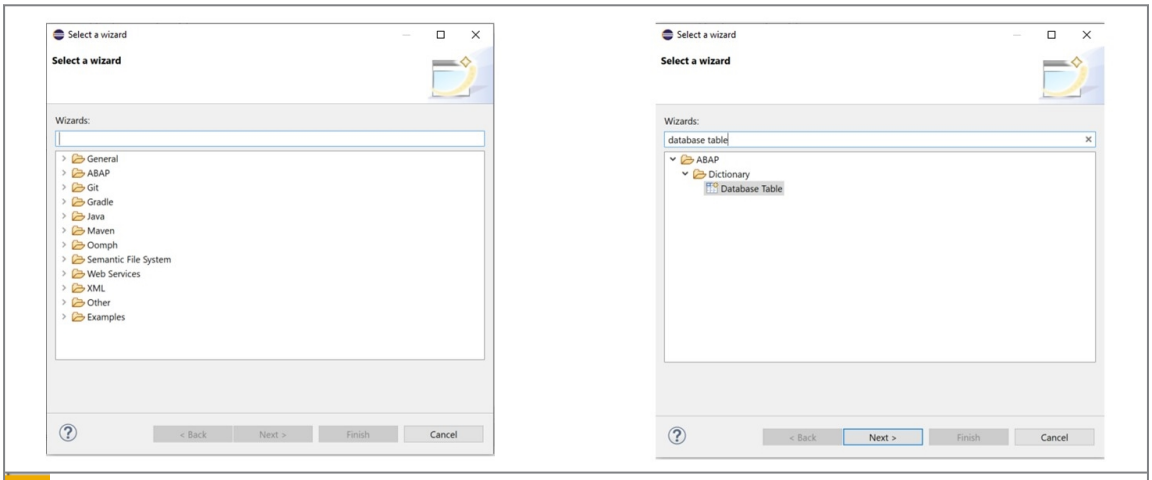


Figure 193: Creating a Database Table

To create a database table, choose *File* → *New* → *Other*. A dialog box appears.

Type **`Database table`** into the filter box. ADT narrows down the list of objects that you can select.

Double-click the *Database Table* entry to create the table.

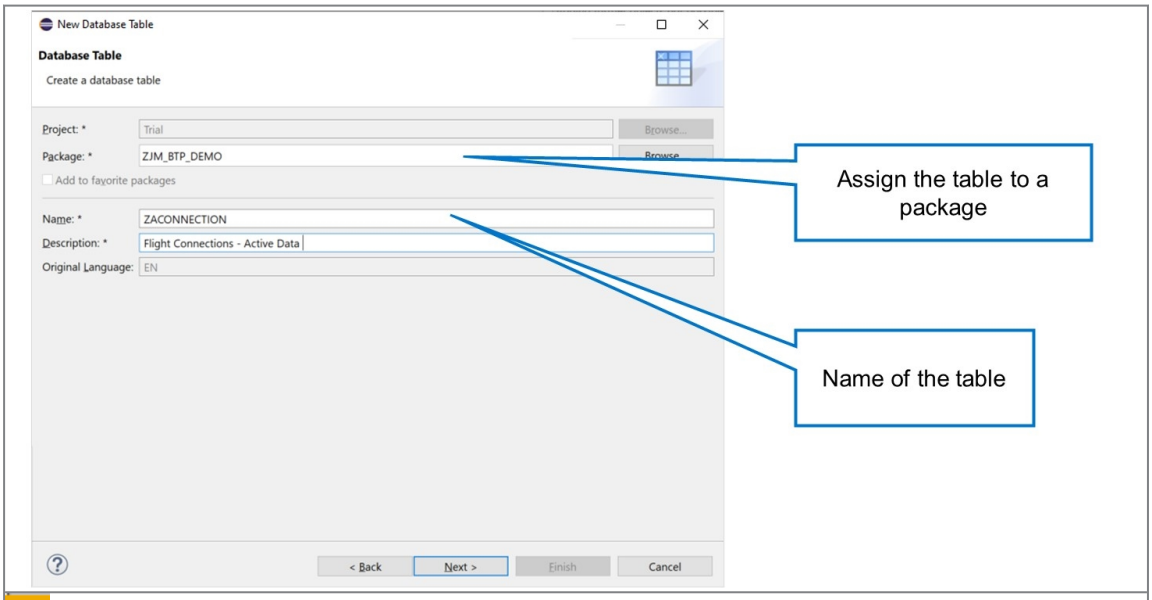


Figure 194: Basic Table Attributes

When you create a table, you must give it a name and assign it to a package.

In the Basic Table Attributes figure, the prefix Z stands for the customer namespace. This is the table in which saved data is stored. Later on, there will be a similar table *ZCONNECTION_D*, in which D stands for draft data. This will allow users to store incomplete data and to resume processing it at a later date.

© Copyright. All rights reserved.

Choose *Next*. ADT prompts you for a transport request. Choose a suitable request and choose *Finish*.

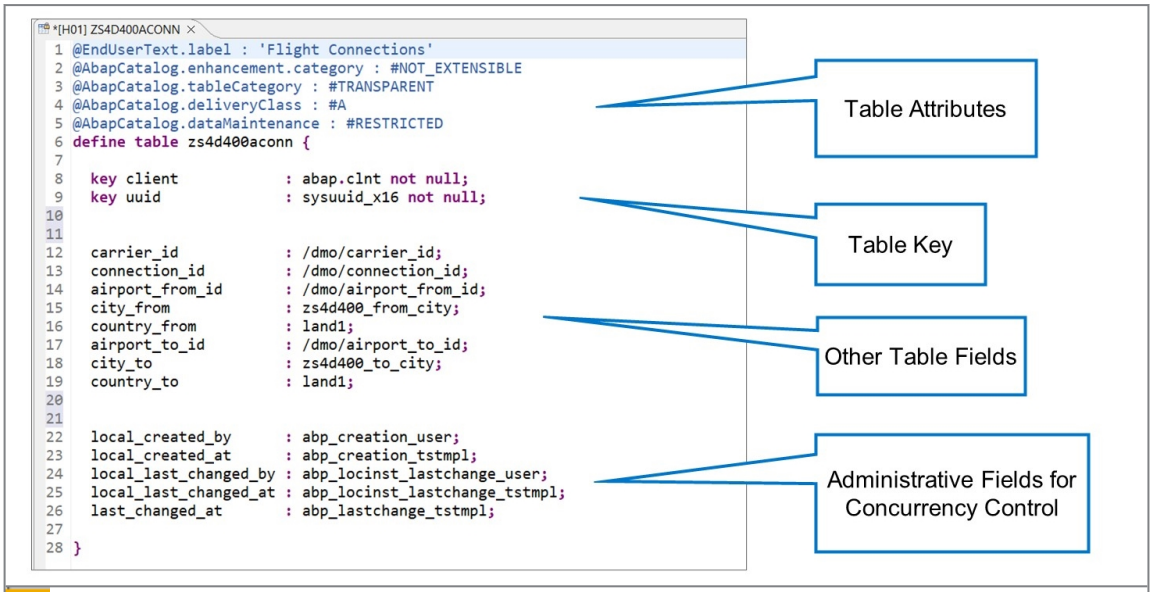


Figure 195: Table Definition

In the table definition, you specify the names and data types of each of the columns of the table. To do this, you use the notation `field_name : data_type ;`.

The data type can be a built-in ABAP Dictionary type - for this, the notation is *abap.data_type*. If the data type is incomplete, you provide the length in parentheses after the data type - for example, *abap.char(10)*. For numeric types with a fixed number of decimal places you would write, for example, *abap.dec(15,2)*. This defines a column of type DEC with 15 digits including two decimal places.

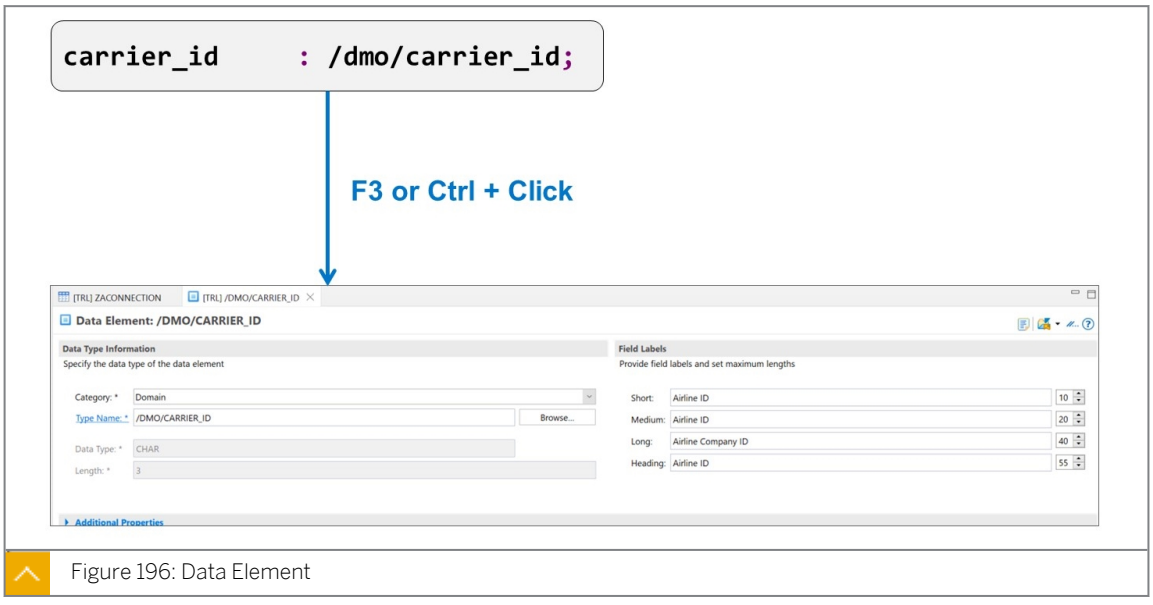


Figure 196: Data Element

If you do not use a built-in ABAP Dictionary type to specify the type of a table column, use a data element instead. Data elements are ABAP Dictionary objects that define the type of a single field such as UUID, or airline code. Data elements contain not only a technical type description but also semantic information such as field labels. If you use data elements to

© Copyright. All rights reserved.

define the types of the table columns, the system can use the field labels automatically on the generated user interface.

The data element in the Data Element figure describes the airline field. It is a character field with length 3; the system will use the field labels automatically on the generated UI.

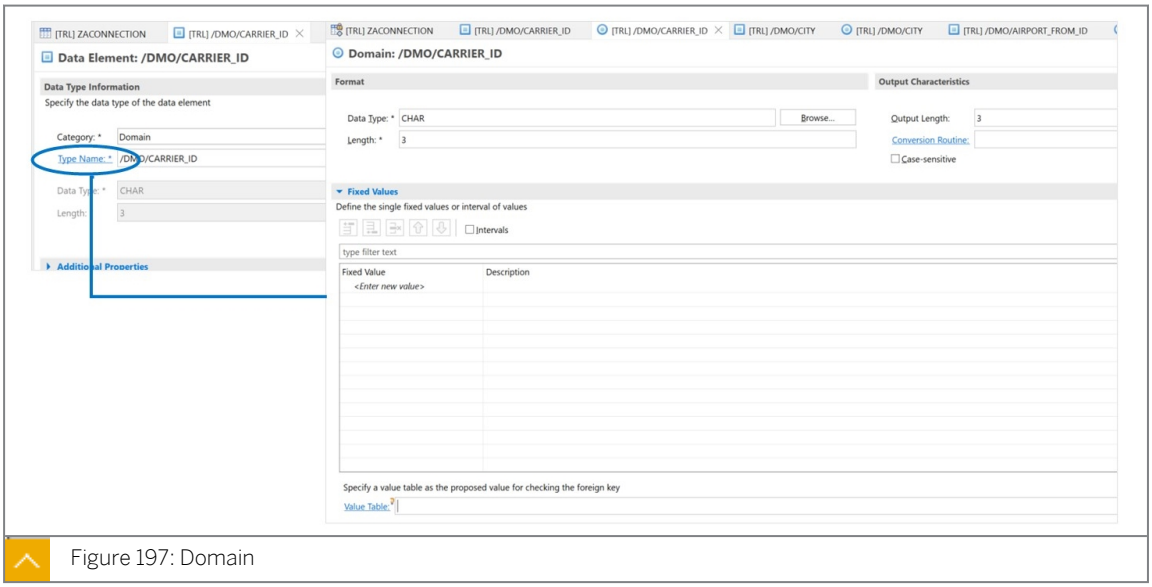


Figure 197: Domain

The technical type of a data element is usually specified by a domain. Here you can see that the domain for an airline code defines a character field with length 3 characters.

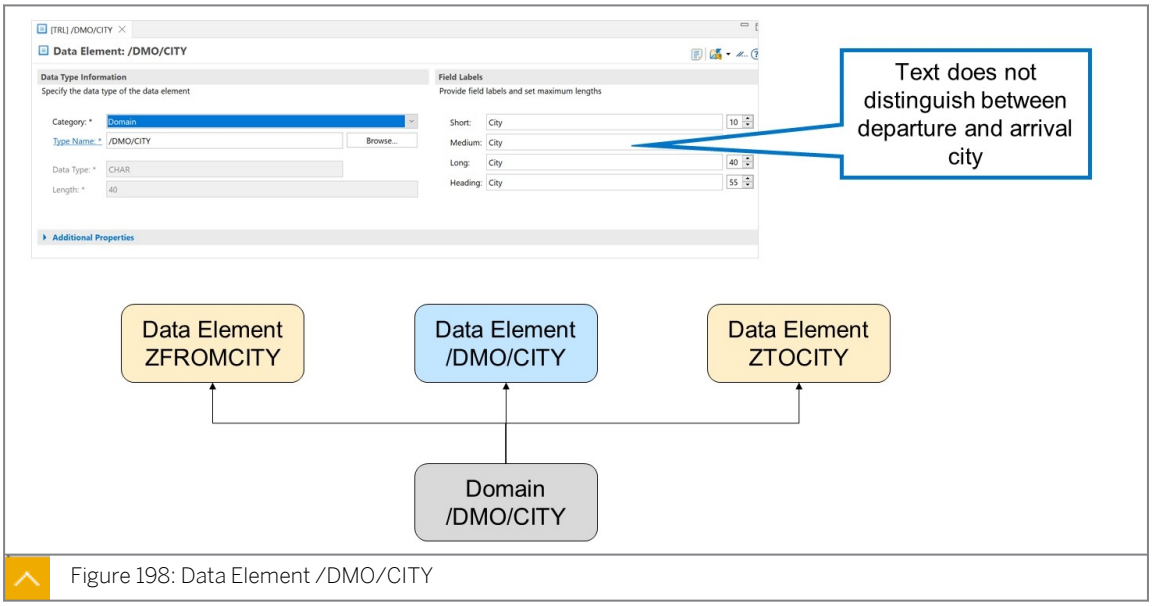


Figure 198: Data Element /DMO/CITY

The data element */DMO/CITY* describes the data type for the columns *city_from* and *city_to* of the database table. However, its field label - the text that will appear in the user interface - is just the word "city". This will make it impossible to distinguish which city is the departure city of the flight, and which is the destination city.

To improve the user experience, it is better not to use the data element */DMO/CITY* in this case. Instead, you create a new data element for the departure city and another for the destination and define the corresponding field labels in each case. As well as the semantic information, you must specify a data type for the new data elements. You do this by assigning

© Copyright. All rights reserved.

a domain. Here, you can re-use the domain */DMO/CITY,* since departure city and destination city should have the same technical attributes.

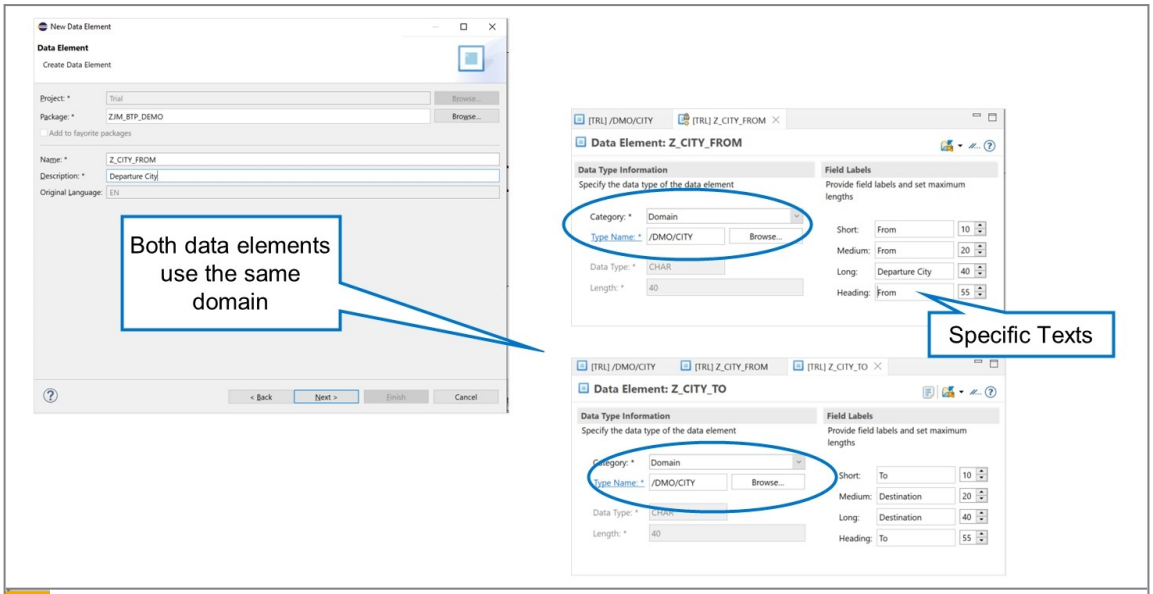


Figure 199: Creating the Data Element

To create a new data element, choose *File* → *New* → *Other*, then type data element into the *Filter Text* field. Enter a package name and the name of the data element, then choose *Next*. Select a transport request and choose *Finish*.

You must then specify a data type for the data element. This should be a domain - the example re-uses the existing domain */DMO/CITY*. Note that both data elements use the same domain. This means that if someone subsequently changes the length of the domain, the change applies to all data elements that use it.

Finally, you enter short, medium, and long field labels, and a label that is used for column headings. Later on, when you generate the Fiori Elements app, the system will use these texts automatically.

```
key client          : abap.clnt   not null;
key uuid            : sysuuid_x16 not null;
```

Figure 200: Table Key

Each table that you define must have a primary key. This is a sequence of fields at the beginning of the table description that identifies each entry in the table uniquely.

The first field in the table (and therefore also the first field in the key definition) must be the client field, and this must have the data type *abap.clnt*. Furthermore, it is convenient for our example to use a UUID for the unique key. If you use the data element sysuuid_x16 to specify the type of this field, you can let the runtime assign the UUID automatically when you create a new record.

© Copyright. All rights reserved.

```
local_created_by      : abp_creation_user;
local_created_at      : abp_creation_tstmpl;
local_last_changed_by : abp_locinst_lastchange_user;
local_last_changed_at : abp_locinst_lastchange_tstmpl;
last_changed_at       : abp_lastchange_tstmpl;
```

Figure 201: Fields for Administration and Concurrency Control

For traceability reasons it is recommended to store some administrative information with the data, like, for example, the user that created or changed the data and timestamps for the creation and the last change. The database table that you create should contain the fields displayed in the figure.

Note:
These fields are mandatory for the generator we are going to use later. For that generator to work properly the fields also have be defined with the data types listed here.

Especially fields*local_last_changed_at* and *last_changed_at* are not only administrative fields. In the ABAP RESTful application programming model they are also used for concurrency control.

Because RESTful applications are stateless, data consistency cannot be ensured by exclusive locks alone. The ABAP RESTful application programming model uses a mixture of exclusive locks and **ETags** to avoid data inconsistencies. An ETag is a field that changes its value whenever a data set is updated. By comparing the value of the ETag field, the framework can ensure that a data set has not been changed since it was read the last time. Timestamps of the last change make perfect ETag fields.

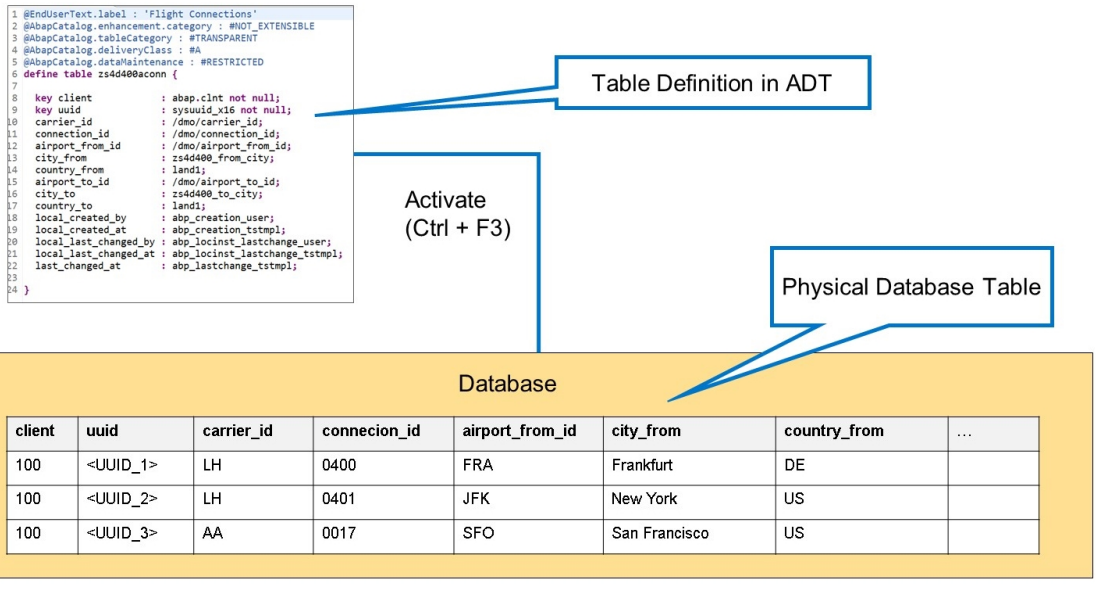


Figure 202: Activating the Table

© Copyright. All rights reserved.

When you activate a database table in ADT, the system creates the corresponding physical table in the database. Once you have done this, you will be able to generate the additional objects that you need for your app automatically.

© Copyright. All rights reserved.

© Copyright. All rights reserved.

Unit 8
Exercise 18

Copy a Database Table

In the exercises of this unit, you will create an OData UI service. This UI service will form the basis for an application that allows you to change the price of flights. Firstly, you create a dedicated package for the required development objects. Then you copy a database table and fill it with data.

Template:

/LRN/S4D400_APT (Database Table)
/LRN/CL_S4D400_APT_COPY (global Class)

Solution:

/LRN/S4D400AFLGT (Database Table)
/LRN/CL_S4D400_APS_COPY (global Class)

Task 1: Create Subpackage
Create a new subpackage under your own package ZS4D400_## (where ## stands for your group number).

The subpackage should have the following attributes:

Attribute	Value
Name	**ZS4D400_##_RAP**, where **##** stands for your group number
Description	Objects for OData UI Service
Add to Favorite Packages	Checked
Superpackage	**ZS4D400_##**, where **##** stands for your group number

1. Create a subpackage under your own package ZS4D400_## (where ## stands for your group number).

Task 2: Copy Database Table
In the new package, create a copy of the database table **/LRN/S4D400_APT**. Name the new database table **Z##FLIGHT**, where ## stands for your group number.

1. Copy database table **/LRN/S4D400_APT** to your own package **ZS4D400_##_RAP**, where ## stands for your group number.
2. Activate the database table definition.

© Copyright. All rights reserved.

Task 3: Fill Database Table

In your *ZS4D400_##* package, create a copy of the global class **/LRN/CL_S4D400_APT_COPY** (suggested name: **ZCL_##_COPY**). In this copy, change the value of the constant **table_name** to the name of your own database table. Then execute the class as a console app. This will fill your database table with data.

1. Copy the class **/LRN/CL_S4D400_APT_COPY** to a class in your own package **ZS4D400_##**.
2. In the copy, adjust the source code of the method **IF_OO_ADT_CLASSRUN~MAIN**. Change the value of the constant **table_name** to the name of your own database table.
3. Activate and execute the class as a console app.

© Copyright. All rights reserved.

Copy a Database Table

In the exercises of this unit, you will create an OData UI service. This UI service will form the basis for an application that allows you to change the price of flights. Firstly, you create a dedicated package for the required development objects. Then you copy a database table and fill it with data.

Template:

/LRN/S4D400_APT (Database Table)
/LRN/CL_S4D400_APT_COPY (global Class)

Solution:

/LRN/S4D400AFLGT (Database Table)
/LRN/CL_S4D400_APS_COPY (global Class)

Task 1: Create Subpackage
Create a new subpackage under your own package ZS4D400_## (where ## stands for your group number).

The subpackage should have the following attributes:

Attribute	Value
Name	**`ZS4D400_##_RAP`**, where **`##`** stands for your group number
Description	Objects for OData UI Service
Add to Favorite Packages	Checked
Superpackage	**`ZS4D400_##`**, where **`##`** stands for your group number

1. Create a subpackage under your own package ZS4D400_## (where ## stands for your group number).
   a) In the *Project Explorer*, right-click on your ABAP package *ZS4D400_##* and choose *New → ABAP Package* .
   b) Enter the package name **`ZS4D400_##_RAP`**, where **`##`** stands for your group number.
   c) Enter the description **`Objects for OData UI Service`**.
   d) Mark the checkbox *Add to favorite packages*.
   e) Ensure that the *Superpackage* field contains the name of your already existing package.

© Copyright. All rights reserved.

f) Choose *Next*.

g) Ensure that the *Software Component* is set to *ZSTUDENTS*.

h) Choose *Next*.

i) Confirm the transport request and choose *Finish*.

Task 2: Copy Database Table

In the new package, create a copy of the database table **`/LRN/S4D400_APT`**. Name the new database table **`Z##FLIGHT`**, where ## stands for your group number.

1. Copy database table **`/LRN/S4D400_APT`** to your own package **`ZS4D400_##_RAP`**, where ## stands for your group number.
   a) Open the definition of the database table **`/LRN/S4D400_APT`** .
   b) Link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click database table **`/LRN/S4D400_APT`** to open the context menu.
   d) From the context menu, choose *Duplicate...*.
   e) Enter the name of your package **`ZS4D400_##_RAP`** in the *Package* field. In the *Name* field, enter **`Z##FLIGHT`**, where ## stands for your group number.
   f) Choose *Next*.
   g) Confirm the transport request and choose *Finish*.
2. Activate the database table definition.
   a) Press **`Ctrl + F3`** to activate the development object.

Task 3: Fill Database Table

In your *ZS4D400_##* package, create a copy of the global class **`/LRN/CL_S4D400_APT_COPY`** (suggested name: **`ZCL_##_COPY`**). In this copy, change the value of the constant **`table_name`** to the name of your own database table. Then execute the class as a console app. This will fill your database table with data.

1. Copy the class **`/LRN/CL_S4D400_APT_COPY`** to a class in your own package **`ZS4D400_##`**.
   a) Open the source code of the global class **`/LRN/CL_S4D400_APT_COPY`**.
   b) If necessary, link the *Project Explorer* view with the editor.
   c) In the *Project Explorer* view, right-click the class **`/LRN/CL_S4D400_APT_COPY`** to open the context menu.
   d) From the context menu, choose *Duplicate...*.
   e) Enter **`ZS4D400_##`** in the *Package* field. In the *Name* field, enter **`ZCL_##_COPY`**, where ## stands for your group number.
   f) Choose *Next*.
   g) Confirm the transport request and choose *Finish*.
2. In the copy, adjust the source code of the method **`IF_OO_ADT_CLASSRUN~MAIN`**. Change the value of the constant **`table_name`** to the name of your own database table.

© Copyright. All rights reserved.

a) Make sure the value of the constant equals the name of your database table.

3. Activate and execute the class as a console app.

   a) Press **`Ctrl + F3`** to activate the class.

   b) Press **`F9`** to run the class.

   c) Check the *Console* view. The last output should read: <...> *was filled with data*. If the output reads <...> *is not a table of the right type.* make sure a database table of that name exists, is active, and is an exact copy of the template table.

© Copyright. All rights reserved.

The Object Generator

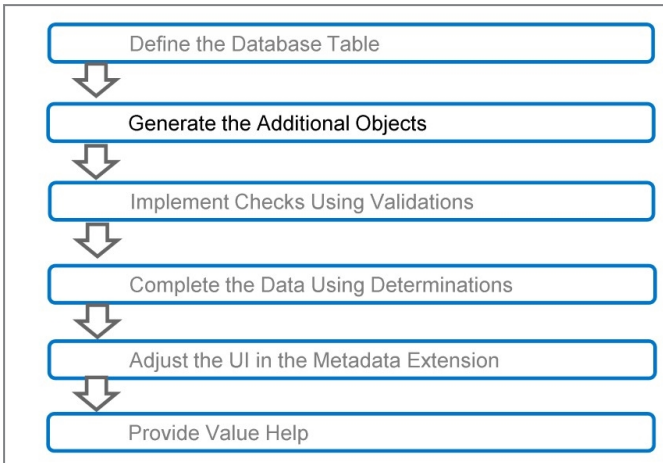


Figure 203: Developing a Simple Application

Generating Additional Objects

We have just defined a database table to hold our data, but the app that we are going to create needs more objects than just this table. We can generate these objects using a wizard in ADT.

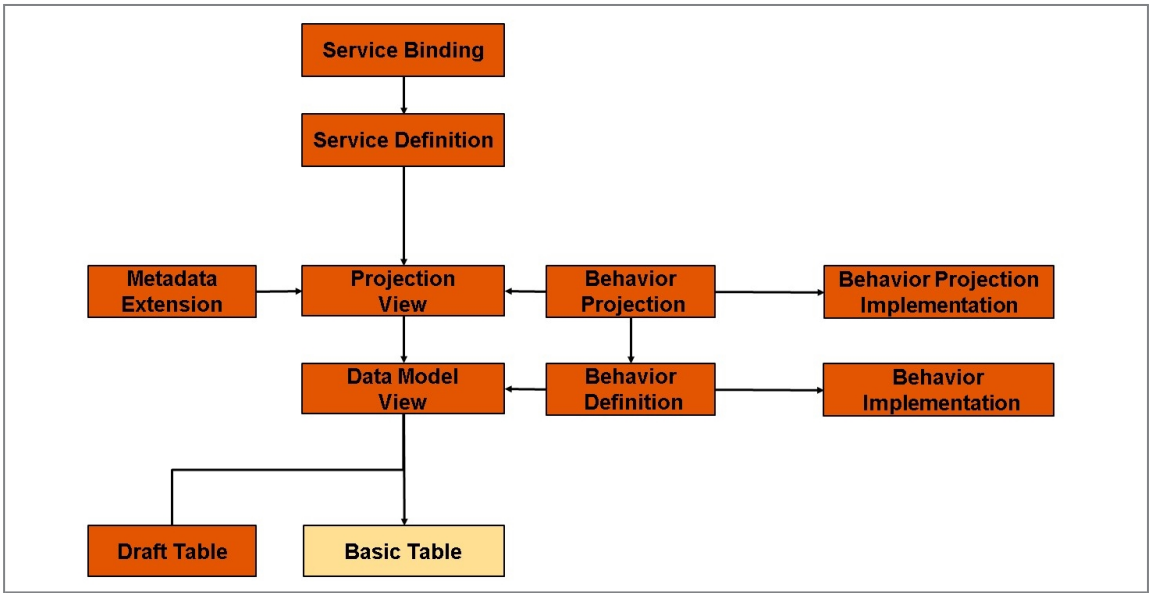


Figure 204: Generating Additional Objects

The generated objects contain all of the information that is necessary to provide a working app with create, read, update, and delete capability. Later on, we will also adjust and extend some of these objects to change the appearance of the user interface and to implement some checks and calculations.

© Copyright. All rights reserved.

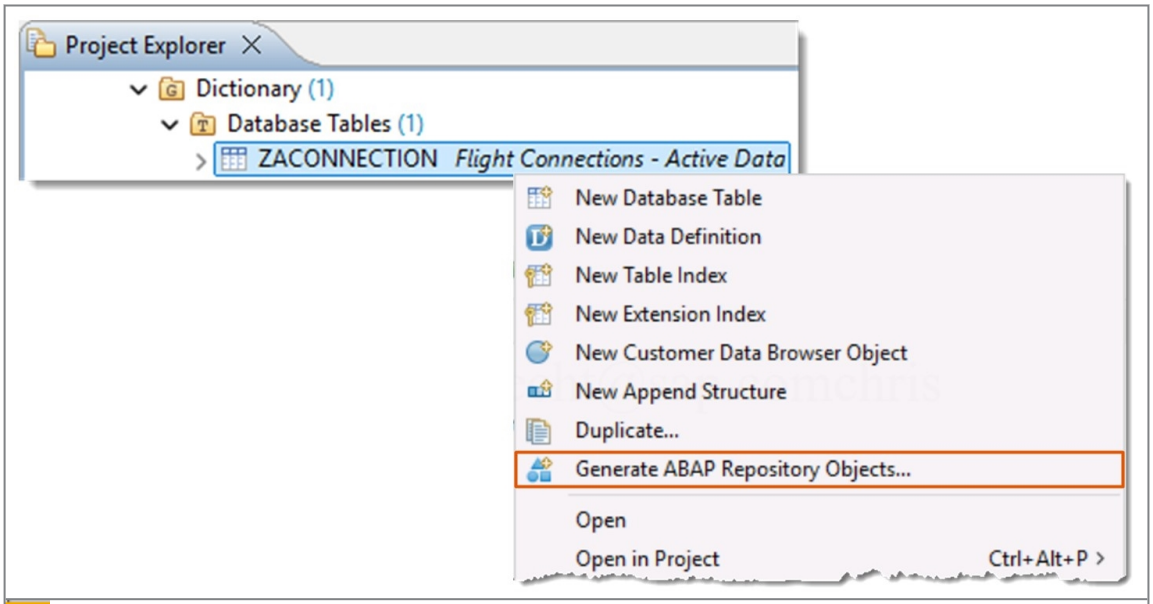


Figure 205: Starting the Wizard

To start the wizard, right-click the table name in the *Project Explorer* and choose *Generate ABAP Repository Objects...*.

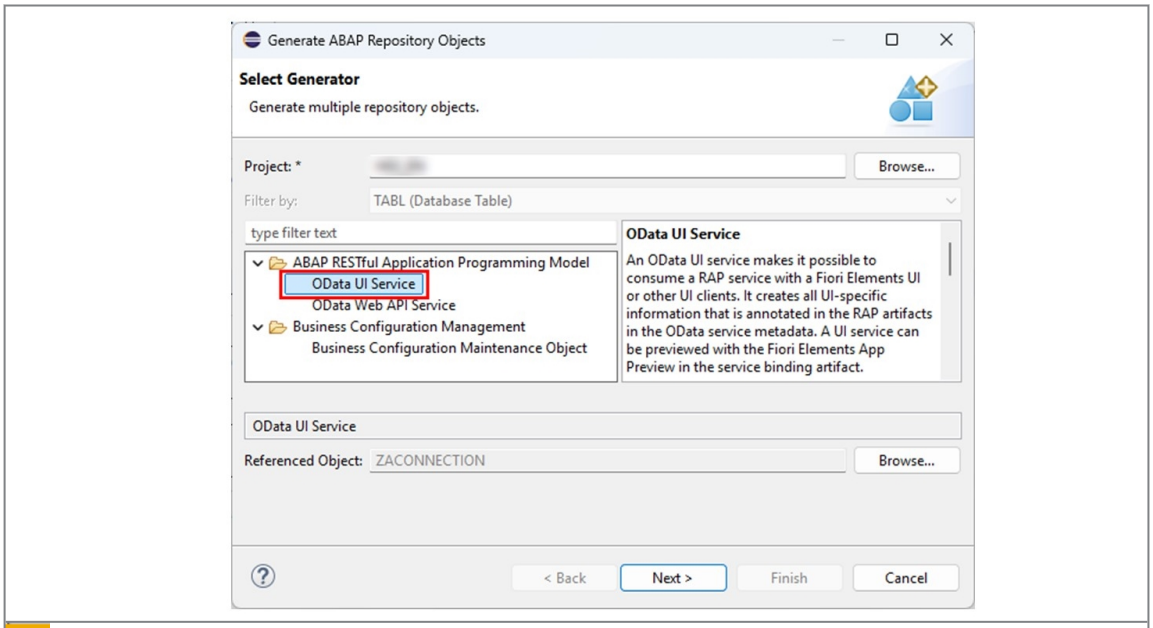


Figure 206: Selecting the Generator

First you have to choose the generator. In our example we want to generate an *OData UI Service*. On the next dialog window, you must enter a package to which all of the new objects will be assigned.

© Copyright. All rights reserved.

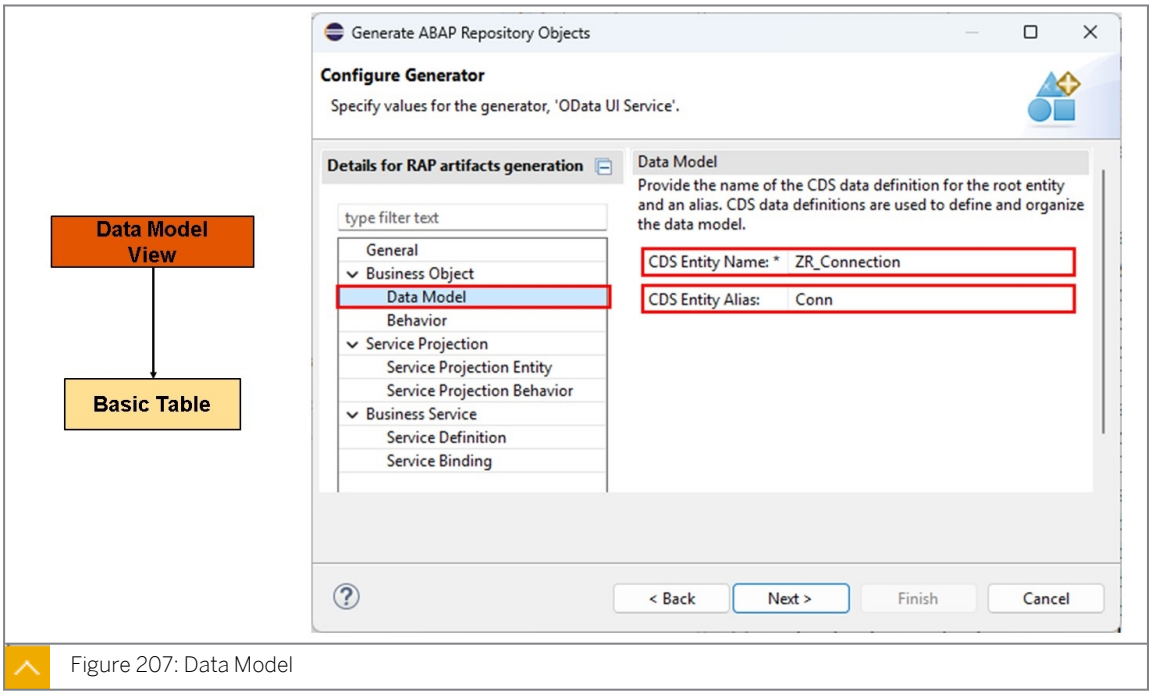


Figure 207: Data Model

In the RESTful Application Programming Model, you do not access database tables directly. Instead, you use CDS entities to define the data model. At this stage, the generator suggests a name for the CDS entity. Since the name of the database table begins with Z, which means it lies in the customer namespace, the generator suggests a name in the same namespace. A further convention is that the namespace is followed by the letter R, and then an underscore.

You can optionally enter an alias name, which is used inside the generated application to identify the generated CDS entity.

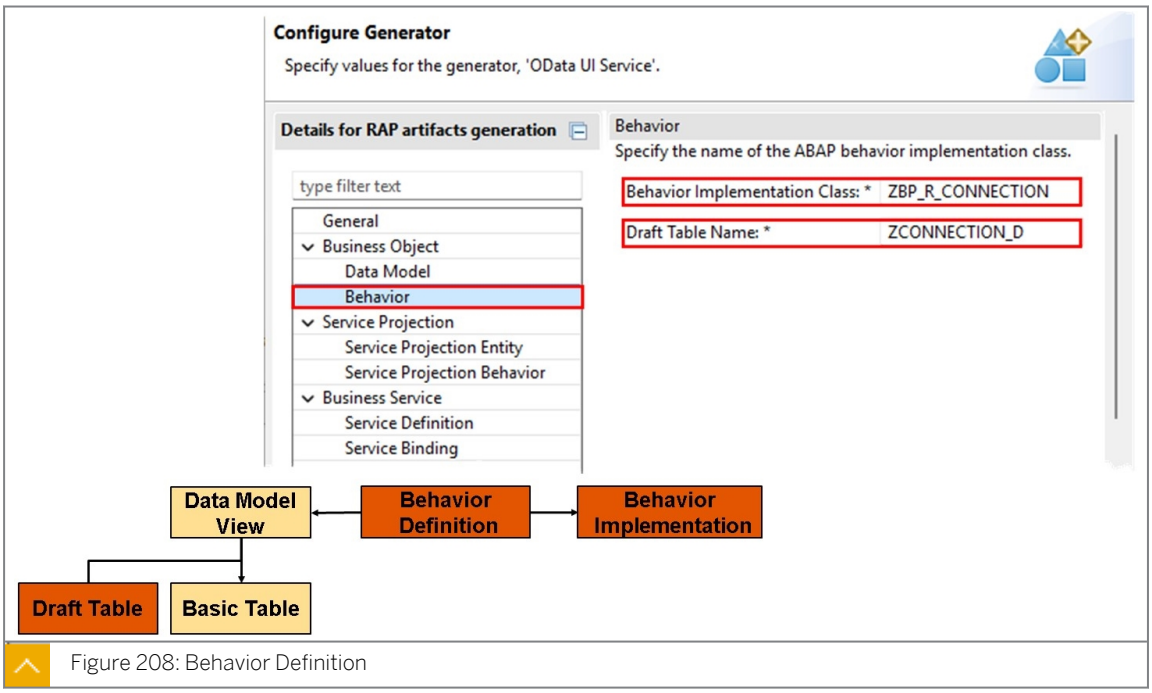


Figure 208: Behavior Definition

With the data model view that you create, you can read data from the database. However, our app should also be able to create, modify, and delete data. In order to make this possible, you must define a behavior definition. This is linked to the data model view and specifies which of

© Copyright. All rights reserved.

the create, update, and delete actions are allowed. For example, some kinds of documents must not be altered once they have been created. In this case, you would allow the create operation, but not update or delete.

As well as specifying which of the create, update, and delete actions should be available, the behavior definition can also contain the following definitions:

- Draft enabling
- Enabling of automatic numbering for the UUID field
- Validations: These are checks that are carried out when the user enters data in the app.
- Determinations: A determination performs a calculation to fill fields in the data record.

The generator creates the behavior definition and switches on automatic numbering for the UUID field. If you need to use validations and determinations, you must add them by hand.

The behavior definition declares which validations and determinations exist. However, they also need an ABAP implementation. The implementations are methods, so you therefore need an implementation class. The naming convention for this class is to use the prefix for the namespace, followed by **`BP_R_`**. **`BP`** stands for behavior pool.

The behavior definition also defines the draft enabling for the entity. Based on the definition of the data model view, the generator creates a corresponding table that will contain the draft data. You therefore need to specify the name of the draft table at this point. The generator suggests a name that is identical to the name of the basic table, but with an additional **`_D`** at the end.

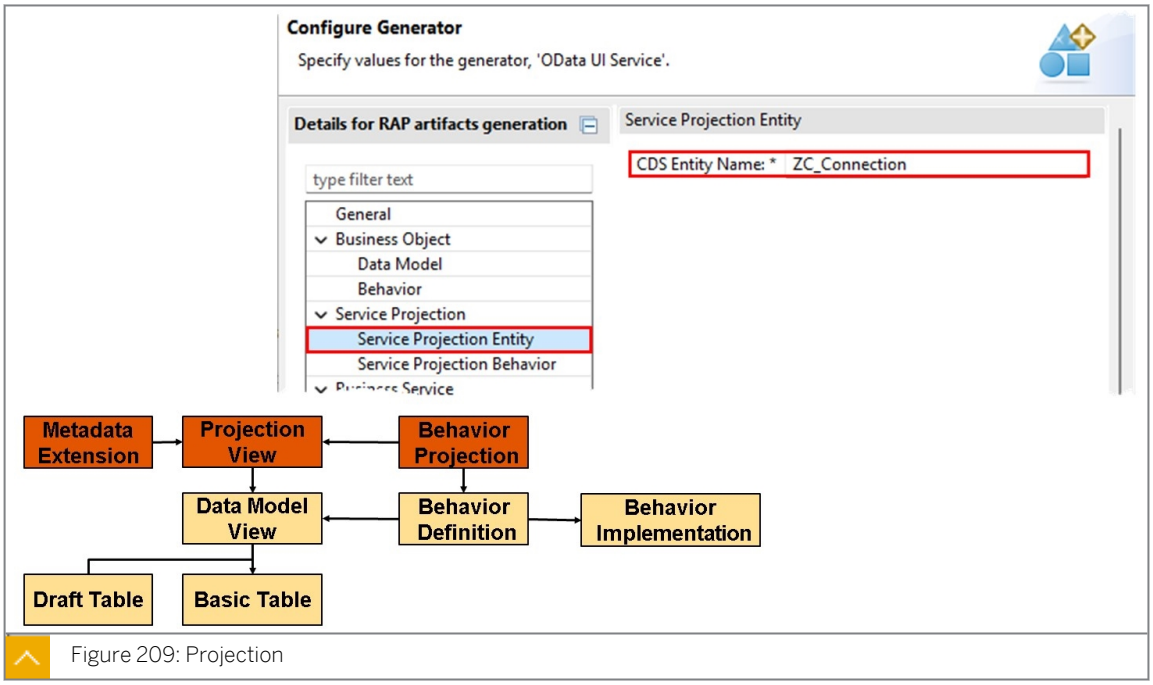


Figure 209: Projection

The data model and its behavior definition are a reusable implementation of a particular business entity. The next step in the object generation is to specify the name of the service projection. The projection contains a view with precisely the fields that are required for a particular app, a behavior definition that specifies which of the defined behaviors should actually be available in the app, and a metadata extension. The metadata extension contains CDS annotations that define how the UI of the app should look.

© Copyright. All rights reserved.

The naming convention for the projection layer is **ZC_<name>**. The projection view, the behavior projection, and the metadata extension will all have the same name.

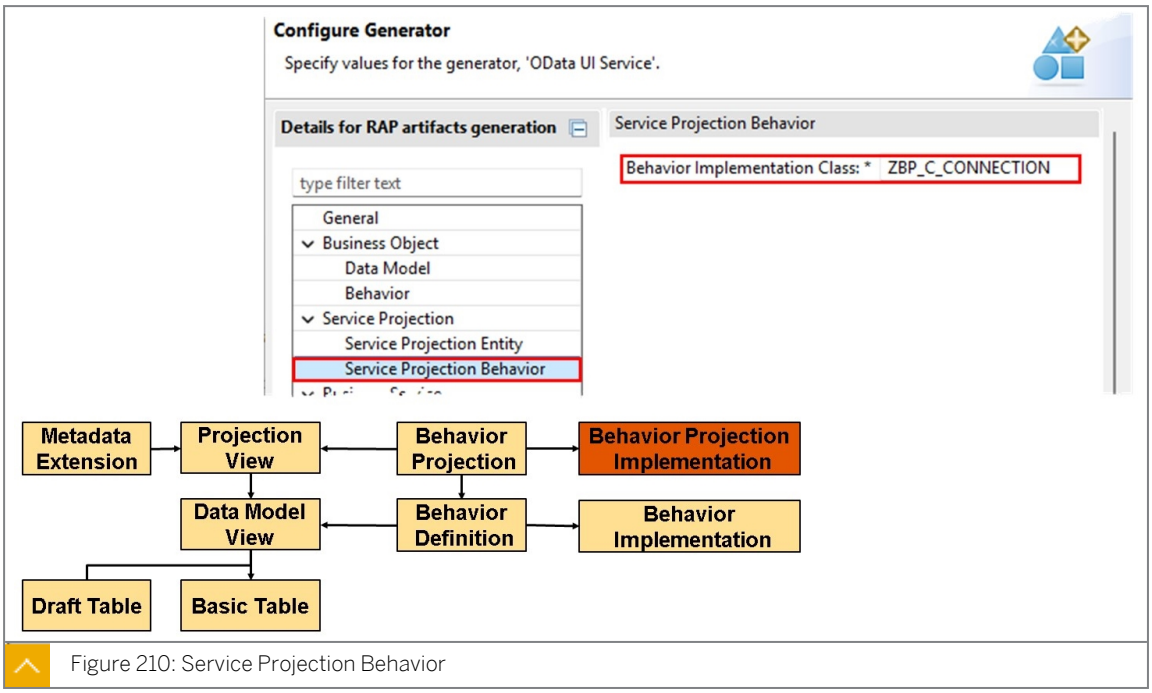


Figure 210: Service Projection Behavior

Sometimes, it can become necessary to provide service specific implementations. The naming convention for the behavior implementation class on projection level is to use the prefix for the namespace, followed by **BP_C_**.

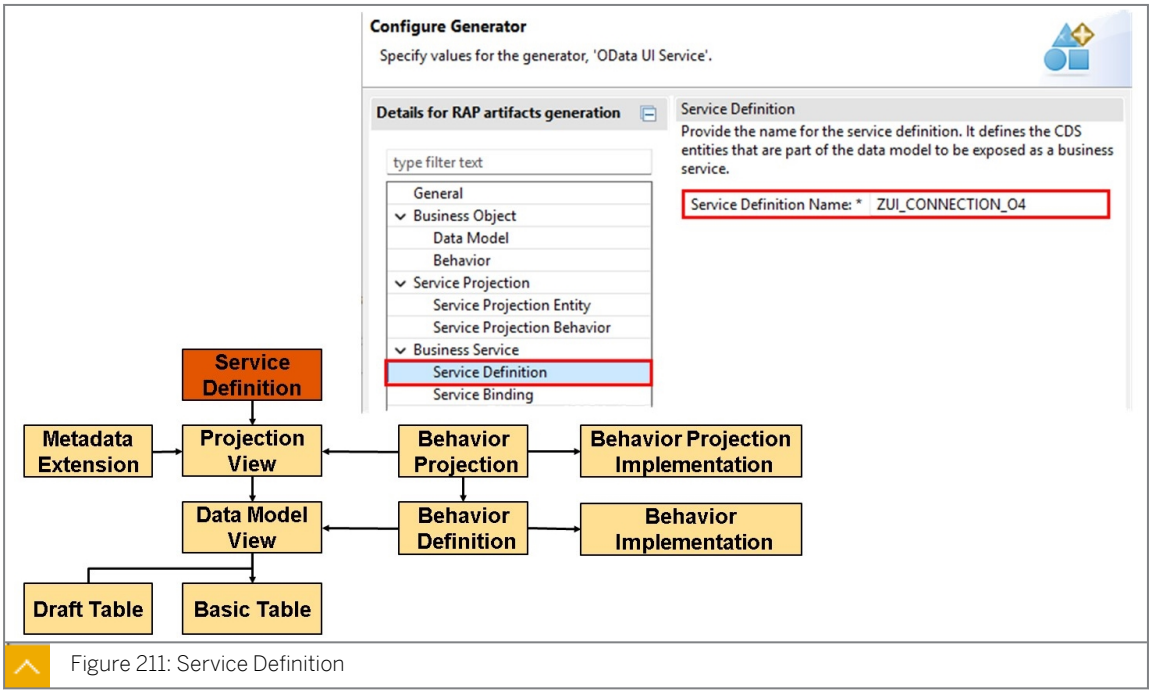


Figure 211: Service Definition

In order to expose the app, you need to create a service definition and a service binding. The service definition specifies the projection view to be exposed in this service.

© Copyright. All rights reserved.

The generator suggests a name for the service definition in the form **ZUI_<...>_O4**. This indicates that the service is for a Fiori elements app and that it uses version 4 of the OData protocol.

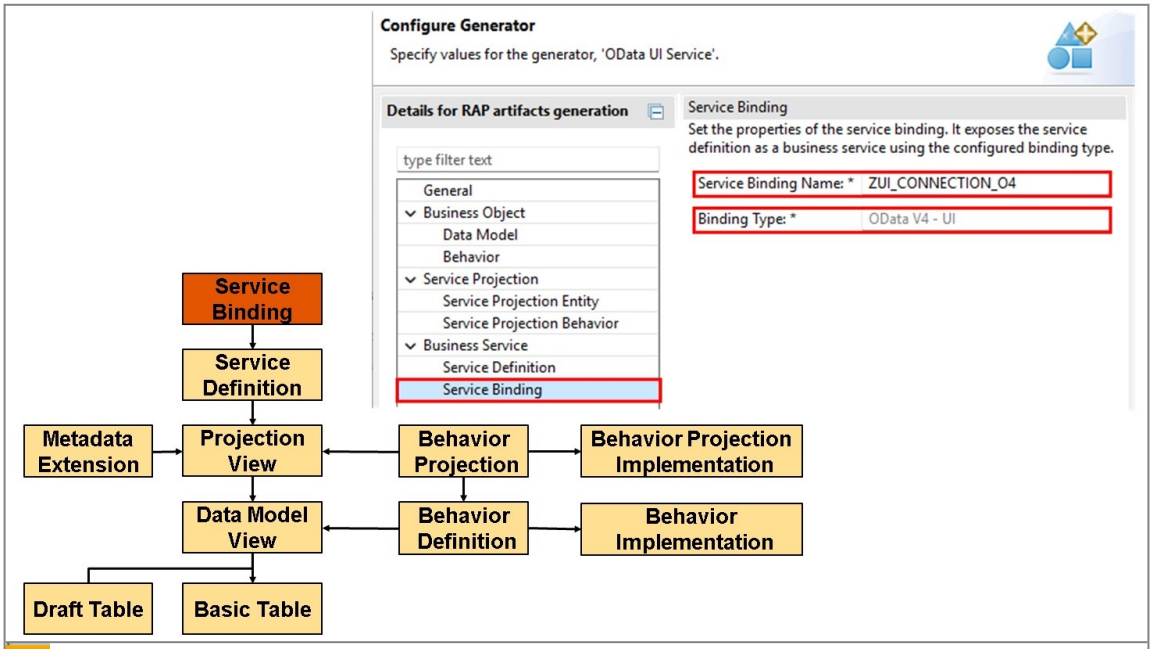


Figure 212: Service Binding

The service binding actually specifies the protocol to be used. The suggested name for the service binding is the same as for the service definition: **ZUI_<...>_O4**.

At the end of the wizard, the system displays a list of all of the objects that it is going to generate. You can check your entries against the naming conventions and look out for typing errors and, if necessary, go back and change any that are incorrect.

Preview of the OData UI Service

Publish and Preview the Service

Using your database table and the object generator, you can generate all of the objects that you need to get a working Fiori Elements app. The only manual step is to publish the service.

© Copyright. All rights reserved.

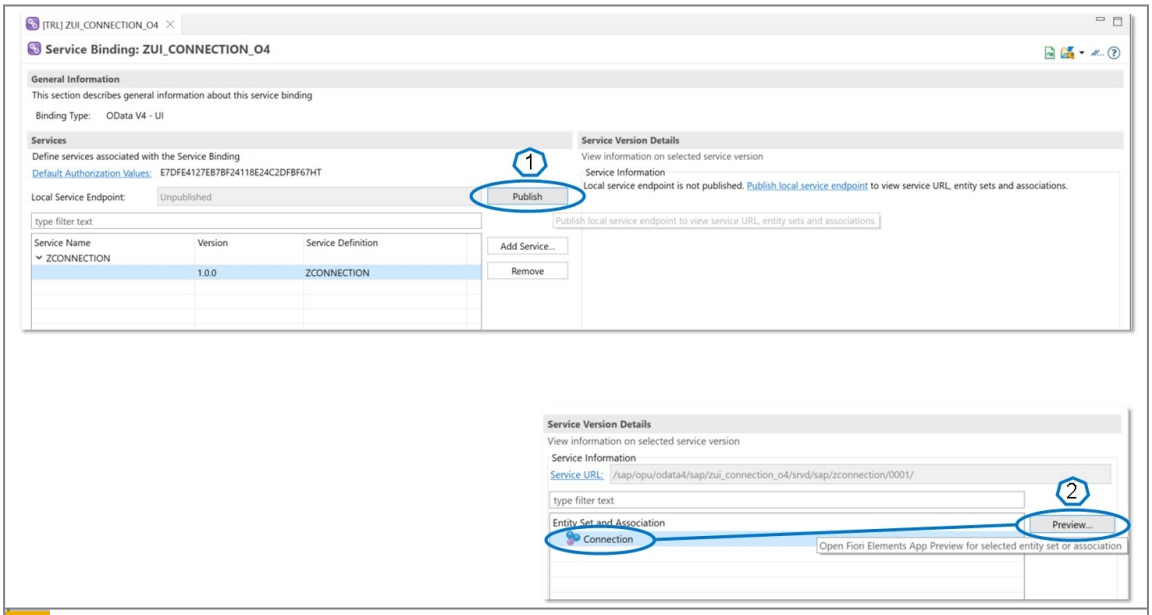


Figure 213: Publish and Preview the Service

You must publish the service before you can test the app. To do so, open the service binding and choose *Publish*. Once you have done this, the entity that you created (in this case, Connection) appears in the list of entities. Mark it and choose *Preview* to test the app.

Test the App

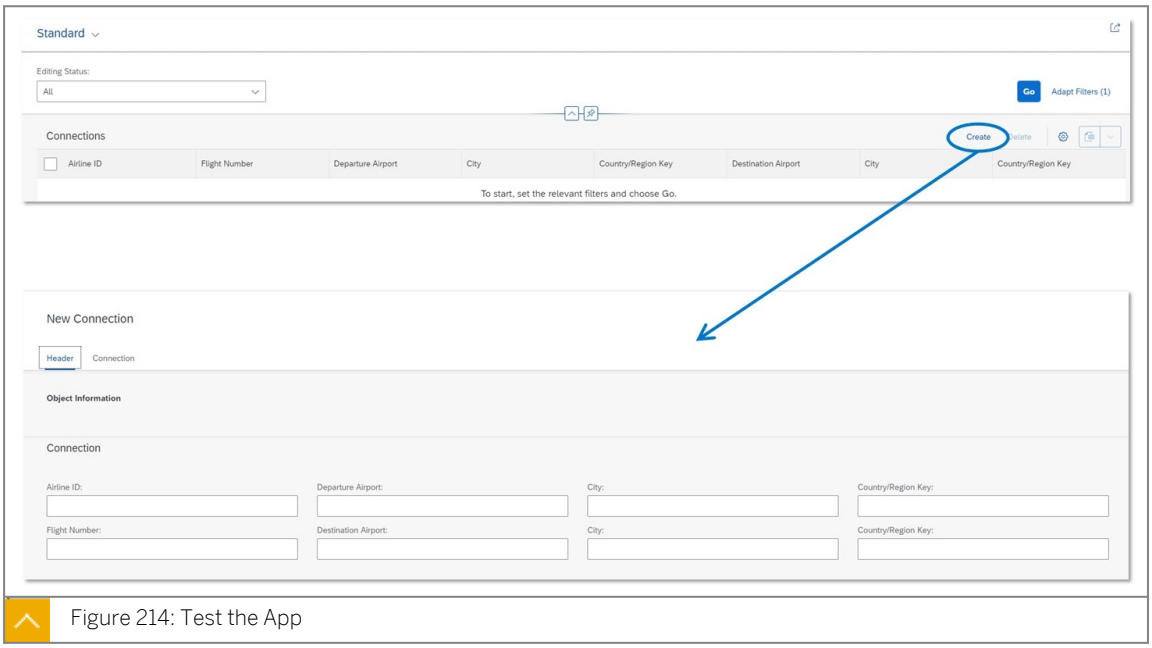


Figure 214: Test the App

When you choose *Preview*, a browser window opens and you can create a new flight connection. However, the app does not implement any checks other than type checks (for example, only digits are allowed in the *Flight Number* field).

© Copyright. All rights reserved.

Unit 8
Exercise 19

Generate and Preview an OData UI Service

In this exercise, you generate the repository objects for an OData UI service and preview the service as an SAP Fiori app.

Template:

none

Solution:

The generated development objects in package **`/LRN/S4D400_EXERCISE_RAP`**.

Note:
Please note that the implementation of the generator used may change. Therefore, the development objects generated in the exercise may differ from those in the solution.

Prerequisites

You completed the previous exercise. You created the database table **`Z##FLIGHT`** (where ## is your group number) and the database table contains data.

Task 1: Use the Generator

Use the generator *ABAP RESTful Application Programming Model → OData UI Service* to generate the required development objects in your subpackage **`ZS4D400_##_RAP`**. To do this, specify the following values in the wizard:

Field	Value
General → Referenced Object	**`Z##FLIGHT`**
General → Project Name	*<empty>*
General → Artifacts Prefix	*<empty>*
General → Artifacts Suffix	*<empty>*
Business Object → Data Model → CDS Entity Name	**`ZR_##Flight`**
Business Object → Data Model → CDS Entity Alias	**`Flight`**
Business Object → Behavior → Behavior Implementation Class	**`ZBP_R_##FLIGHT`**
Business Object → Behavior → Draft Table Name	**`Z##FLIGHT_D`**

© Copyright. All rights reserved.

Field	Value
Service Projection → Service Projection Entity → CDS Entity Name	**ZC_##Flight**
Service Projection → Service Projection Behavior → Behavior Implementation Class	**ZBP_C_##FLIGHT**
Business Service → Service Definition → Service Definition Name	**ZUI_##FLIGHT_O4**
Business Service → Service Binding → Service Binding Name	**ZUI_##FLIGHT_O4**
Business Service → Service Binding → Binding Type	**OData V4 - UI**

1. Start the object generator for your database table **Z##FLIGHT** (where ## is your group number).
2. In the dialog box, select the generator.
3. Ensure that the development objects are created in your subpackage **ZS4D400_##_RAP**.
4. Configure the generator with the data from the table.

Caution:
Do not choose *Next* before you have entered all data.

5. Check the repository objects that are going to be generated and generate the objects.

Note:
Be patient. Generating the objects takes some time. At the end of the generation process, a dialog box appears asking if you want to open the generated Service Binding in the editor. Confirm this dialog by clicking *Open*.

6. Analyze the generator output in the *Project Explorer*.

Task 2: Preview the App

Publish the OData UI service and preview the result as an SAP Fiori app.

1. Publish the local service endpoint.

Note:
Publishing the service may take some time.

2. Preview the OData UI service as an SAP Fiori app.
3. Display the list of flights.

© Copyright. All rights reserved.

Which buttons are available in the toolbar of the displayed table?

4. For one of the flights, navigate to the details view.

   Which buttons are available in the Object Page header?

5. For this flight, switch to change mode.

   Which fields can you edit in change mode?

   Why is it not possible to edit the fields *Airline ID*, *Flight Number*, and *Flight Date*?

6. Enter some text in the field *Flight Price*, for example **a**.

   Is it possible to save the changes?

7. Enter a negative value for the *Flight Price*.

   Can you save the changes now?

© Copyright. All rights reserved.

Unit 8
Solution 19

Generate and Preview an OData UI Service

In this exercise, you generate the repository objects for an OData UI service and preview the service as an SAP Fiori app.

Template:

none

Solution:

The generated development objects in package **`/LRN/S4D400_EXERCISE_RAP`**.

Note:
Please note that the implementation of the generator used may change. Therefore, the development objects generated in the exercise may differ from those in the solution.

Prerequisites

You completed the previous exercise. You created the database table **`Z##FLIGHT`** (where ## is your group number) and the database table contains data.

Task 1: Use the Generator

Use the generator *ABAP RESTful Application Programming Model → OData UI Service* to generate the required development objects in your subpackage **`ZS4D400_##_RAP`**. To do this, specify the following values in the wizard:

Field	Value
General → Referenced Object	**`Z##FLIGHT`**
General → Project Name	*<empty>*
General → Artifacts Prefix	*<empty>*
General → Artifacts Suffix	*<empty>*
Business Object → Data Model → CDS Entity Name	**`ZR_##Flight`**
Business Object → Data Model → CDS Entity Alias	**`Flight`**
Business Object → Behavior → Behavior Implementation Class	**`ZBP_R_##FLIGHT`**
Business Object → Behavior → Draft Table Name	**`Z##FLIGHT_D`**

© Copyright. All rights reserved.

Field	Value
Service Projection → Service Projection Entity → CDS Entity Name	**ZC_##Flight**
Service Projection → Service Projection Behavior → Behavior Implementation Class	**ZBP_C_##FLIGHT**
Business Service → Service Definition → Service Definition Name	**ZUI_##FLIGHT_O4**
Business Service → Service Binding → Service Binding Name	**ZUI_##FLIGHT_O4**
Business Service → Service Binding → Binding Type	**OData V4 - UI**

1. Start the object generator for your database table **Z##FLIGHT** (where ## is your group number).
   a) In the *Project Explorer*, expand the following path: *Favorite Packages → ZS4D400_## → ZS4D400_##_RAP → Dictionary → Database Tables* (where ## is your group number). Alternatively, open the definition of database table **Z##FLIGHT** and, if necessary, link the *Project Explorer* view with the editor.
   b) In the *Project Explorer* view, right-click database table **Z##FLIGHT** to open the context menu.
   c) From the context menu, choose *Generate ABAP Repository Objects...*.
2. In the dialog box, select the generator.
   a) On the left-hand side of the dialog box, choose *ABAP RESTful Application Programming Model → OData UI Service*.
   b) Choose *Next* to navigate to the next step.
3. Ensure that the development objects are created in your subpackage **ZS4D400_##_RAP**.
   a) When the window reads *Enter Package* at the top, ensure that the *Package* field contains **ZS4D400_##_RAP**.
   b) Choose *Next* to navigate to the next step.
4. Configure the generator with the data from the table.

Caution:
Do not choose *Next* before you have entered all data.

   a) Select the items on the left side of the dialog box one by one and fill in the corresponding fields in the right pane. The required values are listed in the table above.
   b) After you entered all data choose *Next*.
5. Check the repository objects that are going to be generated and generate the objects.

© Copyright. All rights reserved.

> Note:
> Be patient. Generating the objects takes some time. At the end of the generation process, a dialog box appears asking if you want to open the generated Service Binding in the editor. Confirm this dialog by clicking *Open*.

a) Check the list of repository objects that are displayed.

b) When you are sure you have entered all names correctly choose *Next*.

c) Confirm the transport request and choose *Finish*.

d) Confirm the dialog window at the end of the generation process by choosing *Open*.

6. Analyze the generator output in the *Project Explorer*.
   a) In the *Project Explorer*, place the cursor on your subpackage **`ZS4D400_##_RAP`** and press **`F5`** to reload the content of the package.
   b) Expand all subnodes of the package to see the names of the generated development objects.

Task 2: Preview the App

Publish the OData UI service and preview the result as an SAP Fiori app.

1. Publish the local service endpoint.

> Note:
> Publishing the service may take some time.

   a) If you have not already, open the generated *ZUI_##FLIGHT_O4* Service Binding.
   b) If the Service Binding is inactive press **`Ctrl + F3`** to activate it.
   c) To publish the service, choose *Publish* in the center of the editor.

2. Preview the OData UI service as an SAP Fiori app.
   a) In the *Service Version Details* section on the right-hand side, choose *Flight* and then *Preview...*.

   Result

   A browser window opens with an SAP Fiori application. If prompted, log on to the application with your ABAP system user.

3. Display the list of flights.
   a) Choose *Go* to display the list of flights that are stored in your database table **`Z##FLIGHT`**.

   Which buttons are available in the toolbar of the displayed table?

   Create, *Delete*, *Copy to Clipboard*, *Show More/Less per Row*, *Settings*, *Export Table*

4. For one of the flights, navigate to the details view.
   a) Choose one of the list entries to navigate to the Object Page with details of that flight.

© Copyright. All rights reserved.

Which buttons are available in the Object Page header?

Edit, *Delete*, *Share*

5. For this flight, switch to change mode.
   a) Choose *Edit*.

   Which fields can you edit in change mode?

   Flight Price, *Currency Code*, and *Plane Type*

   Why is it not possible to edit the fields *Airline ID*, *Flight Number*, and *Flight Date*?

   The Fields *Airline ID*, *Flight Number*, *and Flight Date* belong to the primary key of the database table. Changing the primary key is never a good idea. By default, the generator sets all key fields to write protected during update.

6. Enter some text in the field *Flight Price*, for example **a**.

   Is it possible to save the changes?

   No. The field *Flight Price* is based on a numeric type and the generated application includes type-based input checks.

7. Enter a negative value for the *Flight Price*.

   Can you save the changes now?

   Yes, the check for positive prices is application logic. It has to be implemented by the developer in the behavior of the business object.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Create a database table.
- Generate the development objects for an OData UI service.

© Copyright. All rights reserved.

Unit 8
Lesson 3

Adding ABAP logic

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Implement the behavior of a Business Object.

Validations

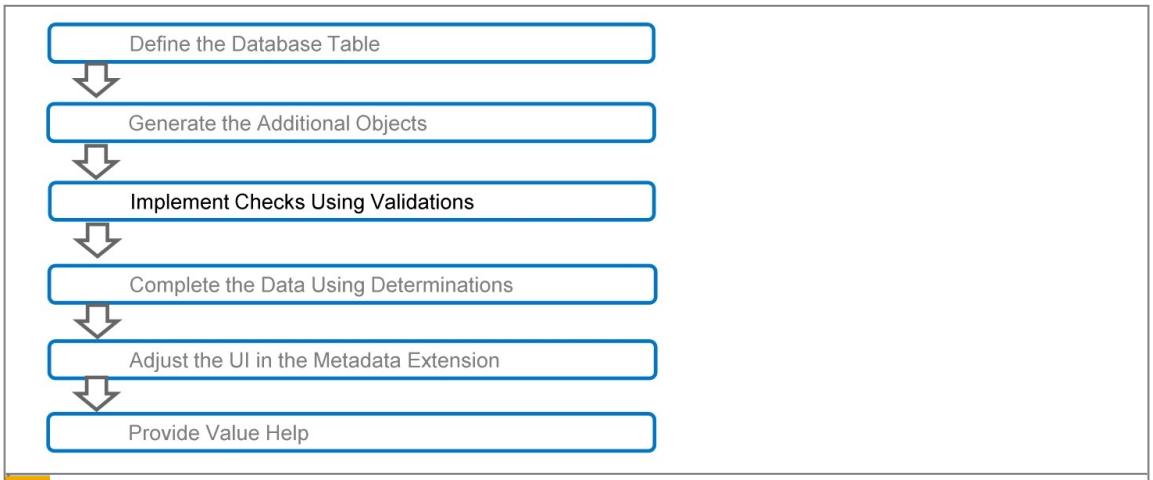
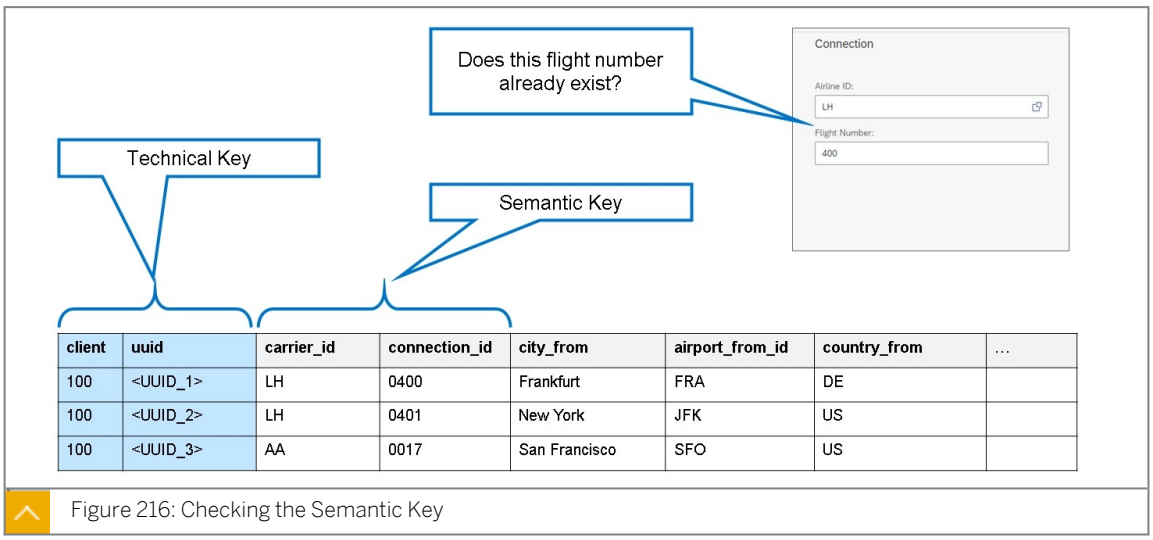


Figure 215: Developing a Simple Application

Checking the Semantic Key



client	uuid	carrier_id	connection_id	city_from	airport_from_id	country_from	…
100	<UUID_1>	LH	0400	Frankfurt	FRA	DE	
100	<UUID_2>	LH	0401	New York	JFK	US	
100	<UUID_3>	AA	0017	San Francisco	SFO	US	

Figure 216: Checking the Semantic Key

© Copyright. All rights reserved.

In the ABAP RESTful application programming model, the key of a database table is often made up of the client field and a UUID field, whose value is assigned automatically by the runtime when you create a new instance of the business object. This field combination is sufficient to ensure that the system can identify each record in the table uniquely. However, as well as this technical key, our object also has a semantic key - in this case the combination of airline and flight number, which must also be unique according to the business logic. In order to ensure the uniqueness of this field combination, you must implement your own check in the form of a validation.

You declare validations in the behavior definition of the CDS view entity, and implement them in the behavior implementation class.

Input Checks in the App

As well as checking the semantic key, there are other checks that you need to perform. For example, although the generated app allows you to create, read, update and delete data, it does not yet contain any consistency checks. Consequently, you can create flight connections for airlines that don't exist, or where the departure and destination airports are the same.

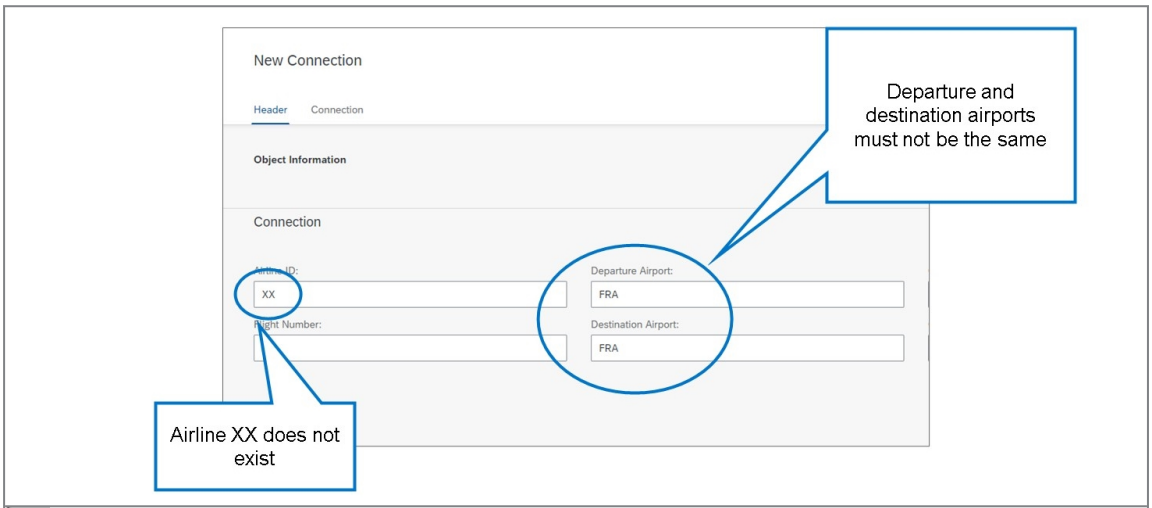
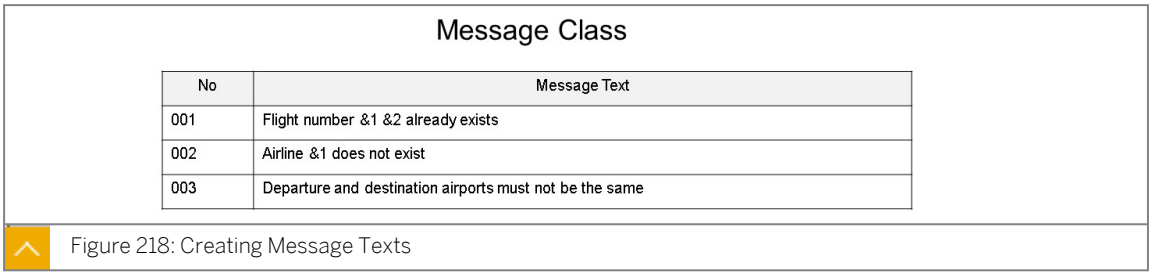


Figure 217: Input Checks in the App

To prevent this from happening, you define further validations in the behavior definition, and implement them in the behavior implementation class.

Creating Message Texts

Before you create the validation, you must create the texts that you want to display. You do this using a message class. A message class is a collection of up to 1000 messages that belong to a particular application area. As shown in the figure, each text has a number which identifies the message uniquely within the message class.


Message Class

No	Message Text
001	Flight number &1 &2 already exists
002	Airline &1 does not exist
003	Departure and destination airports must not be the same

Figure 218: Creating Message Texts

To create a new message class, proceed as follows:

© Copyright. All rights reserved.

1. Choose *File* → *New* → *Other...* and type message into the filter field.
2. Double-click the *Message Class* entry in the hit list, then enter a package, name, and description for the new message class. Choose *Next*.
3. Assign the message class to a transport request and choose *Finish*.

Messages can also contain placeholders, which are replaced with concrete values when the message is displayed. Placeholders are denoted by the ampersand symbol followed by a number. You can use up to four placeholders in each message.

Defining the Validation

To define a validation, you add a validation declaration to the behavior definition of your business object. In this example, the validation should be performed whenever the user saves a data record, and this could be either when they create the record or if they subsequently change it.

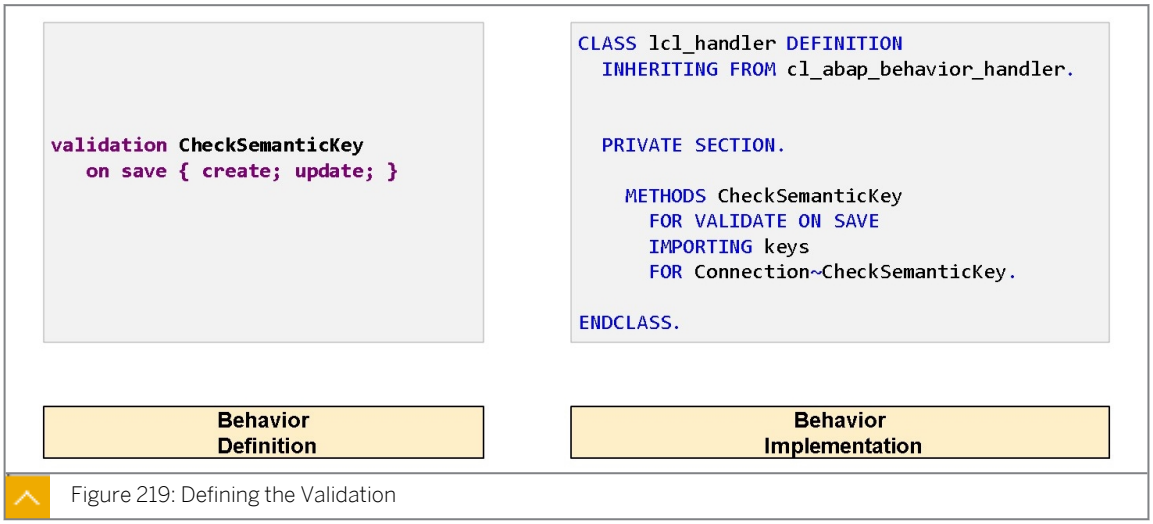


Figure 219: Defining the Validation

When you define the validation in the behavior definition, a warning tells you that the corresponding method does not exist. Use a quick fix (key combination `CTRL + 1`) to add the method to the behavior implementation. The behavior implementation is a local class within your behavior pool. The method definition contains the addition `FOR VALIDATE ON SAVE`, which identifies it as the implementation of the validation. It has an importing parameter *KEYS*. This is an internal table containing the keys of the created or changed objects. You use these to read the actual data that the user has entered.

The addition `FOR Connection~CheckSemanticKey` links the method with the validation *CheckSemanticKey* from the behavior definition. Here, *Connection* is the alias name of the view entity *Z_R_CONNECTION*.

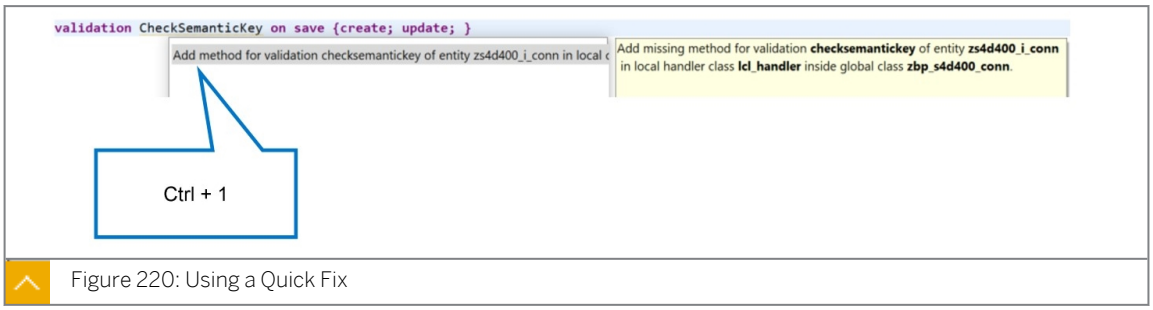


Figure 220: Using a Quick Fix

© Copyright. All rights reserved.

When you define a validation, you must also create its implementation. This is a method in the behavior pool. The easiest way to do this is to use a quick fix. Position the cursor on the name of the validation and press `CTRL + 1`. ADT proposes to create the method. Double-click the proposal to create the method.

The Validation Process

> Note:
> Some code examples in this section use SELECT statements inside of loops. This has been done to keep the examples simple. Note that SELECTs in loops can cause performance problems and should be avoided.

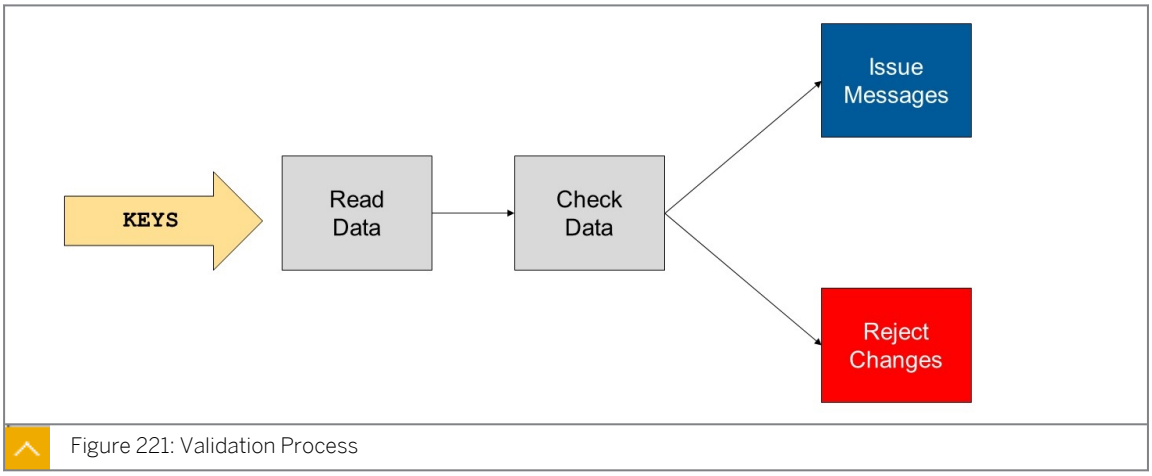


Figure 221: Validation Process

When the system triggers a validation, it calls the corresponding implementation. The importing parameter *KEYS* contains the keys of the data records that have been changed. You use the keys to read the fields of the records that you need using Entity Manipulation Language (EML). EML is a special set of statements in ABAP that allows you to address business objects.

Once you have read the data, you can perform the checks that you need. If the check fails, you will need to issue an appropriate error message and, importantly, tell the framework not to write the changes to the database.

© Copyright. All rights reserved.

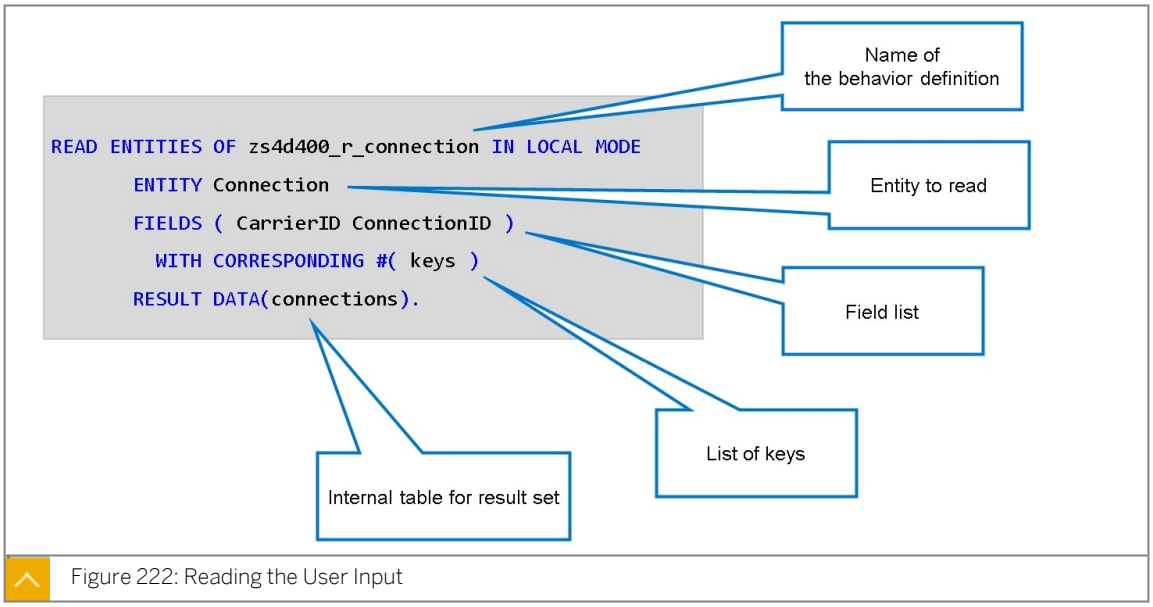


Figure 222: Reading the User Input

The first task in a validation is to read the user input. You do this using the Entity Manipulation Language (EML) statement `READ ENTITIES`. The keys of the corresponding data records are passed to the validation using the importing parameter *keys*.

The fields that you need to validate the semantic key are *CarrierID* for the airline and *ConnectionID* for the flight number.

The code snippet uses the CORRESPONDING key word and an inline declaration for the result set. Below, you see the equivalent code using explicitly-defined variables, which makes it easier to understand the types that are used.

```
DATA read_keys   TYPE TABLE FOR READ IMPORT zs4d400_r_connection.
DATA connections TYPE TABLE FOR READ RESULT zs4d400_r_connection.

read_keys = CORRESPONDING #( keys ).

READ ENTITIES OF zs4d400_r_connection IN LOCAL MODE
       ENTITY Connection
       FIELDS ( CarrierID ConnectionID )
         WITH read_keys
       RESULT connections.
```

Once you have read the user input, you can use the values of *CarrierID* and *ConnectionID* to see if this semantic key has already been used in another data set than the one you are processing just now. Since the key combination could be in either the active table or the draft table, you need to look in both, and the most efficient way to do this is with a **union**.

© Copyright. All rights reserved.

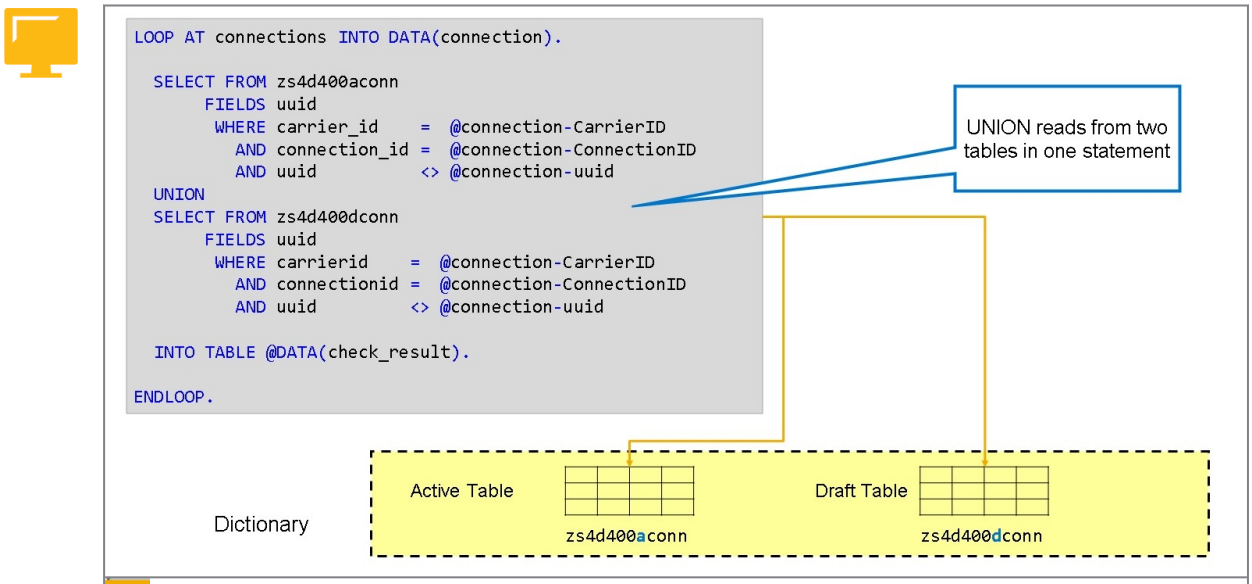


Figure 223: Validating the Key

The result set of this query should always be empty. If not, there are more records with the same combination of *CarrierID* and *ConnectionID*, this means that the record that the user is currently trying to create is a duplicate, and must be rejected.

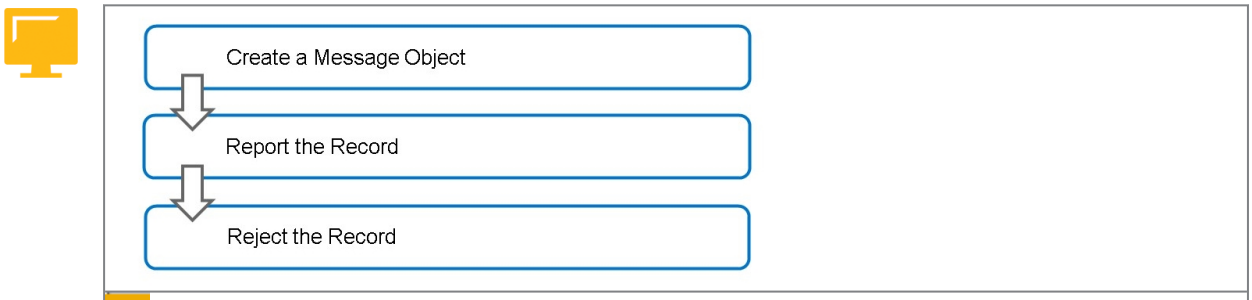


Figure 224: What to Do When the Validation Fails

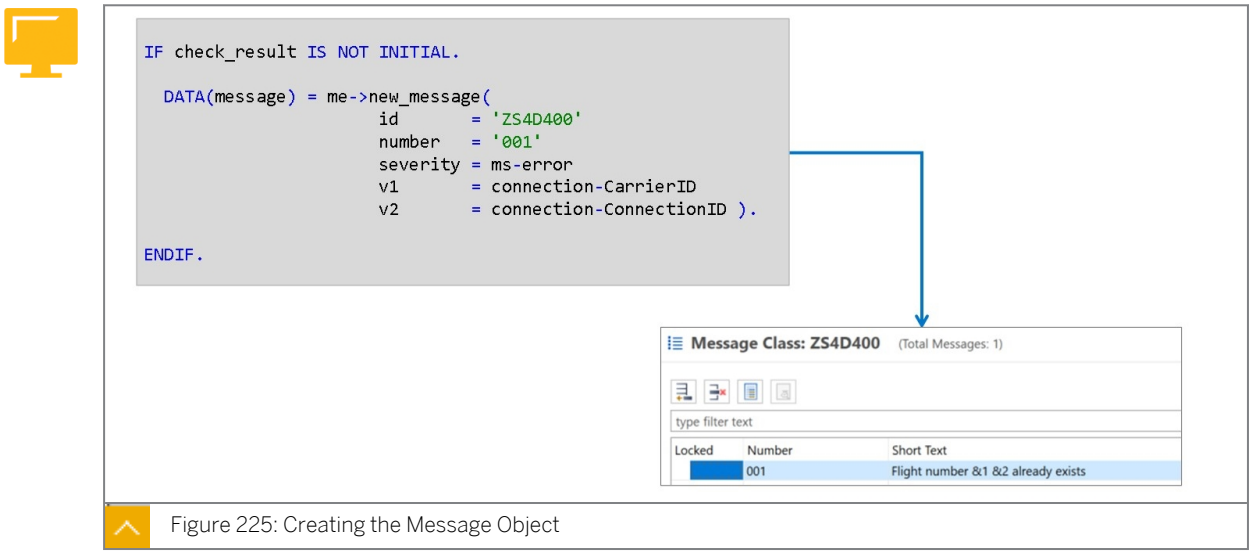


Figure 225: Creating the Message Object

If the combination of carrier ID and connection ID already exists, there will be an entry in the table *check_result*. In this case, you must issue a message.

© Copyright. All rights reserved.

The first step is to create a message object. You do this using the self-reference me and calling the method *new_message( )*. The parameters *ID*, *number*, and *severity* are mandatory. *ID* is the name of the message class that contains the message; *number* is the message number. *Severity* classifies the message as a success, information, warning, or error message. The behavior implementation class contains a structured constant *ms* whose components represent the different severity levels. In this case, you need the severity level *ms-error*.

The method also has optional importing parameters *v1*, *v2*, *v3*, and *v4*. You use these to replace placeholders with concrete values. In this example, the placeholder &1 is replaced with the airline code, placeholder &2 is replaced with the flight number.

The result of the method call is an object reference. In the next step, you will pass the object to the runtime so that the error message is returned the OData service and displayed in the app preview.

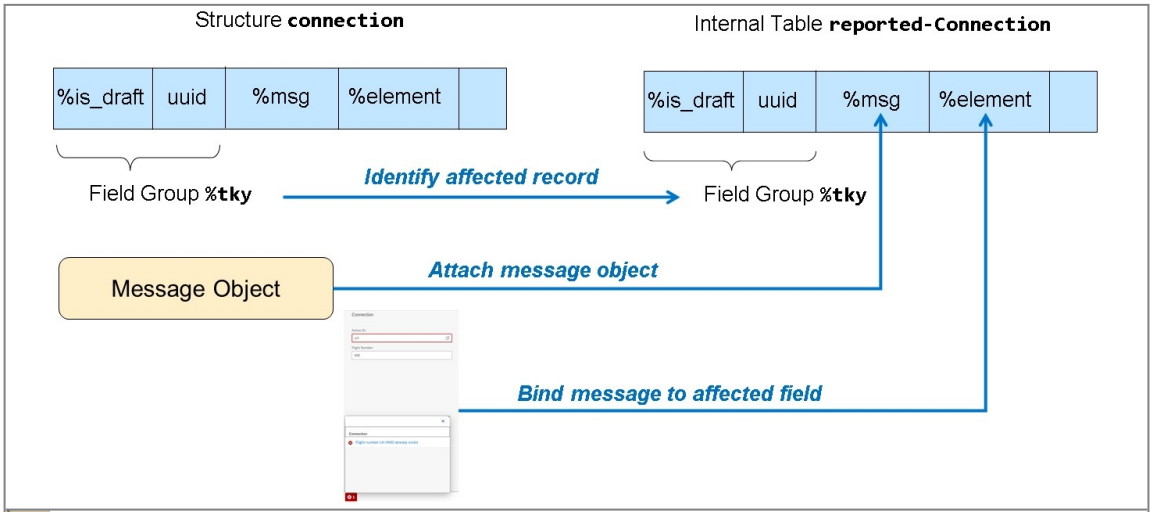


Figure 226: Reporting the Message

To make the runtime display a message, you must report it using the *reported* structure. This is an implicit changing parameter of all validation methods and is a deep structure. It contains a component with the alias name of the entity. This component is an internal table.

To report the message, you must do three things:

1. Add the key of the affected record to the internal table. You can do this using the field group *%tky*. When you group fields like this, you can address the name of the group instead of having to address each field individually.
2. Attach the message object to the table. You do this by assigning the object reference of the message object to the *%msg* component of the internal table.
3. Bind the message to the affected field. This ensures that the field is emphasized in the app. This, in turn, helps the user to navigate the app better. You do this using the *%element* component of the internal table.

© Copyright. All rights reserved.

```
DATA reported_record LIKE LINE OF reported-connection.
```

```
reported_record-%tky = connection-%tky.
reported_record-%msg = message.
reported_record-%element-CarrierID    = if_abap_behv=>mk-on.
reported_record-%element-ConnectionID = if_abap_behv=>mk-on.

APPEND reported_record TO reported-connection.
```

Figure 227: Reporting the Message

In this example, *reported_record* is a structure with the line type of the internal table *reported-connection*. You fill the *%tky* component with the contents of the *%tky* field group in the structure *connection*. This *connection* structure is used as a work area for the internal table containing the data that the user entered. Next, you assign the message object that you created using the *new_message( )* method to the *%msg* component. Finally, to bind the message to the fields *CarrierID* and *ConnectionID*, you use the structure *%element*. *%element* contains a component for each field in the entity. If you set a component to *true*, the corresponding input field will be highlighted in the app. You do this using the structured constant *if_abap_behv=>mk*. This has the component *on* for checked/true and *off* for unchecked/false.

You cannot use the global constants *abap_true* and *abap_false* at this point, as their data types are not compatible.

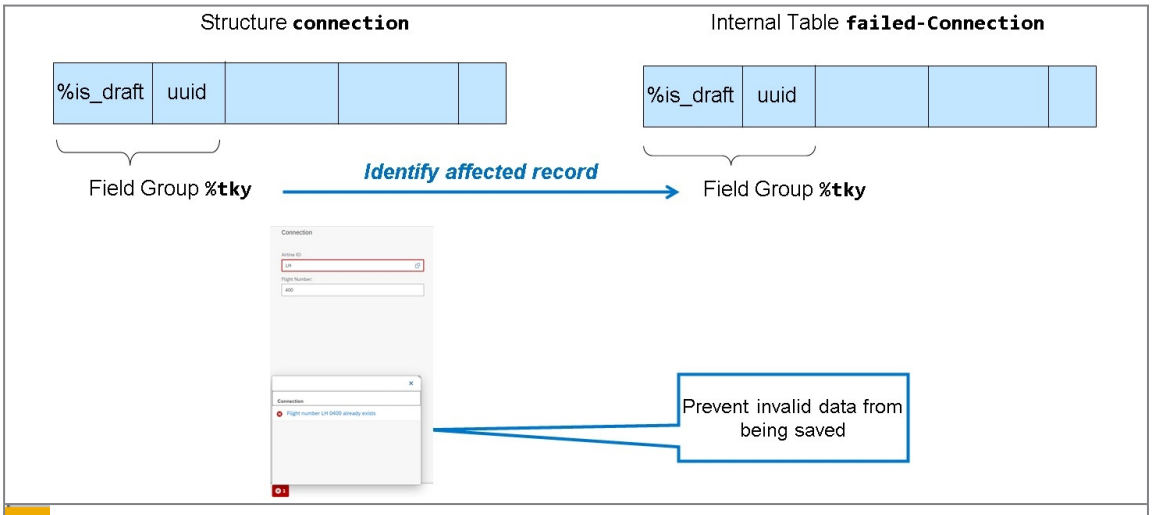


Figure 228: Rejecting the Changes

As well as issuing the message, you must also tell the runtime not to save the incorrect data. To do this, you use the *failed* structure of the validation method. *Failed* is an implicit changing parameter that is present in all validation methods.

To report a record as *failed*, add its field group *%tky* to the field group *%tky* of the internal table *failed-Connection*.

© Copyright. All rights reserved.

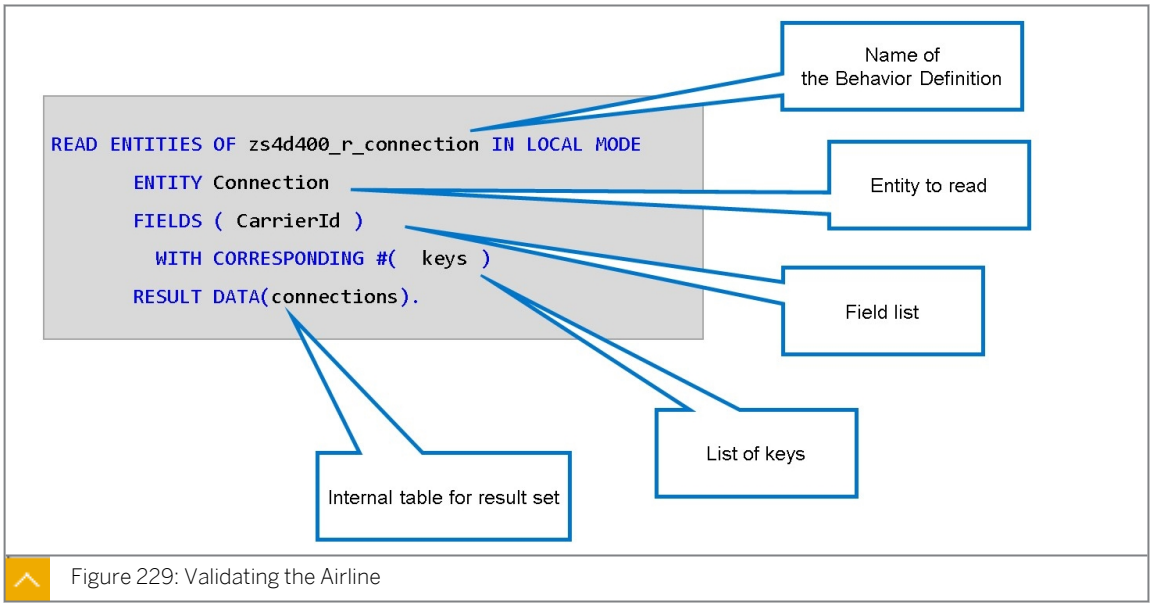


Figure 229: Validating the Airline

The next validation checks that the airline that the user has entered actually exists. The first step is to read the user input using the EML statement READ ENTITIES. This time, you only need to read the field *CarrierID*.

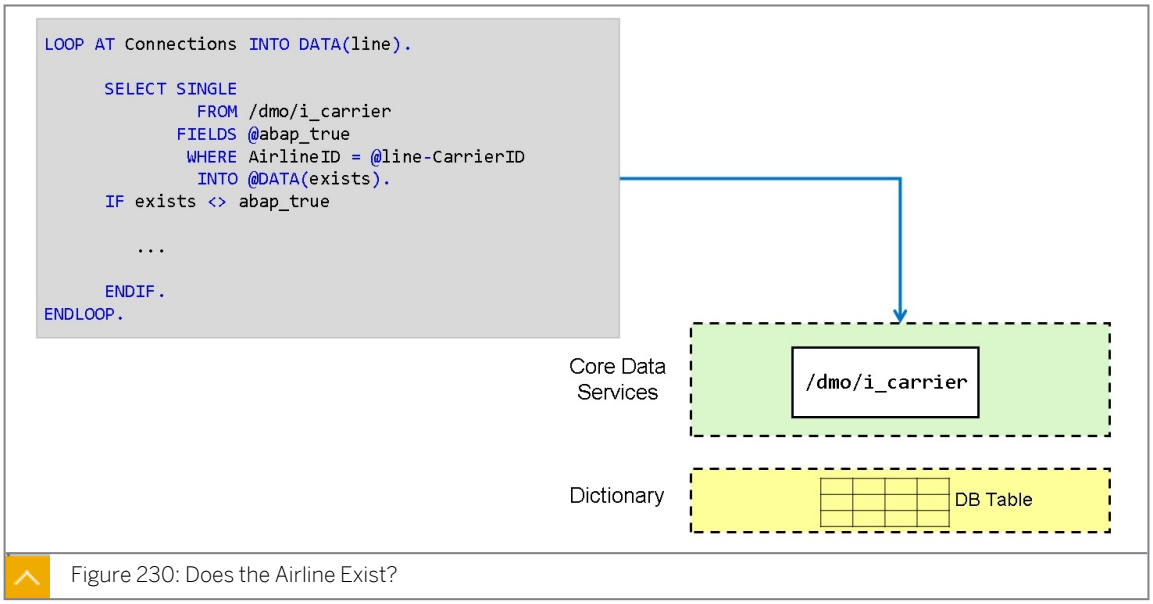


Figure 230: Does the Airline Exist?

The SELECT SINGLE statement reads data using the CDS view entity /dmo/i_carrier and checks whether the given airline exists. If it does, the value of the global constant abap_true ('X') is placed in the field exists. If exists is initial following the SELECT statement, you must issue a message, report it, and add the record to the failed structure as you did in the previous example.

© Copyright. All rights reserved.

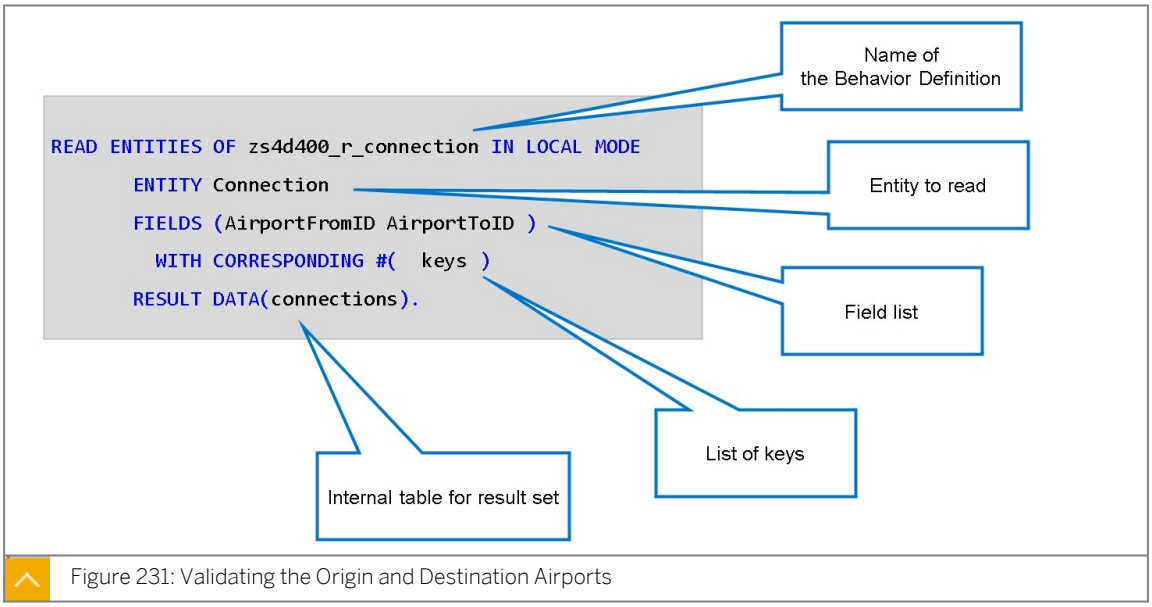


Figure 231: Validating the Origin and Destination Airports

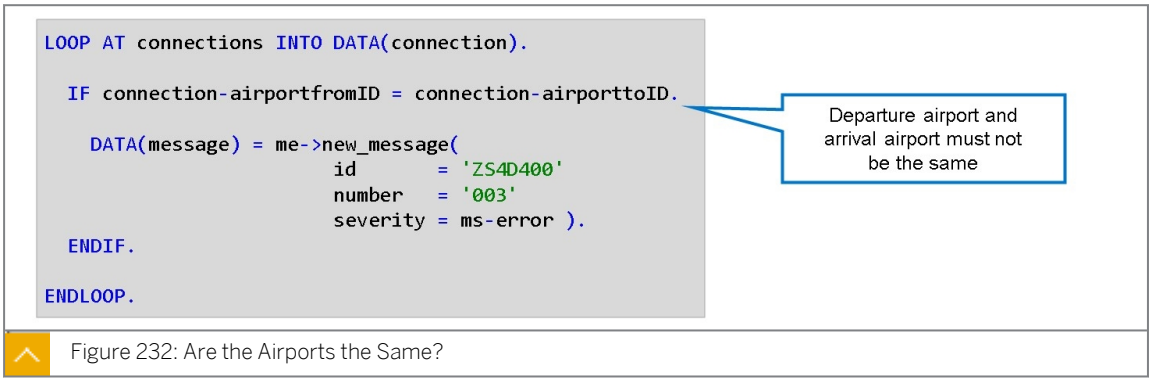


Figure 232: Are the Airports the Same?

If the departure and arrival airports are the same, you must issue the corresponding message and fill the reported and failed structures. The code extract shows the relevant coding to create the message. The coding to fill the reported and failed structures is the same as in the previous examples.

Determinations

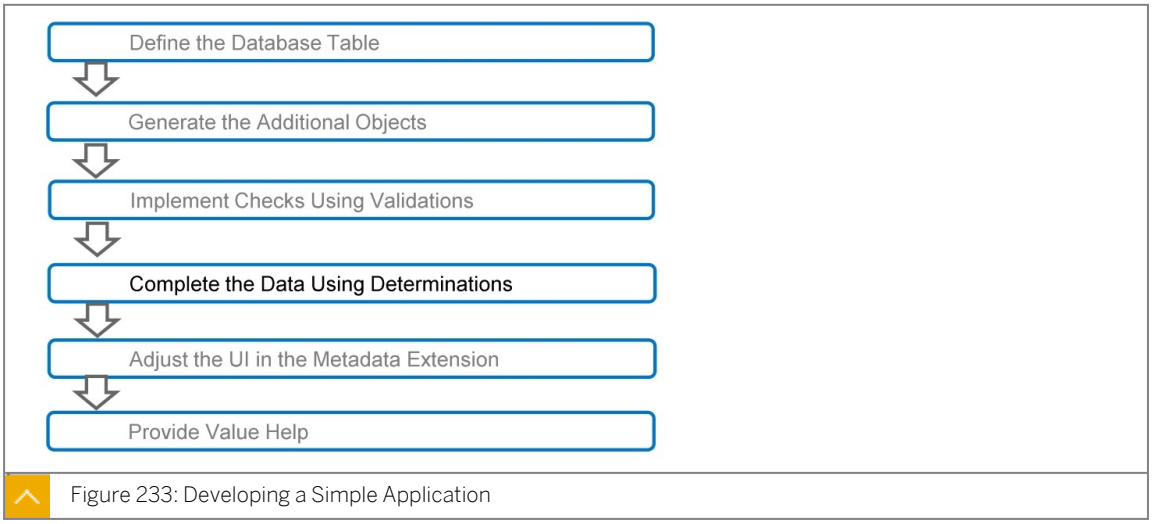


Figure 233: Developing a Simple Application

In this section, you will learn how to complete the data using determinations.

© Copyright. All rights reserved.

Determine Cities Based on Airport Codes

In the example app, the flight connection entity contains a departure airport, city, and country, and an arrival airport, city, and country. While it would be possible to force the user to enter all of this information, it is better in terms of user experience and data consistency to have the user enter only the airport codes and for the app to read the corresponding city and country information from the database. In the ABAP RESTful application programming model, you can perform this kind of task using a **determination**.

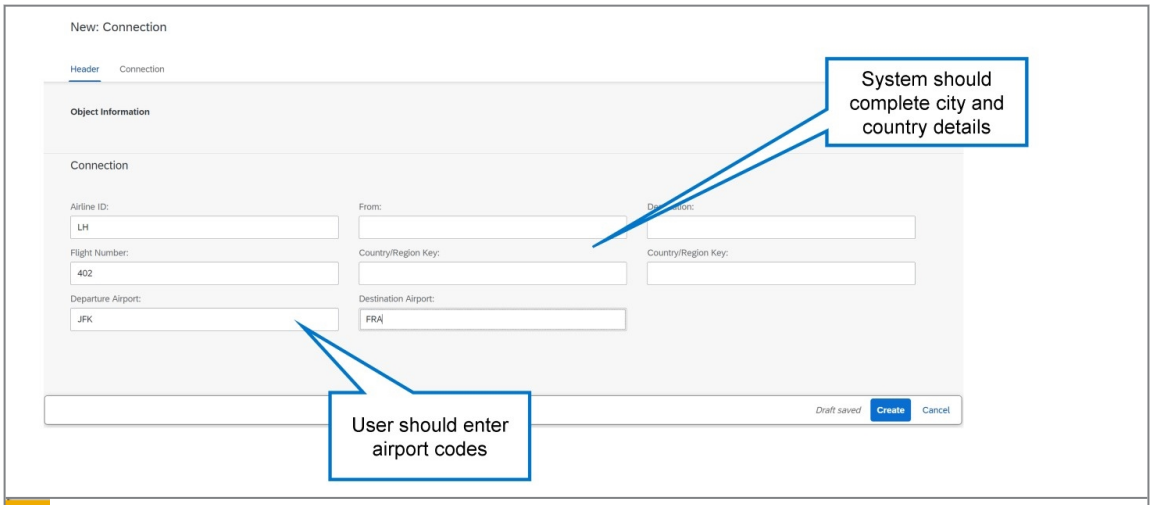


Figure 234: Determine Cities Based on Airport Codes

You will first implement the determination. Afterwards, you will learn how to deactivate input for the fields that will be filled automatically.

Defining the Determination

You define a determination in the behavior definition of a business object. The determination here is called getCities, it will be called whenever the business object is saved and at least one of the *AirportFromID* and *AirportToID* fields has changed. You can use a quick fix in the behavior definition to create the corresponding method in the behavior implementation.

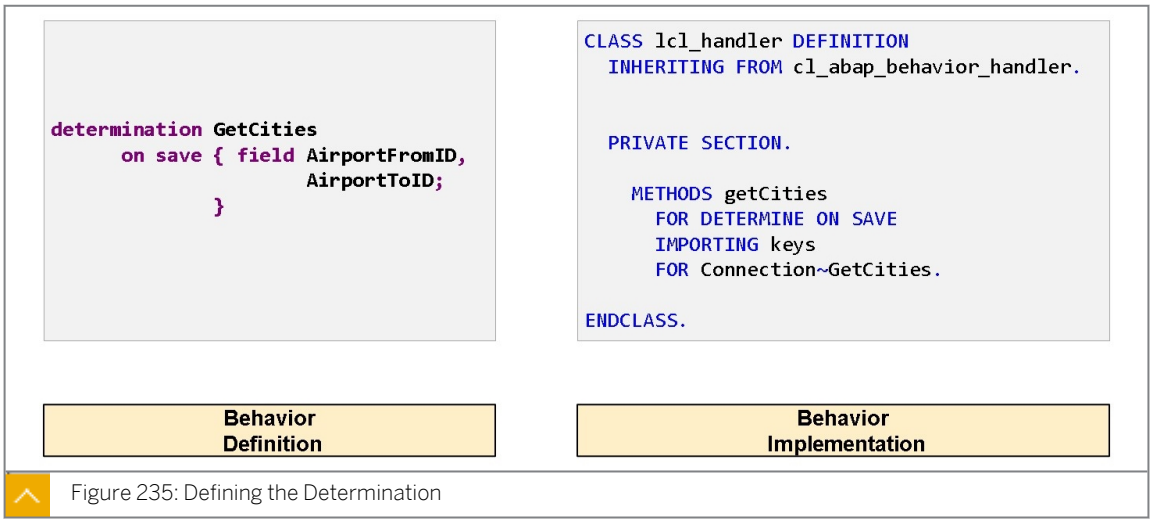


Figure 235: Defining the Determination

The Determination Process

Let's explore each step of the determination process.

© Copyright. All rights reserved.

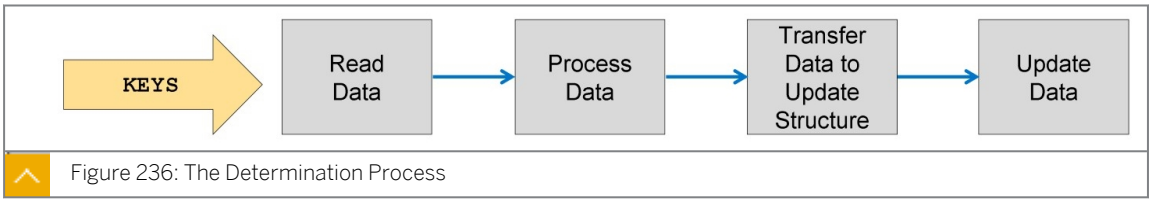


Figure 236: The Determination Process

When the system triggers a determination, it calls the corresponding implementation. The importing parameter KEYS contains the keys of the data records that have been changed. In the determination method, you use EML to read the data based on the keys in exactly the same way that you did in the validations. However, in a determination, you also manipulate the data in the method and you must consequently update the data held by the framework using the EML statement UPDATE.

Note:
Some code examples in this section use SELECT statements inside of loops. This has been done to keep the examples simple. Note that SELECTs in loops can cause performance problems and should be avoided.

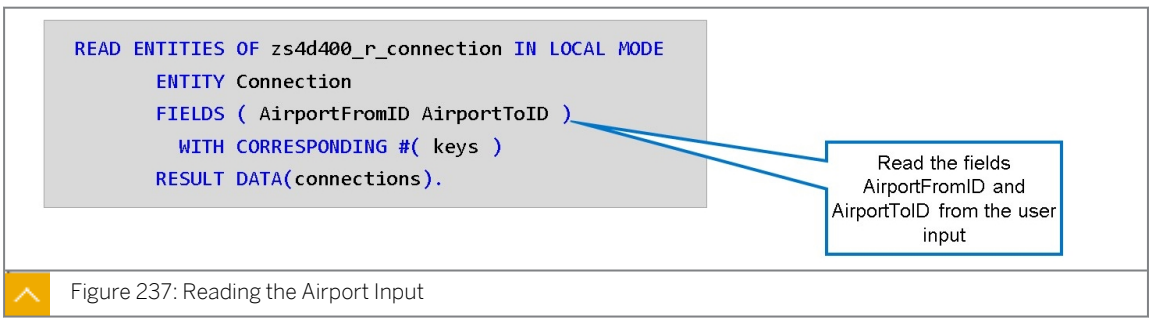


Figure 237: Reading the Airport Input

At the beginning of the determination you read the user input using EML. You need the *AirportFromID* and *AirportToID* fields and will use these to complete the city and country information.

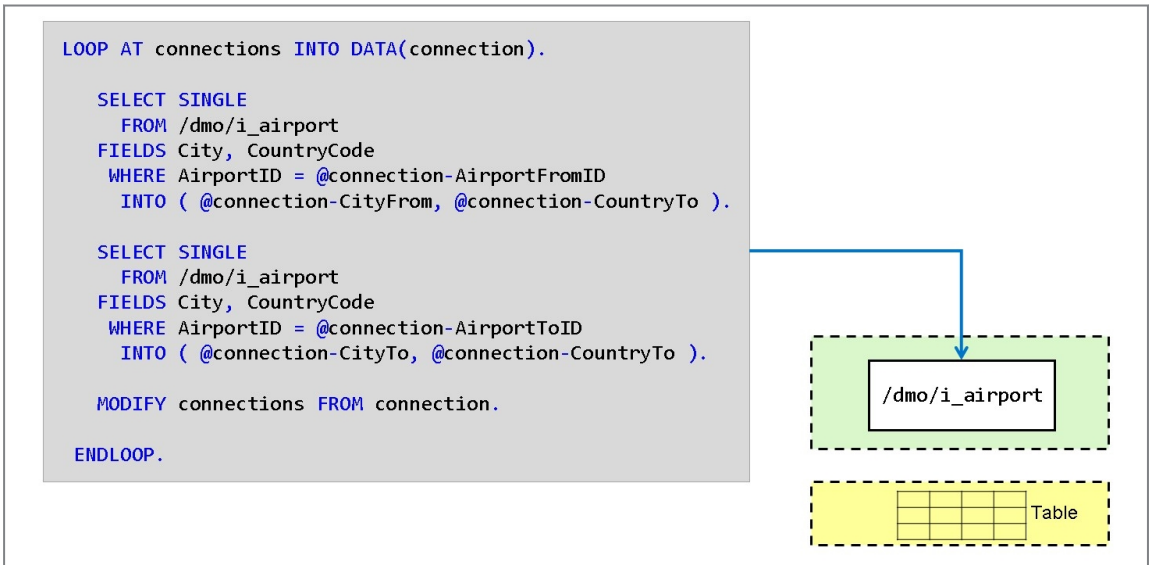


Figure 238: Reading the City and Country Information

© Copyright. All rights reserved.

The demonstration data model provides a CDS view entity */dmo/i_airport* that you can use to read the city and country in which a particular airport is located. The example uses the variant of the INTO clause in which you explicitly specify the fields of the structure that you want to fill. Remember that the changes to the data are in the work area of the internal table and that you must return them to the table itself using the MODIFY statement.

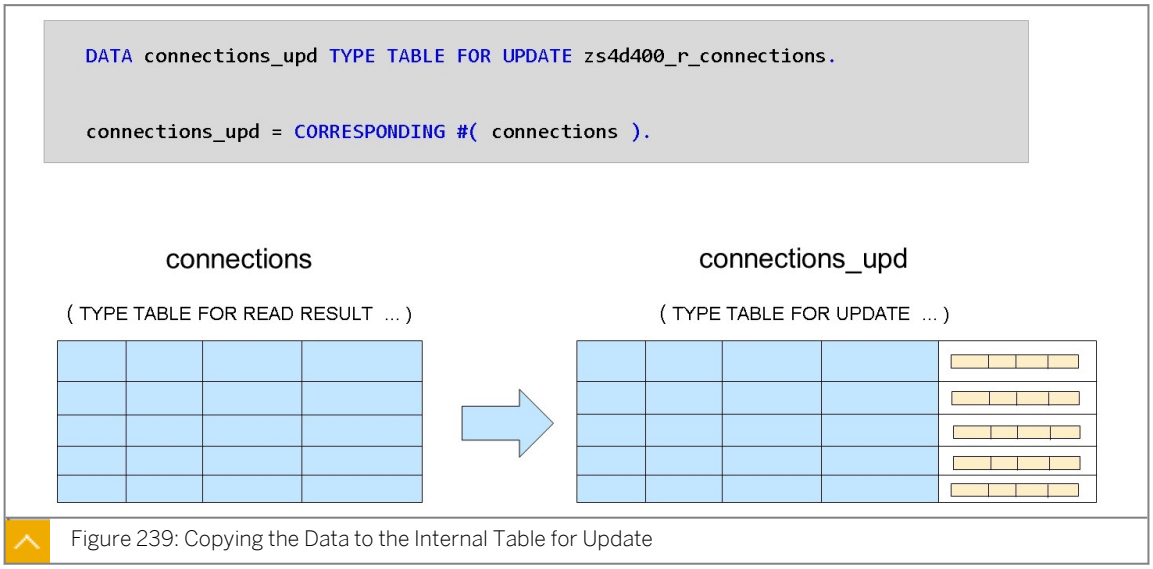


Figure 239: Copying the Data to the Internal Table for Update

The READ ENTITIES statement returns an internal table with the derived type FOR READ RESULT. To change the data in the transactional buffer, you need a MODIFY ENTITIES statement. You pass the data that you want to change to this statement using an internal table with the derived type FOR UPDATE. The data fields are identical in both types, however the FOR UPDATE table has an additional structure called %control that contains administrative information.

You cannot pass the *connections* table to the MODIFY ENTITIES statement. You therefore need to copy your data into an appropriately-typed internal table (*connections_upd*) before you perform the actual modification.

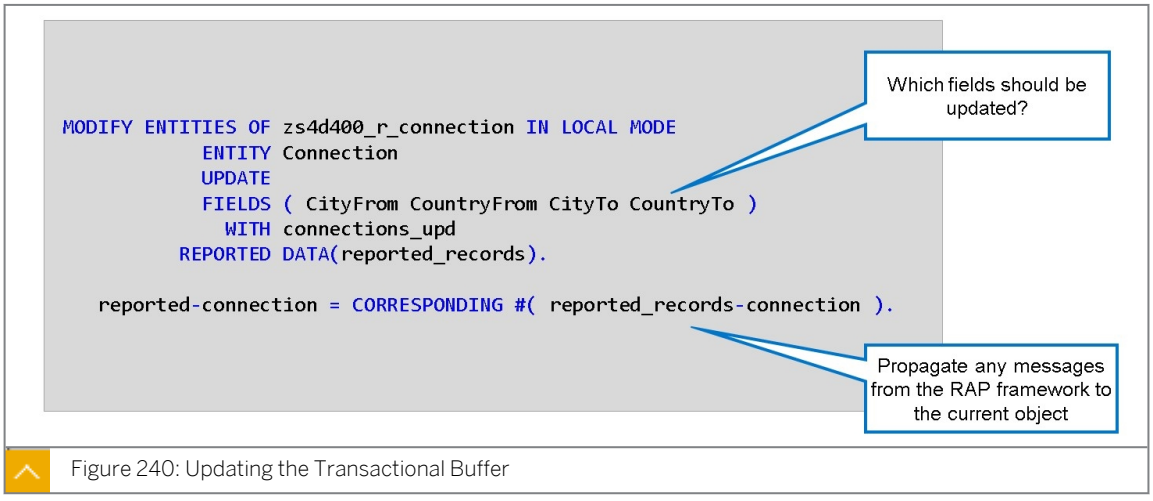


Figure 240: Updating the Transactional Buffer

To update the data with the fields that you filled in the determination, you use the MODIFY ENTITIES statement. In it, you specify which fields should be updated in the FIELDS clause, and pass the data in an internal table using the WITH addition. This table must have the correct derived data type, which in this case would be `TYPE TABLE FOR UPDATE zsd4d400_r_connection`.

© Copyright. All rights reserved.

The MODIFY ENTITIES statement can return messages, which you receive using the REPORTED clause. You then propagate these messages to your own business object by copying the contents of the internal table to the REPORTED structure of the determination method.

© Copyright. All rights reserved.

Unit 8
Exercise 20

Validate the Flight Price

In this exercise you define and implement a validation for the flight price.

Template:

none

Solution:

/LRN/S4D400_R_FLIGHT (Behavior Definition)
/LRN/BP_S4D400_R_FLIGHT (Global Class)

Prerequisites

You completed the previous exercises. You created and filled the database table **`Z##FLIGHT`** (where ## is your group number) and generated the development objects for an OData UI service.

Task 1: Validate the Price

Define and implement a validation to check that the field **`Price`** has a positive value (suggested name: **`validatePrice`**). If the value is negative or equals zero, reject the change and report a suitable error message from message class **`/LRN/S4D400`**.

1. In the behavior definition **`ZR_##FLIGHT`**, define a new validation **`validatePrice`**. Ensure that the validation is always executed during the standard operation **`create`** but only if the value of **`Price`** changed for all other operations.

   Hint:
   Use code-completion wherever possible to enter the code.

2. Activate the behavior definition.

3. Use a quick fix to create the validation implementation method in the behavior handler class.

   Where is the method created?

   _____

   _____

   _____

4. At the beginning of the **`validatePrice`** method, declare a structured data object that you type with the line type of **`failed-flight`** (suggested name: **`failed_record`**). Similarly, declare a structured data object that you type with the line type of **`reported-flight`** (suggested name: **`reported_record`**).

© Copyright. All rights reserved.

Note:
These structures will be used to add rows to **failed-flight** and **reported-flight**, in case the validation finds an error.

5. Use a READ ENTITIES statement to read the user input from the transactional buffer. Use the IN LOCAL MODE addition and ensure that only the key fields and the field **Price** are read. Use an inline declaration for the result set (suggested name: **flights**).

Note:
It is not necessary to list the key fields after the FIELDS addition. READ ENTITIES always reads the key fields.

6. Implement a loop over the data that you just read. Use an inline declaration for the work area (suggested name: **flight**).
7. Inside the loop, check if the **Price** component is greater than zero. If not, fill the structure **failed_record** with the key of the current flight and add it as new row to table **failed-flight**. Similarly, fill structure **reported_record** with the key of the current flight and add it as new row to table **reported-flight**.

Hint:
In draft-enabled business objects, it is recommended that you use the component **%tky** to assign the key.

8. Before the APPEND statement fill the **%msg** component of the **reported_record** structure with a reference to a message object. To create the message object, call the method **new_message** with the following input:

Parameter name	Value
id	'/LRN/S4D400'
number	'101'
severity	ms-error

9. Activate the class.

Task 2: Test and Debug
Set a breakpoint in the validation implementation. Restart the preview of the OData UI service and change the price of an existing flight. Make valid and invalid entries, and debug the validation.

1. Set a breakpoint at the READ ENTITIES statement of the **validatePrice** method.
2. Restart the preview of the OData UI service.
3. In the app, display the list of flights, open the details for one of the flights and switch to change mode.

© Copyright. All rights reserved.

4. Make some changes to the price, and then choose *Save*. Analyze the validation in the debugger.

© Copyright. All rights reserved.

Unit 8
Solution 20

Validate the Flight Price

In this exercise you define and implement a validation for the flight price.

Template:

none

Solution:

/LRN/S4D400_R_FLIGHT (Behavior Definition)
/LRN/BP_S4D400_R_FLIGHT (Global Class)

Prerequisites

You completed the previous exercises. You created and filled the database table **Z##FLIGHT** (where ## is your group number) and generated the development objects for an OData UI service.

Task 1: Validate the Price

Define and implement a validation to check that the field **Price** has a positive value (suggested name: **validatePrice**). If the value is negative or equals zero, reject the change and report a suitable error message from message class **/LRN/S4D400**.

1. In the behavior definition **ZR_##FLIGHT**, define a new validation **validatePrice**. Ensure that the validation is always executed during the standard operation **create** but only if the value of **Price** changed for all other operations.

Hint:
Use code-completion wherever possible to enter the code.

   a) Open the behavior definition **ZR_##FLIGHT**.

   b) Adjust the code as follows:

```
create;
update;
delete;

validation validatePrice on save { create; field Price; }
```

2. Activate the behavior definition.
   a) Press **Ctrl + F3** to activate the behavior definition.
3. Use a quick fix to create the validation implementation method in the behavior handler class.
   a) Position the cursor on the name of the validation and choose *Ctrl + 1*.
   b) Double-click the entry *Add method for validation...*.

© Copyright. All rights reserved.

Where is the method created?

In the local class **lhc_zr_##flight** of global class **ZBP_R_##FLIGHT**.

4. At the beginning of the **validatePrice** method, declare a structured data object that you type with the line type of **failed-flight** (suggested name: **failed_record**). Similarly, declare a structured data object that you type with the line type of **reported-flight** (suggested name: **reported_record**).

Note:
These structures will be used to add rows to **failed-flight** and **reported-flight**, in case the validation finds an error.

a) Add the following code:

```
DATA failed_record   LIKE LINE OF failed-flight.
DATA reported_record LIKE LINE OF reported-flight.
```

5. Use a READ ENTITIES statement to read the user input from the transactional buffer. Use the IN LOCAL MODE addition and ensure that only the key fields and the field **Price** are read. Use an inline declaration for the result set (suggested name: **flights**).

Note:
It is not necessary to list the key fields after the FIELDS addition. READ ENTITIES always reads the key fields.

a) After the declarations, add the following code, replacing **##** with your group number:

```
READ ENTITIES OF ZR_##Flight IN LOCAL MODE
  ENTITY Flight
    FIELDS ( Price )
    WITH CORRESPONDING #(  keys )
    RESULT DATA(flights).
```

6. Implement a loop over the data that you just read. Use an inline declaration for the work area (suggested name: **flight**).

a) After the EML statement, add the following code:

```
LOOP AT flights INTO DATA(flight).

ENDLOOP.
```

7. Inside the loop, check if the **Price** component is greater than zero. If not, fill the structure **failed_record** with the key of the current flight and add it as new row to table **failed-flight**. Similarly, fill structure **reported_record** with the key of the current flight and add it as new row to table **reported-flight**.

Hint:
In draft-enabled business objects, it is recommended that you use the component **%tky** to assign the key.

© Copyright. All rights reserved.

a) Inside the loop, add the following code:

```
IF flight-price <= 0.

  failed_record-%tky = flight-%tky.
  APPEND failed_record TO failed-flight.

  reported_record-%tky = flight-%tky.
  APPEND reported_record TO reported-flight.
ENDIF.
```

8. Before the APPEND statement fill the **%msg** component of the **reported_record** structure with a reference to a message object. To create the message object, call the method **new_message** with the following input:

Parameter name	Value
id	'/LRN/S4D400'
number	'101'
severity	ms-error

a) Adjust the code as follows:

```
LOOP AT flights INTO DATA(flight).
  IF flight-price <= 0.

    failed_record-%tky = flight-%tky.
    APPEND failed_record TO failed-flight.

    reported_record-%tky = flight-%tky.
    reported_record-%msg = new_message(
                      id       = '/LRN/S4D400'
                      number   = '101'
                      severity = ms-error ).
    APPEND reported_record TO reported-flight.

  ENDIF.
ENDLOOP.
```

9. Activate the class.
   a) Press **Ctrl + F3** to activate the class.

Task 2: Test and Debug

Set a breakpoint in the validation implementation. Restart the preview of the OData UI service and change the price of an existing flight. Make valid and invalid entries, and debug the validation.

1. Set a breakpoint at the READ ENTITIES statement of the **validatePrice** method.
   a) In the implementation of the **validatePrice** method, find the READ ENTITIES statement and double-click the area to the left of the row number to set a breakpoint.
2. Restart the preview of the OData UI service.
   a) Close the browser window or browser tab that contains the preview.
   b) Open the **ZUI_##FLIGHT_O4** Service Binding. In the *Entity Set and Association* list on the right-hand side, first choose the *Flight* entry and then choose *Preview...*.
3. In the app, display the list of flights, open the details for one of the flights and switch to change mode.

© Copyright. All rights reserved.

a) Proceed as you have done in previous exercises.

4. Make some changes to the price, and then choose *Save*. Analyze the validation in the debugger.

a) Proceed as you have done in previous exercises.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Implement the behavior of a Business Object.

© Copyright. All rights reserved.

Unit 8
Lesson 4

Improving the User Experience

LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Arrange fields in the app.
- Provide input help.

UI Adjustment

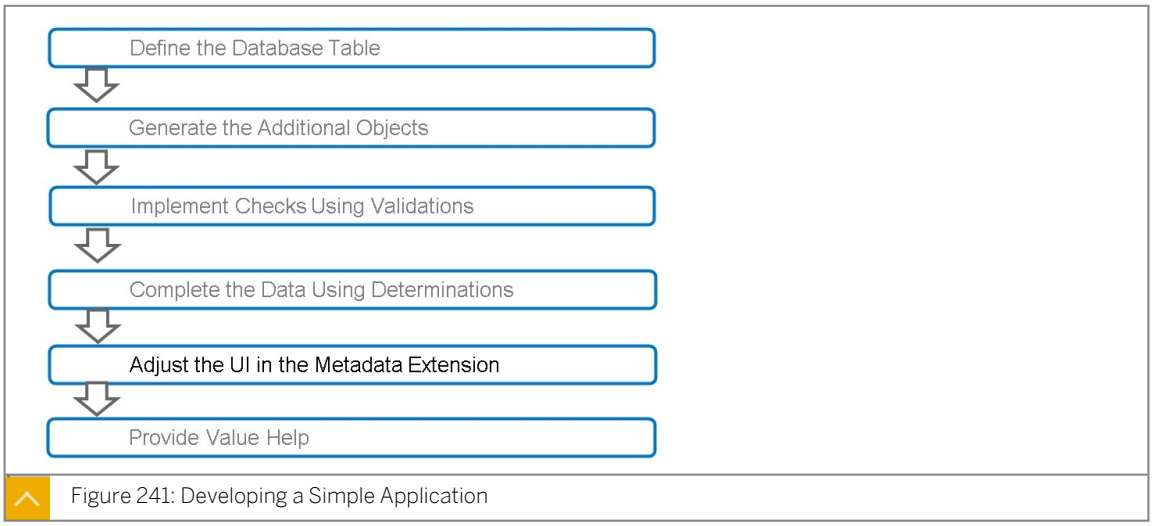


Figure 241: Developing a Simple Application

In this section, you will learn how to control field properties in the behavior definition and how to adjust the UI in the metadata extension.

© Copyright. All rights reserved.

Arranging Fields in the UI

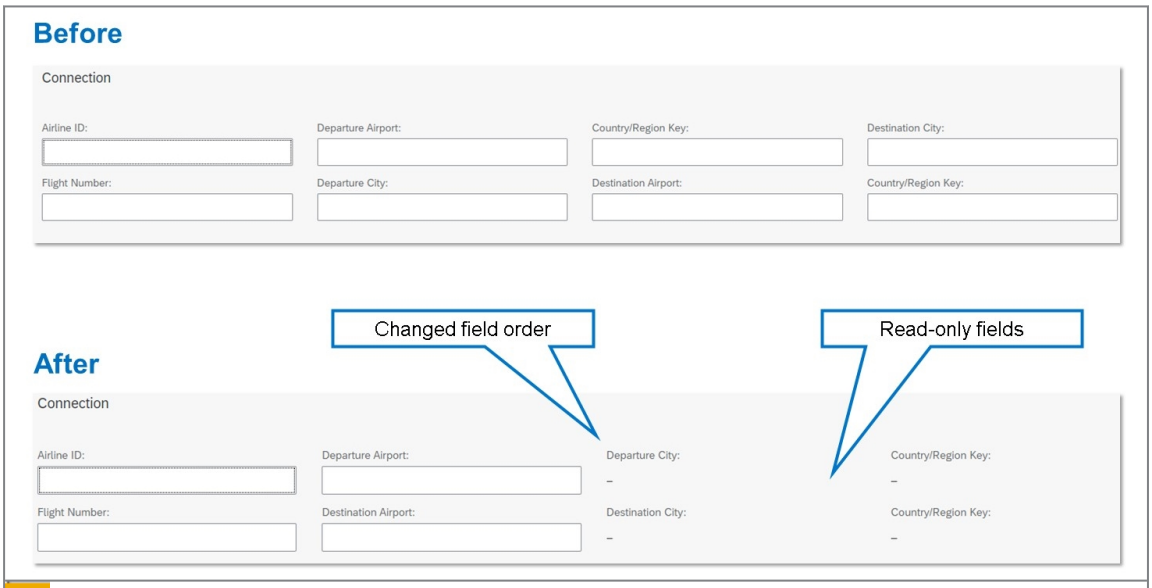


Figure 242: Adjusting the UI

When you generate an SAP Fiori elements app, the system creates a default user interface. The fields are displayed input-ready in the sequence in which they appear in the CDS entity.

In the flight connection example, you can make the information easier to read by changing the order of the fields. You can also make the fields that are filled automatically by the determination read-only.

Making Fields Read-only

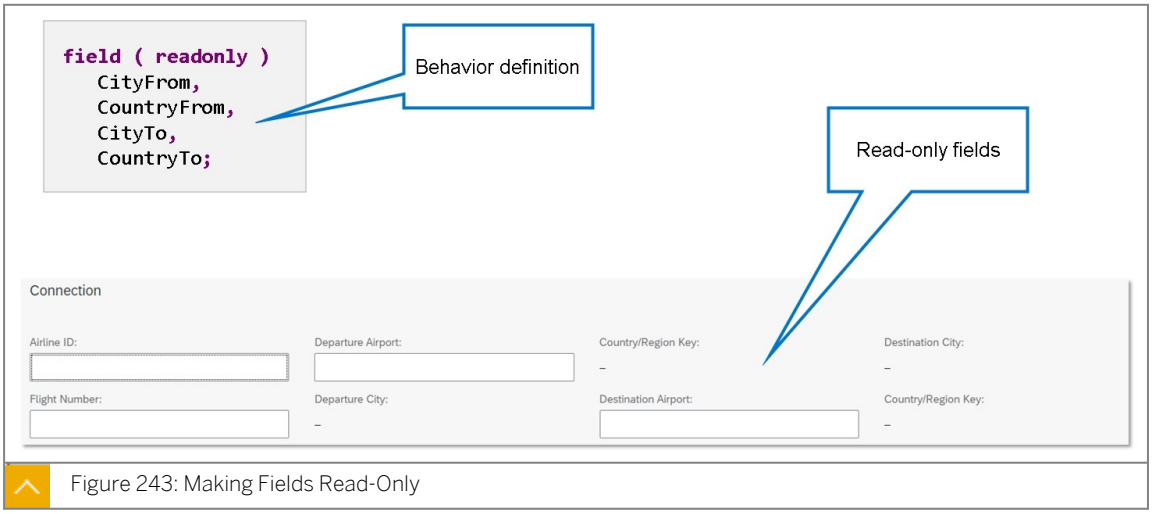


Figure 243: Making Fields Read-Only

You make fields read-only in the behavior definition or the behavior projection. It depends on whether you want them to be read-only in general or only for your particular service.

In our example, we make the city and country fields read-only on the data model level because their values should always be provided by the determination. If we kept them editable we would have to implement a validation to make sure the cities and countries match the airport IDs.

© Copyright. All rights reserved.

To define a list of fields as read-only, use the field (readonly) statement with a comma-separated list of fields. You end the list with a semicolon after the last field.

Once you have made the fields read-only, you can see that it makes sense to rearrange them. You do this by annotating the corresponding fields of the projection. Most commonly, you do this in a metadata extension.

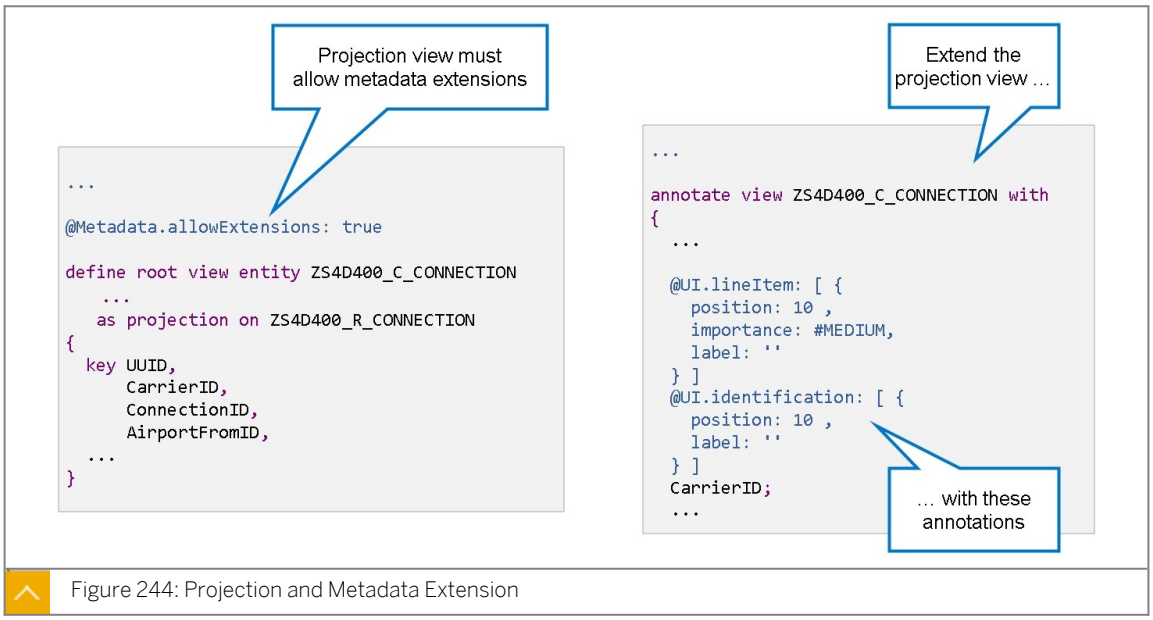


Figure 244: Projection and Metadata Extension

Metadata extensions are standalone repository objects that contain the annotations belonging to a particular CDS entity. The idea behind them is to make it easier to understand both the definition of the CDS entity and the annotations by separating them. Otherwise, the CDS definition would become very long and difficult to read.

If you want to use a metadata extension in conjunction with a CDS view, the CDS view must contain the annotation `@Metadata.allowExtensions: true`. The generator takes care of this for you and automatically generates both a metadata extension for the projection view and the corresponding annotation in the projection view itself.

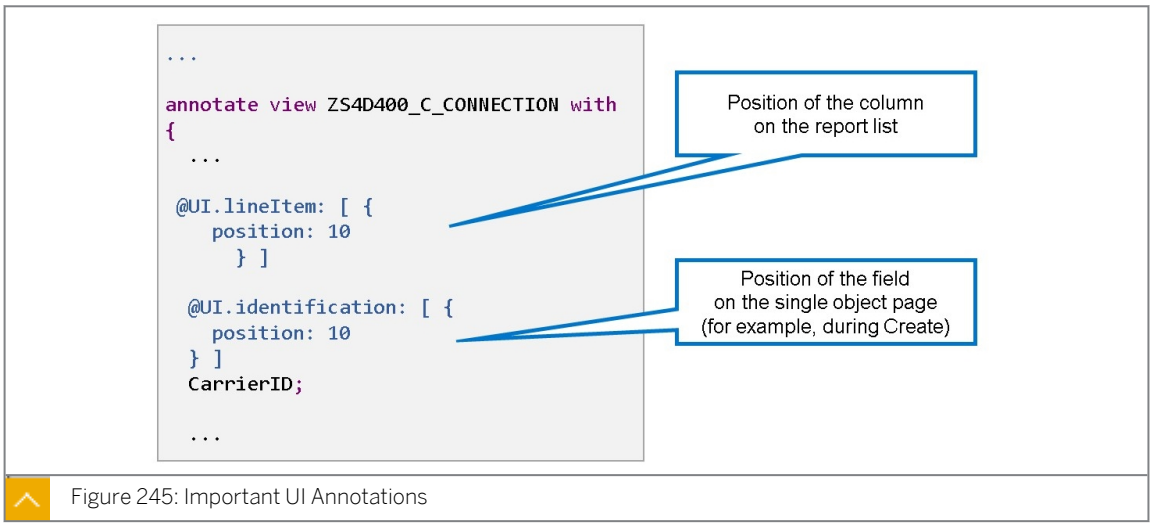


Figure 245: Important UI Annotations

There are two important UI annotations that you might need to change in the metadata extension. The first is `@UI.LineItem`. This is an array in which you set the attribute *position*. This determines the position of the column in the report list page of the app.

© Copyright. All rights reserved.

The second annotation is `@UI.identification`. This is, again, an array in which you also set the attribute *position*. This determines the position of the field in the single object page. that is displayed during the create or update operation.

Rearranging the Fields

Figure 246: Rearranging the Fields

To change the position of the fields, you assign a new value to the attribute`@UI.identification: [ { position: } ]`. To rearrange the fields so that they are displayed as in the "After" figure, you must change the position of the *AirportToID* field from 60 to 40, the position of the *CityFrom* field from 40 to 50, the position of the *CountryFrom* field from 50 to 70, and so on.

Input Help

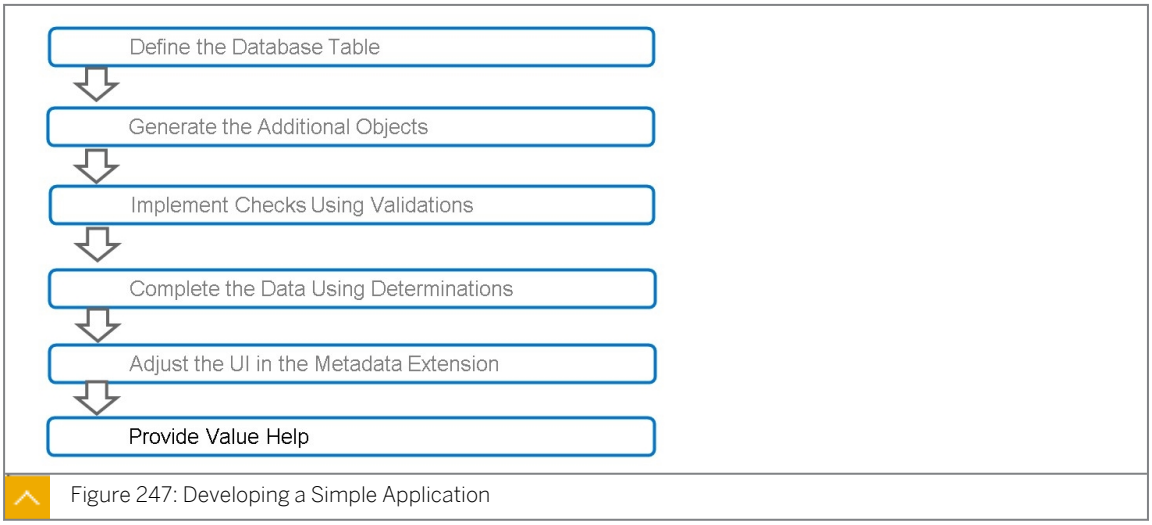


Figure 247: Developing a Simple Application

© Copyright. All rights reserved.

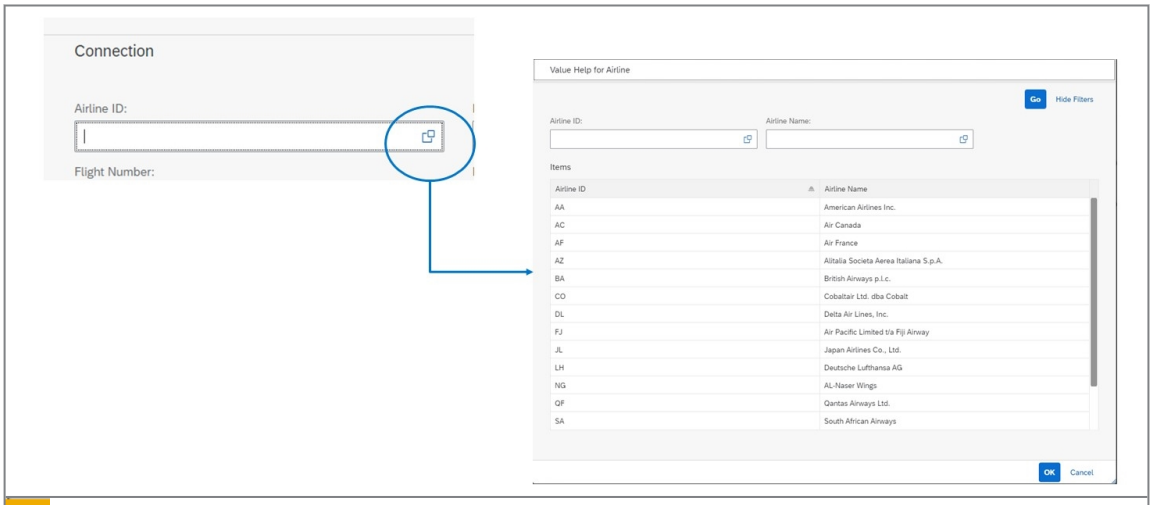


Figure 248: Providing Input Help

A feature of all of SAP's user interface solutions over the years has been value help, which gives the user the possibility to display a list of possible values that can be entered in a field. Typically, this is achieved by reading data from the database. In Fiori Elements, you can provide a value help by defining a CDS View that displays the possible values and attaching it to a field in the app.

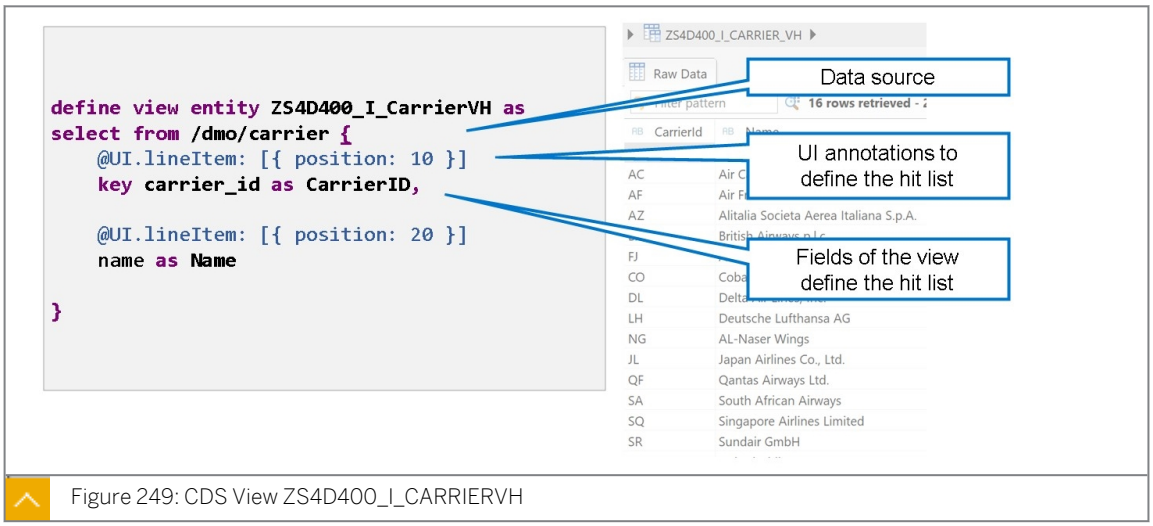


Figure 249: CDS View ZS4D400_I_CARRIERVH

This CDS view selects data from the database table /DMO/CARRIER, which contains a list of airlines. It returns two columns - the airline code and its name. These columns will form the hit list of the value help.

The view also contains `@UI.lineItem` annotations. These define how the hit list will look on the screen - in this case with the airline code first and its name second.

© Copyright. All rights reserved.

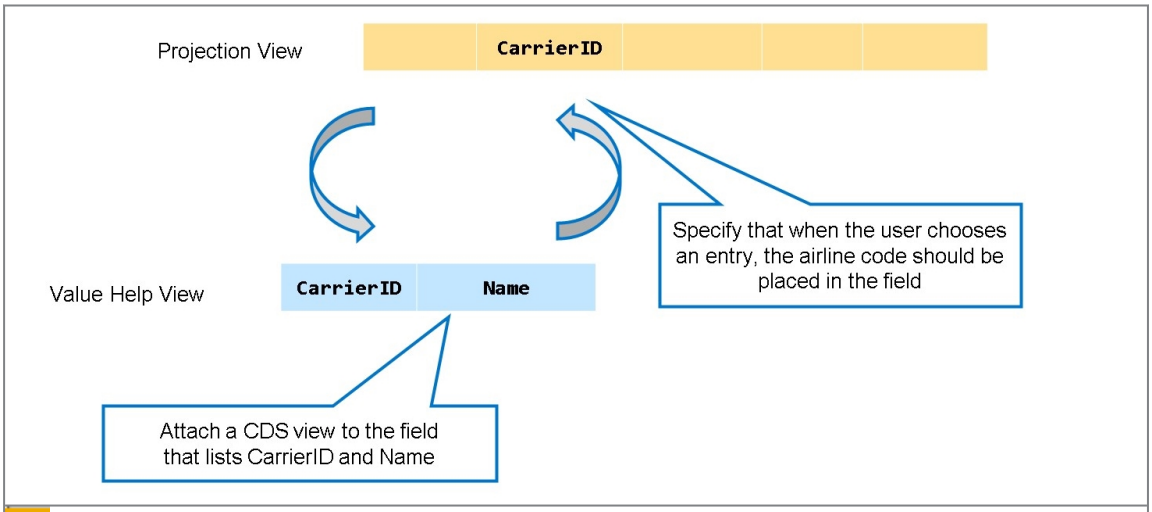


Figure 250: Attaching a Value Help to a Field

To provide a value help, you attach the view that defines the hit list to a field of your projection view using an annotation. The system then displays the field with a value help button. When the user clicks the button, the system reads the possible values using the view and displays the hit list accordingly.

When the user chooses an entry from the hit list, the system must return a single value to the field. In the annotation, you must also specify which field of the value help hit list should be placed in the input field.

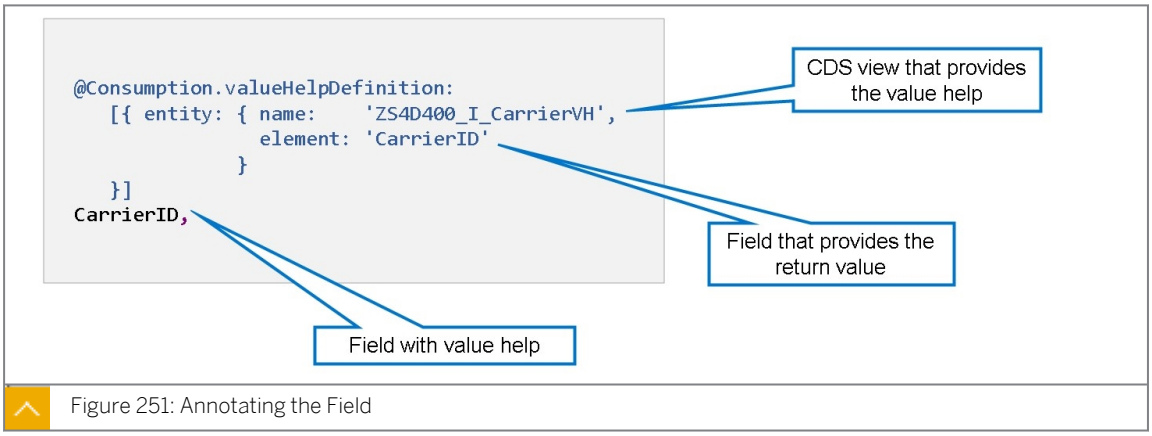


Figure 251: Annotating the Field

To attach the value help to the field, use the annotation `@Consumption.valueHelpDefinition`. This is an array that contains an object *entity*. *entity* has two attributes: *name* is the name of the CDS view that the system should use to provide the value help, *element* is the name of the element of the CDS view that contains the value that should be used when the user chooses an entry from the value help.

© Copyright. All rights reserved.

Unit 8
Exercise 21

Adjust the User Interface

In this exercise, you will explore the value help provided in the application. In addition, you will adjust the user interface of the generated app: either directly by editing the UI metadata, or indirectly by adjusting the business object behavior.

Template:

none

Solution:

/LRN/S4D400_C_FLIGHT (Behavior Projection)
/LRN/S4D400_C_FLIGHT (Metadata Extension)

Prerequisites

You completed the previous exercises. You created and filled the database table **`Z##FLIGHT`** (where ## is your group number) and generated the development objects for an OData UI service.

Task 1: Analyze the Value Help

Verify that a value help is offered for the *Currency* field on the Object Page. Examine where and how this value help is defined.

1. Launch the preview of the OData UI service.
2. Open the Object Page for a flight in edit mode.
3. Verify that a value help is available for the *Currency* field.
4. Return to the development environment and explore the definition of the value help.

   Which annotation is used in the projection view *ZC_##Flight* to define the value help?

   _____
   _____
   _____

   Which CDS view provides the value help?

   _____
   _____
   _____

Task 2: Adjust the Behavior

Exclude the standard operations *create* and *delete*, and set the **`PlaneTypeID`** field to read-only .

© Copyright. All rights reserved.

Note:
These changes should only apply to your OData UI service. The business object itself should still support the creation and deletion of flights, and the changing of the airplane type.

1. In the behavior **projection** for your service, disable the standard operations *create* and *delete*.
2. In the behavior **projection** of your business object, set the **`PlaneTypeID`** field to read-only.
3. Activate the behavior projection.

Task 3: Adjust the UI Metadata

Hide the administrative fields from the UI. Make sure only **`CarrierID`** and **`ConnectionID`** are displayed as selection fields on the report list page. Change the field sequence on the Object Page so that **`PlaneTypeID`** is displayed before **`Price`**.

1. Edit the metadata extension of the projection view **`ZC_##Flight`**. Remove all UI annotations for the fields **`LocalCreatedBy`**, **`LocalCreatedAt`**, **`LocalLastChangedBy`**, **`LocalLastChangedAt`**, and **`LastChangedAt`** and replace them with **`@UI.hidden: true`**.
2. Remove the **`@UI.selectionField`** annotation for all elements except for the elements **`CarrierID`** and **`ConnectionID`**.
3. Change the position of the **`PlaneTypeID`** field on the Object Page. Place it between **`FlightDate`** and **`Price`**.

Hint:
The position on the Object Page is specified with annotation **`@UI.identification`**.

4. Activate the metadata extension.

Task 4: Verify the Result

Restart the preview of the OData UI service and verify the changes to the user interface.

1. Restart the preview of the OData UI service.
2. On the Report List Page, verify that the buttons *Create* and *Delete* have disappeared.
3. Make sure that no more administrative fields are displayed on the Report List Page.
4. Verify that - apart from the *Editing Status* field - only two selection fields are visible at the top of the Report List Page. Test their functionality.

Hint:
Be aware that the selection fields support the entry of multiple values and ranges.

© Copyright. All rights reserved.

5. For any data set, navigate to the Object Page. Verify that the *Delete* button has disappeared.
6. Make sure that no more administrative fields are displayed on the Object Page.
7. Verify that the *Plane Type* field is displayed after the *Flight Date* field and before the *Flight Price* field.
8. Switch to edit mode and verify that the value of the *Plane Type* field cannot be changed.

© Copyright. All rights reserved.

Unit 8
Solution 21

Adjust the User Interface

In this exercise, you will explore the value help provided in the application. In addition, you will adjust the user interface of the generated app: either directly by editing the UI metadata, or indirectly by adjusting the business object behavior.

Template:

none

Solution:

/LRN/S4D400_C_FLIGHT (Behavior Projection)
/LRN/S4D400_C_FLIGHT (Metadata Extension)

Prerequisites

You completed the previous exercises. You created and filled the database table **`Z##FLIGHT`** (where ## is your group number) and generated the development objects for an OData UI service.

Task 1: Analyze the Value Help

Verify that a value help is offered for the *Currency* field on the Object Page. Examine where and how this value help is defined.

1. Launch the preview of the OData UI service.
   a) Proceed as you have done in previous exercises.
2. Open the Object Page for a flight in edit mode.
   a) Proceed as you have done in previous exercises.
3. Verify that a value help is available for the *Currency* field.
   a) Open the value help for the *Currency* field displayed for the flight price.
4. Return to the development environment and explore the definition of the value help.

   Which annotation is used in the projection view *ZC_##Flight* to define the value help?

   `@Consumption.valueHelpDefinition`

   Which CDS view provides the value help?

   `I_CurrencyStdVH`

Task 2: Adjust the Behavior

Exclude the standard operations *create* and *delete*, and set the **`PlaneTypeID`** field to read-only .

© Copyright. All rights reserved.

Note:
These changes should only apply to your OData UI service. The business object itself should still support the creation and deletion of flights, and the changing of the airplane type.

1. In the behavior **projection** for your service, disable the standard operations *create* and *delete*.
   a) Open the behavior projection **ZC_##FLIGHT** (where **##** is your group number).
   b) Add comment markers in front of the statements `use create;` and `use delete;`.
      To add the comment markers, proceed as usual: select the code rows you want to comment and press **Ctrl + <**.
      Your code should then look like this:

```
//  use create;
use update;
//  use delete;
```

2. In the behavior **projection** of your business object, set the **PlaneTypeID** field to read-only.
   a) In the behavior projection, add the following code:

```
field ( readonly ) PlaneTypeID;
```

Note:
The precise position of the statement is not important. We suggest that you place it before the **use...** statements.

3. Activate the behavior projection.
   a) Press **Ctrl + F3** to activate the behavior projection.

Task 3: Adjust the UI Metadata

Hide the administrative fields from the UI. Make sure only **CarrierID** and **ConnectionID** are displayed as selection fields on the report list page. Change the field sequence on the Object Page so that **PlaneTypeID** is displayed before **Price**.

1. Edit the metadata extension of the projection view **ZC_##Flight**. Remove all UI annotations for the fields **LocalCreatedBy**, **LocalCreatedAt**, **LocalLastChangedBy**, **LocalLastChangedAt**, and **LastChangedAt** and replace them with **@UI.hidden: true**.
   a) Open the metadata extension **ZC_##FLIGHT**, where **##** is your group number.
   b) Adjust the code as follows:

```
@UI.hidden: true
LocalCreatedBy;

@UI.hidden: true
LocalCreatedAt;

@UI.hidden: true
LocalLastChangedBy;
```

© Copyright. All rights reserved.

```
@UI.hidden: true
LocalLastChangedAt;

@UI.hidden: true
LastChangedAt;
```

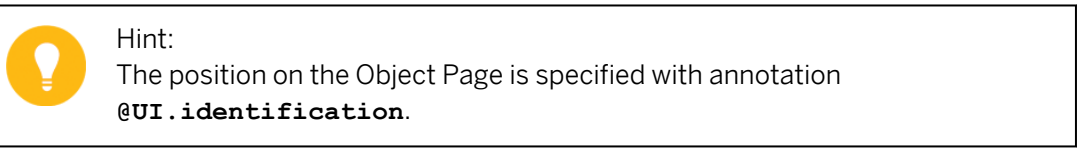
2. Remove the **`@UI.selectionField`** annotation for all elements except for the elements **`CarrierID`** and **`ConnectionID`**.
   a) Adjust the code as follows:

```
@UI.identification: [ {
  position: 30
} ]
@UI.lineItem: [ {
  position: 30
} ]
FlightDate;

@UI.identification: [ {
  position: 40
} ]
@UI.lineItem: [ {
  position: 40
} ]
Price;

@UI.identification: [ {
  position: 50
} ]
@UI.lineItem: [ {
  position: 50
} ]
PlaneTypeID;
```

3. Change the position of the **`PlaneTypeID`** field on the Object Page. Place it between **`FlightDate`** and **`Price`**.



   a) Change the value of annotation **`@UI.identification: [ { position: } ]`** for the **`PlaneTypeID`** element. Choose a value that lies between the values for the elements **`FlightDate`** and **`Price`**.

   b) Adjust the code as follows:

```
@UI.identification: [ {
  position: 35
} ]
@UI.lineItem: [ {
  position: 50
} ]
PlaneTypeID;
```

4. Activate the metadata extension.
   a) Press **`Ctrl + F3`** to activate the metadata extension.

Task 4: Verify the Result

Restart the preview of the OData UI service and verify the changes to the user interface.

© Copyright. All rights reserved.

1. Restart the preview of the OData UI service.
   a) Close the browser window or browser tab that contains the preview.
   b) Open the **ZUI_##FLIGHT_O4** Service Binding. In the *Entity Set and Association* list on the right-hand side, first choose the *Flight* entry and then choose *Preview...*.
2. On the Report List Page, verify that the buttons *Create* and *Delete* have disappeared.
   a) Proceed as you have done in previous exercises.
3. Make sure that no more administrative fields are displayed on the Report List Page.
   a) Proceed as you have done in previous exercises.
4. Verify that - apart from the *Editing Status* field - only two selection fields are visible at the top of the Report List Page. Test their functionality.

Hint:
Be aware that the selection fields support the entry of multiple values and ranges.

   a) Enter a value in the selection field *Airline ID* (for example: **LH**) and choose *Go*.
   b) Repeat with other values.
5. For any data set, navigate to the Object Page. Verify that the *Delete* button has disappeared.
   a) Proceed as you have done in previous exercises.
6. Make sure that no more administrative fields are displayed on the Object Page.
   a) Proceed as you have done in previous exercises.
7. Verify that the *Plane Type* field is displayed after the *Flight Date* field and before the *Flight Price* field.
   a) Proceed as you have done in previous exercises.
8. Switch to edit mode and verify that the value of the *Plane Type* field cannot be changed.
   a) Proceed as you have done in previous exercises.

© Copyright. All rights reserved.

LESSON SUMMARY

You should now be able to:

- Arrange fields in the app.
- Provide input help.

© Copyright. All rights reserved.

Learning Assessment

1. You want to create an application according to the ABAP RESTful Application Programming Model using the Object Generator. What kind of development object do you need as a starting point?

   Choose the correct answer.

   - ☐ A Database Table
   - ☐ B CDS Entity
   - ☐ C Behavior Definition
   - ☐ D Metadata Extension
   - ☐ E Service Definition
   - ☐ F Service Binding

2. Which of the following can you use to specify the data type of a column in a database table?

   Choose the correct answer.

   - ☐ A Domain
   - ☐ B Data element
   - ☐ C Local type

3. You want to generate the development objects for a simple ABAP RESTful application. When you define the database table you must create a client field. Which data type must it have?

   Choose the correct answer.

   - ☐ A The built-in ABAP Dictionary type abap.char(3)
   - ☐ B The built-in ABAP Dictionary type abap.clnt.
   - ☐ C The data element MANDT.

© Copyright. All rights reserved.

4. Which of the following parts of a behavior definition are generated automatically?

   Choose the correct answers.

   - ☐ A Draft enabling
   - ☐ B Validations
   - ☐ C Determinations
   - ☐ D Create, update, and delete operations

5. You need to display an error text. Where do you create it?

   Choose the correct answer.

   - ☐ A In a text pool.
   - ☐ B In a global class.
   - ☐ C In a message class.

6. When you create a validation, what does the system generate automatically?

   Choose the correct answer.

   - ☐ A An empty method in the local class of the behavior implementation.
   - ☐ B An empty method in the global class of the behavior implementation.
   - ☐ C A fully-implemented method in the local class of the behavior implementation.
   - ☐ D A fully-implemented method in the global class of the behavior implementation.

7. In the validation, you use the READ ENTITIES statement to read the data entered by the user. Which parameter of the validation method do you use to ensure that the correct data is retrieved?

   Choose the correct answer.

   - ☐ A REPORTED
   - ☐ B FAILED
   - ☐ C KEYS

© Copyright. All rights reserved.

8. Where do you declare fields as read-only?

   Choose the correct answer.

   - ☐ A In the metadata extension
   - ☐ B In the behavior implementation
   - ☐ C In the behavior definition

9. What does the annotation @Consumption.ValueHelpDefiniton specify?

   Choose the correct answers.

   - ☐ A A list of possible entries
   - ☐ B A CDS view that provides the hit list
   - ☐ C A list of fields in the hit list
   - ☐ D The field that provides the selected value

© Copyright. All rights reserved.

Learning Assessment - Answers

1. You want to create an application according to the ABAP RESTful Application Programming Model using the Object Generator. What kind of development object do you need as a starting point?

   Choose the correct answer.

   - [X] A Database Table
   - [ ] B CDS Entity
   - [ ] C Behavior Definition
   - [ ] D Metadata Extension
   - [ ] E Service Definition
   - [ ] F Service Binding

   Correct! You need a Database Table as a starting point.

2. Which of the following can you use to specify the data type of a column in a database table?

   Choose the correct answer.

   - [ ] A Domain
   - [X] B Data element
   - [ ] C Local type

   That's right.

3. You want to generate the development objects for a simple ABAP RESTful application. When you define the database table you must create a client field. Which data type must it have?

   Choose the correct answer.

   - [ ] A The built-in ABAP Dictionary type abap.char(3)
   - [X] B The built-in ABAP Dictionary type abap.clnt.
   - [ ] C The data element MANDT.

   That's right.

© Copyright. All rights reserved.

4. Which of the following parts of a behavior definition are generated automatically?

   Choose the correct answers.

   - [X] A Draft enabling
   - [ ] B Validations
   - [ ] C Determinations
   - [X] D Create, update, and delete operations

   That's right.

5. You need to display an error text. Where do you create it?

   Choose the correct answer.

   - [ ] A In a text pool.
   - [ ] B In a global class.
   - [X] C In a message class.

   That's right.

6. When you create a validation, what does the system generate automatically?

   Choose the correct answer.

   - [X] A An empty method in the local class of the behavior implementation.
   - [ ] B An empty method in the global class of the behavior implementation.
   - [ ] C A fully-implemented method in the local class of the behavior implementation.
   - [ ] D A fully-implemented method in the global class of the behavior implementation.

   That's right.

7. In the validation, you use the READ ENTITIES statement to read the data entered by the user. Which parameter of the validation method do you use to ensure that the correct data is retrieved?

   Choose the correct answer.

   - [ ] A REPORTED
   - [ ] B FAILED
   - [X] C KEYS

   That's right.

© Copyright. All rights reserved.

8. Where do you declare fields as read-only?

   Choose the correct answer.

   - [ ] A In the metadata extension
   - [ ] B In the behavior implementation
   - [x] C In the behavior definition

   That is correct.

9. What does the annotation @Consumption.ValueHelpDefiniton specify?

   Choose the correct answers.

   - [ ] A A list of possible entries
   - [x] B A CDS view that provides the hit list
   - [ ] C A list of fields in the hit list
   - [x] D The field that provides the selected value

   That's right.

© Copyright. All rights reserved.